

IT の予備知識がない大人のための自習用教材

IT インフラ・ AI 基盤 自習ガイド

予備知識ゼロから、1 ページずつ積み上げる

元資料：会社「IT インフラ」_AI 基盤_統合ガイド（159 枚）を初学者向けに再構築

確認基準日：元資料の 2026 年 10 月 3 日

買い物サイト「サンプル商店」・利用者の佐藤さん・番号などは、すべて説明用の例（架空）です

スマホで「注文履歴を見る」を押すと何が起こる？

佐藤さんがスマホの買い物サイトで「注文履歴を見る」を押すと、数秒で買った商品の一覧が出ます。その間に、スマホ、通信の網、お店のコンピューターが順番に仕事をしています。

灰色の線＝つながり、黒い矢印＝頼みごとが届く向き、緑の破線の矢印＝返事が戻る向き。丸数字は順番。



この資料では、この1つの操作の裏側で「何が」「どこで」「どう動いているか」を、1ページずつ説明していきます。

この資料で説明できるようになること

最後まで読むと、次のことを自分の言葉で説明できるようになります。

- ① コンピューターの中で、命令とデータがどう動いて計算が進むか（第 1 ～ 3 章）
- ② お店のコンピューターが、注文の記録をどう保存し、探し出すか（第 4 ・ 5 ・ 8 章）
- ③ スマホからお店まで、頼みごとと返事がどんな道筋で届くか（第 6 ・ 7 章）
- ④ クラウドや AI のサービスを支える機械・設備・ソフト（第 9 ・ 10 ・ 12 章）
- ⑤ 安全に使い続ける守り方と、仕組みを設計・構築・運用する技術者（SE）の仕事（第 11 ・ 13 章）
- ⑥ IT 業界の会社・製品・給与の情報を、区分と条件に注意して読む方法（第 14 ～ 16 章）

略語や専門用語は、使う前にその場で説明します。わからない言葉が出てきたら、少し前のページに戻ってください。

読み方①：ページの見方と、青字の意味

① 章の名前

② 情報の区分

タイトル：このページの問い

本文と図：文章で説明します

データ ← 青字の例（初出は太字）

③ 出典： S78 | 元資料： p.19

④ 12 / 300

- ① 左上：いま読んでいる章
- ② 右上：情報の区分（技術の説明か、説明用の例か、など。 p.8 ）
- ③ 下：内容のよりどころ（出典の番号）と、もとにした元資料のページ
- ④ 右下：この資料のページ番号

青字の意味

青字は重点的に覚えない用語です（基本情報技術者試験の学習にも役立つ用語）。資料の中で初めて出るところは太字にしています。青字の用語は、同じページか前のページに、名前と意味の説明があります。

読み方②：図の「形」の約束

図の箱は、形と色で「何を表すか」を区別します。全ページで同じ約束です。



スマホ

灰色・太い線の箱：機械
(ハードウェア)

機械の中の部品

薄い灰色・細い線：機械の
中に取り付ける部品



プログラム

紫の箱：ソフトウェア（機
械に仕事をさせる手順）

注文の記録

緑の書類の形：データ（記
録された値）

受付の役

点線の箱：役割（何を担当
するか）。機械の名前では
ない

家の中

破線の大きな枠：場所・建
物・範囲



佐藤さん

オレンジの箱：人・利用者

説明中の箇所

黄色の太い枠：そのページ
で説明している箇所

説明用の例

学習用に簡略化した図

図の右下の札「学習用に簡略化した図」は実物を単純にした図、「説明用の例」は説明のために作った例であることを示します。

読み方③：線と矢印の約束

線は「つながり」、矢印は「情報が動く向き」です。意味が違うので、1つの図に両方が出ることもあります。

灰色の太い線



ケーブル（電線や光ファイバー）で機器が物理的につながっている。矢印なし

黒い矢印



情報が移動する向き。線の上に中身、丸の数字は順番

灰色の破線



電波（無線）でつながっている。矢印なし

緑の破線の矢印



返事（結果）が戻る向き

茶色の極太線



電気（電力）の供給

紫の破線の矢印



本題の前の補助的な問い合わせ（相手の場所を調べる、など）

図ごとに、どの線を使ったかを図の見方の文で説明します。

読み方④：図の3つの種類

伝えたいことに合わせて図を描き分け、図の見方の文に種類を書きます。

🏠 物理的な配置図

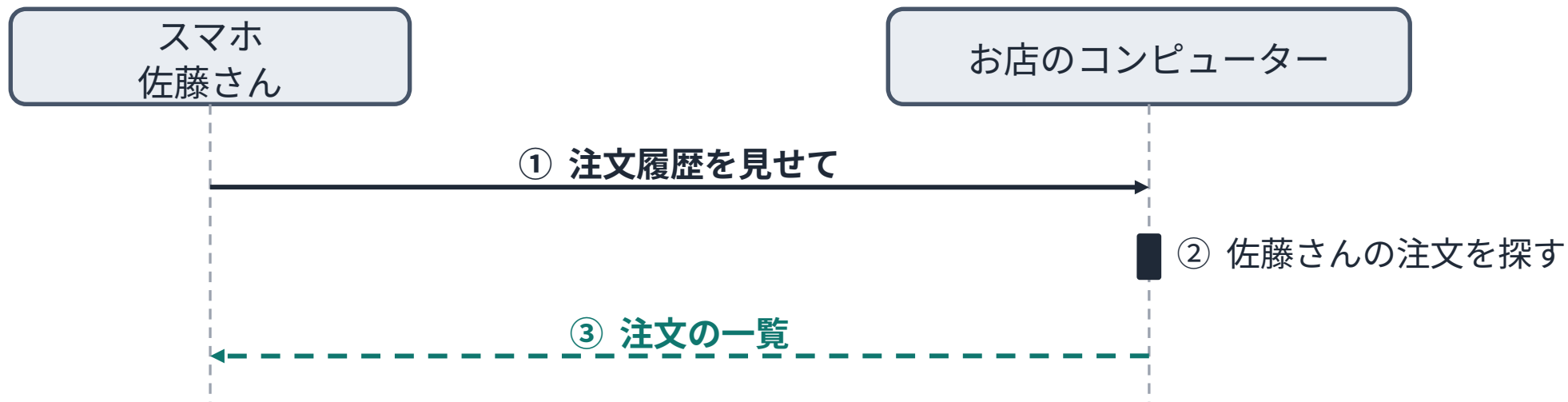
どこに何が置かれ、何の線でつながっているか。例：家の中の機器の位置。

🔄 役割の関係図

どの役割がどの役割に何を頼むか。実物の置き場所や台数とは限らない。

🕒 時間順のやり取り図

上から下へ時間が進む。誰が、どの順で何を送ったか（下の例）。



このほか、上下に積み重ねた「層の図」と、比べるための「表」も使います。

右上の「情報の区分」の意味（1/2）

同じように見える文でも、確かさは区分によって違います。特に数字は、区分と条件を確かめて読みます。

区分	意味	読むときの注意
技術説明	仕組みについての一般的な説明	製品や構成によって細部は違う
一般的な構成例	よくある組み合わせの一例	すべてのシステムがこの形ではない
説明用の例（説明用の仮定）	理解のために作った架空の例（サンプル商店・佐藤さん・番号など）	実在の記録や統計ではない
企業公表	会社が自ら公表している情報	公表した時点と範囲を確かめる
公的統計	国などの公的機関が集計した統計	集計の対象と年を確かめる
公開投稿の集計	社員・元社員などが Web に投稿した内容を集めたもの	投稿した人に偏りがあり、公式の数字ではない

右上の「情報の区分」の意味（2/2）

同じように見える文でも、確かさは区分によって違います。特に数字は、区分と条件を確かめて読みます。

区分	意味	読むときの注意
原資料の提示条件	元資料が計算などの前提として示した条件	条件が変われば結果も変わる
個別の体験談	ある個人の経験	ほかの人にもあてはまるとは限らない
原資料の想定	元資料が置いた見積もりや仮定	実績ではない
学習の提案	学び方や読み方の提案	自分に合わせて変えてよい
今回未確認	作成時点で確かめられなかった情報	事実として扱わず、最新情報を自分で確認する

下の「出典」「元資料」と、確認基準日

- **出典：S78** ... その内容のよりどころを示す番号です。S01～S78は元資料の出典番号をそのまま使い、一覧と確認の状態（依頼者が確認したか、元資料に載っていたが再確認していないか）は付録にまとめています（p.752）。
- **S78とS79** ... S78は元資料の編集者による一般的な技術説明、S79はこの再構築で追加した一般的な基礎説明です。
- **元資料：p.19** ... もとにした159枚の資料の何ページ目の内容かを示します。新しく足した基礎説明のページには付きません。
- **確認基準日** ... 元資料の情報は2026年10月3日の時点で確認されたものです。会社・製品・給与・サービスの情報は、その後に変わっている可能性があります。

目次（１）：学ぶ順番とその理由

技術の土台から順に積み上げ、会社や製品の話は技術を理解した後に置きます。

章	学ぶこと（この順番にした理由）	p.
第 1 章 コンピューターとデータの基本	土台になる「データ」「プログラム」から	p.13
第 2 章 コンピューターの中を見る	命令とデータが部品の中でどう動くか	p.36
第 3 章 ソフトウェアと OS	機械の上で動くソフトの層と中心（OS）	p.104
第 4 章 サーバーと IT インフラの全体像	頼む側・応える側の関係と全体の地図	p.121
第 5 章 データベースと DB サーバー	注文の記録を整理・保存・検索する仕組み	p.137
第 6 章 ネットワークの基本	頼みごとと返事を運ぶ通信の仕組み	p.189
第 7 章 Web サービスの仕組み	第 4 ～ 6 章をつなげ、注文履歴の全体を追う	p.317
第 8 章 データの保存方式と守り方	データを失わないための保存のしかた	p.347
第 9 章 サーバー機器とデータセンター設備	計算機を止めずに動かす機械と建物	p.386

目次（２）：学ぶ順番とその理由

技術の土台から順に積み上げ、会社や製品の話は技術を理解した後に置きます。

章	学ぶこと（この順番にした理由）	p.
第 10 章 仮想化・コンテナ・クラウド	1 台を分けて使う技術と、借りて使う方式	p.410
第 11 章 セキュリティの基本	不正な利用からデータとサービスを守る	p.459
第 12 章 AI と AI 基盤	ここまでの土台の上で、AI がどう動くか	p.485
第 13 章 SE の仕事：設計・構築・運用	仕組みを作り、動かし続ける仕事	p.537
第 14 章 IT 業界の会社と HPE ・ HPE 製品	技術を理解した後で、会社と製品を見る	p.579
第 15 章 働き方・給与・企業比較の読み方	数字の区分と条件に注意して読む	p.623
第 16 章 学習・実習・キャリア	次に何をどう学ぶか	p.656
付録 資料・出典・索引	調べ直すための一覧・出典・用語索引	p.691

第 1 章

コンピューターとデータの基本

この章で学ぶこと

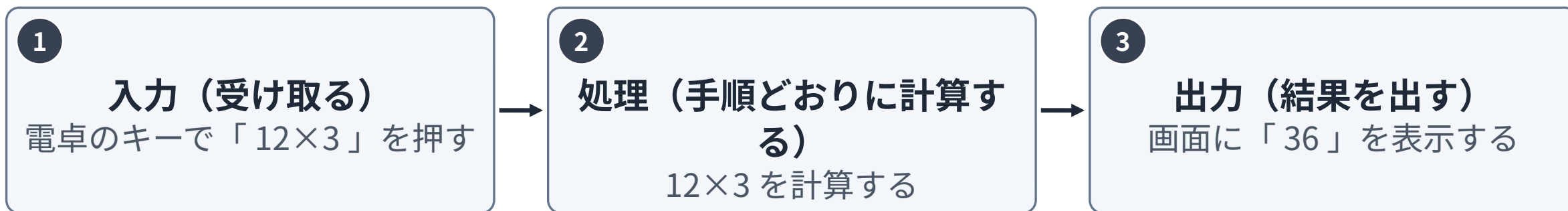
コンピューターが何をする機械か、「処理」「データ」「プログラム」「命令」とは何か、数や文字を 0 と 1 で表す方法（ビット・バイト）、そしてハードウェアとソフトウェアの違いを学びます。

→ 前の章とのつながり

はじめにで見た「注文履歴を見る」の裏側では、いくつものコンピューターが働いています。この章では、それを理解するための最も基本的な言葉を、身近な例で一つずつ確かめます。

コンピューターは何をする機械か

コンピューターは、数や文字などを受け取り、決められた手順に従って計算したり調べたりして、その結果を出す機械です。電卓もスマホも PC も、この意味ではコンピューターの仲間です。



🔑 4 つめの働き：記憶

入力・処理・出力のほかに、途中の値や結果を覚えておく「記憶」の働きもあります。スマホで「注文履歴を見る」を押すときも、押されたことを入力として受け取り、処理し、一覧を画面に出力しています。

「処理」とは何をすることか

日常では「ごみを処理する」のように片づける意味でも使いますが、ITでの**処理**は、受け取った数や文字を、決まった手順で計算したり、比べたり、探したり、形を変えたりすることです。

処理の種類	例（サンプル商店の場合。説明用の例）
計算する	3,000 円と 5,500 円を足して、合計 8,500 円を出す
比べる	300 が 250 より大きいかを調べる
探す	たくさんの注文の中から、佐藤さんの注文だけを選ぶ
並べ替える	注文を新しい順に並べる
形を変える	写真を小さくして、送りやすくする

「注文履歴を見る」も、「探す」「並べ替える」「画面用の形に整える」といった処理の組み合わせです。

データとは：注文番号・商品名・数量のような記録

データは、コンピューターで扱うために記録された、数や文字などの値です。佐藤さんがサンプル商店でノートを3冊買うと、お店の側に「誰が・何を・いくつ・いくらで」買ったかが記録されます。

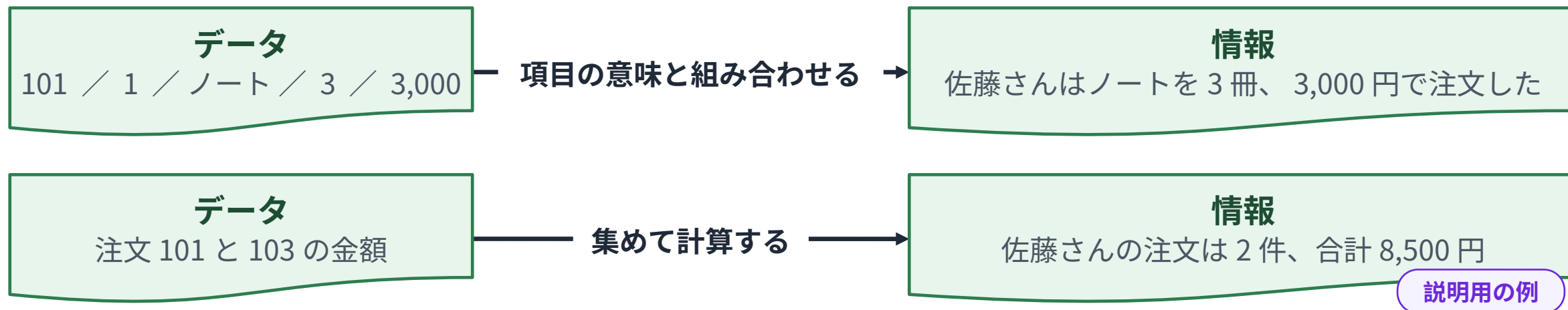
注文番号	顧客番号	商品名	数量	金額（円）
101	1	ノート	3	3,000
102	3	ペン	2	1,200
103	1	ファイル	5	5,500

横の1行が1件の注文、縦の1列が「商品名」などの項目です。顧客番号1は佐藤さん、3は高橋さんを表します。この表は説明用の例で、この後の章でも同じものを使います。

データと情報は違うか

「3」という値だけでは、何の3なのかわかりません。記録された値そのものがデータです。データを目的に合わせて組み合わせたり計算したりして、意味がわかる形にしたものを**情報**といいます。

緑の書類の形はデータや情報、黒い矢印は「手を加えてできるもの」の向きです。



コンピューターが記録・処理するのはデータです。同じデータからも、目的によって違う情報が作れます。

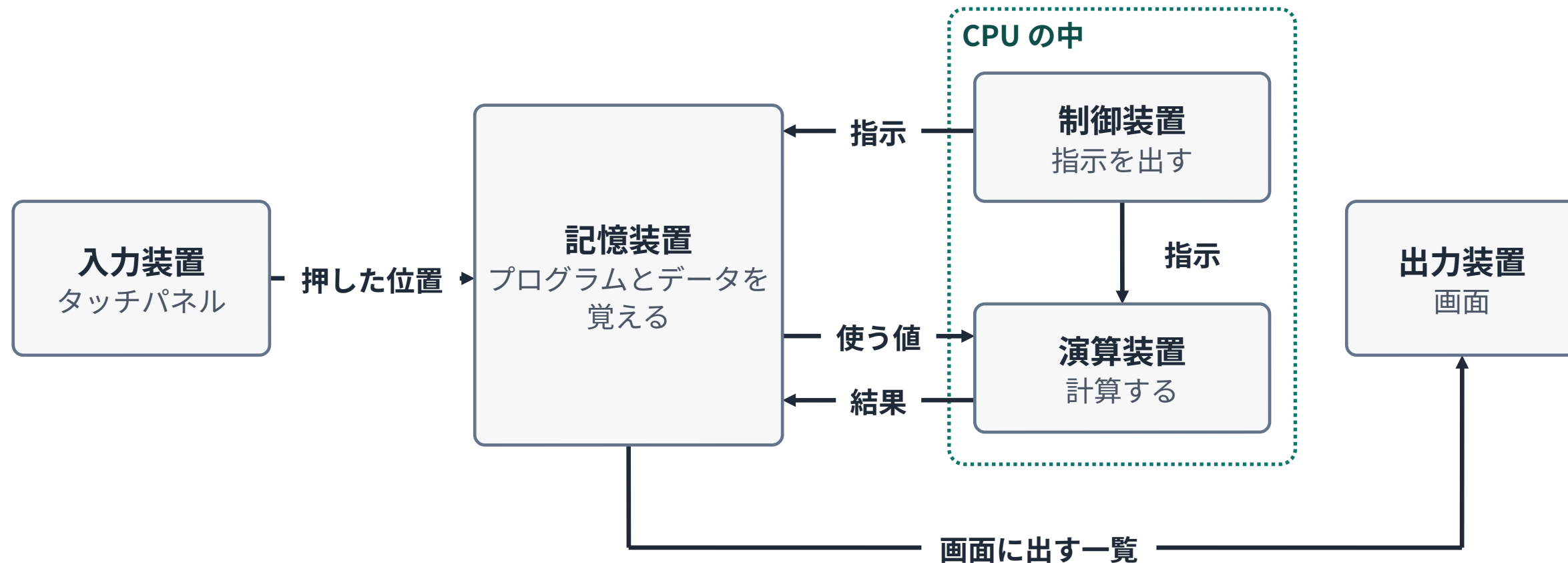
コンピューターの 5 つの働き（五大装置）

コンピューターの働きは、5 つに分けて考えるのが基本で、これを**五大装置**と呼びます。「装置」といっても、1 つの部品が 1 つの働きとは限りません（タッチパネルは入力と出力の両方を担います）。

名前	働き	スマホでの例
入力装置	外から数や文字、操作を受け取る	タッチパネル、マイク、カメラ
出力装置	処理の結果を外へ出す	画面、スピーカー
記憶装置	プログラムやデータを覚えておく	作業用の記憶部品、長く残す記憶部品
演算装置	足し算や比較などの計算をする	命令を実行する部品（CPU）の中
制御装置	ほかの装置に指示を出し、順番を決める	同じく CPU の中（第 2 章で拡大）

5つの働きは、データを受け渡して協力する

役割の関係図。黒い矢印＝データや指示が動く向き、点線の枠＝CPU という1つの部品の中。



学習用に簡略化した図

プログラムとは：仕事の手順書

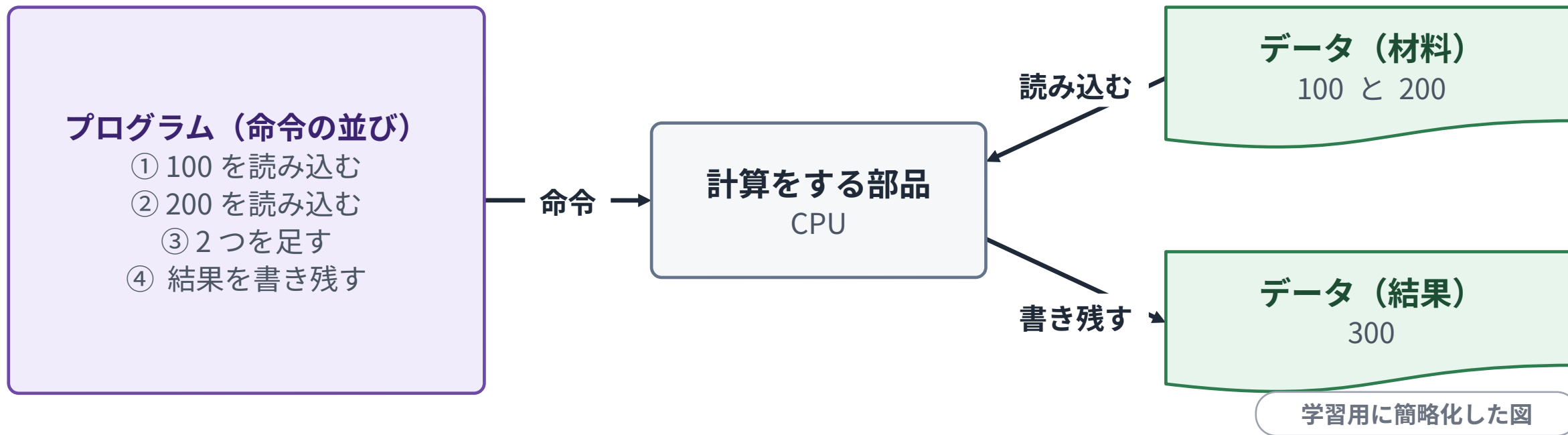
プログラム (ぷろぐらむ)

分類：ソフトウェア

何か	コンピューターにさせる仕事の手順を、決められた形で書き並べたもの。小さな指示（命令）の並び。
なぜ必要か	コンピューターは自分で考えて動くことはない。何を、どの順番で、どう処理するかを、1つ残らず指示する必要がある。
どこにあるか	ふだんは記憶装置にデータとして保存されていて、使うときに読み出されて実行される。
身近な例	電卓アプリ、買い物アプリ、「注文履歴を見る」を押したときに動く手順
似た言葉との違い	「アプリ」は利用者が使う機能をまとめたプログラムの一種。

プログラムとデータの違い：100と200を足す

紫はプログラム（手順）、緑はデータ（材料と結果）、灰色は計算をする部品です。黒い矢印は、手順やデータが動く向きです。



「何を、どの順番でするか」という指示がプログラムで、その1つ1つの小さな指示を**命令**といいます。仕事で扱う材料や結果（100・200・300）がデータです。

たとえば確かめる：レシピと材料

💡 たとえ

プログラムは料理のレシピ、データは材料と出来上がった料理のようなものです。同じレシピ（プログラム）でも、材料（データ）を変えれば、違う料理（結果）ができます。100 と 200 の代わりに 3,000 と 5,500 を渡せば、同じ「足す」手順で 8,500 が出ます。

❗ たとえの限界

人はレシピを読んで、材料が足りなければ工夫できます。コンピューターは、書かれた手順どおりにしか動きません。手順に誤りがあれば、そのとおりに誤った結果を出します。

また、レシピと材料は見た目で区別できますが、コンピューターの中では、プログラムもデータと同じく 0 と 1 の並びとして記憶装置に記録されています。どちらとして使われるかで区別されます。

命令とは：CPU が実行できる小さな指示

プログラムを細かく分けていくと、最後は「読み込む」「足す」「比べる」「書き込む」のような、ごく小さな**命令**の並びになります。命令を実行するのは、コンピューターの中のCPU という部品です。

命令の種類	していること	例
読み込む	記憶装置からデータを取り出す	100 を取り出す
足す	計算する	100 + 200 → 300
比べる	大小や、等しいかを調べる	300 は 250 より大きいかな
書き込む	結果を記憶装置に残す	300 を書き残す

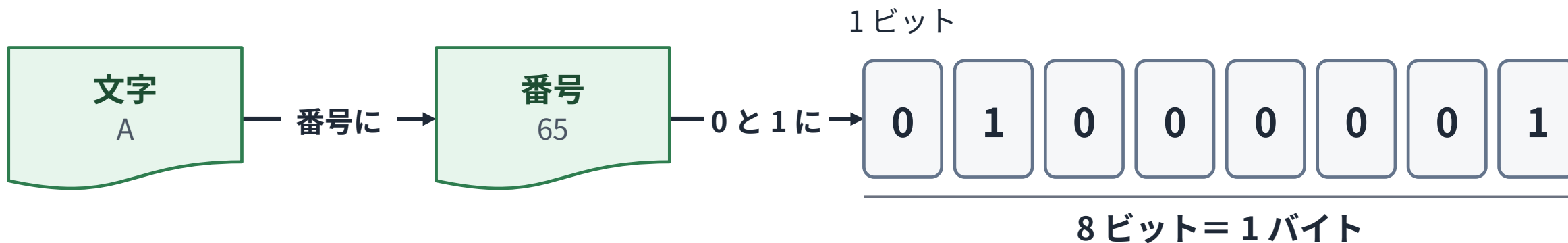
人が書くプログラムとの関係

人がプログラムを書くときは、人に読みやすい言葉（プログラミング言語）を使います。それが命令の並びに翻訳されてから実行されます。

すべてを 0 と 1 で表す：ビットとバイト

コンピューターの中では、数も文字も、最後は「0」か「1」の並びで記録・処理されます。0か1かを表す1桁を**ビット**、8ビットをまとめた単位を**バイト**といいます。

例：文字「A」が、0と1の並び（1バイト）になるまで。黒い矢印は置き換えの順です。



ふだん使う「100」などの数（10進数）も、中では1100100のような0と1の並び（**2進数**）として扱われます。

なぜ 0 と 1 なのか：2 進数の数え方

コンピューターの部品の中には、電気を通す・通さないを切り替える、ごく小さなスイッチが非常にたくさんあります。オンを 1、オフを 0 とすれば、どんな数もスイッチの並びで表せます。

2 つの状態を見分けるだけでよいので、電気の量が少しぶれても間違えにくいという利点もあります。

2 進数では、1 桁上がるごとに重みが 2 倍になります（1、2、4、8、16…）。例： $300 = 256 + 32 + 8 + 4$

10 進数	2 進数
0	0
1	1
2	10
3	11
4	100
8	1000
100	1100100
200	11001000
300	100101100

文字・画像・音も、数に置き換えて扱う

文字

1 文字ごとに番号を決めた表（文字コード）を使う。例：「A」は65。日本語の文字にも番号がある。

画像

画像を細かい点（画素）に分け、点ごとに赤・緑・青の強さを数で表す（例：0～255）。

音

音の波の高さを、1秒間に何万回も測って数の列にする。例：音楽CDは1秒に44,100回。

こうして数にすれば、どれも0と1の並び（データ）として同じように保存・処理・送信できます。注文履歴の一覧も、番号に置き換えたデータとしてスマホに届き、スマホが文字の形に戻して表示します。

番号の決め方をそろえる

送る側と受け取る側で、文字の番号の決め方（文字コード）がそろっていないと、別の文字として表示されてしまいます。これを「文字化け」といいます。

大きさの単位：KB・MB・GB・TB

データの大きさは**バイト**で数えます。大きな量は、1,000 倍ごとに単位を変えて表します。

単位	読み方	大きさ	例
KB	キロバイト	1,000 バイト	短い文章
MB	メガバイト	1,000KB（100 万バイト）	写真 1 枚
GB	ギガバイト	1,000MB（10 億バイト）	映画 1 本、スマホの保存容量
TB	テラバイト	1,000GB（1 兆バイト）	PC 用の大きな保存部品

⚠ 1,000 倍と 1,024 倍

1,024 倍（2 の 10 乗）で数える場合もあります。保存部品の容量は 1,000 倍で、PC の画面は 1,024 倍で計算して表示することがあるため、「1TB なんのに少なく見える」ことがあります。

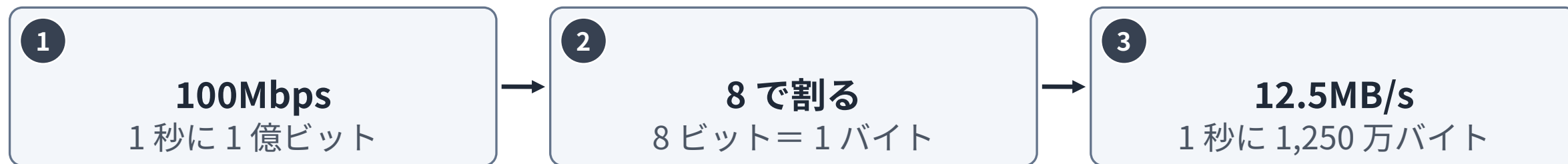
データの大きさの目安

データの例	大きさの目安
英数字 1 文字（例：A）	1 バイト
日本語 1 文字（例：あ）	多くは 3 バイト（UTF-8 という文字コードの場合）
日本語の文章 1,000 文字	約 3,000 バイト（約 3KB）
注文表の 1 行（101、1、ノート、3、3,000）	数十バイト程度
スマホで撮った写真 1 枚	数 MB 程度（画質や設定で変わる）
動画 1 分	数十 MB ～数百 MB 程度（画質で大きく変わる）

表の値はおよその目安です。文字だけのデータは小さく、写真や動画は大きくなります。この違いは、保存する場所の広さや、送るのにかかる時間に直接影響します。

ビット（b）とバイト（B）は8倍違う

通信の速さは、1秒間に何ビット運べるかで表すことが多く、小文字の「b」を使います。「bps」は bits per second（1秒あたりのビット数）の略です。保存の大きさは大文字の「B」（バイト）で表すので、比べるときは8で割ります。



📌 具体例

3MBの写真＝24Mビット。100Mbpsなら理論上0.24秒で送れます。

⚠ 注意

12.5MB/sは理論上の値です。実際は、データに付ける宛先などの付加情報や通信の混雑のため、これより少なくなります。

ハードウェアとソフトウェア

ハードウェアは手で触れる部品や機器、**ソフトウェア**はそれに何をさせるかを定めるプログラムと設定（動き方を決める値。例：画面の明るさ、つなぐ相手）です。どちらか片方だけでは動きません。

ハードウェア（機械）

- PC
- サーバー（サービスを提供するコンピューター）
- CPU（命令を実行する部品）
- メモリ（作業中のデータを置く部品）
- SSD（データを長く保存する部品）
- スイッチ（機器どうしを線でつなぐ通信機器）

ソフトウェア

- アプリ：利用者が使う機能
- ミドルウェア：多くのアプリが共通に使う機能の土台
- OS：ハードウェアを管理する基本ソフト
- ファームウェア：機器の基本動作を制御するソフト

壊れ方・備え方・変え方の違い

比べる点	ハードウェア	ソフトウェア
壊れ方	部品の故障・劣化（すり減る、熱で傷む）	プログラムの誤り（バグ）や、設定の誤り
備え方	二重化・予備部品・RAID	試験・更新・元に戻す手順の用意
変え方	購入・設置・交換	インストール（組み込み）・更新・設定変更

表の言葉

二重化：同じ働きの部品や機器を2つ以上用意し、片方が壊れても動き続けられるようにすること。

RAID（レイド）：複数の保存部品にデータを分けたり写したりして記録し、1台が壊れてもデータを失わないようにする方法（第8章）。

試験：使う前に、正しく動くかを確かめること。

ファームウェア：機器に組み込まれたソフト

ファームウェア (ふあーむうえあ)

分類：ソフトウェア

何か	機器に組み込まれ、その機器の基本的な動作を制御するソフトウェア。
なぜ必要か	電源を入れた直後の部品の確認や、機器ごとの細かい制御は、OS やアプリが動く前から、または別に行う必要がある。
どこにあるか	機器の中の、書き換えができる小さな記憶部品に入っている。
身近な例	PC の電源を入れた直後に部品を確かめ、起動を始めるソフト。家庭の通信機器や SSD の中の制御ソフト。
似た言葉との違い	OS やアプリとは別に、更新が必要なことがある。サーバーの遠隔管理用の部品（第9章）にもファームウェアがある。

第1章のまとめ

✓ ここまでで説明できること

コンピューターは、入力・処理・出力・記憶の働きで仕事をする。五大装置は入力・出力・記憶・演算・制御。

データは記録された値、情報はデータに意味を持たせたもの。

プログラムは命令の並び（手順）、データはその仕事で扱う材料と結果。

中ではすべてが0と1。1ビットは0か1の1桁、8ビット＝1バイト。通信の b と保存の B は8倍違う。

ハードウェアは機械、ソフトウェアはプログラムと設定。壊れ方も備え方も違う。

→ 次の章へ

次の第2章では、コンピューターの中を拡大して、「100と200を足す」命令とデータが、実際の部品の間をどう動くかを見ます。

確認の問い

自分の言葉で答えてから、次のページの解説を読みましょう。

- ① 「100 と 200 を足す」仕事で、プログラムとデータはそれぞれ何ですか。
- ② データと情報の違いを、注文表の例で説明してください。
- ③ 100Mbps の通信は、1 秒間に理論上何 MB 運べますか。

確認の問い：解説

前のページの問いへの答えと、考え方です。

✓ 問1 「100 と 200 を足す」仕事で、プログラムとデータはそれぞれ何ですか。

プログラムは「100を読み込む・200を読み込む・足す・結果を書き残す」という命令の並び（手順）です。データは材料の100と200、結果の300です。 p.21

✓ 問2 データと情報の違いを、注文表の例で説明してください。

「101、1、ノート、3、3,000」のような記録された値がデータです。それを組み合わせて計算した「佐藤さんの注文は2件、合計8,500円」が情報です。 p.17

✓ 問3 100Mbps の通信は、1秒間に理論上何 MB 運べますか。

12.5MB です。bps はビット、MB はバイトの単位で、8ビット＝1バイトなので $100 \div 8 = 12.5$ 。実際は付加情報や混雑のため少なくなります。 p.29

第2章

コンピューターの中を見る

この章で学ぶこと

コンピューターの本体を外から見て、ケースの中、マザーボード、CPU、コア、コアの中へと、見る範囲を少しずつ拡大します。次に「100 と 200 を足して、結果を残す」を例に、命令とデータが部品の間をどう移るかを順番に追います。最後に GPU、保存装置のつなぎ方、性能の測り方を学びます。

→ 前の章とのつながり

第1章では、プログラム（命令の並び）とデータの違い、ビットとバイト、コンピューターの5つの働き（入力・出力・記憶・演算・制御）を学びました。この章では、それが実際のどの部品で行われるかを見ます。

まず外から見る：コンピューター本体の外観

今見ている範囲：

本体の外観

→ ケースの中

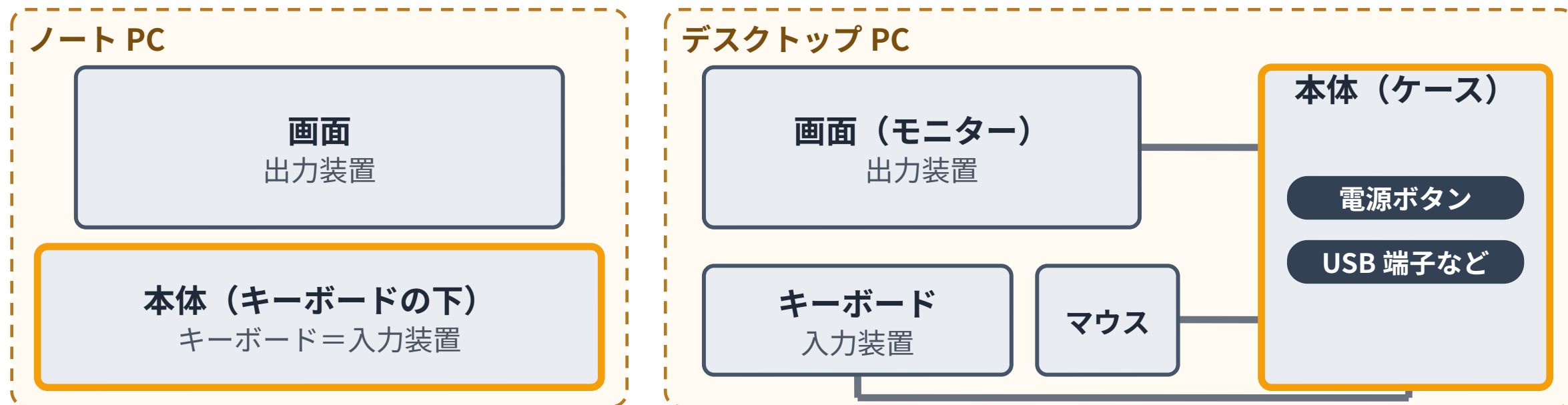
→ マザーボード

→ CPU

→ コア

→ コアの中

正面から見た外観の模式図。灰色の太枠は機械やその部分、灰色の太線はケーブルによる接続です。



画面やキーボードは、第1章の出力装置と入力装置です。計算や記憶をする部品は、黄色の枠の**本体**に収められています。次のページから、この本体の中をのぞきます。

ケースを開けると：本体の中の主な部品

今見ている範囲：

本体の外観



ケースの中



マザーボード



CPU



コア



コアの中

側面を外して見た配置の模式図。茶色の太線＝電力、灰色の太線＝データのケーブル。



ケースを開けると大きな1枚の板が見えます。これが**マザーボード**で、多くの部品はこの板に差し込まれています。

CPU（命令を実行する部品）はファンの下に隠れ、そばにメモリの細長い板が並びます。保存装置は、板に直接差すものと、ケーブルでつなぐものがあります。電源ユニットは、コンセントの電気を各部品へ配ります。部品の位置や形は製品ごとに違います。

マザーボード：部品をつなぐ中心の板

今見ている範囲：

本体の外観



ケースの中



マザーボード



CPU



コア



コアの中

マザーボード（まざーぼーど）

分類：部品

日本語：主基板（しゅきばん）

何か	CPU・メモリ・保存装置・拡張カードを差し込み、配線で互いにつながぐ大きな板（基板）。
なぜ必要か	部品どうしが信号をやり取りし、電力を受け取る通り道が必要。
どこにあるか	本体のケースの中。ケースの内側にねじで固定されている。
身近な例	PCを自作するときは、まずCPUとマザーボードの組み合わせを決める。
似た言葉との違い	「基板」は部品を載せる板全般。その中心となる基板。

マザーボードの上には、部品ごとの差し込み口がある

今見ている範囲：

本体の外観



ケースの中



マザーボード



CPU



コア



コアの中

上から見た配置の模式図。灰色の細い枠は板の上の差し込み口や端子です。

マザーボード

背面の端子

USB・映像・
LAN のケーブル
を差す

CPU ソケット

CPU を載せる台

メモリスロット

メモリの板を差す

電源コネクタ

電力を受ける

SATA 端子

ケーブルで
SSD・HDD を
つなぐ

M.2 スロット

板状の SSD を差す

PCIe スロット：GPU などの拡張カードを差す

PCIe スロット（2 本目）

学習用に簡略化した図

どの部品も、形の合う決まった口に差し込みます。PCIe・M.2・SATA は接続の決まりの名前で、意味は p.84 から説明します。

本体の中の主な部品と、その役割（1/2）

まず名前と役割を並べます。CPU・メモリ・保存装置は、この後1つずつ詳しく見ます。

部品	どこにあるか	何をするか
CPU	マザーボードの CPU ソケット（ファンの下）	プログラムの命令を実行し、計算・比較・データの移動をする
メモリ	CPU の近くのメモリスロット	実行中のプログラムとデータを一時的に置く
SSD・HDD（保存装置）	M.2 スロット、またはケーブルの先	電源を切っても消えないようにデータを保存する
GPU	PCIe スロットの拡張カード、または CPU と同じチップの中	画像の計算や、大量の同じ計算を同時に進める

本体の中の主な部品と、その役割（2/2）

まず名前と役割を並べます。CPU・メモリ・保存装置は、この後1つずつ詳しく見ます。

部品	どこにあるか	何をするか
電源ユニット	ケースの下部など	コンセントの電気を、部品に合った形に変えて配る
ファン（冷却部品）	CPUの上、ケースの前後	部品が出す熱を外へ逃がす
通信の部品	マザーボード上や拡張カード	有線（LAN）や無線（Wi-Fi）で他の機器とデータを送り合う

記憶する部品は2種類：主記憶と補助記憶

今見ている範囲：

本体の外観



ケースの中



マザーボード



CPU



コア



コアの中

第1章の記憶装置は2種類に分かれます。CPUが今使うプログラムとデータを置く**主記憶**と、電源を切っても残るように保存する**補助記憶**です。

比べる点	主記憶（メモリ）	補助記憶（SSD・HDDなど）
電源を切ると	一般に内容が消える	内容が残る
CPUとの関係	CPUが直接、命令とデータを読み書きする	CPUは直接使わず、いったん主記憶へ写す
速さと容量	速いが、容量は少なめ	主記憶より遅いが、容量は多い

❓ なぜ2種類あるのか

速い・大容量・電源を切っても消えない、の全部を満たす記憶装置は作りにくいため、役割を分けて組み合わせます。

メモリ：実行中のプログラムとデータの置き場

今見ている範囲：

本体の外観



ケースの中



マザーボード



CPU



コア



コアの中

メモリ（めもり）

分類：部品

略語：RAM ・ DRAM = Random Access Memory ・ Dynamic RAM 日本語：主記憶装置

何か	実行中のプログラムと作業中のデータを一時的に置く部品（主記憶）。PC では DRAM というチップを並べた細長い板が多い。
なぜ必要か	CPU は保存装置からではなく、メモリから命令とデータを読むため。
どこにあるか	マザーボードのメモリスロット（CPU のすぐ近く）。一般的な DRAM は電源を切ると内容が消える。
似た言葉との違い	「メモリ 16GB」は主記憶の量。スマホの「ストレージ 128GB」は保存装置の量で、別のもの。

ストレージ：電源を切っても消えない保存装置

今見ている範囲：

本体の外観



ケースの中



マザーボード



CPU



コア



コアの中

ストレージ（すとれーじ）

分類：部品

日本語：補助記憶装置・保存装置

何か	電源を切っても消えないように、プログラムやデータを長く保存する装置。補助記憶の実物。
なぜ必要か	メモリの内容は電源を切ると消えるため。
どこにあるか	M.2 スロットに差す、またはケーブルでつなぐ。
身近な例	SSD （チップに電氣的に記録）と HDD （回転する円盤に磁気で記録）が代表例。
似た言葉との違い	保存専用の大きな機器もストレージと呼ぶ（第8章）。

CPU：命令を実行する中心の部品

今見ている範囲：

本体の外観



ケースの中



マザーボード



CPU



コア



コアの中

CPU（しーピーゆー）

分類：部品

略語：CPU = Central Processing Unit 日本語：中央処理装置

何か	プログラムの命令を順に読み取り、計算・比較・データの移動を行う汎用の部品。第1章の演算と制御の働きを担う。
なぜ必要か	OS やアプリの命令を実行する部品がなければ、どのプログラムも動かないため。
どこにあるか	マザーボードの CPU ソケットの上。メモリから命令とデータを受け取り、結果をメモリへ返す。
似た言葉との違い	「汎用」は特定の仕事専用ではなく、どんなプログラムも実行できること。特定の計算が得意な GPU とは違う。

CPU の実物：基板の上の四角いチップ

今見ている範囲：

本体の外観



ケースの中



マザーボード



CPU



コア



コアの中

横から見た断面の模式図。下から順に積み重なっています。



CPU の本体は、小さなシリコンの板（チップ）です。チップは、保護と配線のための四角い台（パッケージ）に収められ、マザーボードのソケットに取り付けられます。

CPU は動くと熱を出すため、上に放熱板とファンを載せて冷やします。熱が逃げないと、CPU は壊れないように自分で動作を遅くします。

CPU・メモリ・ストレージの協力：たとえと実際

💡 たとえ

CPU は作業する人の頭脳、メモリは作業机、ストレージは倉庫、とよくたとえられます。倉庫から書類を机に出し、机の上で作業し、終わったら倉庫へ戻す、という流れです。

たとえ	実際の部品の働き
倉庫から書類を出す	SSD からメモリへ、プログラムとデータを写す（読み込む）
机の上で作業する	CPU が、メモリ上の命令に従い、メモリ上のデータで計算する
倉庫へ戻す	残したい結果を、メモリから SSD へ書き込む（保存する）

次のページで、このたとえで誤解しやすい点を確認してから、実際の部品の動きを追います。

たとえで誤解しやすい点

！ たとえの限界

- ・ CPU は「考える」わけではありません。プログラムに書かれた命令を、書かれたとおりに実行するだけです。
- ・ 机は1つではありません。CPU 中の小さな置き場、メモリ、GPU 専用の記憶などと何段にも分かれ、出し入れは自動で行われます。
- ・ 倉庫から書類を出すほどの時間はかかりません。読み込みは数ミリ秒以下で、必要な一部だけを細かく読むこともあります。
- ・ 倉庫へ戻す（保存する）前に電源が落ちると、机（メモリ）の上の結果は失われることがあります。

🔑 ポイント

たとえば流れをつかむための道具です。ここからは、実際の部品の名前と働きで説明します。

例題「100 と 200 を足して結果を残す」：準備

今見ている範囲：

本体の外観



ケースの中



マザーボード



CPU

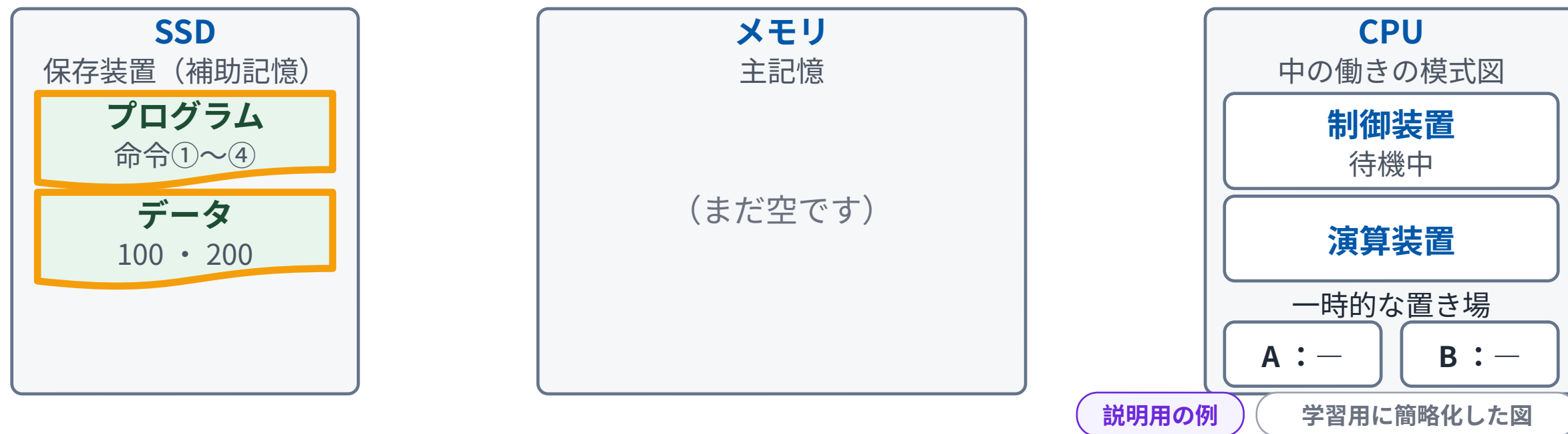


コア



コアの中

学習用の模式図。黒い矢印は命令・データの移動、黄色の枠は今働く場所。



始める前、プログラム（命令①～④の並び）とデータ（100 と 200）は、SSD にファイルとして保存されています。メモリは空で、CPU もまだこの仕事を始めていません。

① 読み込み：SSD からメモリへ写す

今見ている範囲：

本体の外観

→ ケースの中

→ マザーボード

→ CPU

→ コア

→ コアの中

学習用の模式図。黒い矢印は命令・データの移動、黄色の枠は今働く場所。



プログラムを起動すると、OS（コンピューター全体を管理する基本のソフトウェア）の指示で、プログラムとデータが SSD からメモリへ写されます。SSD の中身は残ります。

なぜ、わざわざメモリへ読み込むのか

今見ている範囲：

本体の外観



ケースの中



マザーボード



CPU



コア



コアの中

CPU は、命令とデータをメモリから読む仕組みになっています。SSD は電源を切っても消えない代わりに、メモリと比べると読み書きに時間がかかります。CPU は非常に多くの命令を次々に処理するので、そのたびに SSD を待っているのは、ほとんどが待ち時間になってしまいます。

そこで、使う予定のプログラムとデータを、先に速いメモリへ写しておきます。CPU はメモリとやり取りしながら仕事を進め、残したい結果だけを最後に SSD へ書き込みます。

🔑 ポイント

SSD = 長く残すための場所、メモリ = CPU が今使うための場所、という役割分担です。速さの差は製品によって大きく違うため、ここでは「何倍」とは言いません。

メモリの中身：番号の付いた置き場が並ぶ

メモリの置き場には通し番号が付いていて、これを**アドレス**（番地）と呼びます。CPUは「何番地を読む・書く」と、アドレスで場所を指定します。

アドレス	中身	意味（日本語訳）
0 番地	命令①	10 番地の値を、置き場 A に読み込む
1 番地	命令②	11 番地の値を、置き場 B に読み込む
2 番地	命令③	A と B を足し、答えを A に入れる
3 番地	命令④	A の値を、12 番地に書き込む
10 番地・11 番地	データ	100 ・ 200
12 番地	（空き）	結果を入れる場所

※ 命令の書き方は説明用の例です。実際の命令は 0 と 1 の並び（機械語）で、CPU の設計ごとに違います。また 1 つの番地には 1 バイトが入り、命令や数は複数の番地にまたがるのが普通です。

② 取り出し（フェッチ）：命令を CPU へ

今見ている範囲：

本体の外観



ケースの中



マザーボード



CPU



コア



コアの中

学習用の模式図。黒い矢印は命令・データの移動、黄色の枠は今働く場所。



制御装置は「次に読む番地」が指す0番地から命令①を取り出します。これを命令の取り出し、または**フェッチ**と呼びます。取り出すと、次に読む番地は1に進みます。

③ 解読（デコード）：命令の意味を調べる

今見ている範囲：

本体の外観



ケースの中



マザーボード



CPU



コア



コアの中

学習用の模式図。黒い矢印は命令・データの移動、黄色の枠は今働く場所。



制御装置は、命令①が「10 番地の値を置き場 A へ読み込め」という意味だと調べ、どの部分をどう動かすかを決めます。これを命令の解読、または**デコード**と呼びます。

④ 実行：データ 100 と 200 を CPU の中へ

今見ている範囲：

本体の外観



ケースの中



マザーボード



CPU



コア



コアの中

学習用の模式図。黒い矢印は命令・データの移動、黄色の枠は今働く場所。



命令①を実行すると、10 番地の 100 が CPU の中の置き場 A へ入ります。命令②も同じく取り出し→解読→実行され、200 が置き場 B へ入ります。

⑤ 実行：演算装置が足し算をする

今見ている範囲：

本体の外観



ケースの中



マザーボード



CPU



コア



コアの中

学習用の模式図。黒い矢印は命令・データの移動、黄色の枠は今働く場所。



命令③を解釈した制御装置が「AとBを足せ」と指示し、**演算装置**が $100 + 200 = 300$ を計算して、答えを置き場Aに入れます。ここで初めて計算が行われました。

⑥ 書き戻し：結果をメモリへ

今見ている範囲：

本体の外観



ケースの中



マザーボード



CPU



コア



コアの中

学習用の模式図。黒い矢印は命令・データの移動、黄色の枠は今働く場所。



命令④により、置き場 A の 300 がメモリの 12 番地へ書き込まれます（書き戻し）。CPU 中の置き場はとても小さいので、結果はこうしてメモリへ移します。

⑦ 保存：結果を SSD へ残す

今見ている範囲：

本体の外観

→ ケースの中

→ マザーボード

→ CPU

→ コア

→ コアの中

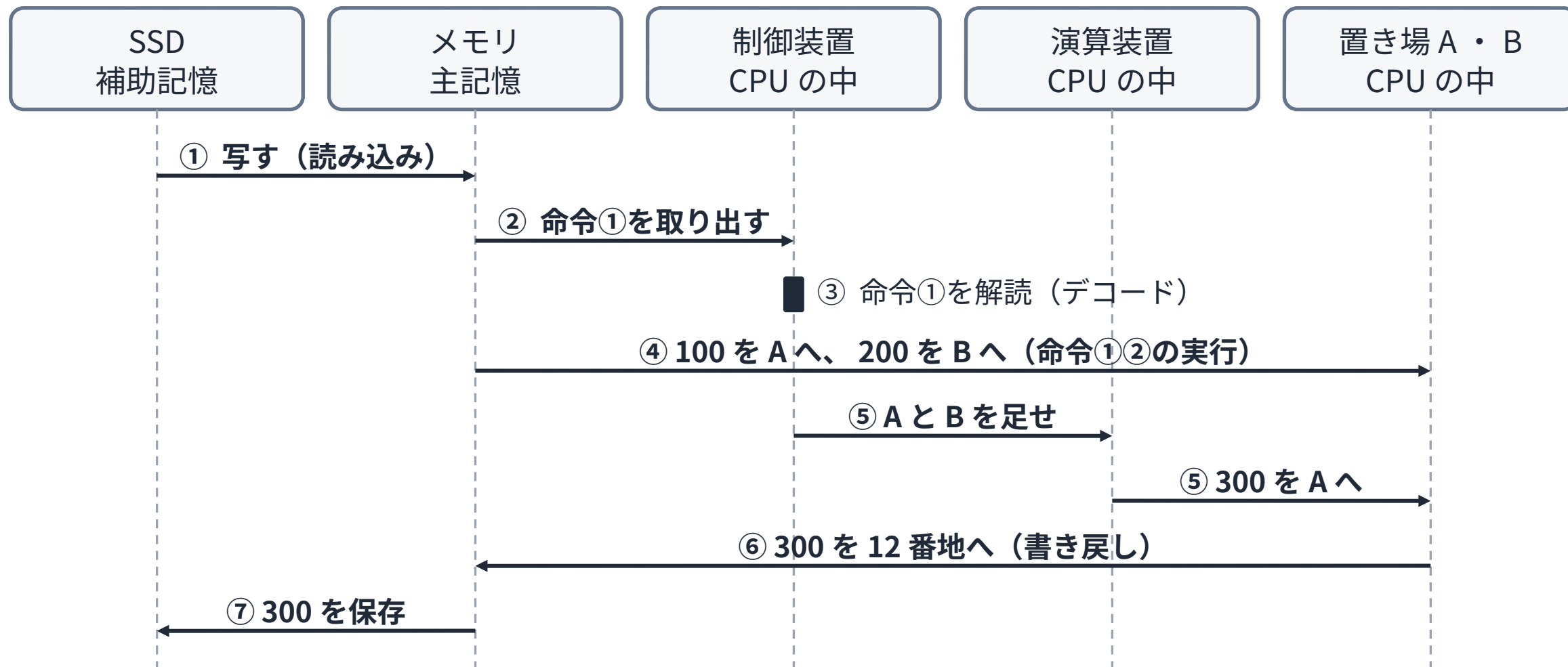
学習用の模式図。黒い矢印は命令・データの移動、黄色の枠は今働く場所。



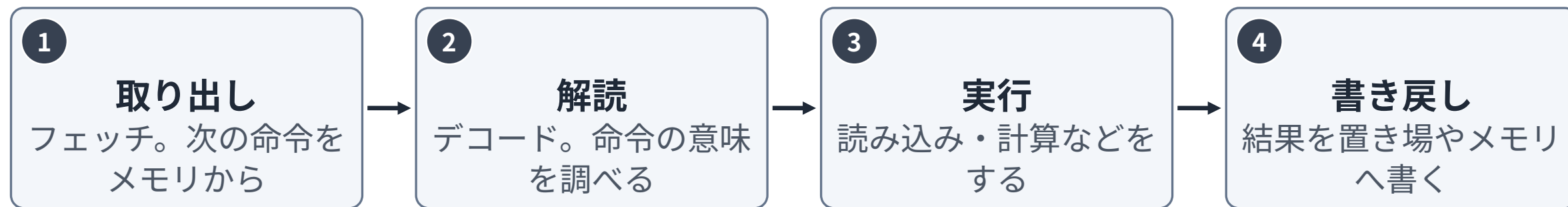
メモリの内容は電源を切ると消えます。プログラムが保存を頼むと、OS がメモリの 300 をファイルとして SSD へ書き込み、電源を切っても結果が残ります。

流れ全体を時間の順に並べると

時間順のやり取り図です。上から下へ時間が進み、黒い矢印は命令やデータの移動を表します。



取り出し→解読→実行を繰り返す：命令実行サイクル



CPU は 1 つの命令でこの段階を終えると「次に読む番地」を進め、次の命令で同じことを繰り返します。これを**命令実行サイクル**と呼びます。足し算 1 回でも 4 命令が必要でした。アプリを動かすと、数十億個にもなる命令が次々に実行されます。

i 補足

段階の分け方や呼び名は、教科書や CPU の設計によって少し違います（書き戻しを実行に含める、など）。大切なのは「命令を 1 つずつ取り出して実行する」という基本の形です。

確認の問い（命令とデータの流れ）

自分の言葉で答えてから、次のページの解説を読みましょう。

- ① 計算の前に、プログラムとデータはどこからどこへ移りますか。理由も。
- ② 命令③では、命令実行サイクルの各段階で何が起きましたか。
- ③ 結果の 300 を保存せずに電源を切ると、どうなりますか。

確認の問い（命令とデータの流れ）：解説

前のページの問いへの答えと、考え方です。

✓ 問1 計算の前に、プログラムとデータはどこからどこへ移りますか。理由も。

SSD からメモリへ移ります（OS の指示で写す）。CPU は命令とデータをメモリから読む仕組みで、SSD はメモリより読み書きに時間がかかるため、毎回 SSD を待たずに済むようにします。
p.52

✓ 問2 命令③では、命令実行サイクルの各段階で何が起こりましたか。

取り出し（2番地から命令③を取り出す）→解読（「A と B を足して A へ」と分かる）→実行（演算装置が $100 + 200 = 300$ ）→書き戻し（300 を置き場 A へ）。 p.61

✓ 問3 結果の 300 を保存せずに電源を切ると、どうなりますか。

300 はメモリと CPU の中の置き場にしかないため、消えてしまいます。残すには、OS を通して SSD へ書き込む（保存する）必要があります。 p.59

CPU を拡大する：中には「コア」が複数ある

今見ている範囲：

本体の外観



ケースの中



マザーボード



CPU



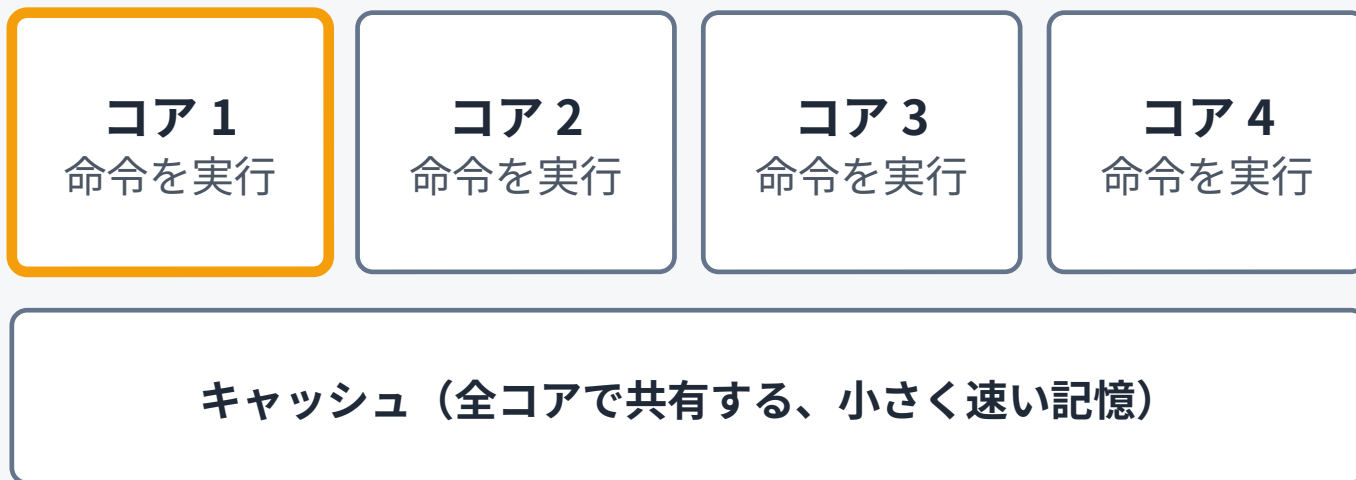
コア



コアの中

上から見た学習用の模式図。配置や個数は製品ごとに違います。

CPU のチップ



命令・データの
行き来



メモリ
CPU の外

学習用に簡略化した図

例題で取り出し→解読→実行を行った一式を**コア**と呼びます。多くの CPU は 1 つのチップにコアを複数持ち、それぞれが別の命令の並びを同時に進めます。

コアの中を拡大する：主な4つの部分

今見ている範囲：

本体の外観



ケースの中



マザーボード



CPU



コア



コアの中

黄色の枠が、このページで説明する部分です。

コア（学習用の模式図）

制御装置
解読と指示

演算装置
計算・比較

キャッシュ
よく使う写し

レジスタ
計算中の値

学習用に簡略化した図

コアの中には、例題で働いた部分があります。命令を解読して指示を出す**制御装置**、計算をする**演算装置**、計算中の値を置くレジスタ、よく使う命令やデータの写しを置くキャッシュです。次のページから1つずつ見ます。

i 補足

「制御装置」「演算装置」は働きで分けた名前です。チップの中に、その名前の箱が並んでいるわけではありません。

制御装置：命令を解読し、各部分に指示を出す

今見ている範囲：

本体の外観



ケースの中



マザーボード



CPU



コア



コアの中

黄色の枠が、このページで説明する部分です。

コア（学習用の模式図）

制御装置
解読と指示

演算装置
計算・比較

キャッシュ
よく使う写し

レジスタ
計算中の値

学習用に簡略化した図

制御装置は、取り出した命令を解読し、演算装置・レジスタ・メモリとのやり取りを、正しい順番で動かす部分です。自分では計算せず、「どこが、いつ、何をするか」を決めます。

自 例題での働き

例題では、命令③「AとBを足してAに入れる」を解読し、演算装置に足し算を指示して、答えを置き場Aへ入れさせました。

次に取り出す命令のアドレスも管理し、命令が1つ終わるたびに次へ進めます。

演算装置：足し算や比較などの計算をする

今見ている範囲：

本体の外観



ケースの中



マザーボード



CPU



コア



コアの中

黄色の枠が、このページで説明する部分です。

コア（学習用の模式図）

制御装置
解釈と指示

演算装置
計算・比較

キャッシュ
よく使う写し

レジスタ
計算中の値

学習用に簡略化した図

演算装置は、制御装置の指示で、レジスタの値を使って計算する部分です。中心部分を**ALU**（Arithmetic Logic Unit：算術論理演算装置）と呼びます。「算術」は足し算などの計算、「論理」は大小の比較や条件の判定です。

具体例

算術： $100 + 200 = 300$ （例題の命令③）

比較：「300 は 250 より大きい」→ はい。この結果で次に実行する命令を変えられます（在庫が0なら「売り切れ」と表示する、など）。

レジスタ：CPU の中のごく小さく速い記憶

今見ている範囲：

本体の外観



ケースの中



マザーボード



CPU



コア



コアの中

レジスタ (れじすた)

分類：部品

何か	コアの中にある、ごく小さく非常に速い記憶。計算に使う値や途中の結果を置く。
なぜ必要か	演算装置は、手元のレジスタにある値を使って計算するため。
どこにあるか	コアの中、演算装置のすぐそば。数は少なく、1つに数値1つ分程度。
身近な例	例題の「置き場 $A \cdot B$ 」がレジスタ。100 と 200 を入れ、答えの 300 も A に入った。
似た言葉との違い	メモリは CPU の外にあり大容量。レジスタは CPU の中にあり、ずっと小さいが最も速い。

次の命令を指すレジスタ：プログラムカウンタ

今見ている範囲： 本体の外観 → ケースの中 → マザーボード → CPU → コア → **コアの中**

プログラムカウンタは、次に取り出す命令のアドレスを覚えている特別なレジスタです。
例題の図で制御装置に書いた「次に読む番地」がこれです。

プログラムカウンタの値	取り出す命令	実行した内容
0	命令①	10 番地の 100 を置き場 A へ
1	命令②	11 番地の 200 を置き場 B へ
2	命令③	$A + B = 300$ を A へ
3	命令④	A の 300 を 12 番地へ

比較の結果でプログラムカウンタを別の番地に書き換えると、「条件に合えば別の命令へ進む」という分かれ道が作れます。

キャッシュメモリ：よく使うものの写しを手元に

今見ている範囲：

本体の外観



ケースの中



マザーボード



CPU



コア



コアの中

キャッシュメモリ（きゃっしゅめもり）

分類：部品

何か	CPU の中の、メモリより小さく速い記憶。最近使った命令やデータの写しを置く。例題の取り出しも実際はここを通る。
なぜ必要か	メモリを待つ間 CPU が止まるため。2 回目からは写しから速く読める。
どこにあるか	CPU の中。各コア専用のもものと全コアで共有するものが何段かある。
似た言葉との違い	通信の DNS キャッシュ、AI の KV キャッシュとは別の仕組み（「写しを手元に置く」点だけ共通）。

記憶の階層：速い記憶ほど小さい

模式図。箱の幅は容量のイメージ。左の矢印の向きほど遅く大容量。



学習用に簡略化した図

CPU は、キャッシュにあればそこから、なければメモリから自動で読みます。メモリにないものは OS が保存装置から読み込みます。よく使うものほど速い記憶にあるので、全体は「大容量なのに速い」に近づきます。この構造を**記憶階層**と呼びます。

複数のコアで速くなる仕事・ならない仕事

今見ている範囲：

本体の外観



ケースの中



マザーボード



CPU



コア



コアの中

1つのCPUに複数のコアを持つ作りを**マルチコア**と呼びます。コアが4つなら、4つの命令の並びを同時に進められます。

灰色の箱はコア、中の文字はそのコアが受け持つ仕事です。

A：分けられる仕事（写真100枚の明るさ調整）

コア1
25枚

コア2
25枚

コア3
25枚

コア4
25枚

→ 同時に進むので早く終わる

B：前の結果を使って次を計算する仕事

コア1
計算
1→2→3

コア2
待機

コア3
待機

コア4
待機

→ 1つのコアでしか進められない

説明用の例

⚠ 注意

コアを2倍にしても速さが2倍とは限りません。順番に依存して並列化できない部分は1つのコアでしか進まず、そこが全体の時間を決めます。

クロック：CPU の動作のリズム

今見ている範囲：

本体の外観



ケースの中



マザーボード



CPU



コア



コアの中

クロック (くろっく)

分類：仕組み

何か	CPU の各部分が動きをそろえるための、一定の間隔で刻まれる電気の合図（リズム）。1秒間に刻む回数を クロック周波数 という。
なぜ必要か	多くの部分が同時に動くので、値を受け渡すタイミングをそろえる合図が必要なため。
身近な例	3GHz = 1秒間に30億回のリズム。G（ギガ）は10億、Hz（ヘルツ）は1秒あたりの回数。
似た言葉との違い	1秒あたりに実行できる命令の数とは同じではない（次のページ）。

クロックの回数と、実行できる命令の数は別

今見ている範囲：

本体の外観



ケースの中



マザーボード



CPU



コア



コアの中

時間を左から右に並べた模式図。上の番号はクロックのリズムの回数、紫の箱は命令です。

クロック

1

2

3

4

5

6

7

8

9

10

A：1回のリズムで2つの命令を進める設計

2 命令

2 命令

2 命令

2 命令

2 命令

2 命令

2 命令

2 命令

2 命令

2 命令

B：数回のリズムを要する命令（割り算や、メモリからデータが届くのを待つ場合など）

命令 1 つ（4 回分）

命令 1 つ

命令 1 つ（待ちを含む）

説明用の例

学習用に簡略化した図

同じ 10 回のリズムでも、A では 20 個、B では 3 個の命令しか終わりません。クロック周波数だけでは、1 秒に実行できる命令の数は決まらないのです。

クロックだけで CPU を比べられない理由

- ① 1 回のリズムで複数の命令を進める設計がある（作業を段階に分けて重ねて進める、演算装置を複数並べる など）。
- ② 逆に、数回のリズムが必要な命令や、メモリからデータが届くのを待つ時間がある。
- ③ そのため、世代や設計が違う CPU どうしは、クロック周波数だけでは比べられない。

1 命令に平均何回のリズムが必要かを **CPI**（Cycles Per Instruction）と呼びます。1 秒に実行できる命令の数は、およそ「クロック周波数 ÷ CPI」です。

説明用の例

CPU 甲：3GHz、CPI 1.5 → $30 \text{ 億} \div 1.5 = 1 \text{ 秒に約 } 20 \text{ 億命令}$

CPU 乙：2.5GHz、CPI 0.5 → $25 \text{ 億} \div 0.5 = 1 \text{ 秒に約 } 50 \text{ 億命令}$

クロックが低い乙のほうが多くの命令を実行できます。実際の CPI は、動かすプログラムによっても変わります。

CPU の性能は、いくつかの要素で決まる

要素	意味	注意
コア数	同時に進められる命令の並びの数	並列化できない処理には効きにくい
クロック周波数	動作のリズムの速さ（GHz）	世代や設計が違えば、これだけで比べられない
世代・設計	1 回のリズムでどれだけ仕事を進められるか	同じクロックでも新しい設計が速いことが多い
キャッシュメモリ	CPU 内の小さく速い記憶の量と段数	足りないとメモリを待つ時間が増える
メモリ	容量と、1 秒に運べる量	CPU が速くても、データが届かなければ待つ

1 つの数字で判断せず、動かしたい仕事で測った結果で比べるのが確実です。

確認の問い（CPUの中）

自分の言葉で答えてから、次のページの解説を読みましょう。

- 1 レジスタ・キャッシュメモリ・メモリを、演算装置に近い順に並べてください。
- 2 「4GHz の CPU は 2GHz の CPU の 2 倍の命令を実行する」は正しいですか。
- 3 コアを 2 倍にしても、2 倍速くならないことがあるのはなぜですか。

確認の問い（CPUの中）：解説

前のページの問いへの答えと、考え方です。

✓ 問1 レジスタ・キャッシュメモリ・メモリを、演算装置に近い順に並べてください。

レジスタ（コアの中。計算中の値）→キャッシュメモリ（CPUの中。最近使った命令やデータの写し）→メモリ（CPUの外。実行中の内容全体）。近いほど速く、小さくなります。 p.71

✓ 問2 「4GHzのCPUは2GHzのCPUの2倍の命令を実行する」は正しいですか。

正しいとは限りません。1回のリズムで複数の命令を進める設計や、数回のリズムを要する命令・メモリ待ちがあるため、世代や設計が違えばクロックだけでは比べられません。 p.75

✓ 問3 コアを2倍にしても、2倍速くならないことがあるのはなぜですか。

前の結果を使って次を計算するような、順番に依存する部分は複数のコアに分けられず、1つのコアでしか進められないからです。 p.72

計算の前に、データを近くの記憶へ運ぶ

今見ている範囲：

本体の外観

→ ケースの中

→ マザーボード

→ CPU

→ コア

→ コアの中

役割の関係図。灰色の箱は部品、黒い矢印はデータの移動、番号は順番です。



どんなに速い CPU でも、データが手元に届くまでは待つしかありません。データは保存装置からメモリへ、メモリから CPU へと運ばれます。どこか 1 か所の通り道が細い（1 秒に運べる量が少ない）と、その先の部品は待たされます。

メモリの「容量」と「1秒に運べる量」は別

今見ている範囲：

本体の外観



ケースの中



マザーボード



CPU



コア



コアの中

📊 容量（GB）

一度に置いておける量。足りないと、入りきらない内容を遅い保存装置へ一時的に退避させるため、待ちが増える。

↔ 1秒に運べる量（GB/s）

帯域、転送速度とも呼ぶ。容量が大きくても、運ぶ速さが足りなければ、CPU や GPU の計算が待たされる。

🧪 説明用の例

16GB のメモリに 20GB 分の内容を置こうとすると、入りきらない分を保存装置へ退避させるので遅くなります（次のページ）。

メモリが足りないとき：仮想記憶とスワップ

仮想記憶 (かそうきおく)

分類：仕組み

何か	OS が保存装置の一部をメモリの延長のように使い、実際のメモリより大きな置き場があるように見せる仕組み。
なぜ必要か	メモリの容量を超えるプログラムやデータを動かしても、すぐに止まらないようにするため。
どこにあるか	仕組みは OS の機能。退避先は SSD など保存装置の一部。
身近な例	メモリが足りないと、しばらく使わない内容を保存装置へ退避させ、必要なときに戻す。この出し入れが スワップ 。アプリを多く開くと遅くなる一因。
似た言葉との違い	保存装置はメモリより遅いので、スワップが多いと全体が遅くなる。容量追加や使い方の見直しで減らす。

SSD と HDD ：記録の仕組みが違う

上から見た構造の模式図（学習用に簡略化）。細い矢印はヘッドが動く向きです。



SSD は、NAND フラッシュというチップに電氣的に記録します（SSD はこのチップと制御部を組み合わせた製品です）。HDD は回転する円盤にヘッドが磁気で記録するため、ヘッドの移動と、目的の場所が回ってくるまでの待ち時間がかかります。

SSD と HDD の得意な使い方

比べる点	SSD	HDD
小さな読み書きを多数	得意（待ち時間が短い）	ヘッドの移動と回転待ちが生じる
同じ費用での容量	一般に少なめ	一般に多め
衝撃	動く部品がない	動作中の衝撃に弱い
寿命の考え方	書き込める量に上限がある	機械部品がすり減る
主な用途	OS ・ データベース ・ アプリ ・ AI のデータ読込	大容量の保管 ・ 写しや長期の保管

注意：速さ・容量・費用は製品と使い方で変わるため、根拠のない「何倍」では比べません。
どちらも故障します。複数台で備える（RAID）・写しを保管する（バックアップ）が必要です（p.366）。

つなぐための決まり①：インターフェース

インターフェース (いんたーふゑーす)

分類：仕組み

何か	2つのものが接する部分と、そこでのやり取りの決まり。部品どうしなら、差し込み口の形・電気信号・データの受け渡し方。
なぜ必要か	決まりがそろっていないと、つないでもデータを受け渡せないため。
身近な例	USB の差し込み口、マザーボードの M.2 スロット、 SATA 端子。
似た言葉との違い	人が操作する画面やボタンの作りは「ユーザーインターフェース」と呼ぶ。この章では主に部品どうしの接続を指す。

つなぐための決まり②：規格と規約（プロトコル）

規格は、多くの会社や団体が合わせる、形・大きさ・信号・手順などの決まりです。乾電池の「単3形」のように、作った会社が違って、同じ規格に合わせればつないで使えます。

規格のうち、データをやり取りする手順（どんな順番で、どんな形の命令を送り、どう返事をするか）を決めたものを、規約、または**プロトコル**と呼びます。プロトコルは通信の章（第6章）でも重要になります。

ポイント

保存装置のつなぎ方は、①形（差し込み口の形と大きさ）、②接続（電気信号の通り道の方式）、③規約（命令の出し方の手順）の3つを区別すると分かりやすくなります。

M.2 と SATA : 形の規格と、接続の規格

名前	分類	何を決めるか	例・注意
M.2 (エムドットツー)	① 形・端子の規格	基板の大きさと差し込み口	2280 = 幅 22mm ・ 長さ 80mm 。 SATA 接続の M.2 もある
SATA (サタ)	② 接続の規格	保存装置と PC をつなぐ方式	2.5 インチの SSD や HDD で広く使われる

M.2 は「どんな形の板を、どんな口に差すか」だけを決めています。その口の奥で、どの接続方式（SATA か PCIe か）を使うかは別に決まります。SATA はケーブルでつなぐ保存装置で長く使われてきた接続方式です。

PCIe と NVMe : 高速な通り道と、命令の出し方

名前	分類	何を決めるか	例・注意
PCIe (ピーシーアイ・エクスプレス)	② 高速な接続の規格	部品どうしの高速な通り道	GPU ・ ネットワーク部品 ・ NVMe SSD などが使う
NVMe (エヌブイエムイー)	③ 通信規約の体系	SSD などへの命令の出し方	多くは PCIe 上で使う。ネットワーク越しに使う NVMe over Fabrics (TCP ・ RDMA など) もある

PCIe は、通り道 (レーン) を何本束ねるかで運べる量が変わります (×4 、 ×16 など) 。 NVMe は、SSD の速さを生かせるように作られた命令の出し方の決まりです。

組み合わせの例：「M.2 だから NVMe」とは限らない

3つの観点の組み合わせを並べた表形式の図。左から形・接続・規約、右端がよく見られる例です。

① 形		② 接続		③ 規約		よく見られる例
M.2 形	+	PCIe 接続	+	NVMe	→	高速な PC 用 SSD に多い
M.2 形	+	SATA 接続	+	AHCI など	→	M.2 でも NVMe ではない
2.5 インチ形	+	SATA 接続	+	AHCI など	→	従来型の SSD ・ HDD

AHCI は、SATA で長く使われてきた命令の出し方の規約です。M.2 の板を買うときは、マザーボードの M.2 スロットが PCIe（NVMe）と SATA のどちらに対応しているかを確認めます。

GPU：同じ計算を大量に並べて進める部品

GPU（じーピーゆー）

分類：部品

略語：GPU = Graphics Processing Unit 日本語：画像処理装置

何か	同じ種類の計算を、多数の計算ユニットで一斉に（並列に）進めるのが得意な部品。もとは画面の画像を作るために生まれた。
なぜ必要か	画面の画素や、AIの計算のように、同じ形の計算を大量に行う仕事は、CPUだけでは時間がかかるため。
どこにあるか	PCIe スロットに差す拡張カード（専用のメモリ付き）、または CPU と同じチップの中。
似た言葉との違い	CPU が段取りをして指示とデータを渡し、GPU が得意な計算を並列に行って結果を返す。すべての処理が速くなるわけではない。

CPU と GPU の作りの違い

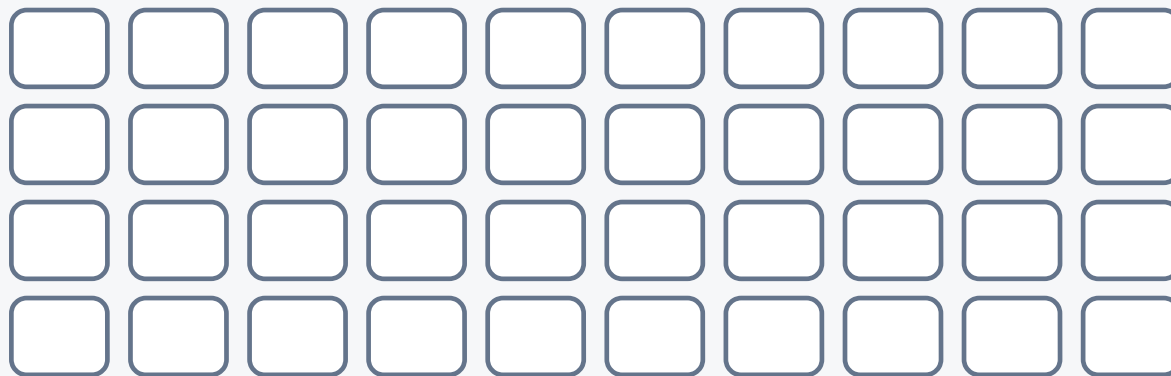
学習用の模式図。箱の大きさや数は実物の比率ではありません。

CPU：少数の高機能なコア



CPU：条件によって分かれ道の多い処理、OS、全体の段取りが得意です。

GPU：多数の計算ユニット



学習用に簡略化した図

GPU：同じ形の計算（数を表の形に並べてまとめて計算する行列計算、画素の処理）を一斉に進めます。

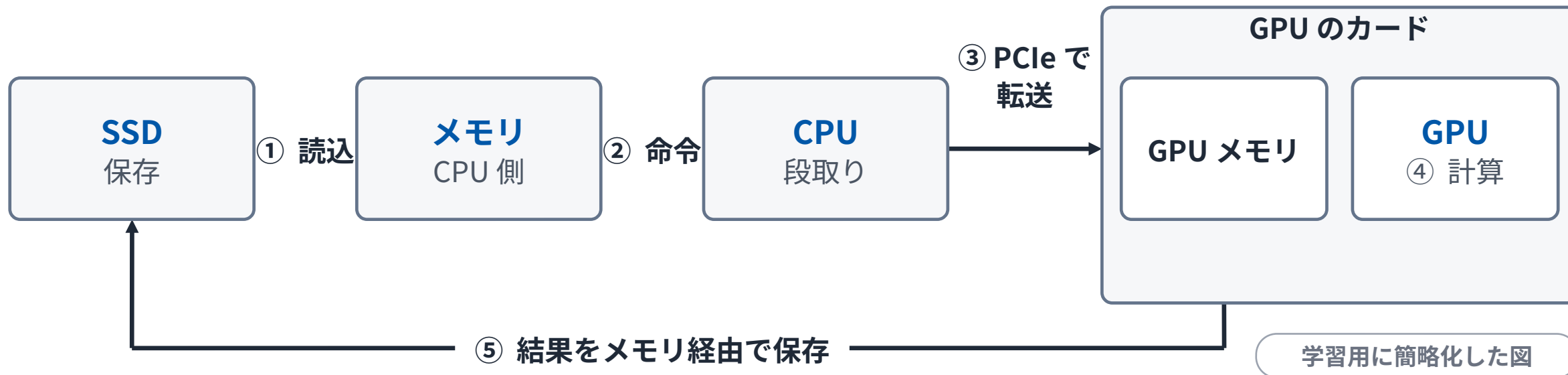
CPU と GPU の比較と、AI での役割

	CPU	GPU
得意なこと	多様な処理・順番に依存する処理	大量の同じ計算の並列処理
AI での役割	データの準備・GPU への指示・通信の管理	AI を作る計算（学習）と、答えを出す計算（推論）の行列計算
向かない例	膨大な行列計算だけを延々とする	分岐が多く並列化できない処理

AI（大量のデータから規則を学ばせたプログラム）は、膨大な行列計算を必要とするため、GPU が重要になります。詳しくは第12章（p.512）で見ます。

GPU を使うときのデータの移動

役割の関係図。黒い矢印はデータの移動、番号は順番。灰色の大きな枠は GPU のカードです。



GPU で計算するときは、データがもう 1 段移動します。CPU 側のメモリから、PCIe の通り道を通して GPU 専用のメモリへ送られます。GPU がどんなに速くても、この転送や保存装置からの読み込みが遅いと、GPU は待たされます。

GPU のメモリと HBM

種類	何を置くか	どこにあるか	注意
RAM ・ DRAM (主記憶)	OS ・ アプリ ・ 作業中のデータ	CPU の近くのメモリスロット	電源を切ると内容は消える
GPU メモリ	GPU が計算に使うデータ ・ AI の数値	GPU のカードやパッケージの上	CPU 側の RAM とは別の領域が一般的
HBM (High Bandwidth Memory)	GPU などへ大量のデータを高速に供給	演算チップのすぐ近くに積み重ねて置く	長期保存用ではない。採用しない GPU もある (GDDR など)

High Bandwidth は「1 秒に運べる量が多い」という意味です。演算チップのすぐ近くに置くことで、大量のデータを速く渡せます。

作業用の記憶と保存用の記憶：HBM と SSD

HBM も SSD の中の NAND も、半導体でできた記憶です。しかし働きは違います。HBM は計算中のデータを速く渡す作業用、NAND は電源を切っても残す保存用です。

	HBM	NAND ・ SSD
主な仕事	計算中のデータを高速に供給	データを大容量で保存
電源を切ると	データは保持されない	データを保持する
重視される点	1 秒に運べる量・容量・電力効率	容量・読み書き性能・耐久性・価格

性能の数字を読む前に：単位の読み方

「速い」を数字で比べるには、まず単位が何を数えているかを読み取ります。

単位	読み方	何を数えるか
GB ・ TB	ギガバイト・テラバイト	量（容量）。1TB = 1000GB（1024で数える場合もある）
GB/s ・ MB/s	ギガバイト毎秒	1秒に運べるバイト数
Gbps ・ Mbps	ギガビーピーエス	1秒に運べるビット数（bit per second）。8ビット = 1バイト
回 / 秒	かいまいびょう	1秒あたりの読み書きの回数
ms ・ μs	ミリ秒・マイクロ秒	待ち時間。1msは1000分の1秒、1μsは100万分の1秒
%	パーセント	部品が働いている時間の割合

「速い」には意味が複数ある：5つの指標

指標	意味	単位の例	主に測る対象	注意
容量	どれだけ入るか	GB ・ TB	メモリ ・ 保存装置	空き容量と速さは別
スループット（帯域・転送速度）	1 秒に運べる量	GB/s ・ Gbps	メモリ ・ 保存 ・ 通信	bit （ b ） と byte （ B ） は 8 倍違う
IOPS	1 秒あたりの読み書き回数	回 / 秒	保存装置	1 回の大きさ ・ 読み書きの割合で変わる
応答時間（レイテンシー）	1 回の処理を待つ時間	ms ・ μs	保存 ・ 通信 ・ アプリ	平均だけでなく遅い側も見る
CPU ・ GPU の使用率	計算部品が働いている割合	%	CPU ・ GPU	低くても、データ待ちのことがある

厳密には、通り道が運べる上限を帯域、実際に運べた量をスループットと呼び分けることがあります。

指標を読むときの例と注意

例：bit と byte の換算

100Mbps は、8 で割って理論上 12.5MB/s に相当します。実際の転送は、通信に付く付加情報や混雑のため、これより少なくなります。

説明用の例

4KB の読み書きを 1 秒に 1 万回できれば、運べる量は $4\text{KB} \times 1\text{万} = \text{約 } 40\text{MB/s}$ （IOPS とスループットの関係）。

平均の応答時間が 1ms でも、100 回に 1 回は 50ms かかることがあります。だから遅い側も見ます。

注意

比べるときは、同じ大きさのデータ・同時の処理数・読み書きの割合で測った値どうしを比べます。

全体の速さは、いちばん待たされる所で決まる

役割の関係図。黒い矢印はデータの流れ、黄色の枠が最も遅い箇所です。数値は説明用の例。



説明用の例

CPUが1秒に1,000件計算できても、結果を送るのが1秒に100件なら、全体も1秒に100件です。このように全体の速さを決めてしまう、いちばん細い所を**ボトルネック**（瓶の首）と呼びます。数値は環境で変わるため、次のページでは症状の見分け方を示します。

ボトルネックの4つの症状と見分け方

症状	見え方	対策の方向
① 計算が足りない	CPU ・ GPU の使用率が高いまま張り付き、処理待ちが増える	計算部品の追加 ・ 処理の分散
② 保存の読み込みが遅い	GPU 使用率が低いのに処理が遅い。データの読み込みを待っている	保存装置 ・ 読み込み方法の見直し
③ メモリが足りない	メモリが満杯になり、スワップで遅くなる	容量の追加 ・ 使い方の見直し
④ 通信を待っている	複数の機器の間で結果の受け渡しを待つ。帯域や遅延が原因	経路 ・ 機器 ・ 配置の見直し

⚠ 注意

使用率が低い＝余裕がある、とは限りません。どの部品が何を待っているかを、測定値と記録から確かめます（p.559）。

部品の組み合わせの条件：PC 自作とサーバー

今見ている範囲：

本体の外観



ケースの中



マザーボード



CPU



コア



コアの中

⚠ 合わせる条件

CPU のソケットの形と世代、メモリの規格、PCIe の世代と本数（レーン数）、電源の容量。どれかが合わないと、差さらない・動かない・性能が出ない、ということが起きます。

🔧 PC 自作とのつながり

部品を選び、相性を確かめ、組み立て、起動を確認する経験は、サーバーの構成を読む練習になる。

☰ サーバーとの違い

サーバー（サービスを提供する側のコンピュータ）は、電源の二重化、交換しやすさ、遠隔管理の専用部品（BMC）、多数のメモリやカードを前提にした設計が多い。

第2章のまとめ

✓ ここまでで説明できること

- ・ 本体→ケースの中→マザーボード→CPU→コア→コアの中の位置関係を説明できる。
- ・ プログラムとデータが SSD→メモリ→CPU へ移り、取り出し→解読→実行→書き戻しで計算され、保存される流れを説明できる。
- ・ 制御装置・演算装置・レジスタ・キャッシュ・コア・クロックの役割と、クロックだけで性能を比べられない理由を説明できる。
- ・ SSD と HDD、M.2 ・ SATA ・ PCIe ・ NVMe の違い、GPU の役割、性能指標とボトルネックの見方を説明できる。

→ 次へのつながり

部品は、プログラムがなければ何もしません。第3章では、部品の上で動くソフトウェアの層と、部品を管理する OS を学びます。

確認の問い（第 2 章）

自分の言葉で答えてから、次のページの解説を読みましょう。

- 1 メモリとストレージの違いを 2 点で説明してください。
- 2 「M.2 の SSD なら必ず NVMe で高速だ」は正しいですか。
- 3 GPU の使用率が低いのに処理が遅いとき、何を疑いますか。

確認の問い（第2章）：解説

前のページの問いへの答えと、考え方です。

✓ 問1 メモリとストレージの違いを2点で説明してください。

メモリ（主記憶）は一般に電源を切ると内容が消え、CPUが直接読み書きします。ストレージ（補助記憶）は電源を切っても残り、CPUはいったんメモリへ写してから使います。 p.43

✓ 問2 「M.2のSSDなら必ずNVMeで高速だ」は正しいですか。

正しくありません。M.2は形と端子の規格で、接続にはPCIeとSATAがあります。M.2形でSATA接続（AHCIなど）の製品はNVMeではありません。 p.88

✓ 問3 GPUの使用率が低いのに処理が遅いとき、何を疑いますか。

GPUに余裕があるのではなく、保存装置からの読み込みやデータの転送を待っている可能性があります。どの部品が何を待っているかを測定値で確かめます。 p.99

第3章

ソフトウェアと OS

この章で学ぶこと

ソフトウェアがどんな層に分かれているか、その中心の OS（基本ソフト）が何をしているか、実行中のプログラム（プロセス）、ファイルとファイルシステム、権限、ミドルウェア、アプリを学びます。

→ 前の章とのつながり

第2章では、CPU・メモリ・ストレージといった機械の中の部品と、命令とデータが部品の間を動く様子を見ました。この章では、その機械の上で動くソフトウェアの側を見ます。

ソフトウェアは層に分けて考える

スマホやPCの中のソフトウェアは、役割ごとに層に分けて考えると整理しやすくなります。上の層ほど利用者に近く、下の層ほど機械に近い仕事をします。

アプリ

利用者が直接使う機能。例：買い物アプリ、地図、チャット

ミドルウェア

多くのアプリが共通に使う機能の土台。例：データを管理するソフト、Webページを返すソフト

OS（基本ソフト）

機械の部品を管理し、アプリを動かす土台。例：Windows、Android

ハードウェア

CPU・メモリ・ストレージなどの機械の部品（第2章）

上の層は、下の層の機能を使って動きます。買い物アプリは、画面に文字を出すときも保存するときも、部品を直接動かさず下の層に頼みます。次のページから、OSを中心に順に見ます。

OS とは：機械を管理する基本ソフト

OS (オーエス)

分類：ソフトウェア

略語：OS = Operating System (オペレーティングシステム) 日本語：基本ソフト

何か	機械の部品（CPU・メモリ・ストレージなど）を管理し、アプリを動かす土台になる基本のソフトウェア。「オペレーティング」は動かす、「システム」はひとまとまりの仕組みの意味。
なぜ必要か	アプリごとに部品の扱い方を作らなくて済む。複数のアプリが部品を取り合わないよう、順番と分け方を決める。
どこにあるか	ハードウェアのすぐ上。電源を入れると最初のほうに動き出し、アプリはその上で動く。
受け取る→する→返す	アプリから「保存して」などの依頼を受け取る→部品を動かす→結果を返す。
身近な例	Windows、macOS、Linux、Android、iOS

OS が管理する「資源」とは

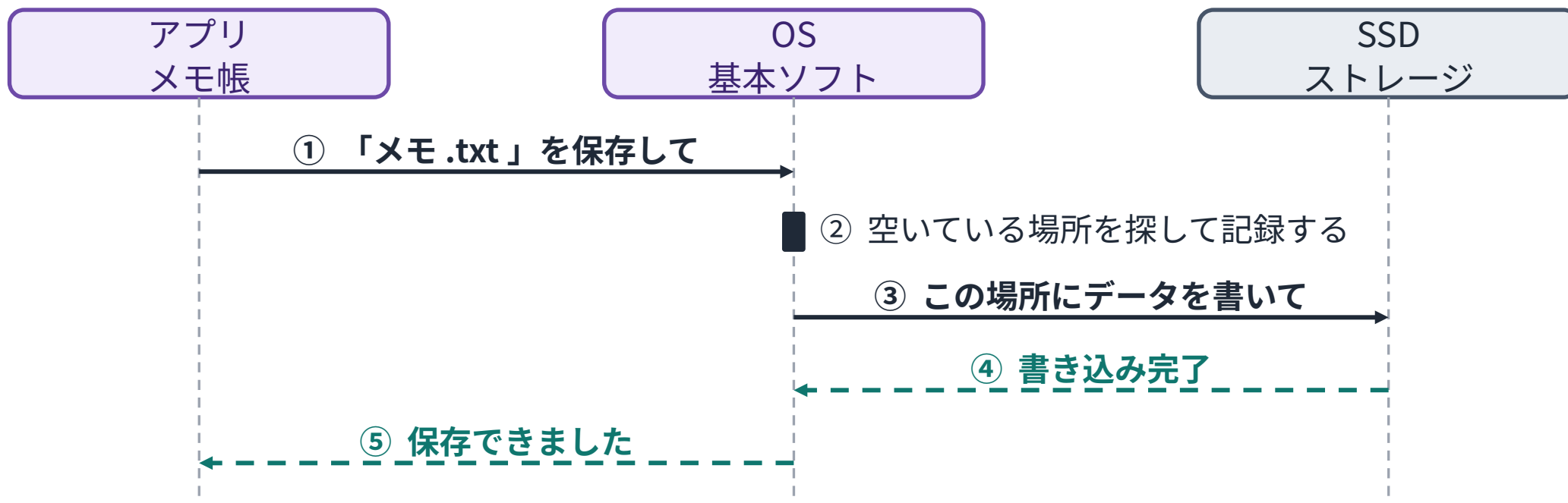
IT でいう**資源**（リソース）とは、プログラムが動くために使う、限りのある機械の能力や場所のことです。OS は、これを複数のアプリに分け与え、取り合いにならないよう管理します。

資源	OS がすること	例
CPU を使う時間	ごく短い時間ずつ順番に割り振る	音楽を流しながら地図を見る
メモリ の場所	どのプログラムがどこを使うか割り当てる	アプリどうしがデータを壊さない
ストレージ の領域	どこに書くかを決め、空きを管理する	保存すると空き容量が減る
入出力・通信の口	どのアプリ宛てかを見て受け渡す	届いた通信を正しいアプリに渡す

このほか、ファイルの管理（ p.112 ）と、誰が何をしてよいかの管理（権限、 p.114 ）も OS の仕事です。

OS がないと何に困るか：保存ボタンの例

ストレージの製品ごとに書き込みの手順は違います。OS がなければ、アプリはすべての部品の扱い方を自分で持つ必要があります。OS があれば「保存して」と頼むだけです。



時間順のやり取り図（学習用に簡略化）。上から下へ時間が進みます。黒い矢印＝依頼、緑の破線の矢印＝返事。

代表的な OS と、カーネル・ディストリビューション

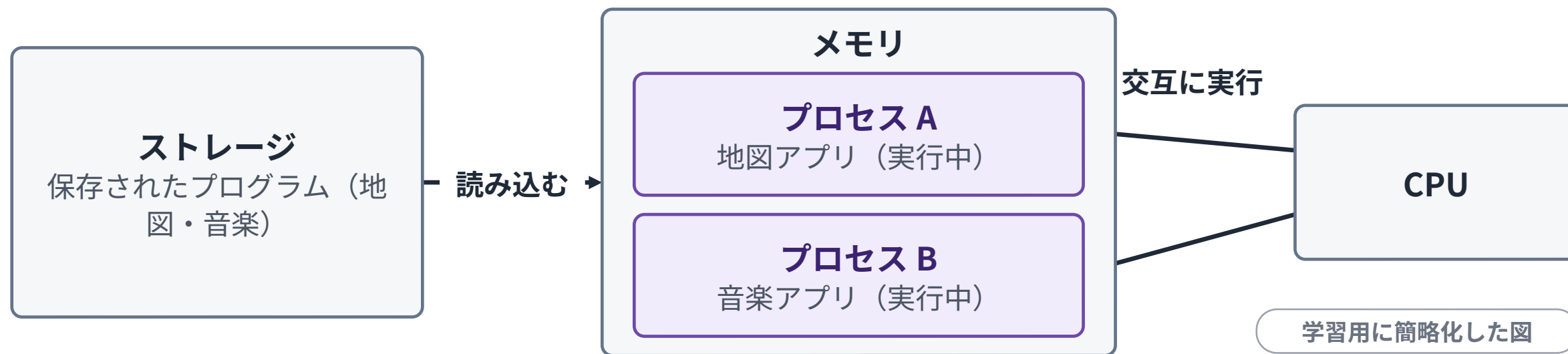
OS	主に使われる場所
Windows	会社や家庭の PC 、一部のサーバー
macOS	Apple 社の PC
Linux	サーバーで広く使われる。組み込み機器にも
Android	多くのスマホ（中心部分に Linux のカーネルを使う）
iOS	iPhone

OS の中心で、CPU ・メモリ ・入出力を直接管理する部分を**カーネル**といいます。Linux は厳密にはこのカーネルの名前で、実際には周辺のソフトと組み合わせて配布される**ディストリビューション**（例：Ubuntu 、 Red Hat Enterprise Linux ）として使います。

プロセス：実行中のプログラム

保存されたプログラムは、ストレージの中のデータにすぎません。実行すると OS がメモリに読み込み、CPU を使う時間やメモリの場所を割り当てます。この「実行中のプログラムと、その状態・資源のまとめり」を**プロセス**といいます。

黒い矢印＝読み込む向きと、CPU が実行する相手。



例：Windows のタスクマネージャーで、動いているプロセスと、それぞれの CPU やメモリの使用量が見られます。

サービス（デーモン）：裏で動き続けるプログラム

画面には出ず、裏で動き続けて機能を提供するプログラムを、Windows では**サービス**、Linux などでは**デーモン**と呼びます。プロセスの一種で、OS の起動とともに動き出すものが多いです。

例	していること
時刻合わせ	時計を正しい時刻に、自動で合わせ続ける
印刷	印刷の依頼を受け付け、順番にプリンターへ送る
依頼の待ち受け	ほかのコンピューターからの依頼を待ち、届いたら応える（お店のコンピューターで注文の依頼を受け付けるプログラムも、この形で動く）

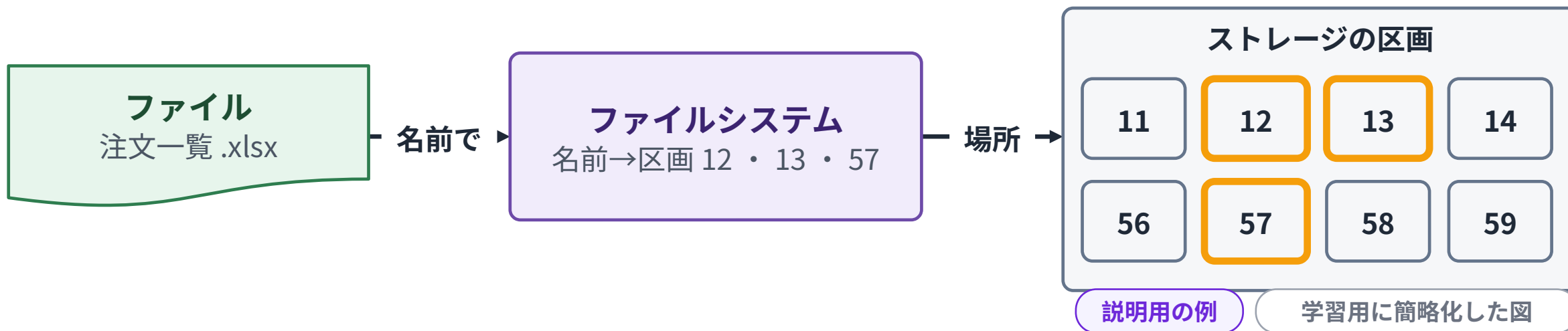
⚠ 似た言葉との違い

このページのサービスは OS の用語です。買い物サイトや動画配信のように、利用者に提供される「ネットのサービス」とは別の意味です。

ファイルとファイルシステム

ファイルは、名前を付けてひとまとまりにしたデータです（例：「注文一覧.xlsx」）。ファイルを入れておく入れ物がフォルダーです。ストレージの中は、番号の付いた小さな区画が並んでいるだけです。どのファイルがどの区画にあるかを記録して管理する仕組みを**ファイルシステム**といい、OSの機能の1つです。

黒い矢印は、ファイル名から実際の区画をたどる向きです。



代表的なファイルシステム：NTFS（Windows）、ext4（Linux）、APFS（macOS・iOS）

ファイルを保存するときの層と、フォーマット

アプリ

ファイルを開く・保存する

ファイルシステム

名前・フォルダー・権限・空き領域を管理（NTFS・ext4・APFS など）

番号付きの区画

ストレージを番号の付いた区画（ブロック）の並びとして読み書きする

保存する装置

SSD / HDD / ネットワークの先にある保存装置（第8章）

⚠ フォーマットの注意

フォーマット（初期化）は、ファイルシステムを作り直す作業です。どの区画に何があるかの記録が新しくなるため、基本的に中のデータは使えなくなります。実行前に、必要なデータが別の場所に残っているか確認します。

ネットワーク越しに保存するときも、ファイル単位でやり取りする方法と、区画単位でやり取りする方法があり、使う約束が違います（第8章）。

権限：誰が何をしてよいか

権限（アクセス権）は、どの利用者やプログラムが、どのファイルや機能に対して、読む・書く・実行するのどれをしてよいかの決まりです。OSは操作のたびに権限を確かめ、許されていない操作を断ります。

操作	意味	例
読む	中身を見る	注文一覧を表示する
書く	中身を変える・消す	注文一覧を書き換える
実行する	プログラムとして動かす	アプリを起動する

 **ポイント**

権限は、必要な人に必要な範囲だけ与えるのが基本です。全員に何でも許すと、操作の誤りや不正な利用の被害が大きくなります。例：注文の記録は、受付のプログラムには「読む・書く」、集計担当の人には「読む」だけを許す（説明用の例）。

ミドルウェア：アプリを支える共通の機能

ミドルウェアは、OS とアプリの間（ミドル＝中間）にあって、多くのアプリが共通に使う機能を提供するソフトウェアです。アプリが毎回同じ機能を作らなくて済みます。

⚙️ データを管理するソフト

大量のデータの保存・検索を引き受ける。例：注文の記録から佐藤さんの分を探す。
（第5章）

⚙️ Web ページを返すソフト

スマホなどからの依頼を受け付け、ページを返す。（第7章）

⚙️ プログラムの実行環境

お店の業務のプログラムを動かすための土台。例：Java という言語で書いたプログラムの実行基盤。（第7章）

どれも、注文履歴の例で後の章で詳しく見ます。ここでは「アプリとOSの間の共通の土台」と覚えておけば十分です。

アプリ（アプリケーション）：利用者が使う機能

アプリケーション（略して**アプリ**）は、利用者の目的に合わせた機能を提供するソフトウェアです。買い物アプリ、表計算、地図、チャットなどがそうです。お店のコンピュータで注文を受け付けて処理するプログラムも、画面はなくても利用者のための機能なのでアプリです。

⚠ 対象の OS が違うと動かない

アプリは特定の OS 向けに作られます。Windows 用のアプリは、そのままでは iPhone で動きません。OS によって、OS への頼み方やプログラムの形式が違うためです。そのため、Windows・Linux・スマホのアプリ開発では、対象の OS・実行環境・使う道具がそれぞれ違います。

層の分け方は、理解のための整理

層の図は役割を理解するための整理です。実際には1台の機械に複数の層が入り、保存や通信も専用の機器とソフトの両方で成り立ちます。構成によって層が増えることもあります。

物理機に直接 OS を入れる構成

アプリ

ミドルウェア

OS

ハードウェア

仮想化を使う構成

アプリ

ミドルウェア

OS（仮想的な機械の中）

機械を分けるソフト（追加）

ハードウェア

右の仮想化では、1台の機械を複数の仮想的な機械に分けるソフト（ハイパーバイザー）が加わり、その上で各OSが動きます。常に必要な層ではありません（第10章）。

第3章のまとめ

✓ ここまでで説明できること

ソフトウェアはアプリ・ミドルウェア・OSの層で考える（理解のための整理）。
OSはCPUの時間・メモリ・ストレージ・入出力などの資源を管理する。中心部分がカーネル。
プロセスは実行中のプログラム。サービス（デーモン）は裏で動き続けるプログラム。
ファイルシステムはファイル名と保存場所の対応を管理する。フォーマットするとデータは使えなくなる。
権限は「誰が何をしてよいか」の決まり。必要な範囲だけ与える。

→ 次の章へ

次の第4章では、複数のコンピューターが「頼む側」と「応える側」に分かれて協力する仕組み（サーバーとクライアント）と、ITの土台の全体像を見ます。

確認の問い

自分の言葉で答えてから、次のページの解説を読みましょう。

- 1 OS が管理する「資源」の例を 3 つ挙げてください。
- 2 保存されたプログラムとプロセスは、どう違いますか。
- 3 フォーマットすると、なぜデータが使えなくなるのですか。

確認の問い：解説

前のページの問いへの答えと、考え方です。

✓ 問1 OSが管理する「資源」の例を3つ挙げてください。

CPU を使う時間、メモリの場所、ストレージの領域、入出力（画面・キー・通信の口）などです。OS はこれを複数のアプリに分け与えます。 p.107

✓ 問2 保存されたプログラムとプロセスは、どう違いますか。

保存されたプログラムはストレージの中のデータです。実行してメモリに読み込まれ、CPU の時間などを割り当てられて動いているまとまりがプロセスです。 p.110

✓ 問3 フォーマットすると、なぜデータが使えなくなるのですか。

フォーマットはファイルシステムを作り直す作業で、どの区画にどのファイルがあるかの記録が新しくなるためです。 p.113

第4章

サーバーとITインフラの全体像

この章で学ぶこと

ネットを通じたサービスで、頼む側と応える側がどんなデータをやり取りするか（要求と応答）、サーバーとクライアントという役割、スマホ・PC・サーバーの違い、そしてアプリを支える土台（ITインフラ）の全体像を学びます。

→ 前の章とのつながり

第1～3章では、1台のコンピューターの中を見てきました。この章からは、複数のコンピューターが協力する世界に進み、はじめにで見た「注文履歴を見る」の流れを少しずつ詳しくしていきます。

サービスには、提供する側と使う側がいる

買い物サイトや地図のように、ネットを通じて利用者に提供される機能を、この章では「サービス」と呼びます。サービスには、機能を提供する側と使う側がいます。

日常の窓口

- 客が窓口で「この荷物を送りたい」と頼む
- 窓口の係が受け付けて手続きをする
- 結果（受付票）を客に返す

ネットのサービス

- スマホが「注文履歴を見せて」と頼む
- お店のコンピューターが記録を調べる
- 結果（注文の一覧）を返す

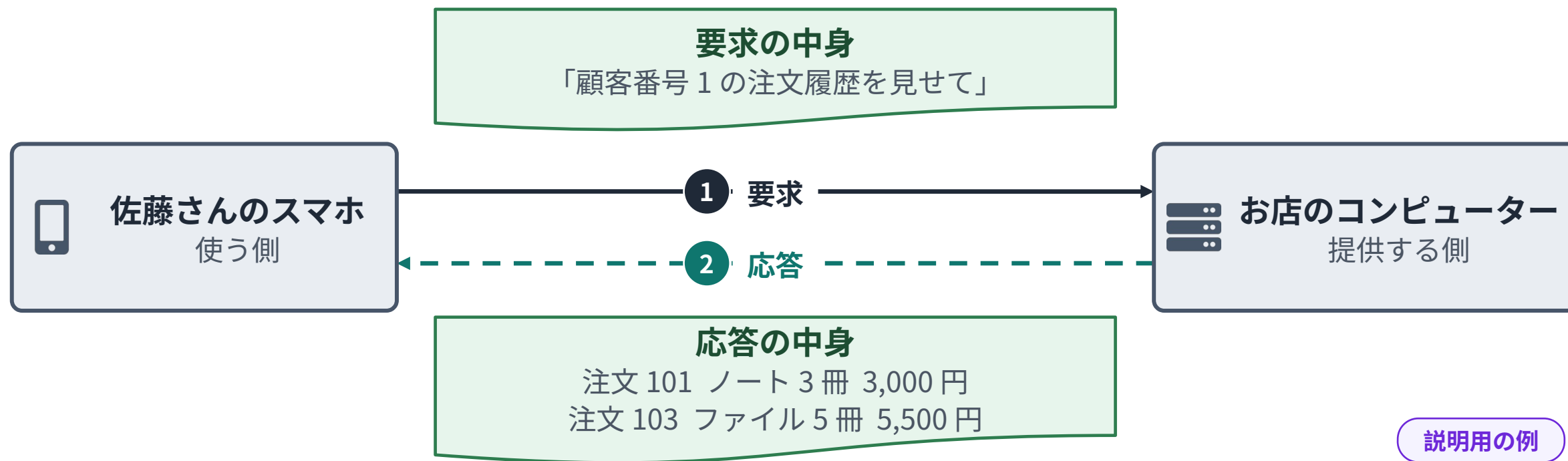
似た言葉との違い

第3章のサービス（デーモン）は、OSの中で裏で動き続けるプログラムでした。この章の「サービス」は利用者に提供される機能で、意味が違います（ただし、お店で依頼を待ち受けるプログラムは、たいていデーモンとして動きます）。

要求と応答：頼むデータと、返すデータ

機器どうしがデータを送り合うことを**通信**といいます。使う側が送る「これをして」という依頼のデータが**要求**（リクエスト）、提供する側が返す結果のデータが**応答**（レスポンス）です。

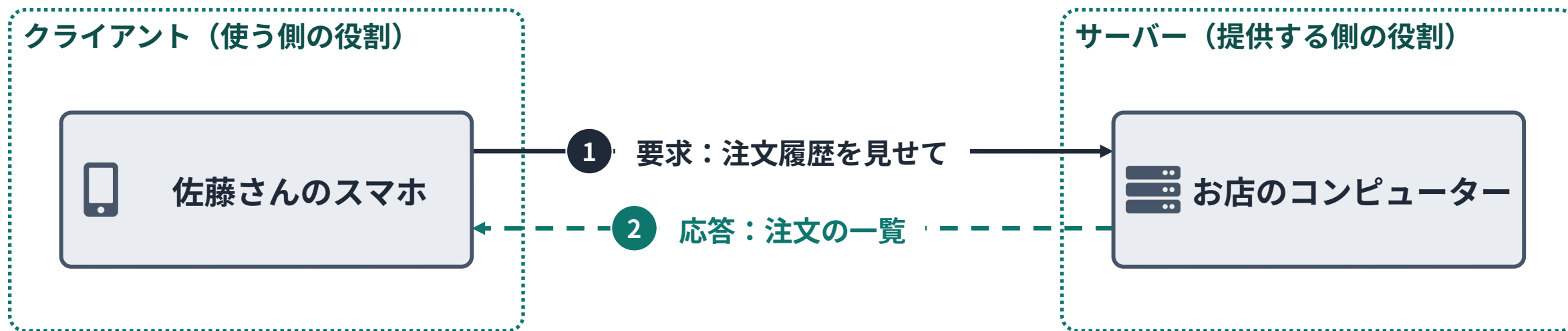
黒い矢印＝要求、緑の破線の矢印＝応答。緑の書類の形は中身（実際は決まった形式で書く）。



サーバーとクライアントは、役割の名前

サービスを提供する側を**サーバー**、利用する側を**クライアント**といいます。要求を送るのがクライアント、要求を受けて応答を返すのがサーバーです。どちらも機械の種類ではなく、やり取りの中での役割の名前です。

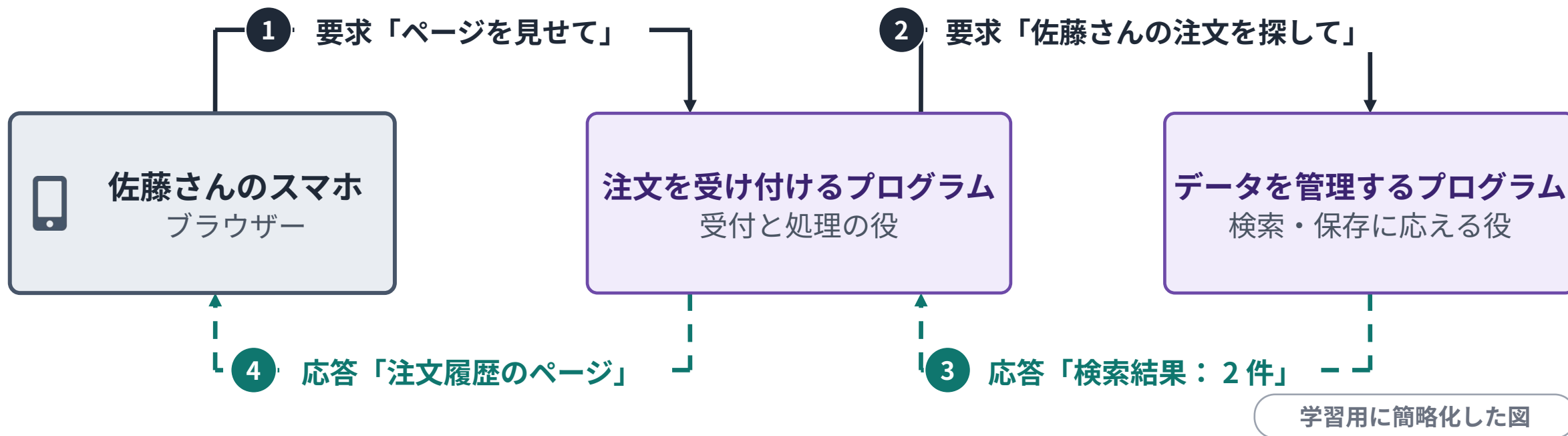
点線の枠は役割、灰色の太枠の箱は機械です。黒い矢印は要求、緑の破線の矢印は応答。



このように役割を分けて作る仕組みを**クライアントサーバシステム**といいます。1台のサーバーが、多くのクライアントからの要求に応えます。

同じプログラムが、サーバーにもクライアントにもなる

役割の関係図（丸数字は順番）。黒い矢印＝要求、緑の破線の矢印＝応答。ブラウザーは Web ページを見るアプリ。



真ん中は、スマホから見ればサーバー、右に頼むときはクライアントです。右は、データの検索や保存の依頼に応える役のサーバーです（第5章）。

「サーバー」という言葉の2つの意味

① 機械としてのサーバー

サーバー用に作られた機械（ハードウェア）。長時間止まらずに動くことや、離れた場所から管理しやすいことを重視して作られている。

② ソフトとしてのサーバー

要求に応えるソフトウェア。例：Web ページを返すソフト。1 台の機械で、複数のサーバーソフトが同時に動くこともある。

たとえ

サーバーは「店員」、クライアントは「客」のようなものです。客が頼み、店員が応えます。

たとえの限界

実際のサーバーは、1 台で何千人もの要求に同時に応えます。また、応えるために裏で別のサーバー（データを管理する役など）に頼むこともあります。「1 人の客に 1 人の店員」という形ではありません。

スマホ・PC・サーバーは、部品は似ていても目的が違う

どれも CPU ・メモリ・ストレージ・通信の部品・電源を持ちます。違いは、誰が使い、どれだけ止まらずに動き、どう管理するかです。

比べる点	スマホ	PC	サーバー
主な使い手	1 人が持ち歩く	1 人が机で使う	多数の利用者・ほかの機器
動く時間	使うときが中心	使うときが中心	長時間・連続が前提のことが多い
故障への備え	買い替え・データの同期	バックアップ（データの写し）	電源や保存部品の二重化、交換しやすさ
管理のしかた	本人・会社の一括管理	本人・会社の管理	遠隔管理用の部品（BMC）・監視・保守契約
置き場所	手元	家・職場	サーバー室・データセンター

BMC は、OS とは別に動き、離れた場所から電源の操作や状態の確認をするための部品です（第 9 章）。二重化は同じ部品を 2 つ以上用意して、片方が壊れても動き続けるようにすることです。

IT インフラ：アプリやサービスを動かす土台

銀行の振込、学校の連絡、工場の生産管理、スマホの地図や決済は、どれも利用者からは見えない場所のコンピューター・保存装置・通信で動いています。アプリやサービスを動かすための、計算・保存・通信・電力と冷却・運用の土台を **IT インフラ** といいます。

銀行

振込・残高照会

学校

成績・連絡・授業資料

工場

生産計画・設備の監視

スマホのサービス

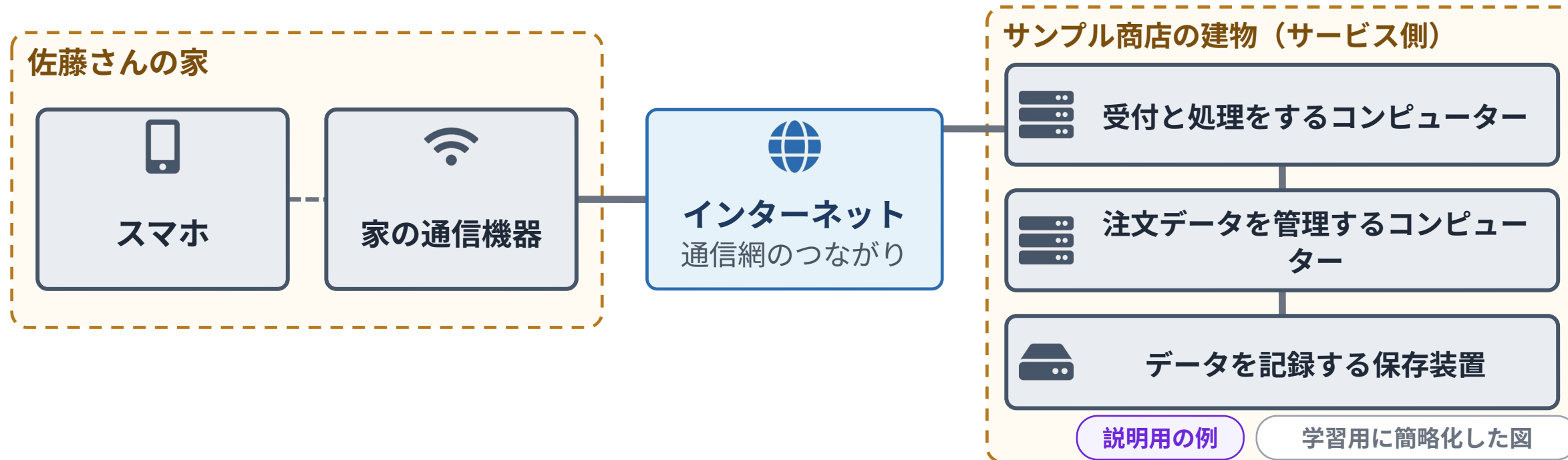
地図・決済・AIチャット

補足

「インフラ」はインフラストラクチャー（infrastructure）の略で、道路・水道・電気のように社会の活動を下で支える土台のことです。IT インフラは、その情報技術版です。

「注文履歴を見る」の全体像（やさしい版）

物理的な配置の図。破線の大きな枠＝場所、灰色の破線＝電波、灰色の太い線＝ケーブル。



地図の3つの区間で、それぞれ何が起こるか

区間	ここで起きること	くわしく
家の中	スマホが電波で家の通信機器につながり、家の外への出口へデータを渡す	p.209
インターネット	相手の場所を shop.example.com のような名前から調べ、多くの通信網を経由して運ぶ。途中で読まれないよう、中身を暗号にして運ぶ	p.247 ・ p.269
サービス側	要求を受け付けて振り分け、注文の処理をし、データを検索・保存する	p.326 ・ p.335

ポイント

図の「受付と処理」「注文データの管理」は機能（役割）の名前です。実際には、1台の機械が複数の機能を兼ねることも、1つの機能を数千台の機械で分担することもあります。役割の数と機械の台数は、一致するとは限りません。

IT インフラを、上から順に見る

利用者

スマホ・PC

アプリ

画面・業務の処理（買い物、チャット、勤怠）

ミドルウェア

データを管理するソフト・Web ページを返すソフトなど

OS

CPU の時間・メモリ・ファイル・権限を管理

計算（サーバー）

CPU ・ GPU ・ メモリ で処理を実行

保存（ストレージ）

データを長く保つ装置

通信

機器どうし・利用者をつなぐ

施設（電力・冷却）

データセンターなどの建物・設備

🔑 運用

上のすべてを動く状態に保つ「運用」という活動が、全部の層にかかわります（次のページ）。

📄 補足

層の分け方は理解のための整理です。実際には1台の機械に複数の層が入り、保存や通信も専用の機器とソフトの両方で成り立ちます。

運用：ITインフラを動く状態に保ち続ける

故障や利用者の増加、更新などに対応しながら、サービスを止めずに動かし続ける活動を**運用**といいます。主な仕事は次の4つです。

仕事	何をするか	注文サイトの例
監視	状態を見続け、異常を早く見つける	応答が遅くなっていないか、保存装置の空きが減っていないかを見る
保守	部品の交換や、ソフトの更新をする	壊れかけた部品を交換する、不具合を直す更新を入れる
バックアップ	データの複製を別の場所に残す	注文の記録を毎日、別の保存装置にも写す
変更管理	変更を計画・確認・記録して安全に行う	手順と、失敗したときに元に戻す手順を事前に決める

バックアップは取るだけでなく、実際に元に戻せることを確かめておくことが大切です。

データセンター：コンピューターを動かす建物

データセンター（でーたせんたー）

分類：場所・施設

何か	多数のサーバーや保存装置、通信機器を、安全に動かし続けるための専用の建物（施設）。
なぜ必要か	機械は大量の電気を使い、熱を出す。停電・火災・不正な侵入から守り、止めずに動かすには専用の設備が要る。
どこにあるか	会社の自前の建物のほか、データセンターを運営する事業者の建物を借りることも多い。
身近な例	電源設備（停電に備えた電池や発電機）、冷却設備、外部との太い通信回線、入退室管理（許可された人だけが入れる）
似た言葉との違い	クラウド（ネット経由で計算や保存の能力を必要なだけ借りる方式）は提供のしかたの名前で、建物とは別物。クラウドの機械も、どこかのデータセンターにある。

第4章のまとめ

✓ ここまでで説明できること

使う側は要求（依頼のデータ）を送り、提供する側は応答（結果のデータ）を返す。データを送り合うことが通信。

サーバー（提供する側）とクライアント（使う側）は役割の名前。同じプログラムが両方になることもある。

ITインフラは、計算・保存・通信・電力と冷却・運用の土台。役割の数と機械の台数は一致するとは限らない。

運用の主な仕事は、監視・保守・バックアップ・変更管理。

→ 次の章へ

次の第5章では、お店の「データを管理する役のサーバー」の中を見ます。注文の記録をどう整理し、どう探し出すのかを、データベースという考え方から説明します。

確認の問い

自分の言葉で答えてから、次のページの解説を読みましょう。

- 1 「注文履歴を見る」で、要求と応答の中身は何ですか。
- 2 受付のプログラムが、サーバーにもクライアントにもなるのはなぜ？
- 3 運用の4つの仕事を挙げてください。

確認の問い：解説

前のページの問いへの答えと、考え方です。

✓ 問1 「注文履歴を見る」で、要求と応答の中身は何ですか。

要求は「顧客番号1（佐藤さん）の注文履歴を見せて」という依頼のデータ、応答は注文101と103の一覧という結果のデータです。 p.123

✓ 問2 受付のプログラムが、サーバーにもクライアントにもなるのはなぜ？

スマホからの要求に応えるときはサーバー、データを管理するプログラムに検索を頼むときはクライアントだからです。サーバー・クライアントは役割の名前です。 p.125

✓ 問3 運用の4つの仕事を挙げてください。

監視（異常を早く見つける）、保守（部品交換・更新）、バックアップ（複製を残して戻せるようにする）、変更管理（変更を計画・確認・記録して安全に行う）です。 p.132

第 5 章

データベースと DB サーバー

この章で学ぶこと

佐藤さんが「注文履歴を見る」を押したとき、サンプル商店の側では注文の記録がどこに蓄えられ、誰が探して返すのでしょうか。データ→データベース→管理するソフトウェア→依頼に応える役割→記録する装置の順に、一つずつ区別して学びます。

→ 前の章とのつながり

第 4 章で、依頼に応える側を「サーバー」、依頼を「要求」、返事を「応答」と呼ぶことを学びました。第 2 章では、電源を切っても記録が残る SSD を見ました。この章はその 2 つを使います。

なぜデータをまとめて管理するのか

サンプル商店では、利用者が注文するたびに「誰が・何を・いくつ・いくらで」買ったかの記録が1件ずつ増えます。仮に1日1,000件なら1年で約36万件です（説明用の例）。この記録を扱うには、次の3つの難しさがあります。

量が増え続ける

何十万件の中から、佐藤さんの注文だけを、毎回すぐに探し出す必要があります。

多人数が同時に使う

佐藤さんが履歴を見ている同じ瞬間に、別の人注文し、店員が発送の状況を書き換えます。

失ってはいけない

記録が消えたり食い違ったりすると、商品が届かない・代金が合わないなどの実害が出ます。

なぜ必要か

そこで、記録を決まった形に整理し、専用の仕組みでまとめて管理します。その中心が、この章で学ぶ「データベース」です。

データとは：注文1件の記録

データとは、コンピュータで扱えるように記録された値のことです（p.16）。佐藤さんがノートを買ったと、右のような1件の記録ができます。

「101」「ノート」「3」という一つ一つの値と、それが何を表すかを示す項目名（注文番号・商品名・数量など）が組になっています。

緑の書類の形＝データ（1件の記録）

注文1件の記録

注文番号：101

顧客番号：1（佐藤さん）

商品名：ノート

数量：3

金額：3,000 円

説明用の例

i 値だけでは意味が決まらない

「3」だけでは、3冊なのか3,000円なのか分かりません。「数量」という項目名と組にして初めて意味が決まるので、記録は項目をそろえて整理します。

表で整理する：1行が1件、1列が1項目

同じ項目をもつ記録が何件もあるときは、**表**（テーブル）の形に並べると整理できます。横の1**行**（レコード）が注文1件、縦の1**列**（フィールド）が1つの項目です。下はサンプル商店の注文表（orders）です。

order_id 注文番号	customer_id 顧客番号	product 商品名	quantity 数量	amount 金額（円）
101	1	ノート	3	3,000
102	3	ペン	2	1,200
103	1	ファイル	5	5,500

具体例

黄色の行＝注文 101 の 1 件、青い列＝数量（quantity）の項目。列名は後で学ぶ命令文で使うため英語です（値は説明用の例）。

1 つのファイルをみんなで使うと困ること

表にした記録を、表計算ソフトのファイル 1 つにして、店員みんなが共有の場所で開いて使うと、次のことに困ります（説明用の例）。

困ること	具体的な場面
同時に書き換えて食い違う	A と B が同じファイルを別々に直して保存すると、後の保存で先の修正が消えることがあります。
探すのが遅い	何十万件の中から毎回探すのは時間がかかります。
見てよい人を分けにくい	ファイルを開ける人には、他人の住所や注文も見えます。
途中で止まると壊れる	保存中に電源が落ちると、中途半端な状態で残ることがあります。

データベース（DB）とは

「DB」は Database の略で、日本語でもそのまま「データベース」と呼びます。

データベース（でーたべーす）

分類：データ

略語：DB = Database 日本語：data（データ） + base（基地・置き場） = データを集めておく基地

何か	目的に合わせて整理し、まとめて蓄えたデータの集まり。サンプル商店なら、顧客表と注文表をまとめたもの。
なぜ必要か	何十万件の記録から必要な行をすぐ探し、多人数が同時に使っても食い違わないようにするため。
どこにあるか	最終的には SSD などの保存装置に記録される。ただし「データベース」が指すのは装置ではなく、整理されたデータの側。
身近な例	買い物サイトの注文履歴、会員名簿、図書館の蔵書目録

データベースの中身：顧客表と注文表

顧客表は「どんな利用者がいるか」、注文表は「誰が何を注文したか」の記録です。

顧客表（customers）

customer_id	name	city
1	佐藤	東京
2	鈴木	大阪
3	高橋	福岡

i 補足

列：customer_id = 顧客番号、name = 名前、city = 都市。
下は注文表（orders）。顧客番号 1 は佐藤さんです。

order_id	customer_id	product	quantity	amount
101	1	ノート	3	3,000
102	3	ペン	2	1,200
103	1	ファイル	5	5,500

データベースを表す「円柱」の記号

この先の図では、データベースを円柱（茶筒のような形）で描きます。これは図の中でデータベースを表すための記号で、IT 分野の図でよく使われる約束事です。実物が円柱の形をしているという意味ではありません。

この資料での形の区別（左から、データ 1 件・データベース・装置）



データ（1 件・1 ファイル）



データの集まり（記号）



機械・装置（ハードウェア）

⚠ 注意

円柱は装置を表しません。データを記録する SSD などの装置は、円柱とは別に灰色の太枠の箱で描き、「データの集まり」と「記録する装置」を図の上でも区別します。

DBMS：データベースを管理するソフトウェア

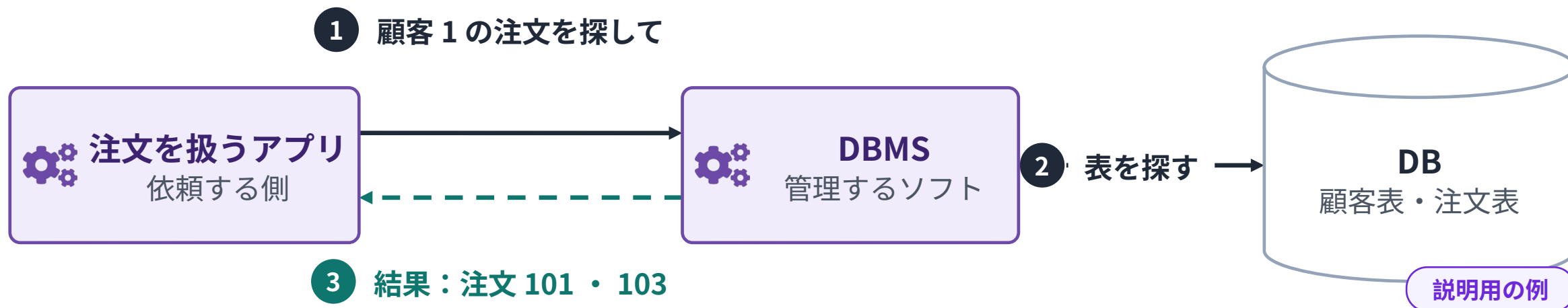
データの集まりは、それだけでは自分で探したり書き込んだりしません。

DBMS は Database Management System の略で、日本語では「**データベース管理システム**」です。3つの語を、サンプル商店の場面に当てはめます。

語	この場面での意味
データベース (Database)	前のページまでで見た、顧客表・注文表などの表の集まり。
管理 (Management)	表から行を探す・追加する・書き換える・消す、という依頼を受け付けて実行すること。あわせて、多人数が同時に使っても食い違わないよう順番を整える、依頼した人に権限があるか確かめる、故障しても記録が壊れないよう守る。
システム (System)	それらの仕事をまとめて行う、ひとそろいのソフトウェア（プログラムと設定）。

DBMS は何を受け取り、何をして、何を返すか

役割の関係図。紫＝ソフトウェア、円柱＝データベース。黒い矢印＝依頼、緑の破線＝返事。



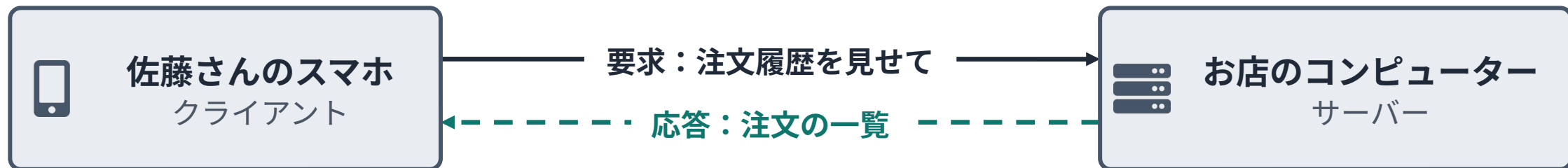
DBMS は、アプリなどの別のプログラムから依頼を受け取ります。依頼は、データベース向けの決まった書き方の命令文（SQL、p.163）で書かれます。DBMS はそれに従って表を探したり書き換えたりし、結果を返します。アプリは、表が SSD のどこに記録されているかを知らなくてもかまいません。

DBMS の製品例： PostgreSQL、Oracle Database、MySQL など（p.181）

復習：「サーバー」は依頼に応える役割の名前

第4章で学んだように、**サーバー**は、依頼（**要求**）を受けて仕事をし、結果（**応答**）を返す側の役割の名前です（p.124）。依頼する側は**クライアント**と呼びます。

役割の関係図。灰色の太枠＝機械。黒い矢印＝要求、緑の破線の矢印＝応答です。



学習用に簡略化した図

🔑 ポイント

「サーバー」は、①サーバー用の機械と、②要求に応えるソフトウェアの両方の意味で使われますが、どちらも「依頼に応える役割」です。同じプログラムが、相手によってサーバーにもクライアントにもなります。

DB サーバー：「DB」＋「サーバー」

「**DB サーバー**」は「**データベースサーバー**」の略です。2つの部分に分けて意味を確かめてから、組み合わせます。

① 「DB」

データベース（Database）の略。整理して蓄えたデータの集まり（p.142）。

② 「サーバー」

要求を受けて仕事をし、応答を返す役割の名前（p.147）。

①＋②

データベースへの依頼（探して・追加して・書き換えて）を受け付け、DBMS で実行し、結果を返す役割。

ポイント

DB サーバーは「データベースへの依頼に応える係」という役割の名前です。実際には、DBMS が動いている機械（またはそれに相当するもの）がこの役割を担います。

DB サーバーの役割を担うものは 1 種類ではない

DB サーバーは役割の名前なので、それを担う「入れ物」にはいくつかの形があります。共通しているのは、中で DBMS が動いていて、データベースへの依頼に応えることです。

物理的な機械

DBMS を動かすサーバー機そのもの。

仮想の機械

1 台の機械をソフトで区切って作った仮想の機械（第 10 章）。

事業者のサービス

事業者が運用する DBMS を通信回線越しに借りる（p.183）。

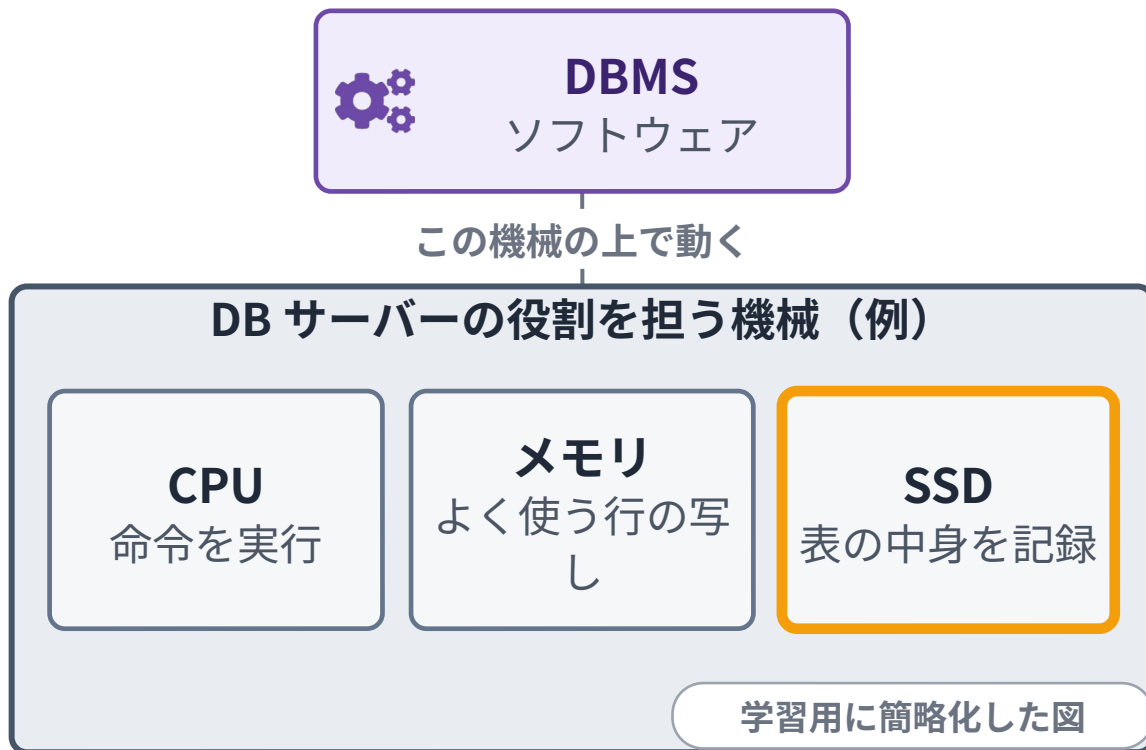
注意

「DB サーバー＝DB 専用の物理的な機械」とは限りません。小さなシステムでは、1 台の機械が画面を作るプログラムと DBMS の両方を動かすこともあります（第 7 章）。

また、注文を扱うアプリは、スマホに対してはサーバーですが、DB サーバーに依頼するときはクライアントです。役割は相手との関係で決まります。

実際に記録するのは保存装置（SSD など）

学習用の模式図。紫＝ソフトウェア、灰色の太枠＝機械、薄い灰色＝部品。

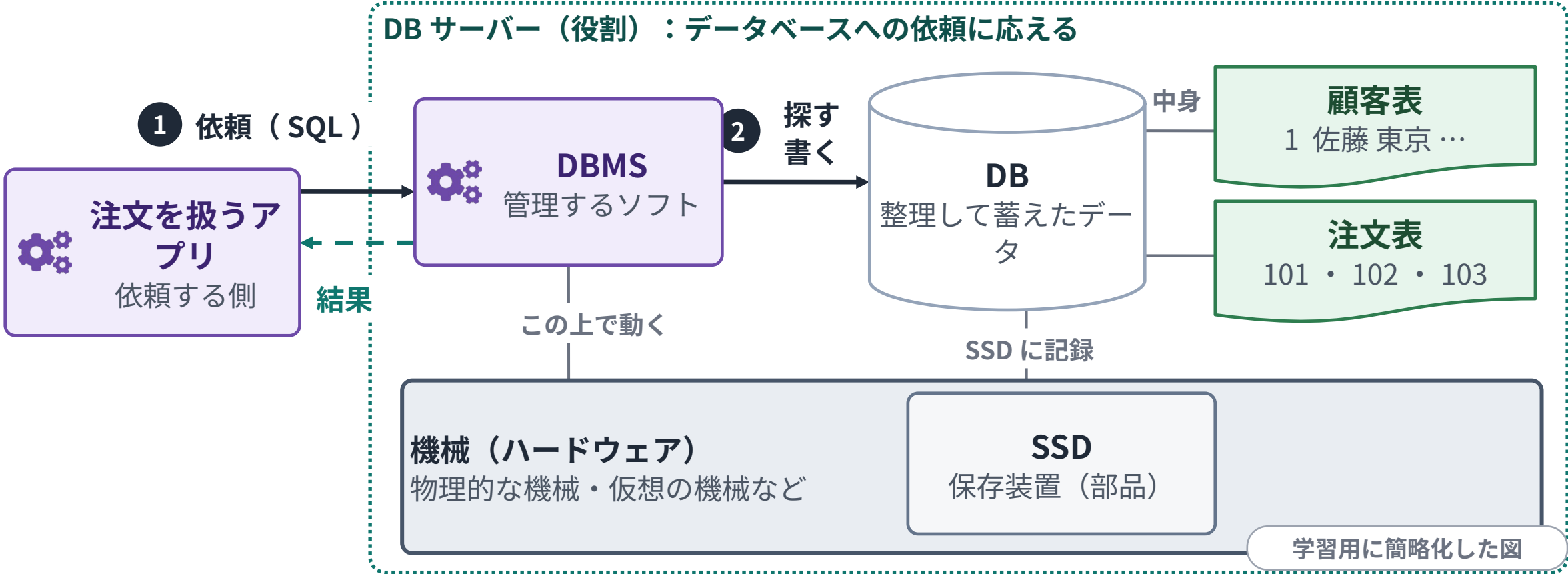


データベースの中身（表の値）は、最終的に SSD や HDD などの**ストレージ**（保存装置、p.82）に記録されます。SSD は電源を切っても記録が消えないので、停電や再起動の後も注文の記録が残ります。

DBMS は、よく使う行を**メモリ**にも写して置き、速く応えます。ただしメモリの中身は電源が切れると消えるため、確定した記録は SSD などに書き込みます。

データ・ソフトウェア・役割・装置の関係図

役割の関係図。緑の円柱＝データベース、緑の書類形＝実際の表、紫＝ソフトウェア、灰色＝機械と部品、点線の枠＝役割。黒い矢印＝依頼、緑の破線＝返事、灰色の細線＝「中身」「この上で動く」「ここに記録」の対応です。



4 つの言葉の正体を表で比べる

言葉	正体（何の種類か）	サンプル商店での例	図の形
DB（データベース）	データ	顧客表・注文表	緑の円柱
DBMS	ソフトウェア	PostgreSQL、Oracle Database など	紫の箱
DB サーバー	役割と、それを担う実行環境（DBMS を動かす機械や OS などの土台）	DBMS が動く物理的な機械・仮想の機械・事業者のサービス	点線の枠
ストレージ（SSD など）	機械・部品（ハードウェア）	表のデータを書き込む SSD	灰色の箱

データそのもの・扱うソフトウェア・依頼に応える役割・記録する装置は、別のものです。
「DB」と言われたら、どれを指すかを確かめましょう。

たとえで整理する：図書館の本・司書・窓口・書庫

図書館では	データベースでは
本と蔵書目録（記録の中身）	DB：顧客表・注文表
司書（本を探す・貸出を記録する人）	DBMS：探す・書き込む仕事をするソフトウェア
「問い合わせに答える」係（司書が担当）	DB サーバー：依頼に答える役割
書庫の棚（本が実際に置かれる場所）	ストレージ：データを記録する SSD など

！ たとえの限界

司書は自分で考えて判断しますが、DBMS は命令文に書かれたとおりに動くプログラムです。また、DBMS と SSD が 1 台の機械の中に収まっていることも多く、図書館のように場所がはっきり分かれてはいません。

確認の問い（データベースの4つの言葉）

自分の言葉で答えてから、次のページの解説を読みましょう。

- 1 図の円柱の記号は何を表していますか。実物の形と関係がありますか。
- 2 注文表・PostgreSQL・SSD・DBサーバーは、それぞれ何の種類ですか。
- 3 「DBサーバー＝DB専用の物理的な機械」と言い切れないのはなぜですか。

確認の問い（データベースの4つの言葉）：解説

前のページの問いへの答えと、考え方です。

✓ 問1 図の円柱の記号は何を表していますか。実物の形と関係がありますか。

円柱はデータベース（データの集まり）を表す図の約束事で、実物の形ではありません。SSDなどの装置は灰色の箱で描きます。 p.144

✓ 問2 注文表・PostgreSQL・SSD・DBサーバーは、それぞれ何の種類ですか。

注文表＝データ、PostgreSQL＝ソフトウェア（DBMSの製品）、SSD＝装置（ストレージ）、DBサーバー＝役割です。 p.152

✓ 問3 「DBサーバー＝DB専用の物理的な機械」と言い切れないのはなぜですか。

DBサーバーは役割の名前で、物理的な機械・仮想の機械・事業者のサービスのどれでも担え、1台が他の仕事と兼ねることもあるからです。 p.149

「要求」「応答」「検索」の意味

DB サーバーとのやり取りを読むために、3 つの言葉を確認します。

言葉	意味	注文履歴の例
要求 (リクエスト)	「これをして」という依頼として送られるデータ (p.123)	アプリ→ DB サーバー：「顧客番号 1 の注文を探して」
応答 (レスポンス)	要求に対する結果として返されるデータ	DB サーバー→アプリ：「注文 101 と 103 の 2 行」
検索	たくさんの行の中から、条件に合う行を探して取り出すこと	注文表の 3 行から、customer_id が 1 の行を探す

要求と応答は、通信（機器やプログラムどうしがデータを送り合うこと）で運ばれる中身の名前です。検索は、DB サーバーの中で DBMS が行う仕事の名前です。

検索の例：顧客番号 1 の注文だけを取り出す

条件は「customer_id（顧客番号）が 1」。DBMS は、下の注文表（orders）の行を 1 つずつ確かめ、条件に合う行（黄色）だけを取り出します。

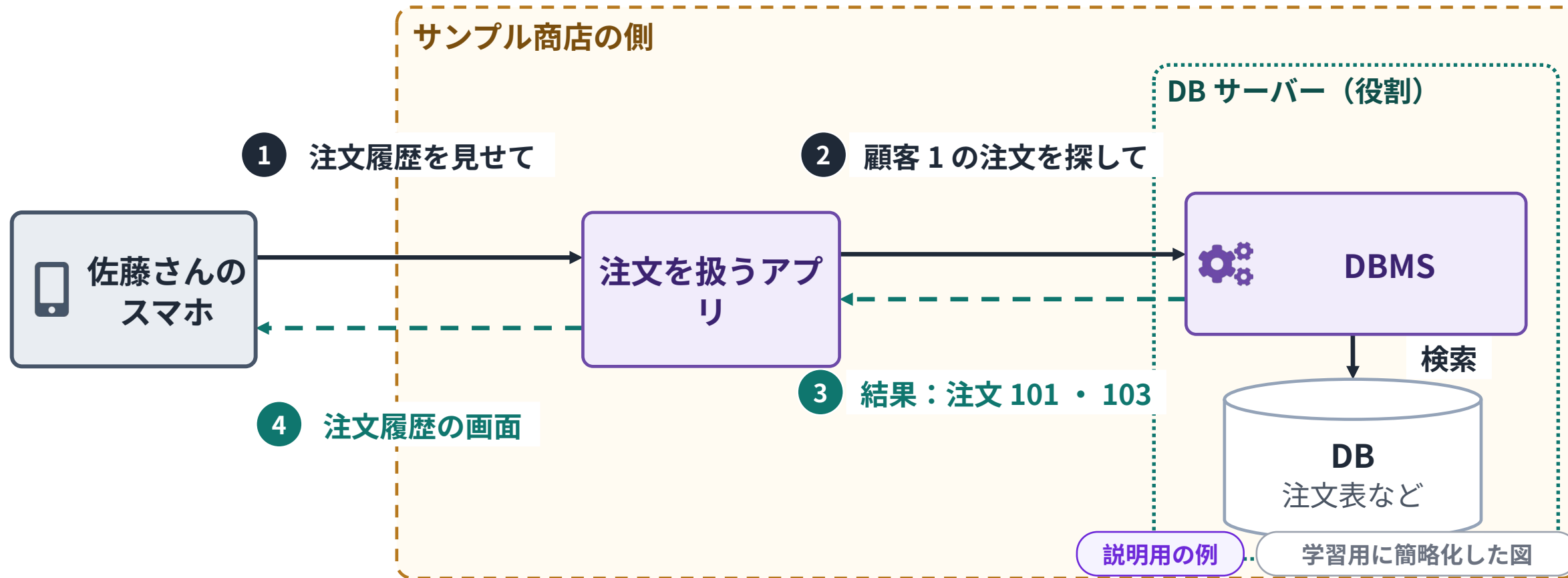
order_id	customer_id	product	quantity	amount
101	1	ノート	3	3,000
102	3	ペン	2	1,200
103	1	ファイル	5	5,500

↓ 検索の結果（条件に合う 2 行＝佐藤さんの注文履歴）

order_id	customer_id	product	quantity	amount
101	1	ノート	3	3,000
103	1	ファイル	5	5,500

注文履歴を見るときの依頼と返答（関係図）

役割の関係図。黒い矢印＝要求（依頼）、緑の破線＝応答（返事）、丸数字は順番。矢印の文字が運ばれる中身です。スマホとお店の間の通信の道すじは第6章で扱います。



前ページの矢印①～④で運ばれる中身

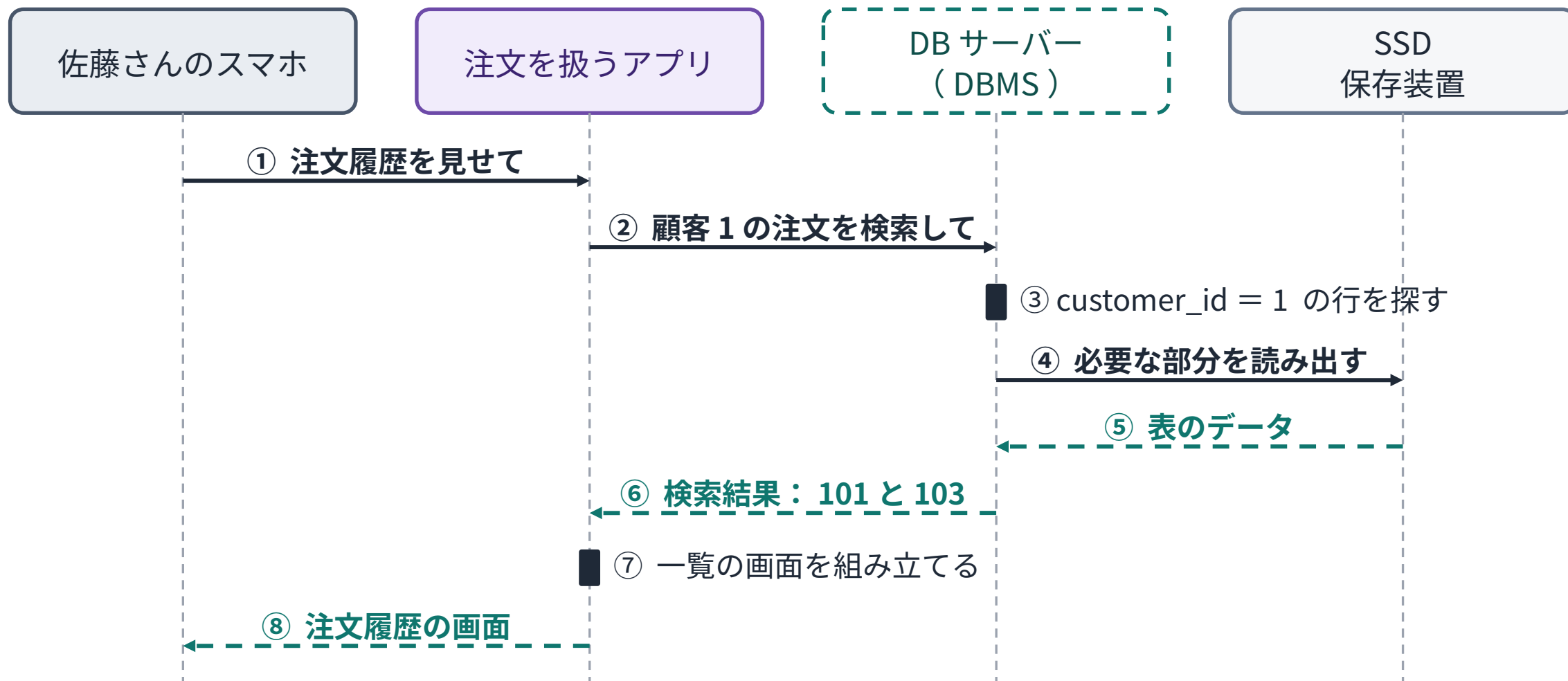
番号	誰から誰へ	種類	中身の例（説明用）
①	スマホ→アプリ	要求	佐藤さん（顧客番号 1）が「注文履歴を見る」を押した、という依頼
②	アプリ→DB サーバー	要求	「注文表から、customer_id が 1 の行を探して」（SQL という命令文で書く）
③	DB サーバー→アプリ	応答	検索結果の 2 行： 101 ノート 33,000 ／ 103 ファイル 55,500
④	アプリ→スマホ	応答	2 件の注文を見やすく並べた「注文履歴」の画面

🔑 ポイント

アプリは、スマホから見ればサーバー、DB サーバーから見ればクライアントです。DB サーバーとやり取りするのはアプリで、スマホが直接 DB サーバーに依頼するものではありません。

注文履歴を表示するまで（時間順のやり取り図）

上から下へ時間が進みます。黒の矢印＝要求、緑の破線＝応答です。



時間順の図を読む：各段階で起きていること

- ①② 佐藤さんの操作が要求としてアプリに届き、アプリはそれを「顧客 1 の注文を検索して」という DB サーバーへの要求に言い換えます。
- ③～⑤ DB サーバーの中で DBMS が検索します（縦の細い帯は、その中での処理）。よく使う行がメモリにあればそれを使い、なければ SSD から読み出します（④⑤はその場合）。
- ⑥～⑧ 検索結果の 2 行が応答として返り、アプリがそれを見やすい画面にしてスマホへ返します。

i 補足

この図は学習用に簡略化しています。実際のサイトでは、受付を担当する別のサーバーが間に入ることがあります。その分担は第 7 章で、スマホとお店の間の通信の道すじは第 6 章で扱います。

リレーショナルデータベース（RDB）

リレーショナルデータベース（りれーしょなるでーたべーす）

分類：データ

略語：RDB = Relational Database

何か	データを行と列の表で持ち、表どうしを共通の値（例：顧客番号）で結び付けて使うデータベース。
なぜ必要か	顧客の名前を顧客表に1回だけ書いて番号で指せば、書き換えが1か所で済み、食い違いが起きにくい。
身近な例	サンプル商店の顧客表と注文表は、customer_id で結び付いている。

補足

名前の「リレーション」は数学の用語で、厳密には表そのものを指します。表どうしを結び付けない方式（NoSQL、p.180）もあります。

SQL：DBMS への依頼を書くための言語

SQL (えすきゅーえる)

分類: 言語

何か	DBMS に「何を・どの表から・どの条件で」探す・追加する・変える・消すかを指示する言語（命令の書き方の決まり）。
なぜ必要か	決まった書き方があるので、どのアプリも同じ書き方で依頼でき、DBMS は迷わず実行できる。
どこにあるか	アプリの中で組み立てられ、要求として DB サーバーへ送られる。
身近な例	SELECT name FROM customers WHERE city = ' 東京 ';
似た言葉との違い	製品の名前ではない。PostgreSQL・MySQL などの DBMS 製品が SQL を受け付ける（細部は製品ごとに違う）。

SQL の基本の 4 つの命令

SQL の命令は、最初の英単語で「何をしたいか」を表します。よく使う 4 つを先に一覧で見て、次のページから 1 つずつ、小さな表・命令・日本語訳・実行結果を対応させて確かめます。

命令	意味	この後の例
SELECT	取り出す	東京の顧客の名前／佐藤さんの注文履歴を取り出す
INSERT	追加する	鈴木さんの注文 104 を注文表に追加する
UPDATE	変更する	注文 102 の金額を訂正する
DELETE	削除する	注文 103 を取り消す

i 補足

「探す（検索）」は SELECT で、「書き換える」は INSERT ・ UPDATE ・ DELETE で依頼します。命令の単語は大文字で書く習慣がありますが、小文字でも動く製品がほとんどです。

SELECT：表から必要な部分を取り出す

① 元の表：顧客表（customers）

customer_id	name	city
1	佐藤	東京
2	鈴木	大阪
3	高橋	福岡

② 命令（SQL）と日本語訳

```
SELECT name
FROM customers
WHERE city = '東京';
```

名前（name）の列を取り出す
顧客表（customers）から
都市（city）が東京の行だけ

③ 実行結果

name
佐藤

WHERE の後に、対象にする行の条件を書きます。city が「東京」の行は 1 行（黄色）だけなので、その行の name = 「佐藤」が結果として返ります。

佐藤さんの注文履歴を取り出す SELECT

命令（SQL）と日本語訳

```
SELECT order_id, product,  
       quantity, amount  
FROM orders  
WHERE customer_id = 1;
```

注文番号・商品名・
数量・金額の列を取り出す
注文表（orders）から
顧客番号が 1 の行だけ

実行結果

order_id	product	quantity	amount
101	ノート	3	3,000
103	ファイル	5	5,500

これが、佐藤さんの「注文履歴を見る」で、アプリが DB サーバーに送る要求の中身にあたります（説明用の例）。結果の 2 行が応答として返ります（p.160 の②と⑥）。

INSERT：表に新しい行を追加する

鈴木さん（顧客番号 2）の新しい注文 104 を追加する例です（説明用）。

命令（SQL）と日本語訳

```
INSERT INTO orders
VALUES (104, 2, '消しゴム', 4, 800);
```

注文表（orders）に行を追加する
値：104・顧客 2・消しゴム・4 個・800 円

実行結果：注文表の最後に 1 行増える（黄色が追加された行）

order_id	customer_id	product	quantity	amount
101	1	ノート	3	3,000
102	3	ペン	2	1,200
103	1	ファイル	5	5,500
104	2	消しゴム	4	800

UPDATE ： 行の値を変更する

注文 102 （高橋さんのペン） の金額を 1,500 円に訂正する例です（説明用）。

命令（SQL）と日本語訳

```
UPDATE orders
SET amount = 1500
WHERE order_id = 102;
```

注文表（orders）を変更する
金額（amount）を 1500 にする
注文番号が 102 の行だけ

実行結果： 102 の行の金額だけが変わる（黄色）

order_id	customer_id	product	quantity	amount
101	1	ノート	3	3,000
102	3	ペン	2	1,500 （元は 1,200 ）
103	1	ファイル	5	5,500

DELETE：行を削除する

注文 103（佐藤さんのファイル）の取り消しで、その行を削除します（説明用）。

命令（SQL）と日本語訳

```
DELETE FROM orders
WHERE order_id = 103;
```

注文表（orders）から行を消す
注文番号が 103 の行だけ

実行結果：103 の行がなくなり、2 行が残る

order_id	customer_id	product	quantity	amount
101	1	ノート	3	3,000
102	3	ペン	2	1,200

これらの例は、どれも最初の表（101 ～ 103）に 1 つずつ実行した結果です（続けて実行したものではありません）。

注意：WHERE を書き忘れると全部の行が変わる

条件のない DELETE

```
DELETE FROM orders;
```

注文表の行をすべて消す（条件なし）→ 1 行も残らない

条件のない UPDATE

```
UPDATE orders  
SET amount = 1500;
```

注文表を変更する
すべての行の金額が 1,500 になる

⚠ 注意

WHERE（条件）を書き忘れると、表のすべての行が変わります。実務では、①先に同じ条件の SELECT で対象の行を確かめる、②トランザクション（p.174）の中で実行し、誤りなら取り消す、③バックアップを取っておく、といった備えをします。

主キー：表の中で1行を決める値

名前は同じ人がいるかもしれません。仮に大阪の別の佐藤さんが登録すると（赤い行）、「name が佐藤」では2行が当てはまります。

customer_id	name	city
1	佐藤	東京
2	鈴木	大阪
3	高橋	福岡
4	佐藤	大阪（仮に追加した場合）

ポイント

重ならないように割り当てた customer_id を使えば、1 の行は必ず佐藤（東京）の1行だけです。このように表の中で1行を特定する値（列）を**主キー**と呼びます。重複せず、空にもできません。注文表では order_id が主キーです。

外部キー：別の表の行を指す値

注文表の customer_id は、その注文をした人を、顧客表の主キー（customer_id）で指しています。このように、別の表の主キーを指す列を**外部キー**と呼びます。

説明用の例。矢印は「この値が、あの行を指している」という対応を表します（情報の移動ではありません）。注文表は一部の列だけを表示しています。

注文表（orders）

order_id	product	customer_id
101	ノート	1
102	ペン	3
103	ファイル	1

顧客表（customers）

customer_id	name	city
1	佐藤	東京
2	鈴木	大阪
3	高橋	福岡

1 → 佐藤さんの行

1 → 佐藤さんの行

3 → 高橋さん

注文 101 と 103 の customer_id = 1 は、どちらも顧客表の「1：佐藤（東京）」の行を指します。だから佐藤さんの注文履歴は 101 と 103 の 2 件です。

外部キーがあると、存在しない顧客への注文を防げる

顧客表には customer_id が 1・2・3 の3人しかいません。そこへ、顧客番号9の注文を追加しようとする、どうなるでしょうか（説明用の例）。

命令（SQL）と日本語訳

```
INSERT INTO orders  
VALUES (105, 9, 'ペン', 1, 600);
```

注文表に行を追加しようとする
顧客番号9（顧客表に存在しない）

⚠ 実行結果

DBMS の返事：エラー。顧客表に customer_id = 9 の行がないため、追加を拒否します（実際のエラーの文言は製品によって違います）。

この行が入ると「誰の注文かわからない注文」ができてしまいます。外部キーの決まりをDBMSに登録しておく、と、矛盾した行をDBMSが自動で拒否します。データどうしに矛盾がない状態を「整合性が保たれている」と言います。

トランザクション：複数の操作をひとまとまりに

振込では、①佐藤さんの口座から1万円を引き落とし、②相手の口座に1万円を入金します（説明用の例）。①の直後に故障して②が行われないと、1万円が消えてしまいます。

成功した場合



途中で失敗した場合



この「全部成功か、全部取り消すか」の仕組みを**トランザクション**と呼びます。

トランザクションが守る 4 つの性質（ACID 特性）

トランザクションが満たすべき性質は、4 つの英語の頭文字をとって **ACID 特性** と呼ばれます。振込の例で 1 行ずつ見ます。

性質	意味	振込の例
原子性 （ Atomicity ）	全部行うか、全く行わないかのどちらか	引き落としだけ行われて入金されない、ということは起きない
一貫性 （ Consistency ）	決まり（ルール）に反する状態にならない	振込の前後で、2 つの口座の合計金額が変わらない
独立性 （ Isolation ）	同時に動く他のトランザクションの、途中の状態に影響されない	同時に別の振込があっても、途中の金額を読んで計算を誤らない
耐久性 （ Durability ）	確定した結果は、その後に故障しても失われない	「振込完了」の後に停電しても、記録は SSD などに残っている

トランザクションは保存装置ではなく DBMS の機能

トランザクションは、SSD などの保存装置が自動でする機能ではありません。どの操作をひとまとまりにするかをアプリが指示し、DBMS が確定や取り消しを行います。

サンプル商店での例（説明用。書き方は製品によって少し違う）

```
BEGIN;  
INSERT INTO orders VALUES (104, ...);  
UPDATE stock SET ... ;  
COMMIT;
```

ここからひとまとまり

注文 104 を注文表に追加

在庫表で消しゴムを 4 個減らす

まとめて確定（失敗したら ROLLBACK で全部取り消す）

具体例

鈴木さんの注文 104 を受けるとき、注文表への追加と在庫の減算をひとまとまりにすれば、「注文は入ったのに在庫が減っていない」食い違いが起きません。WHERE を書き忘れた命令も、確定（COMMIT）前なら取り消せます。

結合（JOIN）：2つの表をつないで取り出す

注文表には顧客の名前がなく、顧客番号だけがあります。「誰がいくら買ったか」を名前付きで見たいときは、2つの表の customer_id が同じ行どうしをつなぎ合わせます。これを**結合（JOIN）**と呼びます。

注文表（一部の列）

order_id	customer_id	amount
101	1	3,000
102	3	1,200
103	1	5,500

顧客表（一部の列）

customer_id	name
1	佐藤
2	鈴木
3	高橋

結合した結果

name	amount
佐藤	3,000
高橋	1,200
佐藤	5,500

色が同じ行どうしが、customer_id でつながっています。外部キー（p.172）が指す行を引いてくる、と考えると分かりやすいでしょう。

結合（JOIN）のSQLと実行結果

命令（SQL）と日本語訳

```
SELECT c.name, o.amount  
FROM orders o  
JOIN customers c  
  
ON o.customer_id = c.customer_id;
```

顧客の名前と注文の金額を取り出す
注文表（orders）を、短く o と呼ぶ
顧客表（customers）を c と呼んでつ
なぐ
つなぐ条件：両方の customer_id が同
じ

実行結果

name	amount
佐藤	3,000
高橋	1,200
佐藤	5,500

「c.name」は「c（顧客表）の name 列」という意味です。佐藤さんの分だけにしたいときは、最後の「;」の前に「WHERE c.customer_id = 1」を加えます。

表計算ソフト（Excel など）とデータベースの違い

どちらも表ですが得意なことが違います。整合性＝データに矛盾がないこと。

比べる点	表計算ソフトのファイル	データベース（+ DBMS）
主な使い方	少人数での集計・グラフ	多人数・多数のアプリで同時に使う
同時の書き換え	上書きや食い違いが起きうる	DBMS が順番を整える
整合性	表どうしの矛盾は防ぎにくい	主キー・外部キーで守る
見てよい人の管理	ファイル単位が中心	表・操作ごとに権限を設定
途中で失敗したとき	中途半端な状態が残りうる	トランザクションで取り消せる

i 補足

集計・グラフで情報を整理する「データ分析」では表計算ソフトも便利で、DB から取り出した結果を表計算ソフトで分析することもよくあります。

DB 製品は「方式」と「提供のしかた」で整理する

方式（データをどんな形で持つか）

- RDB：表で持ち、表どうしを結び付ける（この章で学んだ方式）
- **NoSQL**：RDB 以外の方式のまとめた呼び名。項目名と値を書き並べた「文書」で保存する文書型、キーと値の組で保存する型など

提供のしかた（どう手に入れて使うか）

- 商用ソフト：会社から利用の権利（ライセンス）を買って使う
- **オープンソースソフトウェア**（OSS）：元の文章（ソースコード）が公開され、条件の下で誰でも使える
- **マネージドサービス**：運用の一部を事業者が担い、通信回線越しに使う

⚠ 注意

NoSQL は「SQL が一切ない」とは限りません。OSS は無償のものが多くても、運用・保守の手間や費用は別にかかります。

主な DB 製品の整理（ 1/2 ）： RDB

製品	方式	提供のしかた	特徴・注意
Oracle Database	RDB	商用ソフト・クラウド	大企業の基幹系（業務の中心のシステム）が多い。ライセンス条件の確認が必要
PostgreSQL	RDB	オープンソース	無償で使えるが、運用・保守の手間や費用は別
MySQL	RDB	オープンソース・商用版	Web サービスで広く使われる
Microsoft SQL Server	RDB	商用ソフト・クラウド	Windows ・ Microsoft 製品との連携
IBM Db2	RDB	商用ソフト・クラウド	金融などの大規模システムで使われる

主な DB 製品の整理（2/2）：RDB 以外とサービス

製品・サービス	方式	提供のしかた	特徴・注意
MongoDB	文書型 （ NoSQL の 一種）	オープンソース・ クラウド	JSON（項目名と値を並べる書き方）に似た文書で保存。SQL が一切ないとは限らない
AWS（Amazon のクラウドサービス）の DB サービス	サービスの集合	マネージドサービス	RDS ・ Aurora（RDB）、DynamoDB（キーと値）など方式が異なる

🔑 ポイント

製品名を見たら、「方式は RDB か、それ以外か」と「自分で入れて運用するのか、サービスとして使うのか」の 2 つを確かめると整理できます。

表の「クラウド」は、事業者の設備を通信回線越しに必要な分だけ借りて使う形のことです（第 10 章）。

マネージドでも、データと権限の責任は利用者に残る

事業者が受け持つことが多い	利用者に残ること
機械・保存装置の用意と交換	どんなデータを入れるか（個人情報を入れてよい か）
DBMS のインストールと更新	誰にどの表の読み書きを許すか（権限）
バックアップの仕組みの提供	どの頻度で取り、どこまで戻せばよいかの設定と、 戻せるかの確認
建物・電源・冷却	どこから接続できるかななどの設定、パスワードの管 理

⚠ 注意

分担はサービスや契約によって違います。「事業者のサービスだから全部任せて安心」とは考
えず、契約と設定を確認します（第 10 章 p.449）。

第5章のまとめ

✓ ここまでで説明できること

データ：注文番号・商品名などの記録。表の1行が1件、1列が1項目。

DB：整理して蓄えたデータの集まり。図では円柱の記号。

DBMS：検索・追加・変更・削除、同時利用、権限、保護を行うソフト。

DBサーバー：DBへの依頼に応える役割（機械・仮想の機械・サービス）。

ストレージ：データを実際に記録するSSDなどの装置。

SQL・主キー・外部キー・結合・トランザクション：依頼の書き方と、食い違いを防ぐ仕組み。

→ 次へのつながり

第6章で、要求と応答がスマホからお店までどの道を通るかを学び、第7章でDBサーバーを全体の中に位置づけます。

確認の問い（第 5 章 その 1）

自分の言葉で答えてから、次のページの解説を読みましょう。

- ① 注文履歴を表示するとき、アプリと DB サーバーの間の要求と応答は？
- ② `SELECT order_id, amount FROM orders WHERE customer_id = 3;` の訳と結果は？

確認の問い（第5章 その1）：解説

前のページの問いへの答えと、考え方です。

✓ 問1 注文履歴を表示するとき、アプリとDBサーバーの間の要求と応答は？

要求は「注文表から customer_id が 1 の行を探して」という SQL の命令、応答は検索結果の 2 行（101 ノート 3 3,000 / 103 ファイル 5 5,500）です。 p.159

✓ 問2 SELECT order_id, amount FROM orders WHERE customer_id = 3; の…

「注文表から、顧客番号が 3 の行の注文番号と金額を取り出す」。結果は 1 行： order_id 102、amount 1,200（高橋さんのペン）です。 p.165

確認の問い（第 5 章 その 2）

自分の言葉で答えてから、次のページの解説を読みましょう。

- 1 注文表の `customer_id`（外部キー）は何を指し、何を防げますか。
- 2 トランザクションとは何で、どのソフトウェアの機能ですか。

確認の問い（第5章 その2）：解説

前のページの問いへの答えと、考え方です。

✓ 問1 注文表の `customer_id`（外部キー）は何を指し、何を防げますか。

顧客表の主キー `customer_id` を指します（101・103の1は佐藤さんの行）。存在しない顧客（例：顧客番号9）への注文をDBMSが拒否して防げます。 p.172

✓ 問2 トランザクションとは何で、どのソフトウェアの機能ですか。

振込の引き落としと入金のような複数の操作をひとまとまりにし、全部成功なら確定、途中で失敗なら全部取り消す仕組みです。DBMSの機能です。 p.174

第 6 章

ネットワークの基本

この章で学ぶこと

佐藤さんがスマホで「注文履歴を見る」を押したとき、その依頼が家の機器、回線、インターネットを通してサンプル商店へどう届くかを学びます。図の読み方から始め、家庭の機器、宛先の番号、補助の通信、暗号化へと進みます。

→ 前の章とのつながり

第 4 章で要求と応答、第 5 章でデータベースから注文を探す仕組みを学びました。この章では、その要求と応答のデータが、離れた機器の間をどう運ばれるかを見ます。

通信とは：離れた機器へデータを運ぶこと

第4章の「要求」と「応答」は、どちらもデータです。

佐藤さんが「注文履歴を見る」を押すと、スマホは「注文履歴を見せて」という**要求**のデータを作ります。ところがサンプル商店のコンピューターは、遠く離れた建物の中にあります。このデータを相手まで運び、相手が作った**応答**（注文101と103の一覧）を持ち帰ることが**通信**です。

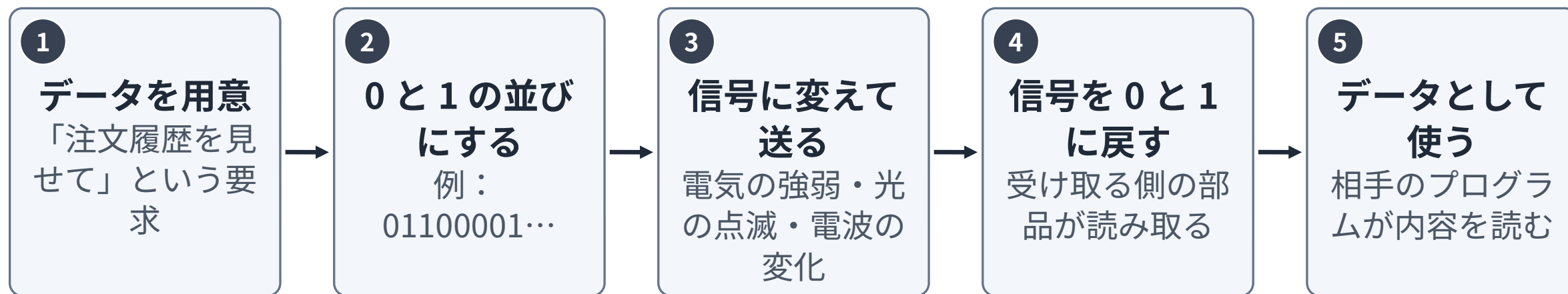
比べる点	手紙	機器どうしの通信
運ぶもの	紙に書いた文章	0と1で表したデータ
運び方	配達員が手で運ぶ	電気・光・電波の信号にして運ぶ
宛先	住所と名前	機器を区別する番号
決まりごと	宛名の書き方・切手	通信の約束（この章で説明）

🔑 ポイント

考えることは「何を・何に乗せて・どこへ・どんな決まりで」運ぶかの4つです。

2 台の機器が情報を送り合うとは

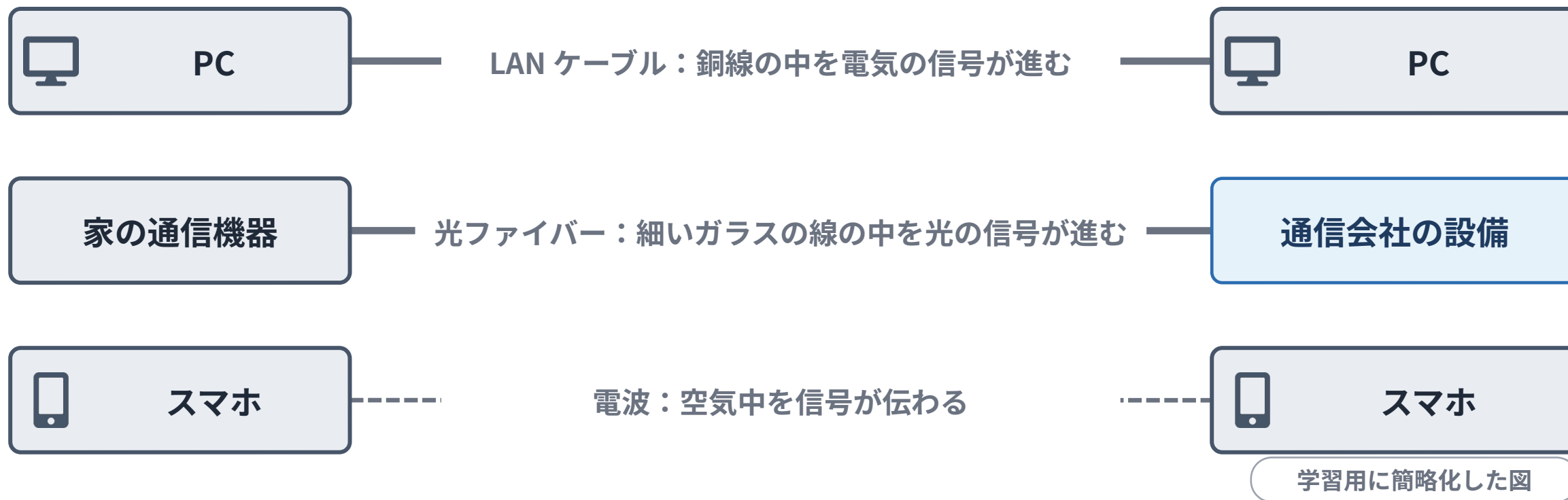
文字も画像も、コンピュータの中では0と1（第1章のビット）です。



0と1は、そのままでは線の中を進めません。送る側の機器は、通信用の部品で0と1を「電圧が高い・低い」「光が点く・消える」のような信号に変えて送り出します。受け取る側は信号を読み取って0と1に戻し、元のデータを組み立て直します。信号の通り道には、ケーブルと電波があります（次のページ）。

信号の通り道：ケーブルと電波

物理的な配置図。灰色の太い実線＝ケーブルでの接続、灰色の破線＝電波での接続（電波そのものは目に見えません）。線の長さは実際の距離ではありません。



どの道でも運ぶ中身は同じ 0 と 1 のデータで、違うのは信号の種類と線や機器です。

有線と無線の特徴

通り道	運ぶもの	主な使われ方	特徴
LAN ケーブル (有線)	銅線を流れる電気の信号	家やオフィスの PC ・ プリンター	安定しやすい。ケーブルが 届く範囲だけ
光ファイバー (有 線)	細いガラスなどの線 を通る光の信号	家と通信会社を結ぶ回 線、建物どうし	長い距離でも弱まりにくく、 大量のデータを運べる
電波 (無線)	空気中を伝わる電波	Wi-Fi ・携帯電話	ケーブルがいらず持ち歩け る。壁・距離・混雑で品質 が変わる

ポイント

有線か無線かは「道」の違いです。佐藤さんの依頼も、家の中では電波、家から外へは光ファイバー、というように途中で道を乗り継いで運ばれます。

NIC：機器の中で通信を受け持つ部品

NIC（ニック）

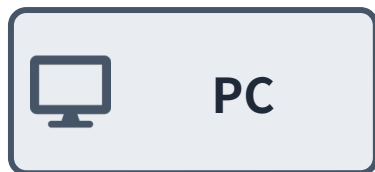
分類：部品

略語：NIC = Network Interface Card 日本語：ネットワークインターフェースカード

何か	機器をネットワークにつなぐ通信用の部品。ネットワークとの インターフェース （つなぎ口）にあたる。
なぜ必要か	CPU やメモリは信号を直接ケーブルや電波に出せない。NIC がないと他の機器とデータをやり取りできない。
どこにあるか	PC ではマザーボード上の部品や差し込むカード（LAN 端子につながる）。スマホでは無線用の部品として内蔵。板の形とは限らず、有線用と無線用など 1 台に複数あることも多い。
受け取る→する→返す	OS から受け取ったデータを信号にして送り出し、届いた信号は 0 と 1 に戻して OS へ渡す。

ネットワーク図の読み方：記号の約束

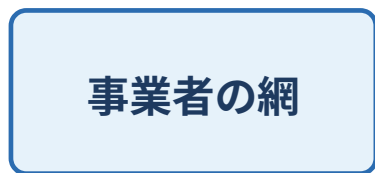
この章の図は、すべて下の約束で描きます。図の上下の位置や線の長さは、実際の距離や大きさを表しません。



灰色の太枠の箱＝機械（機器）。
中の文字が名前



点線の箱＝役割・機能（機械1台とは限らない）



青い箱＝通信会社などの網やサービス



茶色の破線の大枠＝場所・範囲
（家・建物）



灰色の太い実線＝ケーブルでの接続



灰色の破線＝電波での接続



黒の矢印＝情報の移動（依頼・本文）。丸数字＝順番



緑の破線の矢印＝返事（応答）の移動

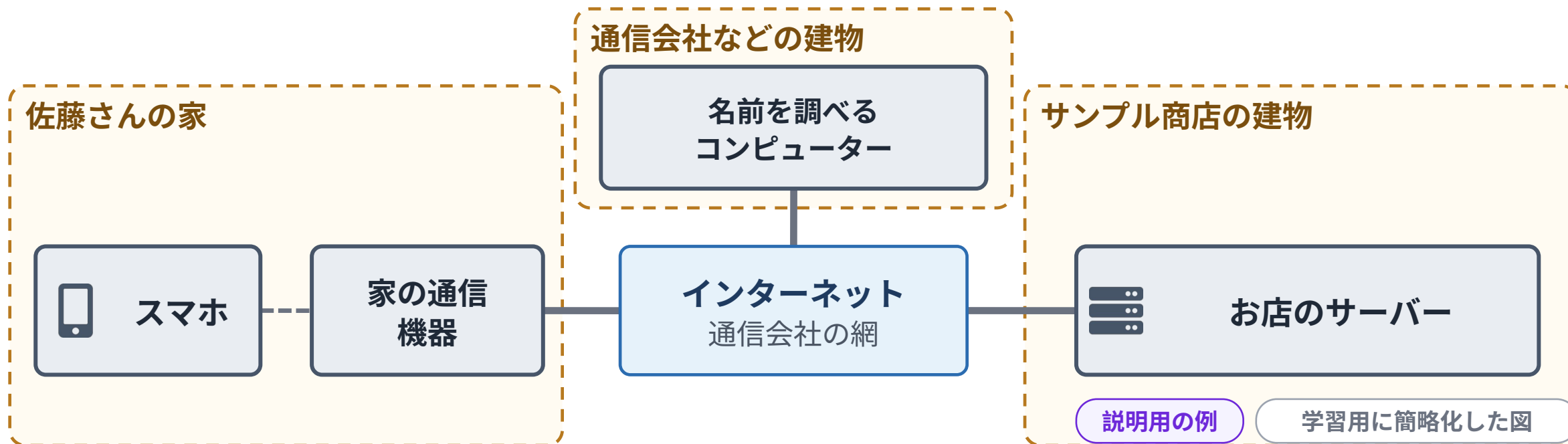


紫の破線の矢印＝補助の問い合わせと答え（本文とは別）

同じ場面を 3 種類の図で①：物理的な配置図

場面：佐藤さんがスマホで注文履歴を見る。まず置き場所とつながりを描きます。

破線の大枠＝場所、灰色の箱＝機械、太い実線＝ケーブル、灰色の破線＝電波。矢印はありません。
配置図は置き場所とつながりだけを表し、何を頼んだかは表しません。

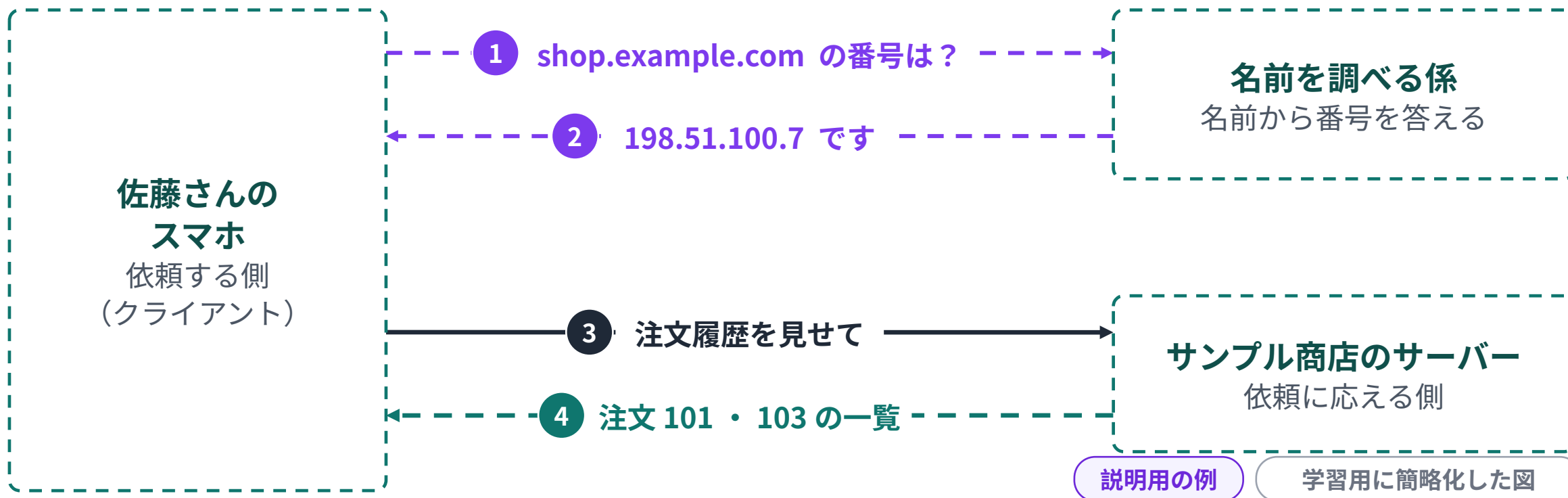


「名前を調べるコンピューター」は、shop.example.com というお店のサイトの名前から、お店のサーバーの番号を調べて教える係です（p.247 で詳しく見ます）。

同じ場面を 3 種類の図で②：役割の関係図

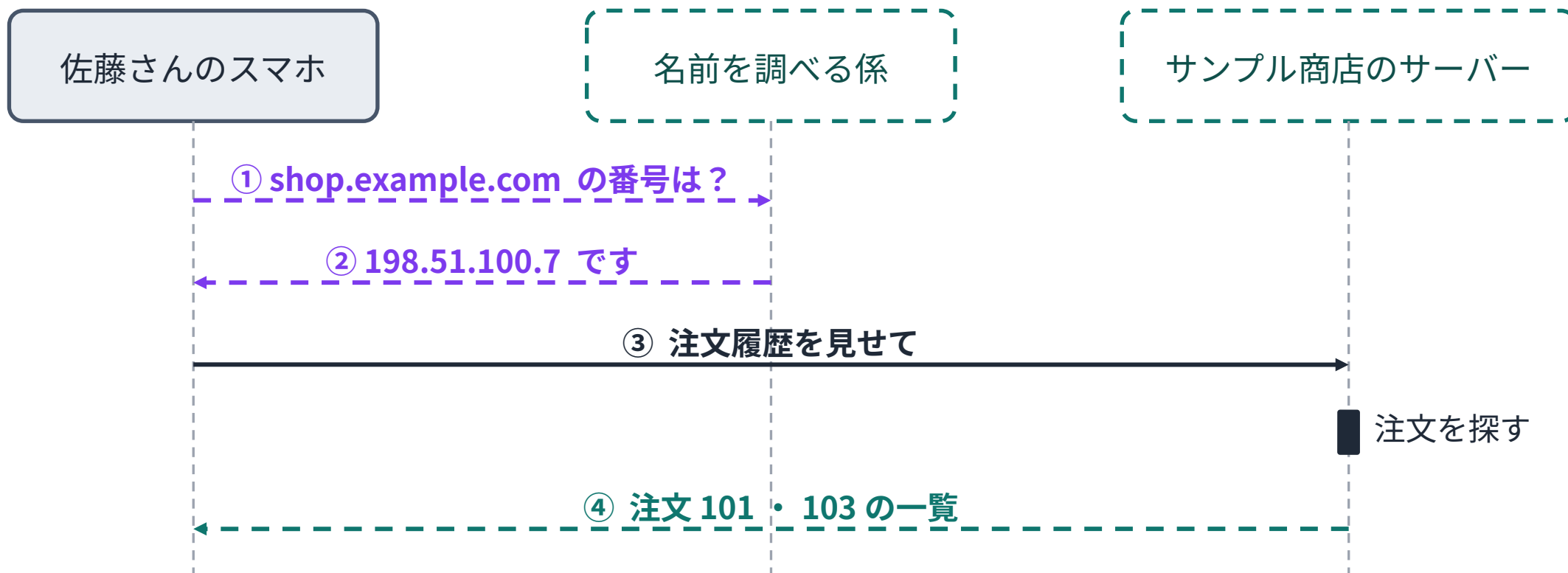
次に、同じ場面を「だれが、だれに、何を頼み、何が返るか」で描きます。

点線の箱＝役割。黒の矢印＝依頼（要求）、緑の破線の矢印＝返事（応答）、紫の破線の矢印＝本文の前に行う補助の問い合わせ。丸数字は順番です。途中の機器やケーブルは省いています。矢印は「直接つながった線」ではなく「情報の行き先」を表します。



同じ場面を 3 種類の図で③：時間順のやり取り図

最後に、同じやり取りを時間の順に並べます。上から下へ時間が進みます。



縦の点線はその機器の時間の流れで、上の矢印ほど先に起きます。③の依頼の前に、①②で相手の番号を調べ終わっていることが分かります。

3種類の図は、答える問いが違う

図の種類	答える問い	主な記号	この図からは分らないこと
物理的な配置図	どこに何が置かれ、何でつながっているか	場所の枠・機械・ケーブル・電波	何をどの順番で頼んだか
役割の関係図	だれがだれに何を頼み、何が返るか	役割の箱・矢印・番号	途中の機器や置き場所
時間順のやり取り図	どの順番で、どんな中身が行き来するか	縦の点線（時間）・横の矢印	置き場所やケーブルの種類

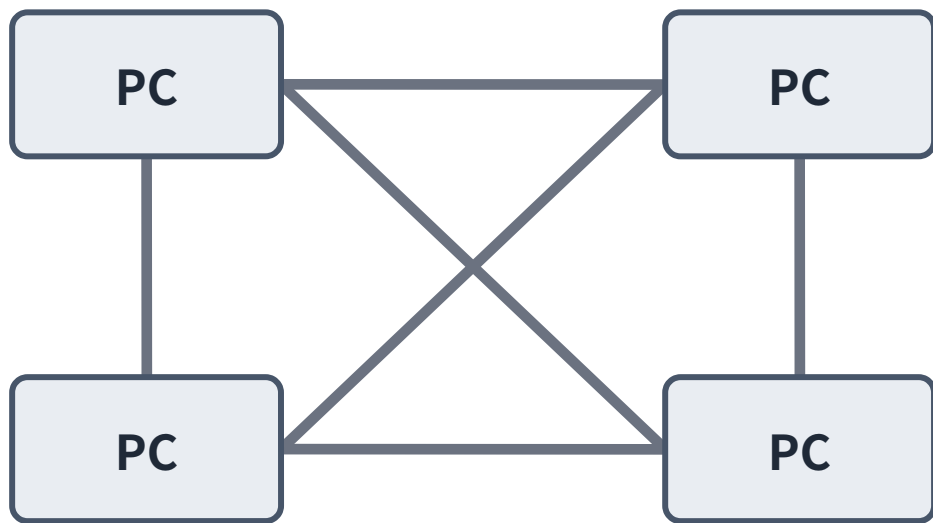
 ポイント

図を見たら、まず図のすぐ上の説明で「どの種類の図か」「線と矢印が何を表すか」を確かめます。同じ場面でも、知りたいことによって描き方が変わります。

ネットワーク：3 台以上をつなぐと中継役が要る

ネットワークは、複数の機器をケーブルや電波でつなぎ、その間でデータを運べるようにした仕組み全体のことです。2 台なら 1 本でつなげますが、台数が増えると、全部を直接つなぐのは現実的ではありません。そこで、データを受け取って次へ渡す「中継する機器」を置きます。

物理的な配置図（学習用の模式図）。灰色の箱＝機械、太い実線＝ケーブル。



4 台を直接つなぐと 6 本（10 台なら 45 本）



中継する機器につなげば、各 1 本で済む

プロトコル：通信の約束ごと

プロトコル（ぷろところ）

分類：通信の約束

日本語：通信規約（通信の約束）

何か	機器どうしが通信するための約束ごと。信号の形、データの区切り方、宛先の書き方、順番、失敗時の扱いなどを決める。NIC や OS、アプリの中のプログラムが従う。
なぜ必要か	作った会社が違う機器どうしても、同じ約束に従えば話が通じる。約束がないと受け取った 0 と 1 を読めない。
身近な例	Wi-Fi の約束、宛先の番号の約束（IP）、確実に届ける約束（TCP）、Web の約束（HTTP）
似た言葉との違い	規格 （第 2 章）は部品の形や性能も含む広い取り決め。プロトコルは通信の手順の取り決め。

約束は1つではなく、役割ごとに重なっている

💡 たとえ

手紙は、封筒の決まった位置に住所と名前を書く約束があるから、中身を読まずに配達できます。通信でも、データの先頭に宛先を決まった形で書くので、途中の機器は中身を見ずに運べます。

❗ たとえの限界

通信の約束は役割ごとに何段も重なり、1回の通信で複数のプロトコルが同時に使われます。

1回の通信で使われる約束の例（上ほどアプリに近い）

Web の約束

ページを頼む・返す決まり
(HTTP)

届け方の約束

順番どおり確実に届ける (TCP など)

宛先の約束

最終的な相手の番号を書く
(IP)

線や電波の約束

信号の形・すぐ隣の機器への渡し方

それぞれの中身は、この章の後のページで1つずつ見ます。

パケット：小分けにして付け札を付けたデータ

パケットは、ネットワークで運ぶデータの小さなまとまりです。大きなデータは小分けにされ、それぞれに宛先・送り元・順番の番号などの情報（付け札）を付けて送られます。受け取った側は番号を見て並べ直し、元のデータに戻します。

データの形を表す模式図（機器の配置ではありません）。緑の書類形＝データ、白い箱＝付け札。矢印は処理の流れです。



なぜデータを小分けにして運ぶのか

- ❶ **1本の線を多くの機器で交代に使える**：大きなデータが線を独り占めせず、他の機器の packets も合間に流れます。
- ❷ **壊れた部分だけ送り直せる**：失われた packets だけを再び送ればよく、全部をやり直さずに済みます。
- ❸ **途中の機器は付け札だけ見て渡せる**：中継する機器は中身を読まず、宛先を見て次へ渡します。

💡 たとえ

引っ越しの荷物を段ボールに分け、宛先と箱の番号を書くのに似ています。

❗ たとえの限界

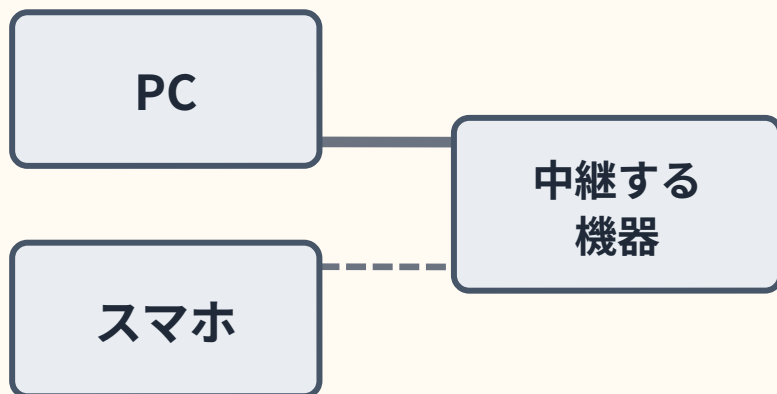
パケットは別々の道を通り、届く順番が入れ替わることもあります。並べ直しや送り直しは届け方の約束（p.258）が受け持ちます。

LAN と WAN ：近い範囲の網と、離れた拠点をつなぐ網

LAN（ラン、Local Area Network）は、家や1つの建物の中など「その場所（ローカル）」の範囲の機器をつなぐネットワークです。ケーブルでも電波（Wi-Fi）でもかまいません。**WAN**（ワン、Wide Area Network）は、本社と工場のように離れた拠点どうしをつなぐ広い範囲のネットワークです。

物理的な配置図。破線の大枠＝場所、灰色の箱＝機械、太い実線＝ケーブル、灰色の破線＝電波。

LAN：家・1つの建物



WAN：離れた拠点どうし



学習用に簡略化した図

インターネットと Web は別のもの

インターネットは、通信会社・企業・大学などの多数のネットワークを、中継する機器が互いにつないだ全体です。Web（ウェブ）は、ブラウザでページを見るための仕組みで、インターネットの上で動くサービスの 1 つです。

用語	ひとことで	混同しやすい点
LAN	近い範囲の機器をつなぐ網	有線でも Wi-Fi でもよい
WAN	離れた拠点をつなぐ広域の網	インターネット経由の VPN（暗号で守った専用の通り道）で作ることもある
インターネット	世界中の網を相互につないだ仕組み	特定の会社の網 1 つではない
Web	ブラウザでページを見る仕組み（HTTP などの約束を使う）	インターネット上のサービスの 1 つ。メールやゲームの通信は別

確認の問い（図の読み方と基本の言葉）

自分の言葉で答えてから、次のページの解説を読みましょう。

- 1 物理的な配置図と役割の関係図は、それぞれ何を表しますか。
- 2 灰色の破線と、紫の破線の矢印は何が違いますか。
- 3 インターネットと Web の違いは何ですか。

確認の問い（図の読み方と基本の言葉）：解説

前のページの問いへの答えと、考え方です。

✓ 問1 物理的な配置図と役割の関係図は、それぞれ何を表しますか。

配置図は、どこに何が置かれ、ケーブルや電波で何とつながっているか。関係図は、だれがだれに何を頼み何が返るかで、途中の機器は省きます。 p.199

✓ 問2 灰色の破線と、紫の破線の矢印は何が違いますか。

灰色の破線は電波での接続（つながり）、紫の破線の矢印は名前の問い合わせなど補助の通信での情報の移動です。 p.195

✓ 問3 インターネットと Web の違いは何ですか。

インターネットは多数の網を相互につないだ通り道、Web はその上で動く「ブラウザでページを見る」サービスの1つです。 p.206

家庭の配置図①：まず家の中の端末だけ

ここから、佐藤さんの家のネットワークを、物理的な配置図で1つずつ組み立てます。

物理的な配置図。破線の大枠＝家の範囲、灰色の箱＝機械。

佐藤さんの家の中



スマホ
佐藤さん



PC
家族が使う

スマホとPCは、まだどこにもつながっていません。
次のページから、つなぐ機器を1つずつ足します。

学習用に簡略化した図

利用者が直接操作するスマホやPCを「端末」と呼びます。通信は端末のアプリが始めます。ただし端末だけでは、互いにも家の外にもデータを送れません。

家庭の配置図②：アクセスポイント（AP）を足す

物理的な配置図。灰色の破線＝電波での接続。黄色の太枠＝今回足した機器。

佐藤さんの家の中



アクセスポイント
(AP)

PC は有線でつなぐ機器なので、
次のページでつなぎます。



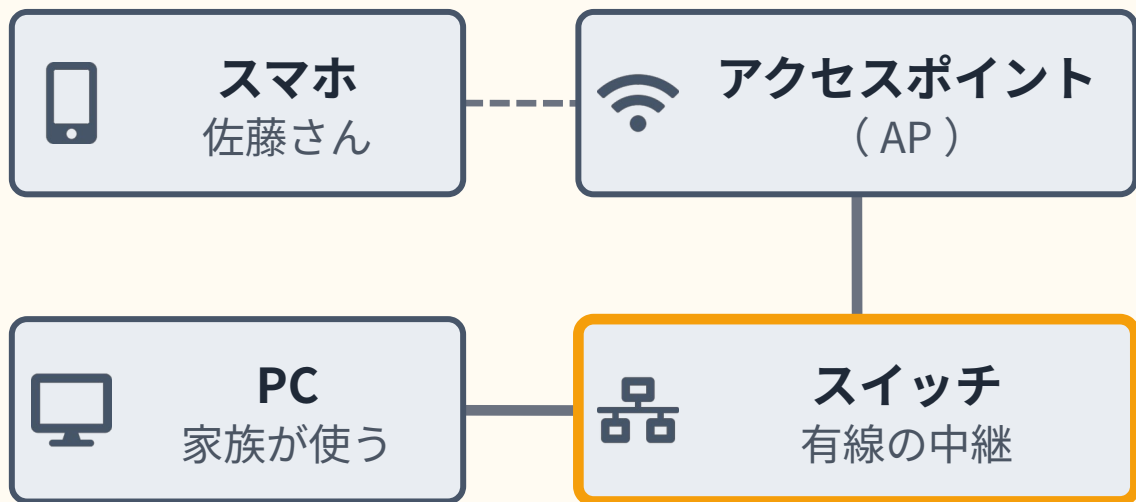
学習用に簡略化した図

アクセスポイント（AP）は、スマホなど無線の端末を電波で受け止め、家のネットワークにつなぐ機器です。スマホにはケーブルを挿す口がないため必要です。電波でつなぐLANを**無線 LAN**といい、代表的な規格の名前が**Wi-Fi**です。

家庭の配置図③：スイッチを足す

物理的な配置図。太い実線＝ケーブル、灰色の破線＝電波。黄色の太枠＝今回足した機器。

佐藤さんの家の中



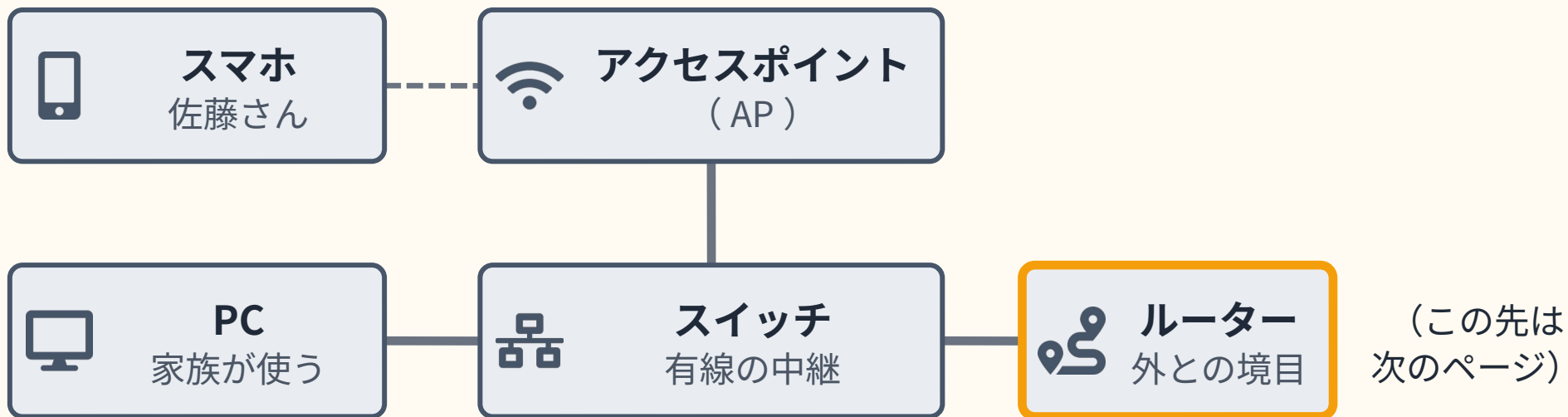
学習用に簡略化した図

スイッチは、LAN 端子（ケーブルを挿す口）を複数持ち、届いたデータを宛先の機器がつながった口へ送り出して、同じ LAN の中を中継します。これで PC・アクセスポイント・スマホの間でデータを送れます。

家庭の配置図④：ルーターを足す

物理的な配置図。太い実線＝ケーブル、灰色の破線＝電波。黄色の太枠＝今回足した機器。

佐藤さんの家の中

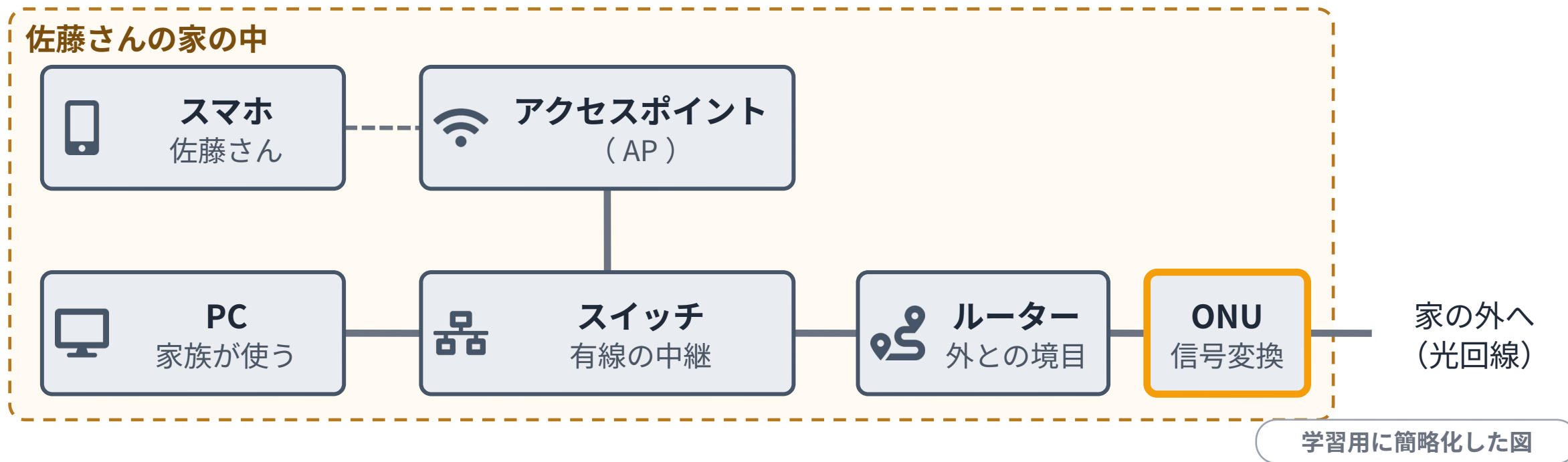


学習用に簡略化した図

ルーターは、家の LAN と外のネットワークの境目に置き、データの宛先を見て「次にどこへ渡すか」を決めて中継する機器です。スイッチが中継するのは家の中だけです。サンプル商店のサーバーのような家の外の相手へのデータは、ルーターが外へ送り出します。

家庭の配置図⑤：ONU（またはモデム）を足す

物理的な配置図。太い実線＝ケーブル、灰色の破線＝電波。黄色の太枠＝今回足した機器。



ONU（光回線終端装置）は、家の外から来た光ファイバーの光の信号と、家の中の機器が扱う電気の信号を相互に変換します。テレビ用のケーブルなどを使う回線では、**モデム**が同じように信号を変換します。機器の並び方は契約や機種で異なります。

家庭の配置図⑥：家の外の回線・ISP・インターネット

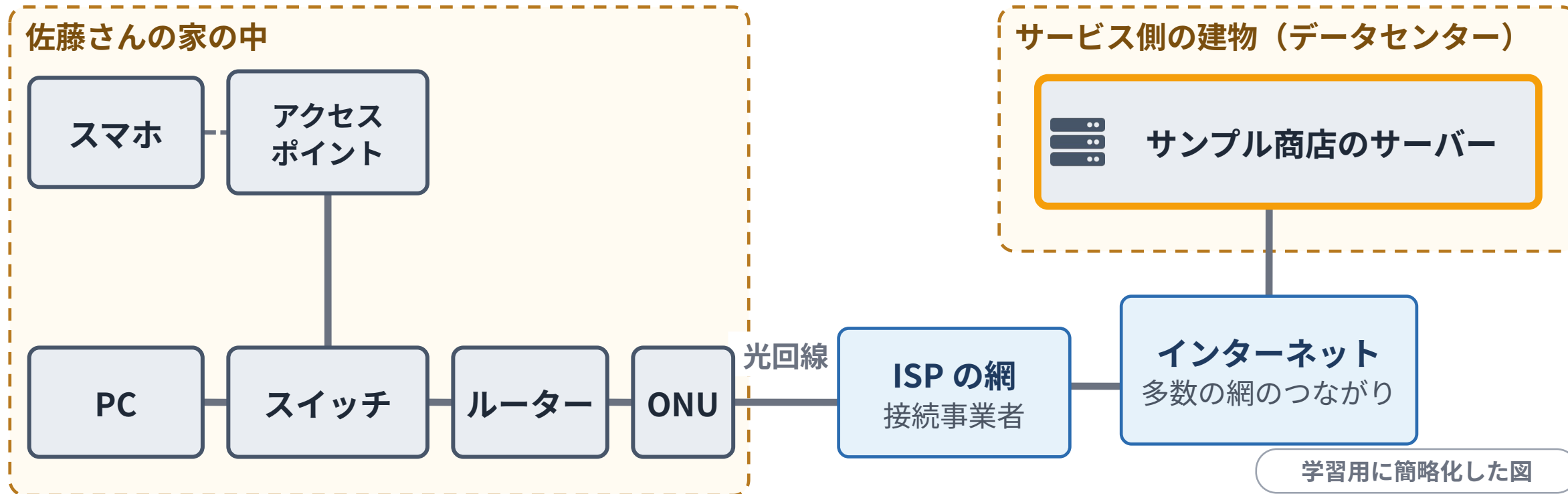
物理的な配置図。青い箱＝事業者の網、太い実線＝ケーブル、灰色の破線＝電波。



家から出た光回線は **ISP**（インターネット接続事業者、プロバイダー）の網につながり、そこから**インターネット**へ出ます。回線の会社とISPが別の契約のこともあります。

家庭の配置図⑦：サービス側の建物まで

破線の大枠＝建物、青い箱＝事業者の網、太い実線＝ケーブル、灰色の破線＝電波。



お店のサーバーはデータセンター（第4章）のような建物にあり、そこもインターネットにつながっています。依頼はスマホから電波とケーブルを乗り継いで届きます。

家庭の通信機器の役割と置き場所

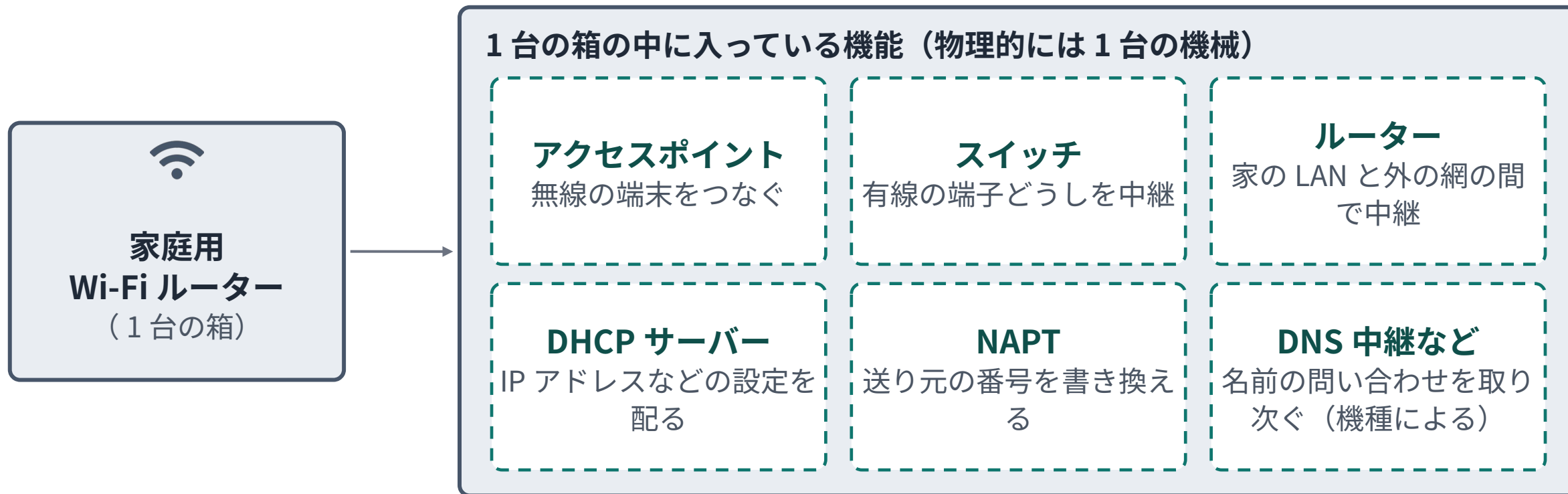
機器	役割	置き場所
端末（スマホ・PC）	利用者が操作する機器。通信を始める	家の中
アクセスポイント（AP）	無線の端末をネットワークにつなぐ	家の中
スイッチ	同じ LAN の中の機器どうしを中継する	家の中
ルーター	家の LAN と外の網の間で渡し先を決める	家の中（外との境目）
ONU ／ モデム	回線の信号を家の機器が扱える形に変換	家の中（回線の入口）
ISP	回線をインターネットへつなぐ事業者の網	家の外

⚠ 注意

並びや数は契約・機種で異なり、複数の役割を1台にまとめた機器もあります。

家庭用 Wi-Fi ルーター：1つの箱に複数の機能

左は外から見た姿、右は中の働き。灰色の太枠＝物理的な箱（1台の機械）、点線の箱＝中で働く機能で、別々の機械ではありません。



下の段の3つ（設定を配る・番号を書き換える・問い合わせを取り次ぐ）は、宛先の番号を学んだ後で1つずつ説明します。ONU まで1台に入った機種もあります。

Wi-Fi につながる ≠ インターネットに届く

黄色の破線の枠＝Wi-Fi の表示が示す区間。灰色の破線＝電波、太い実線＝ケーブル。

Wi-Fi の表示が示すのはここだけ



Wi-Fi のマークは、スマホとアクセスポイントの間の無線がつながったことを示すだけです。その先のルーターの設定、外の回線、名前の問い合わせ（p.247）のどこかが止まると、外へは届きません。

補足：業務用では、アクセスポイント・スイッチ・ルーターを別々の機器にすることが多く、ルーターの働きも持つスイッチ（L3 スイッチ）もあります。

家の中の接続と、外へ出る回線は別の区間

有線 LAN と Wi-Fi は家の中、光・CATV・携帯は外へ出る回線です。

区間	方式	使う媒体・機器	特徴と注意
家の中	有線 LAN	LAN ケーブル・スイッチ	安定しやすい。ケーブルや端子の規格で速さの上限が変わる
家の中	Wi-Fi（無線 LAN）	電波・アクセスポイント	壁・距離・干渉で変わる。2.4GHz 帯は届きやすく、5GHz 帯は速いが壁に弱い傾向

GHz（ギガヘルツ）は電波の周波数の単位です。同じ Wi-Fi でも、使う周波数の帯（2.4GHz 帯・5GHz 帯）で届きやすさや速さが変わります。

 ポイント

「遅い」と感じたとき、家の中の区間（端末～アクセスポイント）の問題か、外へ出る回線の問題かを分けて考えると、原因を探しやすくなります。

家から外へ出る回線の種類

区間	方式	使う媒体・機器	特徴と注意
家→外	光回線	光ファイバー・ONU	固定回線の代表。集合住宅では建物内の配線方式で差が出る
家→外	CATV 回線	同軸ケーブル・ケーブルモデム	テレビ用の配線を使う。上りが下りより遅い契約もある
家→外	4G / 5G （携帯回線）	基地局・SIM	持ち運べる。混雑・電波状況・契約で速さが変わる

上りは家から外へ送る向き、下りは外から受け取る向きです。基地局は携帯電話の電波の受け口、SIM は契約者を識別する部品で、携帯回線のくわしい仕組みは第6章の後半で扱います。

区間を分けて考える：在宅勤務と「速さ」

家の中と外の区間を分けて考える見方は、在宅で働くときの確認や、「速さ」の数値を読むときにも役立ちます。

リモート就業で確認すること

- 会社の規定（働く場所・使う端末・回線）
- 回線の安定性
- Wi-Fi のアクセスポイントの位置
- 障害時の連絡手段

「速さ」の見分け方

- リンク速度：PC とアクセスポイントの間で決まった通信の速さの上限
- 実効速度：インターネットの相手との間で実際に出る速さ
- 2 つは別の数値（ p.295 ）

確認の問い（家庭の通信機器）

自分の言葉で答えてから、次のページの解説を読みましょう。

- ① スイッチとルーターの違いを説明してください。
- ② Wi-Fi のマークが出ていればページは必ず開けますか。
- ③ 家庭用 Wi-Fi ルーターを「箱」と「機能」で説明してください。

確認の問い（家庭の通信機器）：解説

前のページの問いへの答えと、考え方です。

✓ 問1 スイッチとルーターの違いを説明してください。

スイッチは家の LAN の中で機器どうしを中継し、ルーターは家の LAN と外の網の境目で次の渡し先を決めます。 p.212

✓ 問2 Wi-Fi のマークが出ていればページは必ず開けますか。

開けるとは限りません。マークは無線がつながったことだけを示し、ルーターの設定・回線・名前の問い合わせが止まると届きません。 p.218

✓ 問3 家庭用 Wi-Fi ルーターを「箱」と「機能」で説明してください。

物理的には1台の箱ですが、中にアクセスポイント・スイッチ・ルーター・設定を配る機能などが入っています。 p.217

宛先を表す番号は 3 種類ある

パケットの付け札には宛先が書かれます（p.203）。宛先の番号は 3 種類です。

IP アドレス

どの機器へ届けるか（最終的な相手）

- ・例：サンプル商店のサーバー 198.51.100.7

MAC アドレス

今いる区間で、次にだれへ手渡すか

- ・例：まず家のルーターへ

ポート番号

届いた機器の中の、どのプログラムへ渡すか

- ・例：注文サイトのプログラム（443 番）

手紙でいえば「最終的な住所」「次の郵便局への手渡し」「家の中のだれ宛てか」です。次のページから 1 つずつ見ます。

IP アドレス：最終的な相手の機器を示す番号

IP アドレス（あいびーあどれす）

分類：データ

略語：IP = Internet Protocol address

何か	ネットワーク上の機器（正確には機器の NIC などの接続口）を区別する番号。データの最終的な宛先と送り元に使う。
なぜ必要か	番号がないと、パケットをどの機器へ届けばよいか、途中の機器が判断できない。返事を送り返す先も分からない。
どこにあるか	パケットの付け札に「宛先 IP」「送り元 IP」として書かれる。各機器の OS が自分の番号を設定として持つ。
身近な例	スマホ 192.168.1.21、PC 192.168.1.20、サンプル商店のサーバー 198.51.100.7（説明用の例）
似た言葉との違い	機器に一生固定の番号とは限らず、接続のたびに変わることもある。製造時に決まる MAC アドレス（p.227）とは別。

IPv4 の形と、IP アドレスを使う場面

広く使われている **IPv4**（IP の第 4 版）は、32 ビットの IP アドレスを 8 ビットずつ区切り、0～255 の 4 つの数で書きます。

パケットの模式図。白い箱＝付け札、緑の書類形＝中身。佐藤さんのスマホからサーバーへの依頼の例です。



説明用の例

- **宛先**：途中のルーターは宛先 IP を見て、次の渡し先を決めます。
- **送り元**：サーバーは送り元 IP を見て、返事の宛先にします。
- 家の外へ出るとき、送り元はルーターで書き換えられます（p.253）。

MAC アドレス：区間の中で次に渡す相手の番号

MAC アドレス（まっくあどれす）

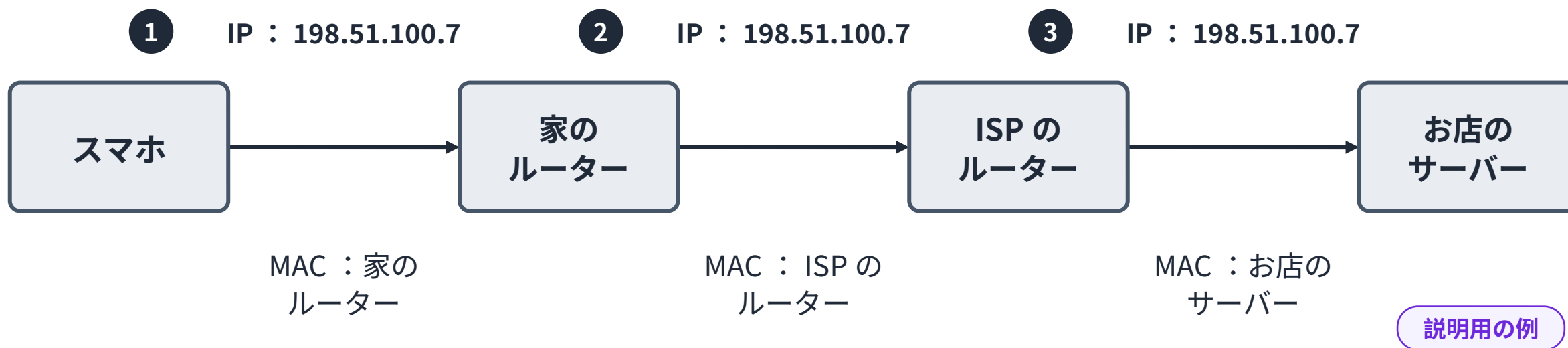
分類：データ

略語：MAC = Media Access Control address

何か	NIC ごとに付けられた番号。同じ LAN など 1 つの区間の中で、データを次に手渡す相手の機器を指定するために使う。
なぜ必要か	スイッチはこの番号を見て、どの口へ送るかを決める。
どこにあるか	NIC に製造時に書き込まれている（設定で別の番号を使うスマホなどもある）。
身近な例	AA-BB-CC-11-22-33（説明用の例。0～9 と A～F の 16 進数、2 けた × 6 組）
似た言葉との違い	IP アドレスは最終的な相手で、途中で変わらないのが基本。MAC は区間ごとに付け替わる。

MAC の宛先は区間ごとに変わり、 IP の宛先は変わらない

役割の関係図。黒の矢印＝パケットの移動（区間ごと）。矢印の上は、その区間での宛先です。途中の区間は省略しています。



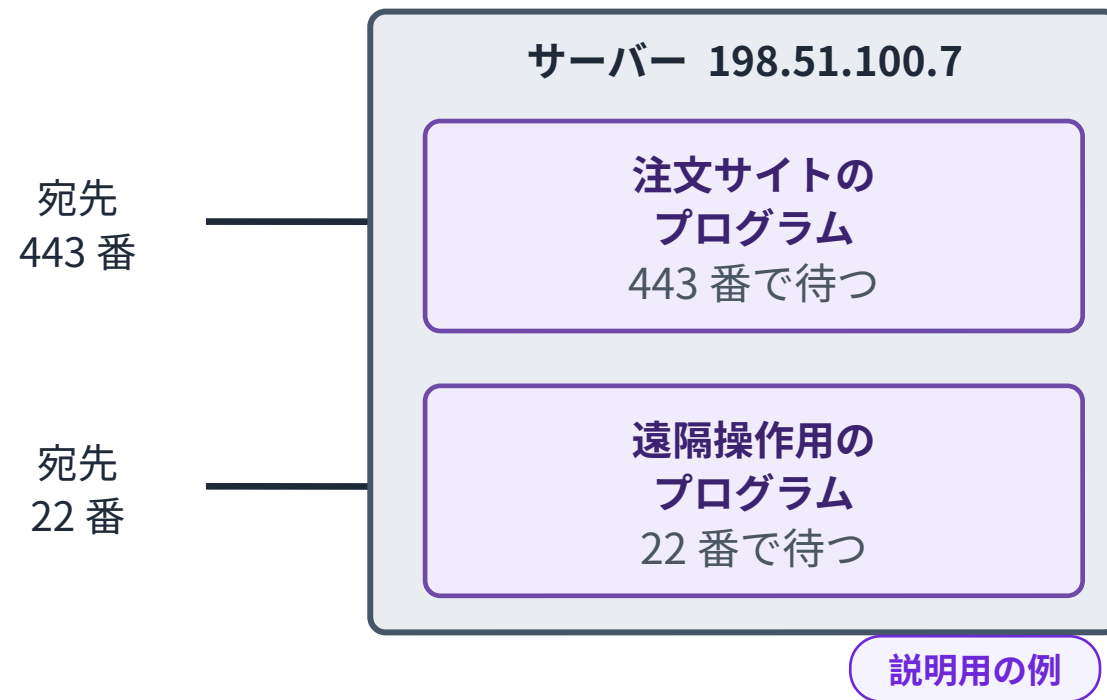
ルーターはパケットを受け取るたびに、次の区間の相手の MAC アドレスを宛先にした新しい包みに入れ直します。最終的な相手を示す IP の宛先はそのままです。区間によっては MAC 以外の番号を使うこともありますが、「区間ごとに付け替える」考え方は同じです。

ポート番号：機器の中のどのプログラム宛てか

1 台の機器の中では、複数のプログラム（第 3 章の **プロセス**）が同時に通信します。届いたデータをどれに渡すかの目印が必要です。

ポート番号は、機器の中のどのプログラムの通信の受け口宛てかを示す番号です（0 ～ 65535）。サーバーのプログラムは、決まった番号の受け口で依頼を待ちます。例えば暗号化した Web の通信（HTTPS）では 443 番がよく使われます。

紫の箱＝プログラム、灰色の太枠＝機械。黒の矢印＝届いたデータ。



ポート番号を使う場面

スマホが依頼を送るとき、付け札には「宛先ポート 443（注文サイトのプログラム）」と「送り元ポート 50000（ブラウザーが今回だけ使う番号）」が書かれます。返事は 50000 番へ返るので、スマホの中で正しいアプリに届きます。

よく使われる番号	受け口のプログラム（通信の種類）
443	暗号化した Web の通信（HTTPS）
80	暗号化しない Web の通信
53	名前の問い合わせ（DNS）
22	遠隔操作（離れた機器をコマンドで操作する）

⚠ 注意

「ポート」は LAN 端子のことではなく、プログラムの受け口を表す番号です。

IP ・ MAC ・ ポートを比べる

	IP アドレス	MAC アドレス	ポート番号
指すもの	IP 通信の相手（機器の接続口）	同じ区間の中の機器	機器の中のプログラムの受け口
使う範囲	端から端まで（NAPT で変わることがある）	1 つの区間だけ	TCP/UDP の通信ごと
例	192.168.1.20	AA-BB-CC-11-22-33	443（HTTPS）

💡 **たとえ**

IP = 最終の住所、MAC = 次の中継局への手渡し、ポート = 家の中の部屋番号。

❗ **たとえの限界**

IP は固定の住所とは限らず、接続のたびに変わることもあります。「ポート」は LAN 端子ではありません。

3つの番号は入れ子の付け札

入れ子の模式図（実物の形ではなく付け札の関係を表します）。内側ほど端から端まで変わらず、外側ほど区間ごとに付け替えられます。

フレーム（区間の中を運ぶ包み）宛先 MAC：家のルーター ← 区間ごとに付け替え

パケット 宛先 IP：198.51.100.7（最終的な相手の機器）

TCP/UDP の包み 宛先ポート：443（相手の中のプログラム）

中身：注文履歴を見せて

説明用の例

送る側は内側から順に包み、受け取る側は外側から開けます。途中のルーターは外側の2つ（フレームとパケットの付け札）だけを見て運びます。

家庭 LAN の番号：中で使う番号と外で通じる番号

簡略化した物理的な配置図。箱の下の段が各機器の IP アドレス。家の機器は Wi-Fi ルーターの中のスイッチや AP の機能につながります。



LAN の中では**プライベート IP**（LAN の中だけで使う番号。192.168.～などの範囲が決められ、別の家でも同じ番号を使える）を使います。インターネット上で通じる番号は**公開 IP**（グローバル IP アドレス。世界で重ならないように割り当てられる）で、家庭では多くの場合 ISP からルーターに 1 つ割り当てられます。

サブネット：「同じ LAN」とみなす番号の範囲

サブネットは「同じ LAN とみなす IP アドレスの範囲」です。192.168.1.0/24 の「/24」は、32 ビットのうち先頭の 24 ビット（4 つの数のうち前の 3 つ）が同じなら同じ LAN とみなす、という意味です。同じことを**サブネットマスク**という形で書くと 255.255.255.0 になります（255 の部分がそろっていれば同じ LAN）。

宛先	前の 3 つの数	自分（192.168.1.20）から見た判断
192.168.1.30（プリンター）	192.168.1	同じ → 同じ LAN。相手へ直接渡す
203.0.113.80（外のサーバー）	203.0.113	違う → LAN の外。出口（ルーター）へ渡す

 ポイント

端末は、宛先がサブネットの中か外かで、最初に渡す相手を変えます（p.237）。

端末が持つ 4 つのネットワーク設定

設定項目	例（ PC ）	意味
IP アドレス	192.168.1.20	この端末の LAN 内での番号（プライベート IP ）
サブネットマスク（ /24 ）	255.255.255.0	192.168.1.xxx を同じ LAN とみなす
デフォルトゲートウェイ	192.168.1.1	LAN の外へ出るときに渡す出口（ルーター）
DNS サーバー	192.168.1.1 など	名前（ shop.example.com など）から番号を調べるときの問い合わせ先

デフォルトゲートウェイの「ゲートウェイ」は出入り口、「デフォルト」は特に決まっていないときの既定の、という意味です。これら 4 つの設定は、ふつうは自動で配られます（ p.244 ）。

IPv6：番号の数を大きく増やした新しい版

IPv4 の番号は 32 ビットなので、約 43 億通りしかありません。インターネットにつながる機器が増えて足りなくなったため、128 ビットで表す **IPv6** が作られ、今は IPv4 と並んで使われています。

- 表記が長い：例 2001:db8::1（「:」で区切り、0～9 と a～f を使う）
- 端末が、インターネットで通じるアドレスを持つ構成も多い
- そのため、家庭で番号を書き換える仕組み（NAT、p.252）の考え方がそのまま当てはまるとは限らない

補足

203.0.113. ～ ・ 198.51.100. ～ ・ 2001:db8:: ～ は、説明用に予約された番号です。192.168. ～ は LAN の中で使うために予約された範囲です。

宛先が同じ LAN か外かで、最初に渡す相手が変わる

端末の OS は「経路表」（宛先の範囲ごとに、だれへ渡すかを書いた表）を見て、宛先が同じ LAN なら相手へ直接、外ならデフォルトゲートウェイ（ルーター）へ渡します。渡す相手の MAC アドレスは、**ARP**（IP アドレスから MAC アドレスを調べる約束）で「この番号の機器の MAC は？」と LAN 全体に問い合わせ調べてみます。

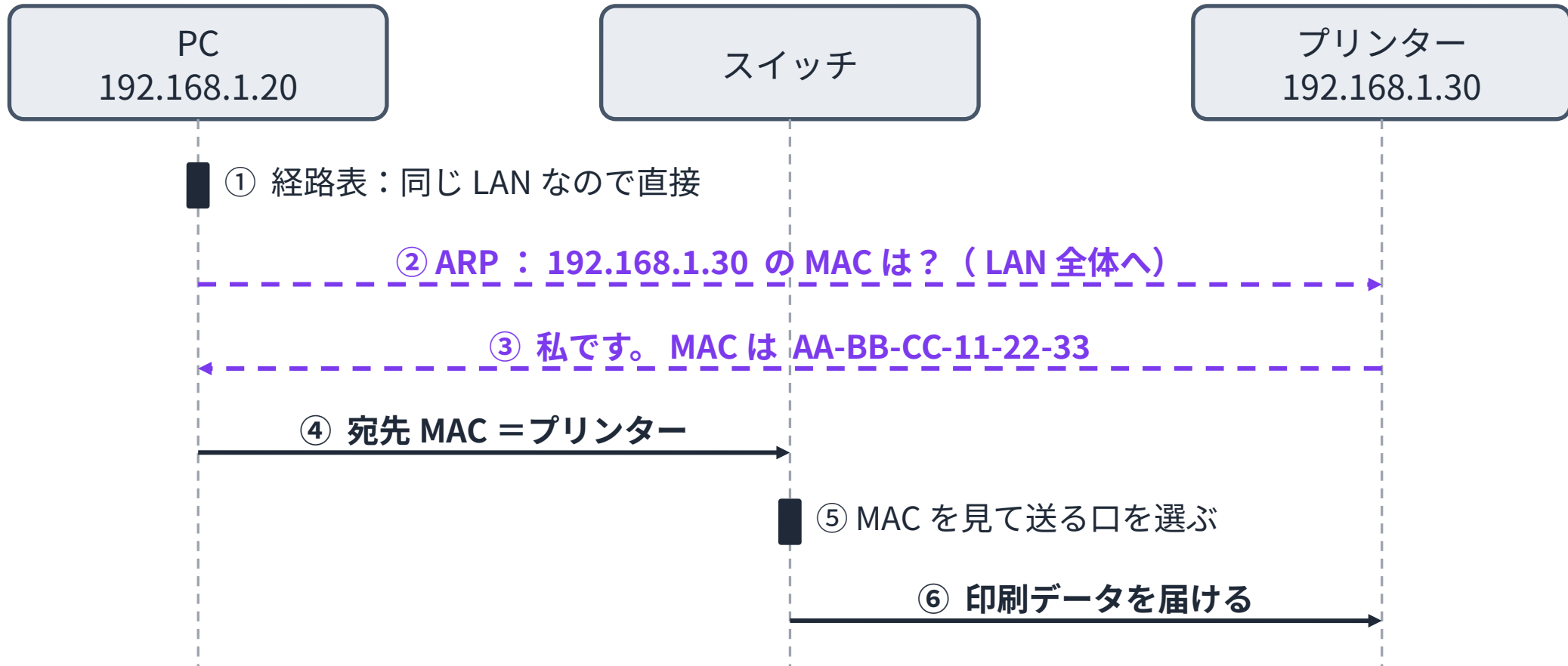
宛先の範囲	渡す相手
192.168.1.0/24（同じ LAN）	直接、相手へ
それ以外（既定の出口）	192.168.1.1（ルーター）

ポイント

次の 2 ページで、同じ LAN のプリンターへ送る場合（ケース A）と、外のサーバーへ送る場合（ケース B）を、時間順の図で比べます。

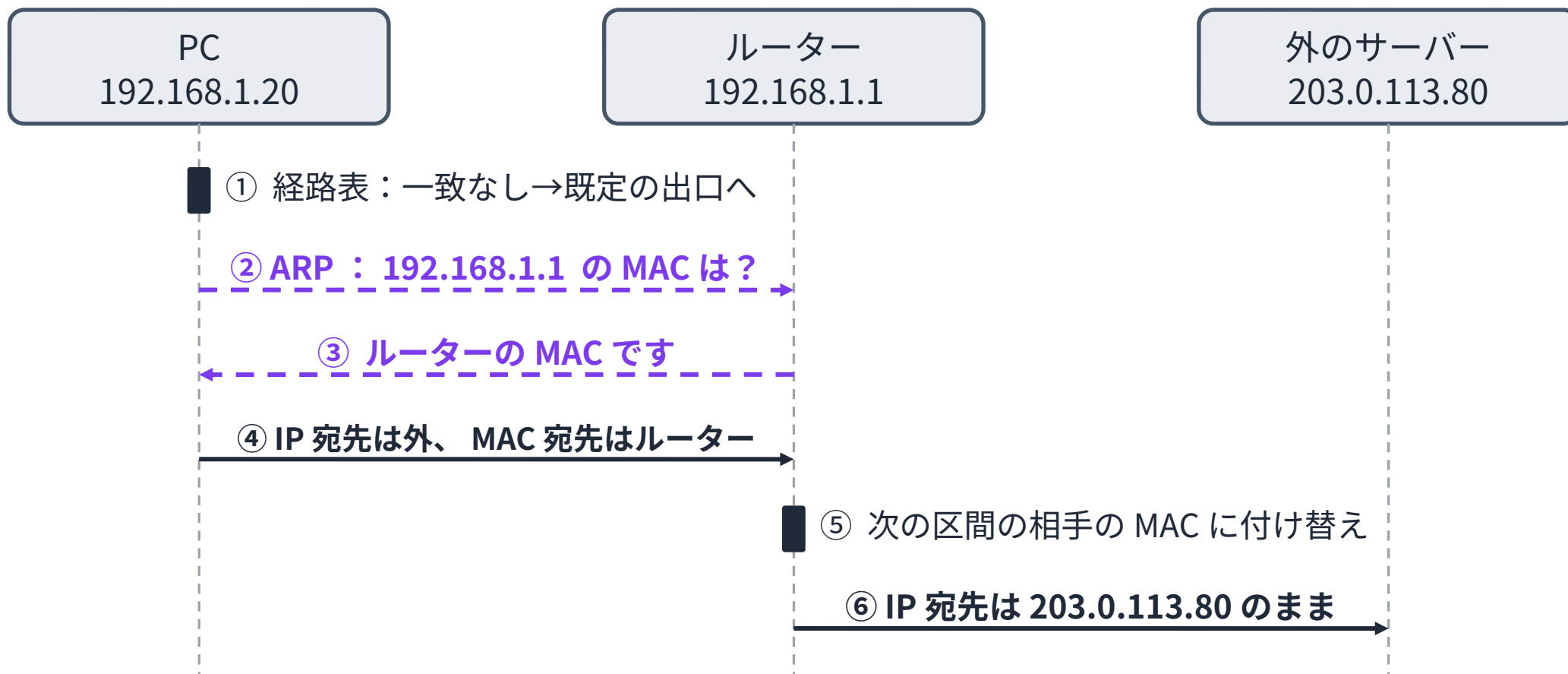
↑ PC の経路表の例（説明用・簡略化）

ケース A：同じ LAN のプリンターへ送る



時間順のやり取り図。紫の破線＝補助の問い合わせ（ARP）、黒の矢印＝本文（印刷データ）、縦の棒＝機器の中の処理。②③もスイッチを通りますが省いています。

ケース B：外のサーバーへ送る



時間順のやり取り図。④で IP の宛先は外のサーバー、MAC の宛先はルーターです。ルーターは⑤で包みを入れ直して中継します（⑥の先にも多くの中継があります）。

ケース A と B の違い： IP の宛先と MAC の宛先

	IP の宛先	フレーム（MAC）の宛先
ケース A	192.168.1.30（プリンター）	プリンターの MAC
ケース B	203.0.113.80（外のサーバー）	ルーターの MAC

IP の宛先は「最終的にだれへ」、MAC の宛先は「この区間でまずだれへ」です。宛先が LAN の外なら、MAC の宛先は出口のルーターになります。

補足

ARP は IPv4 の仕組みです。IPv6 では近隣探索（NDP）という別の仕組みで、相手の区間の中の番号を調べます。

スイッチ・ルーター・アクセスポイントが見る情報

機器	主に見る情報	働く範囲
スイッチ	MAC アドレス（同じ LAN の中の宛先）	同じ LAN の中
ルーター	IP アドレス（最終的な宛先）と経路表	異なるネットワークの間
アクセスポイント	無線の接続・無線の暗号化	電波と有線 LAN の間

アドレスを学んだので、家庭の機器の違いを「何を見て中継するか」で言い直せます。スイッチは区間の中の宛先（MAC）を見て口を選び、ルーターは最終的な宛先（IP）を見て次のネットワークへの渡し先を選びます。

確認の問い（宛先の番号）

自分の言葉で答えてから、次のページの解説を読みましょう。

- 1 IP ・ MAC ・ ポートは、それぞれ何を区別しますか。
- 2 PC が外のサーバーへ送るとき、宛先 MAC はだれのものですか。
- 3 /24 （ 255.255.255.0 ） は何を意味しますか。

確認の問い（宛先の番号）：解説

前のページの問いへの答えと、考え方です。

✓ 問1 IP・MAC・ポートは、それぞれ何を区別しますか。

IP は最終的な相手の機器、MAC は同じ区間で次に手渡す機器、ポートは機器の中のどのプログラムの受け口かを区別します。 p.231

✓ 問2 PC が外のサーバーへ送るとき、宛先 MAC はだれのものですか。

出口のルーター（192.168.1.1）の MAC です。IP の宛先は外のサーバー（203.0.113.80）のままです。 p.239

✓ 問3 /24（255.255.255.0）は何を意味しますか。

前の3つの数（先頭24ビット）が同じアドレスを同じ LAN とみなす意味です。192.168.1.30 は同じ LAN、203.0.113.80 は外です。 p.234

DHCP：ネットワークの設定を自動で配る

DHCP（でいーえいちしーピー）

分類：通信の約束

略語：DHCP = Dynamic Host Configuration Protocol 日本語：動的ホスト構成プロトコル

何か	機器がネットワークにつながったとき、IP アドレス・サブネットマスク・デフォルトゲートウェイ・DNS サーバーなどの設定を自動で配る仕組み。
なぜ必要か	手で設定すると手間がかかり、番号の重なりや入力ของ誤りが起きる。
どこにあるか	家庭では多くの場合、Wi-Fi ルーターの中の DHCP サーバーの機能が配る。
受け取る→する→返す	「設定をください」を受け取る → 空いている番号を選ぶ → 期限つきで渡す。
似た言葉との違い	本文（ページや質問）を中継するものではない。設定を配るだけ。

DHCP のやり取り：探索・提案・要求・承認

佐藤さんのスマホ
つないだばかり

DHCP サーバー
多くはルーターの中

① 探索：設定をくれるサーバーはありますか

② 提案：192.168.1.21 を使えます

③ 要求：では 192.168.1.21 を使います

④ 承認：期限つきで割り当てます

📖 受け取る設定の例

IP アドレス／サブネットマスク／デフォルトゲートウェイ（出口）／DNS サーバー／使ってよい期間（リース）

⚠ 注意

DHCP が働くのは、接続したときと期間が切れる前の更新のときです。注文履歴の依頼やページの本文は、DHCP サーバーを経由しません。

ドメイン名：番号の代わりに使う名前

IP アドレスは数字の並びなので、人が覚えるのは大変です。また、お店がサーバーを移すと番号が変わることもあります。そこで、shop.example.com のような名前を使います。この名前を**ドメイン名**といいます。名前は「.」で区切られ、右ほど大きな区分です。

shop

その組織の中のサービスや機器の名前（ここではサンプル商店の買い物サイト）

example

組織などに割り当てられた名前。example.com は説明用に予約された名前

com

一番右の大きな区分（トップレベルドメイン、TLD）。ほかに jp ・ org など

ただし通信の宛先に書けるのは IP アドレスです。名前から番号を調べる仕組みが必要になります（次のページ）。

DNS：名前から IP アドレスを調べる仕組み

DNS（でいーえぬえす）

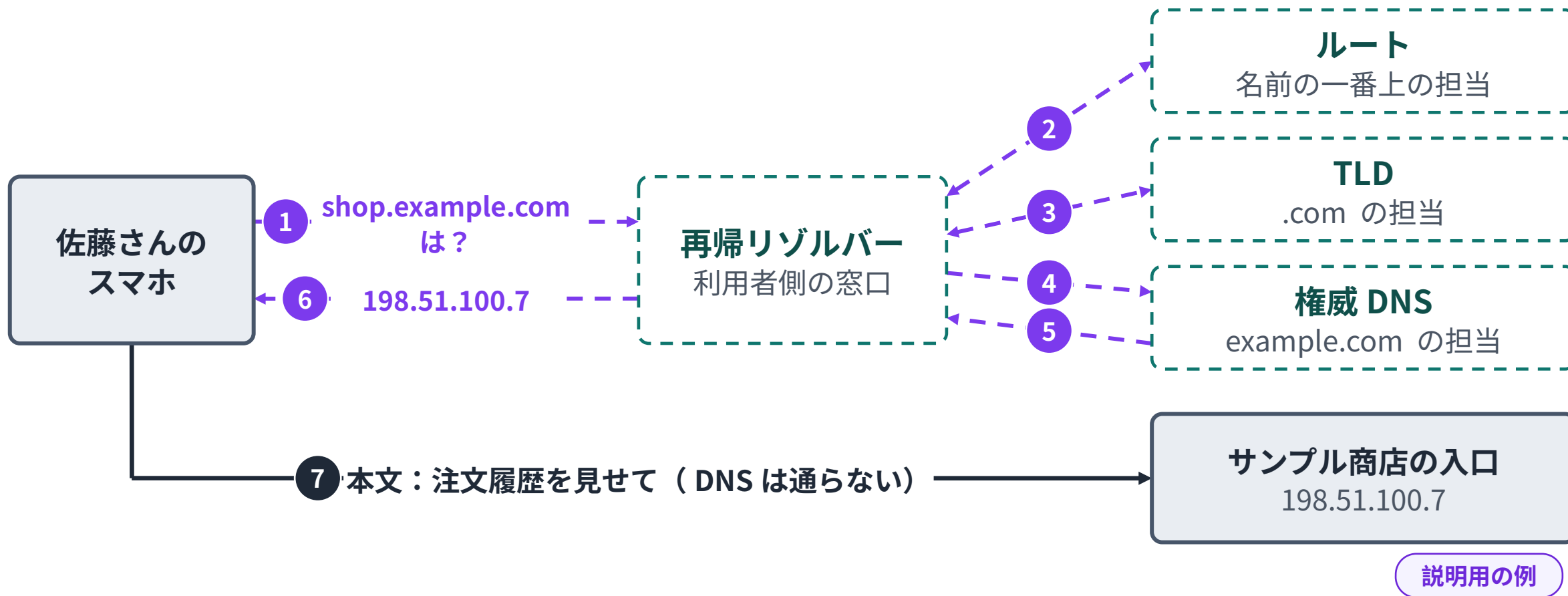
分類：仕組み

略語：DNS = Domain Name System 日本語：ドメインネームシステム

何か	ドメイン名に対応する IP アドレスなどを調べて答える仕組み。名前から番号を調べることを「名前解決」という。
なぜ必要か	通信の宛先には IP アドレスが必要だが、人が使うのは名前。名前と番号の対応を、世界中で分担して管理する。
どこにあるか	利用者側の窓口（再帰リゾルバー。ISP などが用意）と、名前ごとの担当（権威 DNS）が、別々の場所にある。
受け取る→する→返す	「shop.example.com の IP アドレスは？」を受け取る → 調べる → 「198.51.100.7 です」と返す。
似た言葉との違い	名前を調べるだけで、本文は DNS サーバーを通らない。暗号化や番号の書き換えとも別の仕組み。

DNS の問い合わせは、担当を順にたどる

役割の関係図。紫の破線の矢印＝名前の問い合わせと答え（補助の通信）、黒の実線の矢印＝本文の通信。丸数字は順番、②～⑤の中身は次のページです。



①～⑦で、だれが何をたずねるか

番号	だれ → だれ	中身
①	スマホ → 再帰リゾルバー（窓口）	「shop.example.com の IP アドレスは？」
②	リゾルバー ⇄ ルート	「.com の担当は？」 → 案内
③	リゾルバー ⇄ TLD	「example.com の担当は？」 → 案内
④	リゾルバー → 権威 DNS	「shop.example.com の IP アドレスは？」
⑤	権威 DNS → リゾルバー	「198.51.100.7 です」
⑥	リゾルバー → スマホ	「198.51.100.7 です」
⑦	スマホ → お店の入口	本文「注文履歴を見せて」（DNS は通らない）

ルート＝名前が一番上の担当、TLD＝.com など一番右の区分の担当、権威 DNS＝example.com の正式な答えを持つ担当。窓口が上から順に教わります。

DNS のキャッシュ・確かめ方・混同しやすい点

🕒 キャッシュ

一度調べた結果を、決められた時間だけ端末やリゾルバーが覚えておくこと。毎回①～⑥をすべて調べ直すわけではない。

🔍 確認の方法

nslookup というコマンドで DNS の答えを見られる。ただしブラウザーが使う DNS と結果が違うこともある。

⚠️ 混同しやすい点

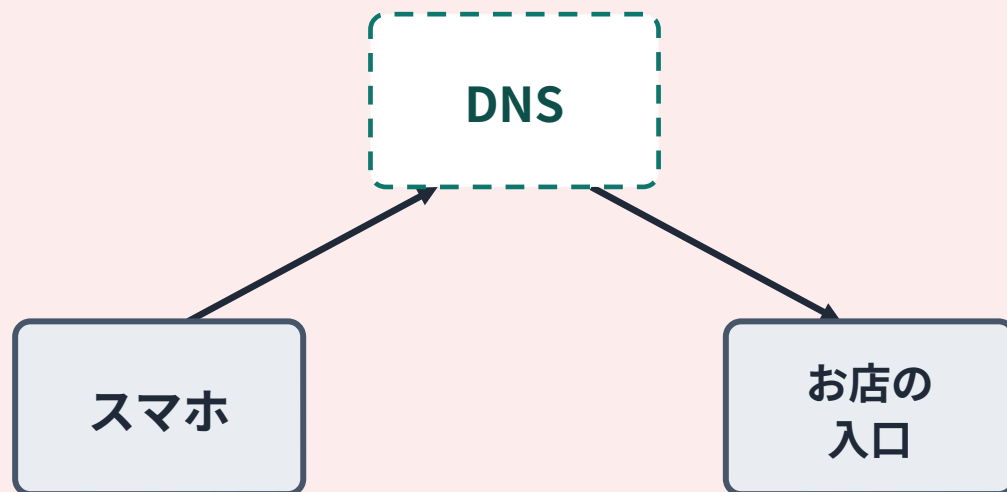
DNS は名前解決だけを受け持つ。暗号化（TLS）や番号の書き換え（NAT）とは別の仕組み。

キャッシュという言葉は、第2章のキャッシュメモリと同じく「よく使うものを近くに一時的に置いて、取りに行く手間を減らす」という意味で使われています。

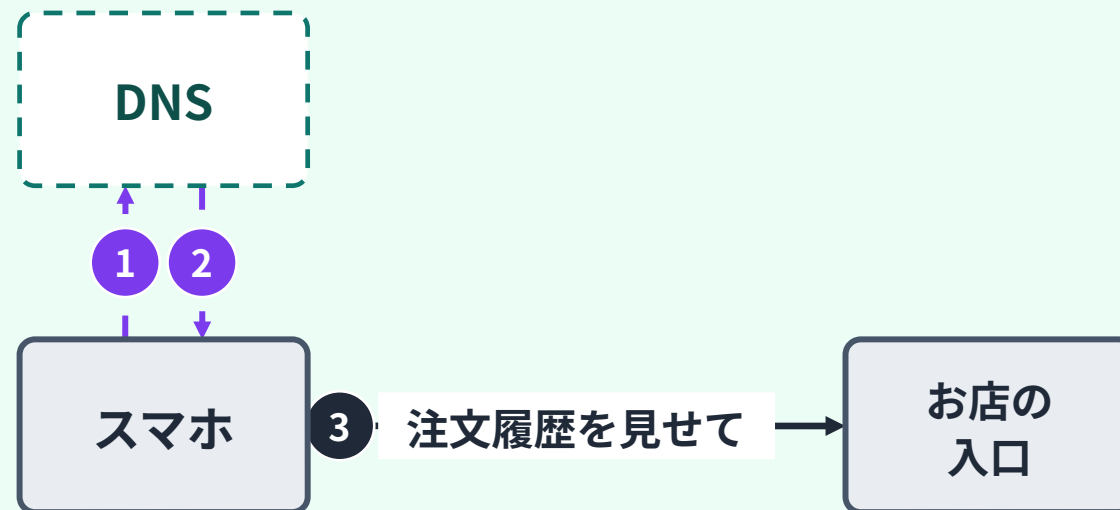
DNS の後、本文は DNS サーバーを通らない

役割の関係図。黒の矢印＝本文の依頼、紫の破線の矢印＝名前の問い合わせと答え。左は誤った理解、右が実際の流れです。

× 誤った理解：本文が DNS を通る



○ 実際：本文は相手へ直接向かう



DNS に聞くのは相手の番号だけで、本文はその番号へ直接送られます。家のルーターや ISP の中継機器は通りますが、DNS サーバーは通り道に入りません。

なぜ家の中の番号を書き換えて外へ出すのか

IPv4 の番号（約 43 億通り）は、世界中の機器すべてに配るには足りません。そこで家庭では、ISP から受け取った 1 つの公開 IP（例 203.0.113.50）を、家の中の複数の端末で共有します。

ところが、家の中のプライベート IP（192.168.1.21 など）は、別の家でも使われている番号です。そのままインターネットへ送ると、相手は返事をどこへ返せばよいか分かりません。そのため家のルーターが、外へ出るパケットの送り元を自分の公開 IP に書き換えます。この仕組みを **NAT**（アドレス変換）といいます。

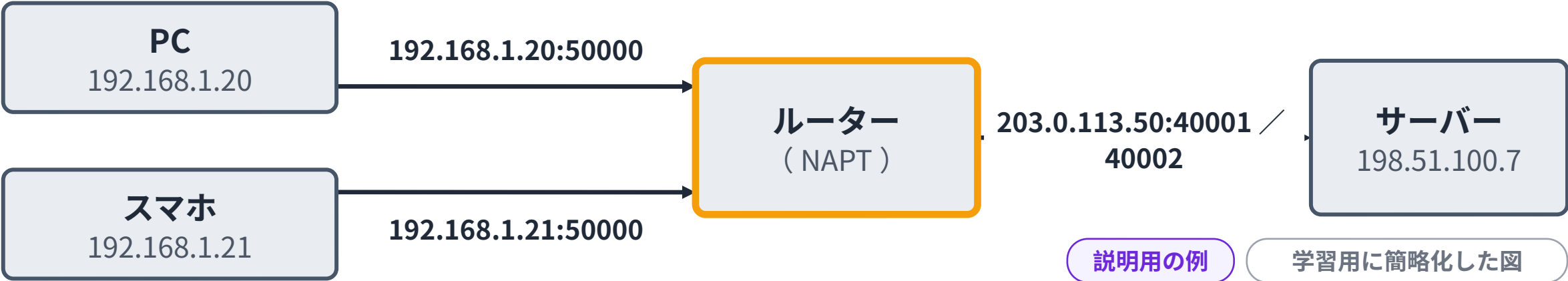
ポイント

書き換えたままでは、返ってきた返事を家の中のどの端末へ渡せばよいか分かりません。そこでルーターは「だれの通信をどう書き換えたか」を対応表に記録します（次のページ）。

NAPT：送り元のIPとポートを書き換える

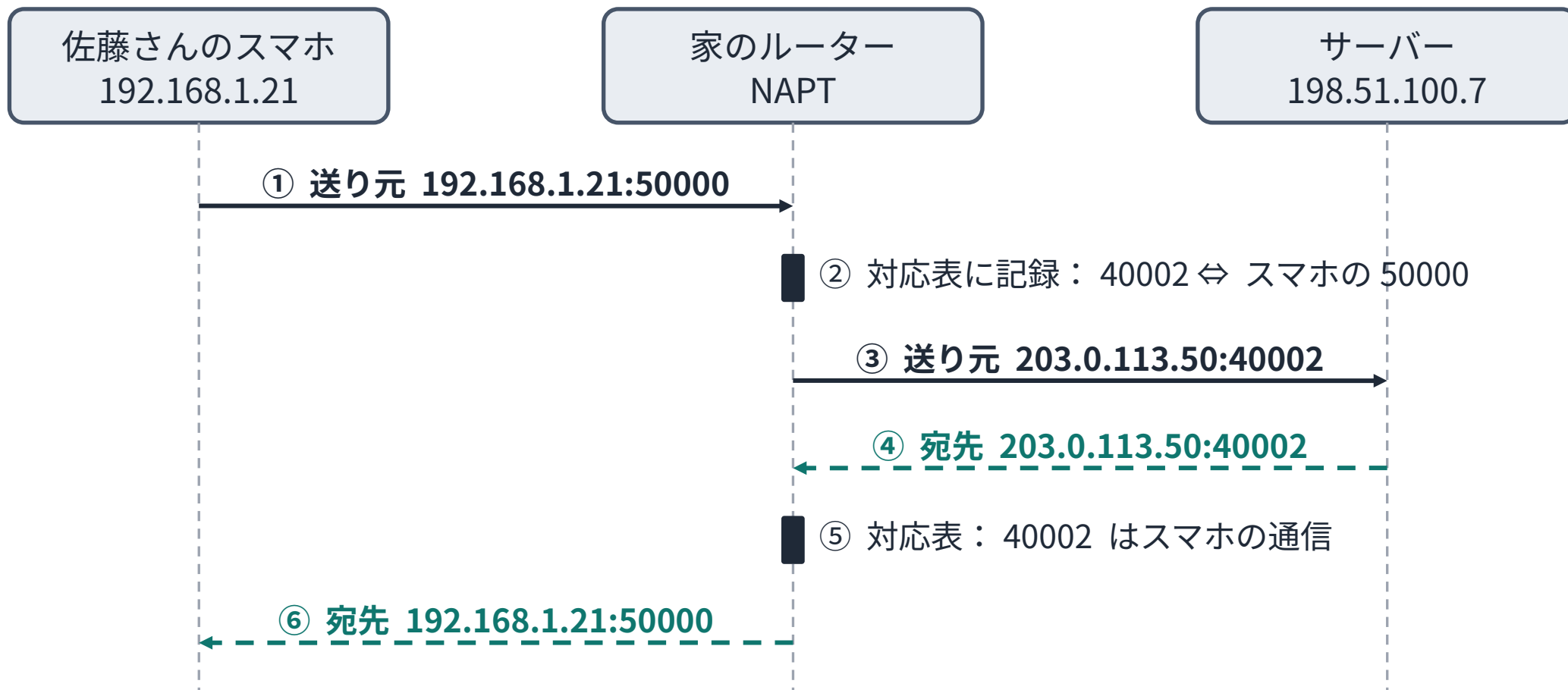
家庭のルーターは、IP アドレスだけでなくポート番号も書き換えます。これを **NAPT** といいます。端末ごとに違うポート番号を割り当てるので、1つの公開IPでも返事を区別できます。

役割の関係図。黒の矢印＝外へ出るパケット。矢印の上は送り元の番号です。



端末	書き換え前の送り元	書き換え後の送り元	宛先
PC	192.168.1.20 : 50000	203.0.113.50 : 40001	198.51.100.7 : 443
スマホ	192.168.1.21 : 50000	203.0.113.50 : 40002	198.51.100.7 : 443

返事是对応表を使って元の端末へ戻す



時間順のやり取り図。黒の矢印＝依頼、緑の破線＝返事、縦の棒＝ルーターの中の処理。サーバーから見える相手は 203.0.113.50 だけです。

NAT がしないこと・環境による違い

⚠ NAT がしないこと

- 名前解決（DNS）ではない
- 暗号化（TLS、p.262）ではない
- 中身を隠す仕組みではない（書き換えるのは付け札の番号だけ）

⇄ 環境による違い

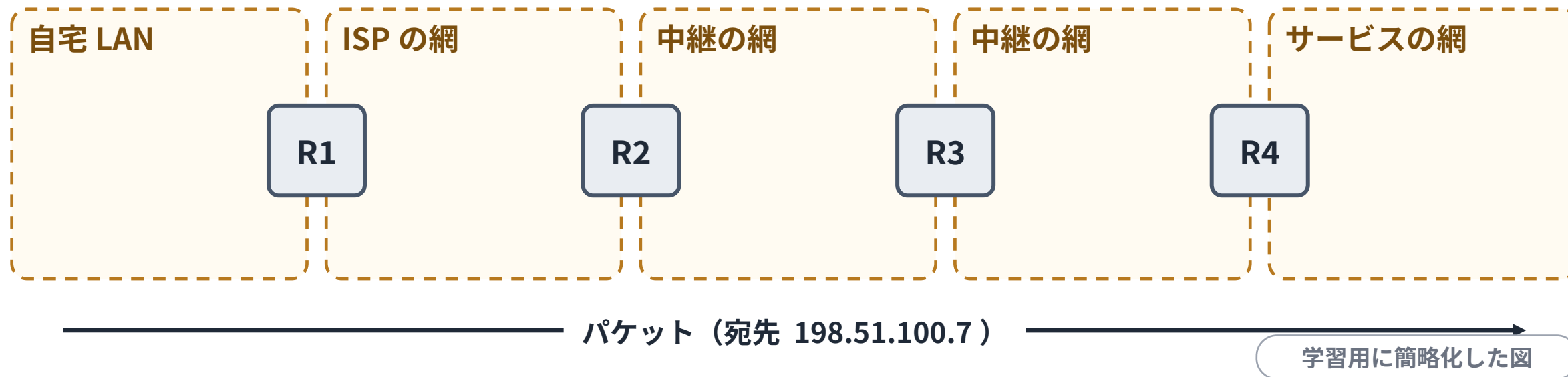
- IPv6 では、家庭と同じ構成とは限らない
- 携帯電話会社の網では、事業者の側で番号を変換する場合もある

NAT・NAPTは「1つの公開IPを家の中で共有するための番号の付け替え」です。安全のための暗号化や、名前から番号を調べる仕組みとは、別の役割だと区別しておきましょう。

経路制御：ルーターが次の網へ順に渡す

パケットは、通るネットワークごとにルーターが「次はどこへ渡すか」を決めて中継されます。宛先に応じて次の渡し先を決めることを**経路制御（ルーティング）**といいます。

物理的な配置図を簡略化。破線の大枠＝それぞれのネットワーク、R1～R4＝境目のルーター。黒の矢印＝パケットの進む向き。



経路は1本に固定されず、混雑や故障などの状況で変わることがあります。

パケット・経路制御・ホップ

用語	意味
パケット	ネットワークで運ぶデータのまとまり。宛先・送り元などの情報が付く
経路制御（ルーティング）	宛先に応じて次の渡し先を決めること。ルーターどうしが経路の情報を交換する
ホップ	ルーターを1つ通過すること。tracert（経路上のルーターを順に表示する Windows のコマンド）で経路上の中継点を観察できる

⚠ 注意

tracert の結果に「*」が出ても、故障とは限りません。問い合わせに応答を返さない設定のルーターもあります。

TCP ・ UDP ・ QUIC ： 届け方の約束が違う

TCP は順番と送り直しを管理して確実に届け、**UDP** は軽いまとまりをそのまま送り、**QUIC** は UDP の上で送り直し・暗号化・複数の流れを受け持ちます。IP ・ TCP ・ UDP など、インターネットで使う約束の一式を **TCP/IP** と呼びます。

HTTP/2 までの重なり

HTTP/1.1 ・ 2 Web の要求と応答

TLS 暗号化・相手の確認

TCP 順序・送り直し・量の調整

IP 宛先へパケットを届ける

HTTP/3 の重なり

HTTP/3 Web の要求と応答

QUIC 送り直し・暗号化（TLS 1.3）

UDP 小さなまとまりを運ぶ

IP 宛先へパケットを届ける

層の図：上ほどアプリに近い約束。HTTP は Web の約束、TLS は 2 ページ後に説明します。

TCP ・ UDP ・ QUIC の比較と使われ方

	TCP	UDP	QUIC
順序の保証	あり	なし（アプリ側で対応）	流れごとにあり
失ったデータの再送	あり	なし（アプリ側で対応）	あり
暗号化	別に TLS を使う	別に用意	仕様に組み込み
注意	混雑時は送る量を調整	「いつでも速い」ではない	UDP の上でも配送の管理を行う

📖 使われ方の例

TCP ： Web （ HTTP/1.1 ・ HTTP/2 ） 、メール、ファイル転送

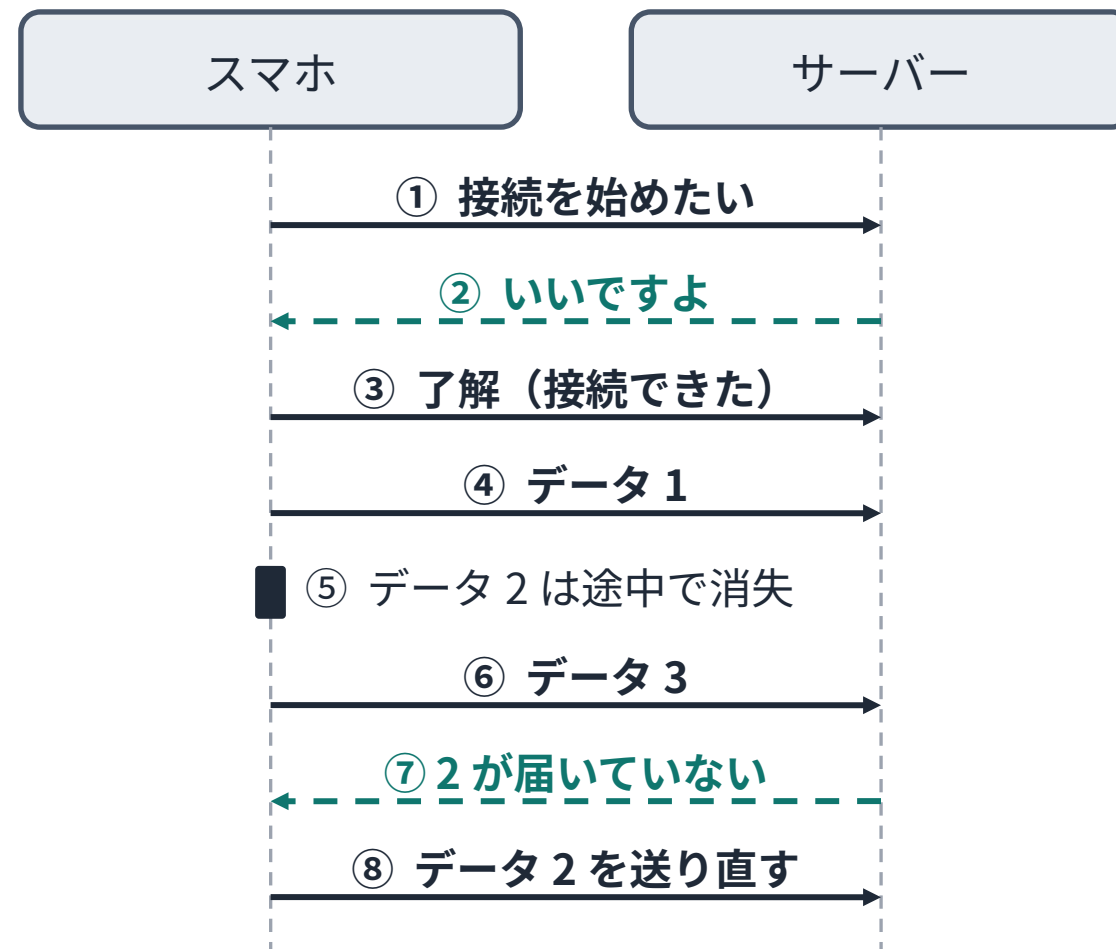
UDP ： DNS の問い合わせの多く、音声・映像、対戦ゲームの一部

QUIC ： HTTP/3 に対応した Web サービス

TCP の確実な届け方：順番の確認と送り直し

TCP では、まず相手と「接続」を始める合図を交わします。その後、データに順番の番号を付けて送り、受け取った側は「どこまで届いたか」を返します。届かなかった分だけを送り直すので、順番どおり確実に届きます。

UDP はこの確認をしないため軽く、多少欠けても待つより先へ進みたい音声・映像などで使われます。



時間順のやり取り図（簡略化）。黒＝送るデータ、緑の破線＝返事。⑤は届かなかったため矢印なし。

TLS の前に：暗号化と証明書

インターネットでは、データが家のルーター、ISP、中継の網など多くの機器を通ります。途中で中身を読まれたり、偽物の相手につながったりしないようにする道具が2つあります。

暗号化

暗号化は、決まった手順と鍵を使って、データを第三者に読めない形に変えることです。鍵を持つ相手だけが元に戻せます（復号）。途中の機器には意味のない0と1の並びにしか見えません。

証明書

証明書（デジタル証明書）は、「このサーバーは shop.example.com の正式な持ち主のもの」と、信頼できる発行元が電子的な署名で保証するデータです。

たとえ

暗号化は「中身が見えない封筒に入れて封をする」、証明書は「公的な機関が発行した身分証明書」に近い働きです。

TLS：相手を確認め、中身を暗号化する

TLS（ていーえるえす）

分類：通信の約束

略語：TLS = Transport Layer Security

何か	通信の相手が本物かを証明書で確かめ、やり取りする中身を暗号化し、途中での書き換えに気づけるようにする約束。
なぜ必要か	途中の機器に中身を読まれたり書き換えられたりしないため。
どこにあるか	端末のブラウザやアプリと、サービスの入口の機器の間で働く。
受け取る→する→返す	接続の始めに証明書を確認めて暗号の鍵を決め、以後のデータを暗号化して送る。
身近な例	https:// で始まるページ（鍵マーク）
似た言葉との違い	NAT（番号の書き換え）やDNS（名前の調べ）とは別の仕事。

TLS が行う 3 つのことと、HTTPS

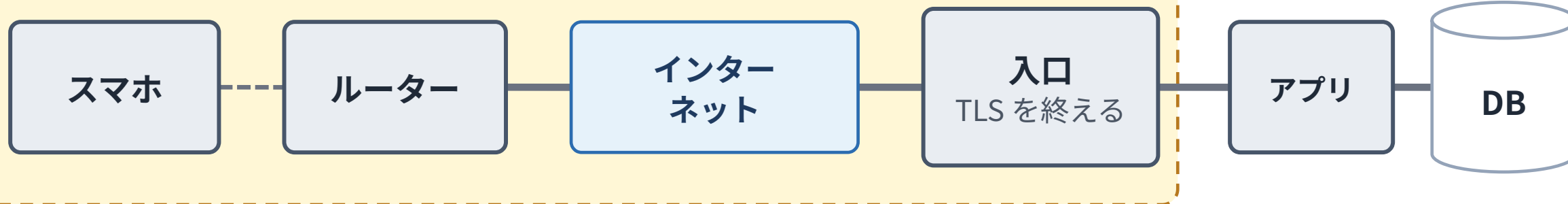
TLS が行うこと	内容
相手の確認	サーバーの証明書を、信頼できる発行元の署名で確かめる
暗号化	途中の機器から中身を読まれないようにする
改ざんの検知	途中で書き換えられたら気づけるようにする

HTTPS は、Web のページなどをやり取りする約束（HTTP。第7章で詳しく見ます）を、**TLS** で保護した通信です。佐藤さんの「注文履歴を見せて」という依頼と、返ってくる注文の一覧は、HTTPS で暗号化されて運ばれます。

TLS で守られる区間には区切りがある

物理的な配置図を簡略化。太い実線＝ケーブル、灰色の破線＝電波。黄色の破線の枠＝TLS で保護される区間。サービス側は一般的な構成例です。

TLS で保護される区間（端末～入口）



入口から先は別の通信。
暗号化するかは設計による

⚠ 注意

鍵マークは「通信路が保護されている」という意味で、サービスそのものが安全・正しいことの保証ではありません。また、家のルーターの NAT は TLS とは別の仕事です。

NTP：機器の時刻を合わせる

NTP（Network Time Protocol）は、ネットワークを通じて時刻サーバーから正しい時刻を受け取り、機器の時計を合わせる約束です。スマホや PC は、定期的に自動で時刻を合わせています。

証明書の期限

証明書には有効期限がある。時計がずれていると、本物の証明書を「期限切れ」と誤って判断することがある。

ログの順番

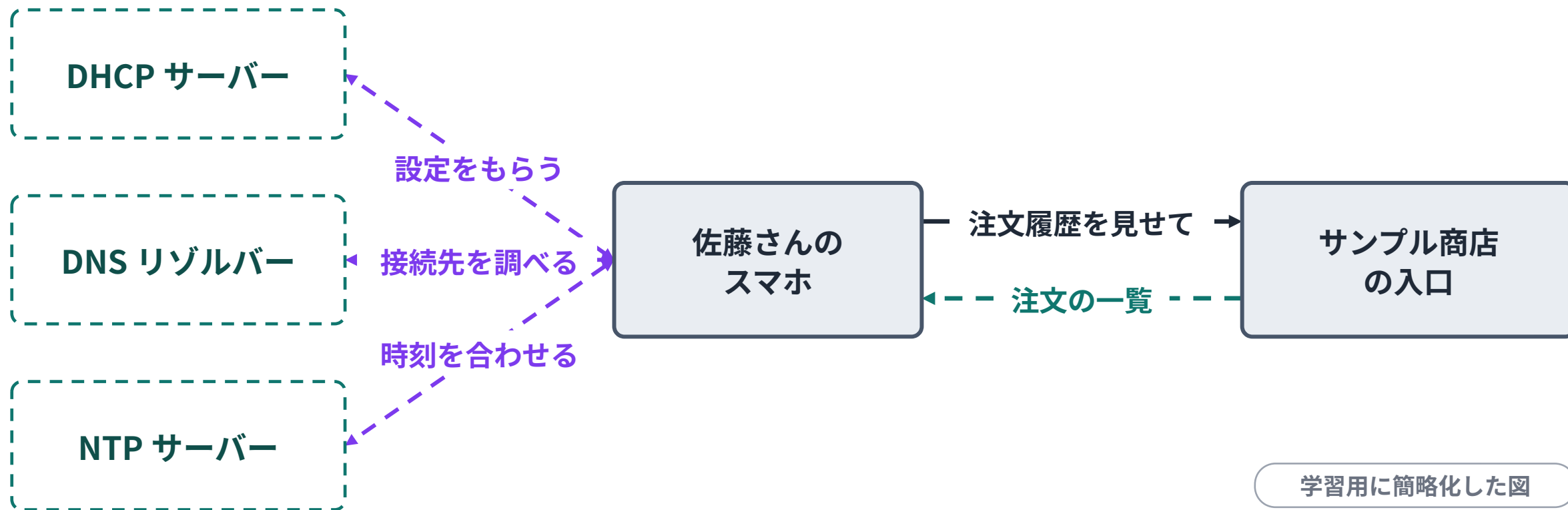
複数の機器の記録（ログ）を時刻で並べて原因を探す。時計がずれると順番が狂う。

認証の仕組み

時刻を使って本人確認をする仕組みがある。時計がずれると確認に失敗する。

DHCP ・ DNS ・ NTP は補助の通信。本文の通り道ではない

役割の関係図。紫の破線＝補助の通信（本文とは別の時期・別の相手）、黒の矢印＝本文の依頼、緑の破線＝本文の返事。補助の相手は本文の通り道に入っていません。



DHCP の機能はルーターの箱の中にあることが多いですが、その機能自体は本文を中継しません。本文は **HTTPS** で入口へ運ばれます。

補助の仕組みと NAT ・ TLS を一覧で比べる

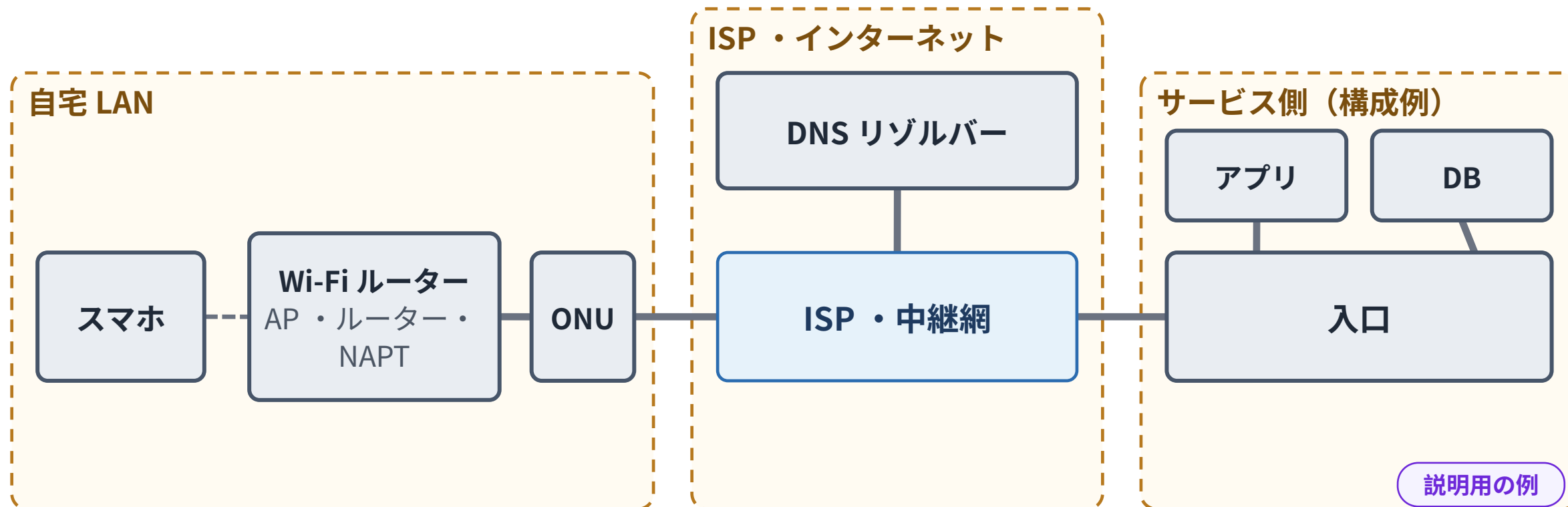
	DHCP	DNS	NTP	NAT	TLS
役割	設定の配布	名前→ IP	時刻合わせ	IP ・ ポートの変換	暗号化 ・ 相手確認
いつ	接続時 ・ 更新時	接続先を初めて使うとき等	定期的	外への通信のため	接続の開始時に合意
どこで	ルーター等	リゾルバー等	時刻サーバー	ルーター	端末と入口の間

DHCP ・ DNS ・ NTP は本文とは別の相手と行う補助の通信です。 NAT は本文が通るルーターで付け札の番号を書き換える仕事、 TLS は本文そのものを守る仕事です。 5 つとも役割が違うので、混ぜずに区別します。

全手順の舞台：家からサービスまでの配置

ここまでの部品をつなげ、「注文履歴を見る」から表示までを数ページで追います。

簡略化した物理的な配置図。破線の大枠＝場所、灰色の箱＝機械、青い箱＝事業者の網。太い実線＝ケーブル、灰色の破線＝電波。



端末からサービスまでの全手順（8段階）

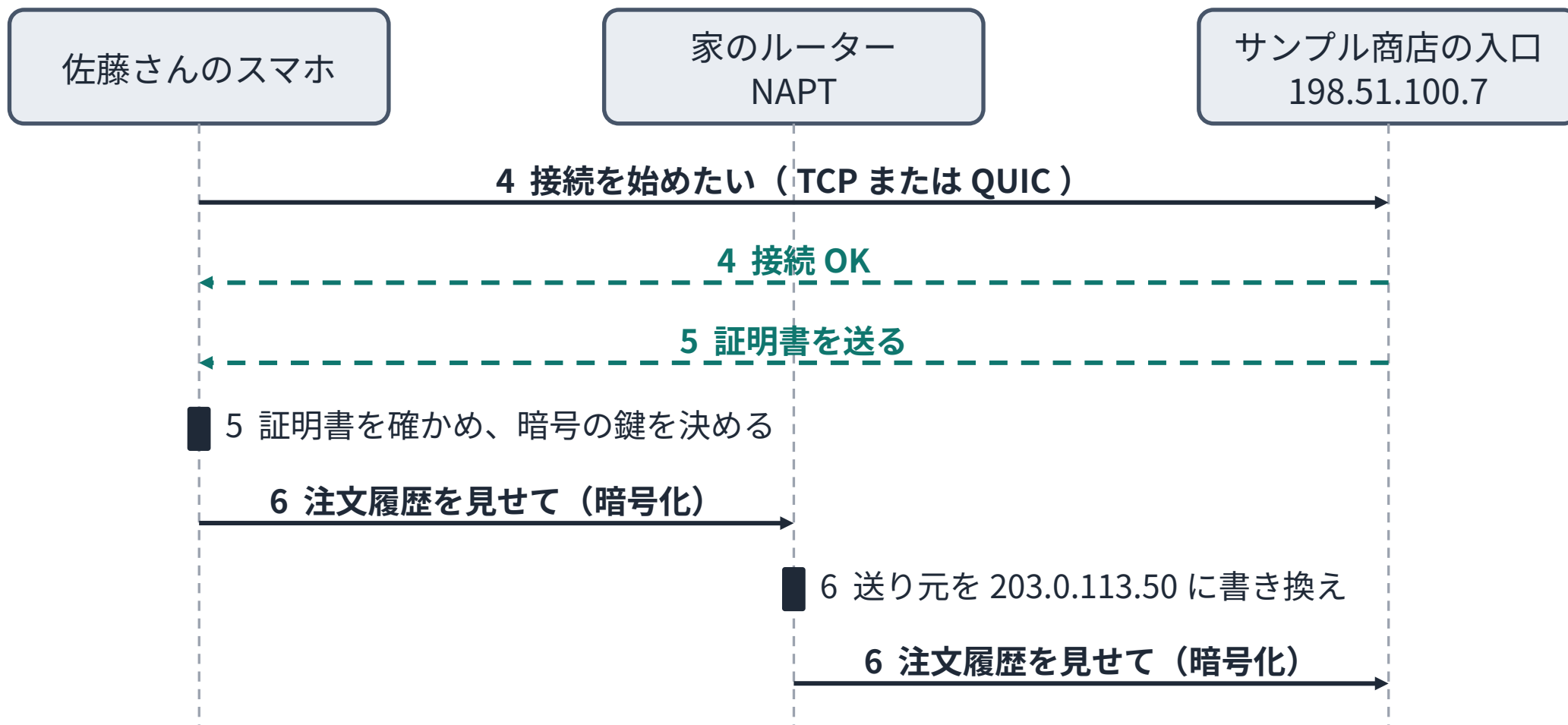
段階	何をするか（佐藤さんの例）	種類
1 Wi-Fi 接続	アクセスポイントに参加し、暗号化した無線でつなぐ	準備
2 DHCP	IP アドレス・出口・DNS サーバーの設定を受け取る	補助
3 DNS	shop.example.com から 198.51.100.7 を調べる	補助
4 接続の確立	TCP または QUIC で、入口との接続を始める	本文の準備
5 TLS	証明書を確認、暗号化の鍵を決める	本文の準備
6 依頼を送る	「注文履歴を見せて」を HTTPS で送る（途中で NAPT）	本文
7 サービス側の処理	誰からの依頼か確認、アプリが DB から注文を探す	サービス内部
8 表示	返ってきた注文の一覧を受け取り、画面に表示する	本文の返事

全手順①：準備（1 Wi-Fi 接続・2 DHCP・3 DNS）



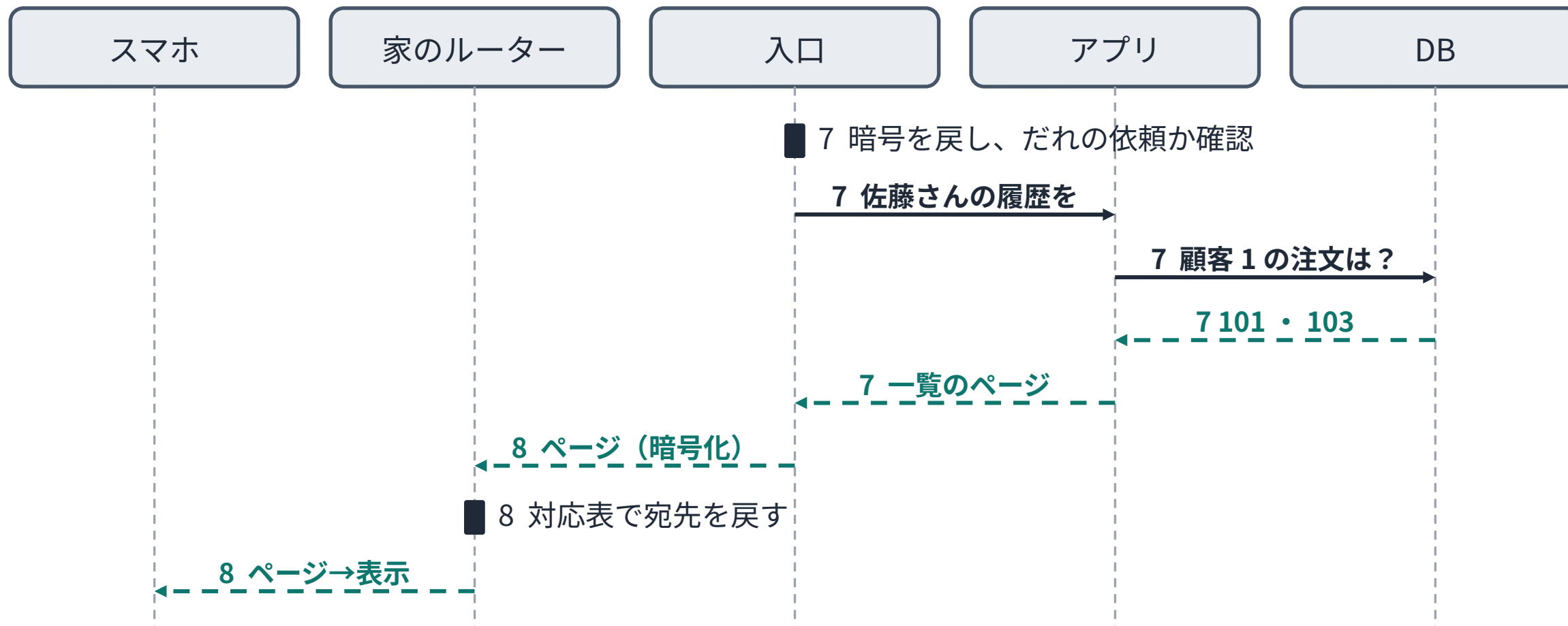
時間順のやり取り図。紫の破線＝本文の前に行う準備・補助のやり取り。まだ注文履歴の依頼は送っていません。DNSの問い合わせは、家のルーターが取り次ぐこともあります。

全手順②：接続・TLS・依頼の送信（4～6）



黒＝本文と接続、緑の破線＝返事。本文は DNS を通らず入口へ直接向かいます。

全手順③：サービス側の処理と表示（7～8）



時間順のやり取り図。黒の矢印＝依頼、緑の破線＝返事。サービス側の分担は一般的な構成例で、第7章で詳しく見ます。

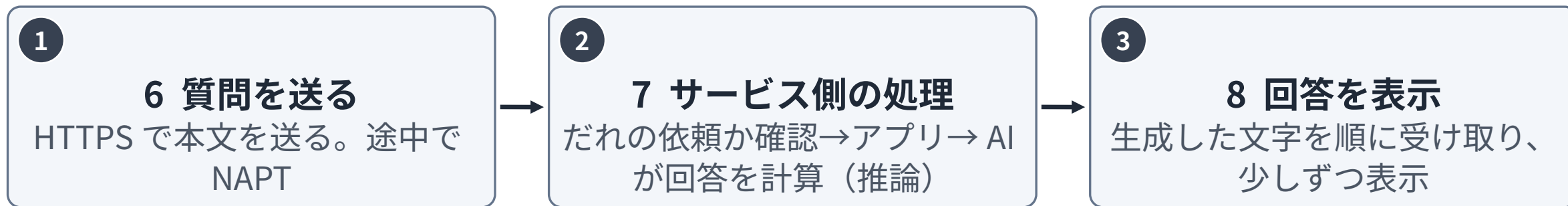
1 ～ 5 は、毎回やり直すとは限らない

段階	2 回目以降の通信では
1 Wi-Fi 接続	つながったままなら、やり直さない
2 DHCP	受け取った設定を期限まで使う。期限が切れる前に更新する
3 DNS	結果がキャッシュされ、覚えている時間のうちは調べ直さない
4 接続・5 TLS	一度作った接続をしばらく再利用する。そのあいだは手順を省ける

そのため、2 回目に「注文履歴を見る」を押したときは、多くの場合 6 ～ 8 だけで表示されます。最初の表示が少し遅く、2 回目から速く感じるのは、このためでもあります。

同じ流れは AI サービスでも使われる

対話型の AI サービスに質問するときも、1～6 までの流れは同じです。違うのは 7 と 8 です。



⚠ 注意

ChatGPT などの実際の内部構成は、公開されていない部分があります。ここでは一般的な構成例として描いています。推論（学習済みの AI で答えを計算すること）は第 12 章で扱います。

第6章前半のまとめ

✓ ここまでで説明できること

0 と 1 を信号にして送る仕組みと、3 種類のネットワーク図の読み方
家庭の機器・ISP・インターネットの配置と、1 台に複数の機能が入ること
IP・MAC・ポートの区別、補助の通信、NAPT・TLS の役割
「注文履歴を見る」が届いて表示されるまでの 8 段階

→ 次へのつながり

後半では、携帯電話の通信、通信の速さや遅れを表す指標、オンラインゲームの通信、
「遅い・つながらない」ときの切り分けを扱います。第7章では、入口から先のサービス側（Web・アプリ・DB）の分担を詳しく見ます。

ネットワークの用語を整理する（1/2）

用語	意味	区別・注意	詳しく
LAN / WAN	近い範囲の網／離れた拠点をつなぐ網	インターネットは多数の網の相互接続	p.205
スイッチ／ルーター／AP	同じ LAN の中継／網と網の中継／無線の接続	家庭用 Wi-Fi ルーターは複数機能を 1 台に	p.217
ONU / モデム・ISP	回線の信号変換装置／接続を提供する事業者	Wi-Fi 接続とインターネット到達は別	p.213
IP ・ MAC ・ ポート	最終の相手／この区間の相手／機器内のプログラム	MAC は区間ごとに付け替わる	p.231
サブネット・デフォルトゲートウェイ	同じ LAN とみなす範囲／外への出口	/24 は 255.255.255.0	p.234

ネットワークの用語を整理する（2/2）

用語	意味	区別・注意	詳しく
ARP / NDP	IP に対応する MAC を調べる （IPv4 / IPv6）	LAN 外宛てはルーターの MAC を調べる	p.237
DHCP ・ DNS ・ NTP	設定配布・名前解決・時刻合 わせ	本文の通り道ではない補助の 通信	p.266
NAT / NAT	IP（とポート）を書き換えて 対応付ける	暗号化でも名前解決でもない	p.253
TCP ・ UDP ・ QUIC ・ TLS	届け方の約束と、暗号化・相 手確認	QUIC は UDP の上で再送・暗 号化を行う	p.258

確認の問い（第 6 章前半）

自分の言葉で答えてから、次のページの解説を読みましょう。

- 1 依頼の前に行う補助の通信を 2 つ挙げてください。
- 2 DNS で番号を調べた後、依頼は DNS サーバーを通りますか。
- 3 HTTPS の鍵マークが出ていれば、そのお店は安全ですか。

確認の問い（第 6 章前半）：解説

前のページの問いへの答えと、考え方です。

✓ 問 1 依頼の前に行う補助の通信を 2 つ挙げてください。

DHCP で IP アドレス・出口・DNS サーバーの設定を受け取り、DNS で shop.example.com の番号（198.51.100.7）を調べます。 p.270

✓ 問 2 DNS で番号を調べた後、依頼は DNS サーバーを通りますか。

通りません。家のルーターや ISP の中継機器は通りますが、本文は調べた番号の入口へ直接向かいます。 p.251

✓ 問 3 HTTPS の鍵マークが出ていれば、そのお店は安全ですか。

言えません。鍵マークは通信路が TLS で保護されていることを示すだけで、サービスそのものの安全さの保証ではありません。 p.264

確認の問い（第 6 章前半・続き）

自分の言葉で答えてから、次のページの解説を読みましょう。

- 1 家庭のルーターが NAT で書き換えるものは何ですか。
- 2 TCP と UDP の届け方の違いは何ですか。

確認の問い（第6章前半・続き）：解説

前のページの問いへの答えと、考え方です。

✓ 問1 家庭のルーターが NAT で書き換えるものは何ですか。

外へ出るパケットの送り元の IP アドレスとポート番号です。返事是对応表で元の端末へ戻します。暗号化はしません。 p.253

✓ 問2 TCP と UDP の届け方の違いは何ですか。

TCP は順番の確認と送り直しで確実に届けます。UDP はそれをせず軽く送り、音声・映像や DNS の多くで使われます。 p.259

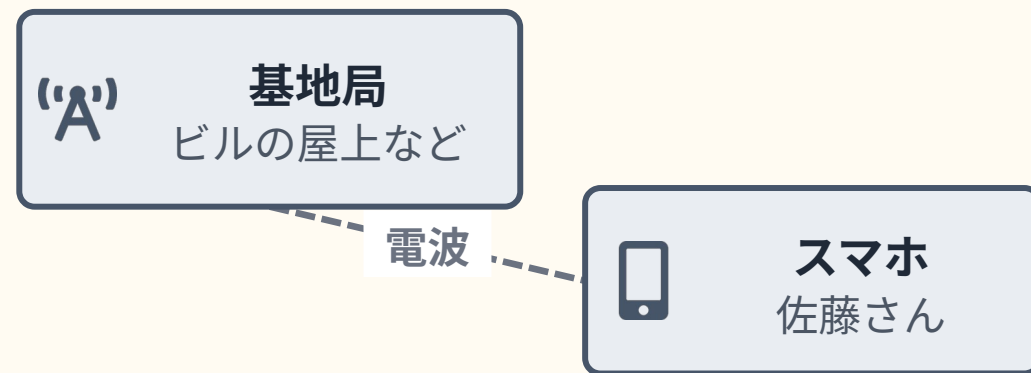
家の外では、スマホは基地局とつながる

外出先で佐藤さんがスマホの「注文履歴を見る」を押すとき、家のアクセスポイント（AP）は近くにありません。代わりにスマホは、携帯電話会社が街に置いた**基地局**と電波で通信します。

基地局は、電波を送受信するアンテナと、電波を有線の回線へつなぐ装置の組です。ビルの屋上・鉄塔・駅の構内などに置かれ、周りの一定の範囲を受け持ちます。

物理的な配置図。灰色の太枠＝機械、破線＝電波（無線の接続）。

街の中（屋外）

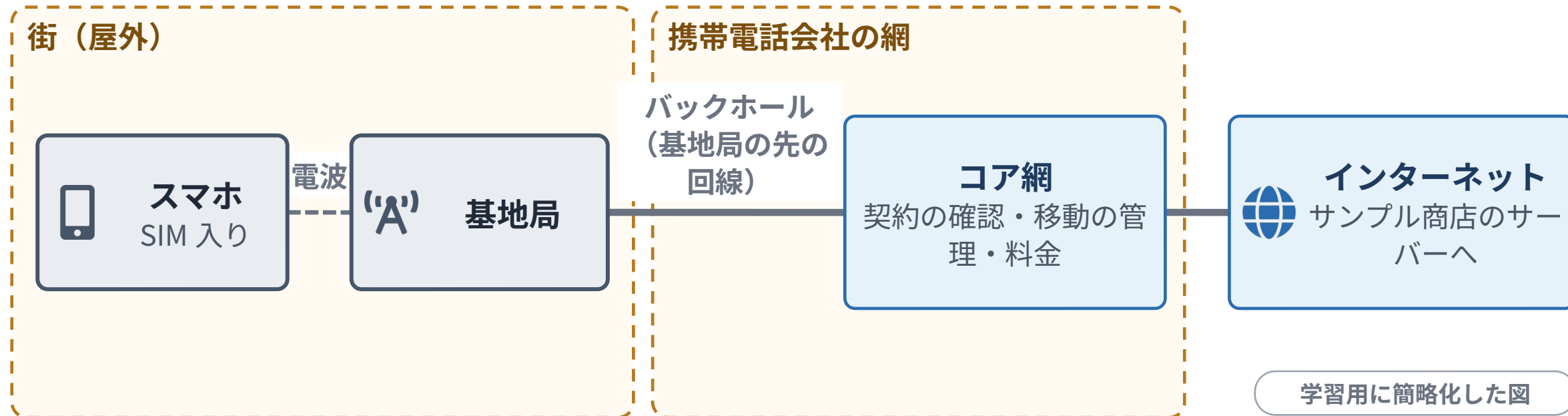


🔑 家との対応

家では AP が電波の受け口でした。外では基地局がその役をします。

携帯の通信が外へ出るまでの道すじ

物理的な配置図。破線＝電波、灰色の太線＝ケーブルなどの有線の回線。左から右へ、佐藤さんの依頼が外へ出ていく順に並べています。



基地局の先は有線の回線（**バックホール**）で携帯電話会社の中心の設備（**コア網**）へつながり、そこからインターネットへ出ます。4G・5Gは携帯電話の通信方式の世代の呼び名で（4Gは第4世代、5Gは第5世代）、どちらもこの道すじは同じです。

区間ごとに、誰が何をしているか

区間・設備	何をするか	家の通信に近い役
電波	スマホと基地局がデータを電波で送り合う	Wi-Fi（スマホ～AP）
基地局	電波を受け取り、有線の回線へ渡す	AP（アクセスポイント）
バックホール	基地局とコア網を結ぶ回線	家の外の回線
コア網	SIMで契約を確認、移動の管理、料金の計算	ISP（利用者の確認）

！ たとえの限界

右の列は目安です。たとえば「移動の管理」は家の通信にはない、携帯電話ならではの仕事です。

SIM：契約者を確認するための情報

SIM（しむ）

分類：部品

略語：SIM = Subscriber Identity Module（加入者を識別する部品）

何か	携帯電話の契約者（加入者）を識別する情報を持つ部品。スマホに差す小さなカード型のほか、本体に組み込まれた型（eSIM）もある。
なぜ必要か	携帯電話会社が「この通信はどの契約のものか」を確認め、つないでよいか、料金を誰に請求するかを決めるため。
どこにあるか	スマホの中（カードを差す口、または本体に内蔵）。
受け取る→する→返す	通信を始めるとき、コア網からの確認に対して、SIM の情報で応える → 契約が確認されると通信できる。

SIM の情報で、通信してよい契約かを確かめる

役割の関係図。黒い矢印＝送る情報、緑の破線の矢印＝返事（要点だけの図）。



📋 具体例

機種を変えるとき SIM を新しいスマホへ差し替えると、同じ契約・電話番号で使えます。SIM は端末ではなく契約に結び付くからです。

⚖️ 似た言葉との違い

Wi-Fi のパスワードは「AP につなぐ許可」で、契約の確認とは別です。

自宅の固定回線と 4G ・ 5G を比べる

「固定回線」は、光回線のように家まで引いた、決まった場所の回線です。

	自宅の固定回線	4G ・ 5G
最初の接続	Wi-Fi で AP ・ ルーターへ	電波で基地局へ
外への出口	ルーター→ ONU→ISP	携帯電話会社のコア網
利用者の識別	回線の契約 ・ ISP の認証（利用者の確認）	SIM の情報で契約を確認
速さに影響するもの	家の Wi-Fi ・ 回線 ・ 混雑	電波の強さ ・ 混雑 ・ 周波数 ・ 契約

⚠ 注意

「5G」と表示されていても、速さや反応の速さが一定に保証されるわけではありません。電波の強さや混み具合で変わります。

地下で電波が弱くなるのはなぜか

断面のつमりの模式図。破線＝電波、灰色の太線＝ケーブル。地上の基地局から地下へ向かう電波は、土やコンクリートを通るうちに弱まります。



電波は土やコンクリートで弱まり、金属に当たると反射します。そのため地下街や駅には、電波を届けるための専用のアンテナ設備が置かれることがあります。

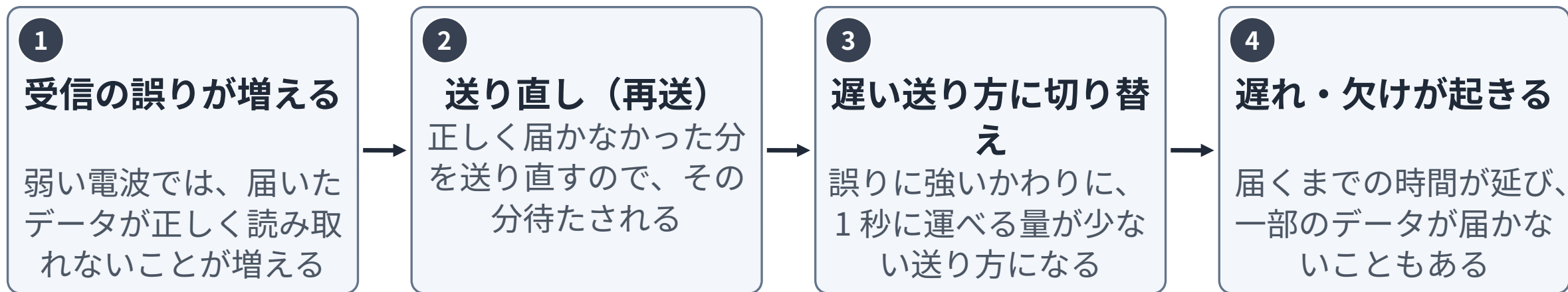
電波が届きにくくなる 6 つの原因

地下だけでなく、建物の奥やエレベーター、人の多い場所でも電波の状態は悪くなります。

原因	起きていること	場所の例
遮蔽物（さえぎる物）	土やコンクリートで電波が弱まる	地下・建物の奥
金属	電波を反射して届きにくい	エレベーター
距離	基地局や AP から遠い	受け持つ範囲の端
建物の構造	入口や窓から入る電波が奥まで届かない	地下街・半地下
干渉	他の電波が重なって受信の邪魔をする	混雑した Wi-Fi
設備の配置	その場所を受け持つ設備がない	山間部など

電波が弱いと「つながっているのに遅い」

アンテナの表示が出ていても、遅く感じる場合があります。電波が弱いと、次のことが重なるからです。

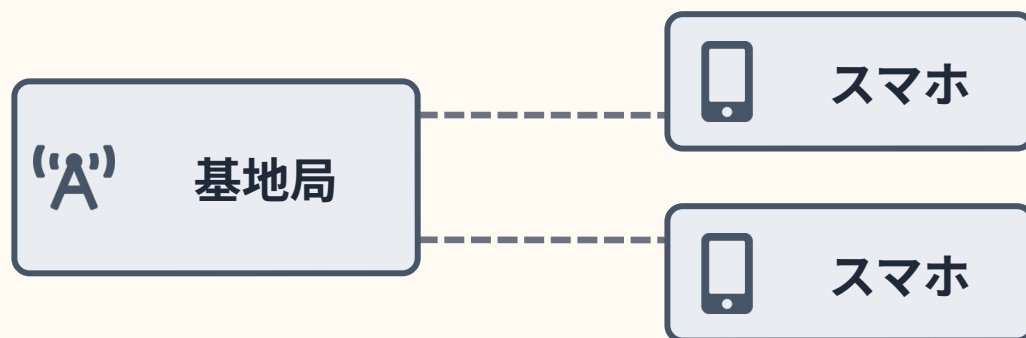


このように、再送・低速化・届くまでの時間の増加・データの欠けが重なった結果が「つながっているのに遅い」状態です。「届くまでの時間」と「データの欠け」には正式な名前があり、p.295 からの数ページで説明します。

電波が強くても、混雑していると遅くなる

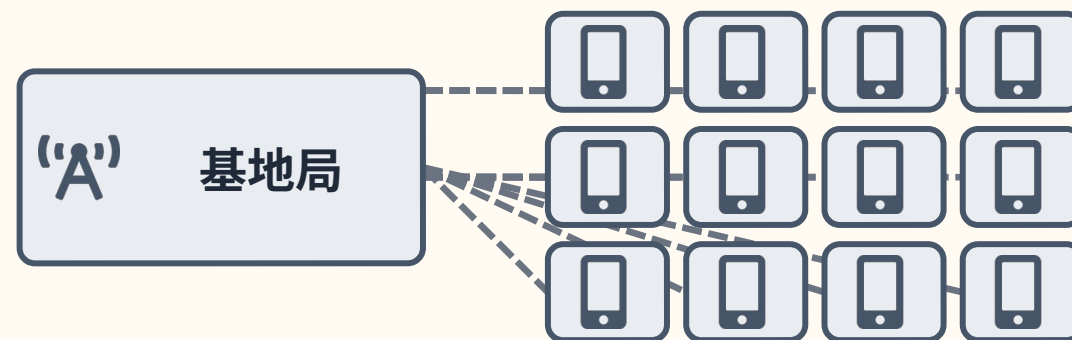
物理的な配置図を単純にしたもの（学習用）。破線＝電波。基地局が運べる量は、同じ範囲にいる利用者と分け合います。

利用者が少ない



1人あたりの取り分が大きい

利用者が多い（駅・イベント）



同じ能力を多くの人で分ける

🔧 説明用の例（単純化した数）

基地局が1秒に運べる量を仮に「100」とすると、2人なら1人あたり約50、50人なら約2です。実際の分け方は方式や状況で変わりますが、人が増えるほど取り分が減る点は同じです。

周波数：電波の種類によって届き方が違う

電波は波の一種で、1秒間に何回振動するかを**周波数**といいます（単位はHz：ヘルツ）。第2章のクロック周波数と同じく「1秒あたりの回数」という意味です。携帯電話会社は、いくつかの周波数の電波（周波数帯）を使い分けています。

周波数	届き方	運べる量
低い周波数	遠くや建物の中まで届きやすい	比較的少なめ
高い周波数	障害物に弱く、届く範囲が狭い	多く運びやすい

ポイント

電波の強さだけでは速さは決まりません。どの周波数の電波につながっているか、何人で分け合っているか（混雑）も影響します。

確認の問い（携帯の通信と電波）

自分の言葉で答えてから、次のページの解説を読みましょう。

- 1 外出先のスマホの依頼は、どんな設備を通して外へ出るか。
- 2 電波の表示が出ているのに遅いとき、考えられる原因を2つ挙げよ。

確認の問い（携帯の通信と電波）：解説

前のページの問いへの答えと、考え方です。

✓ 問1 外出先のスマホの依頼は、どんな設備を通して外へ出るか。

スマホ→（電波）→基地局→バックホール（基地局の先の回線）→携帯電話会社のコア網→インターネットの順です。コア網はSIMの情報で契約を確かめます。 p.283

✓ 問2 電波の表示が出ているのに遅いとき、考えられる原因を2つ挙げよ。

例：電波が弱く、再送や遅い送り方への切り替えが起きている（地下・建物の奥など）。同じ基地局を多くの人で分け合っていて混雑している。周波数の違いも影響します。 p.290

帯域と遅延：「速い」には2つの意味がある

「この回線は速い」というとき、実は2つの違う意味が混ざっています。

⇄ 帯域：一度に運べる量

1秒あたりに運べるデータの量です。単位はMbps（1秒に100万ビット）など。大きいほど、動画や大きなファイルが早く届きます。「回線が速い」はこちら。

🕒 遅延：届くまでの時間

データを送ってから相手に届くまでの時間です。単位はms（ミリ秒＝1000分の1秒）。小さいほど、操作への反応が早くなります。「反応が速い」はこちら。

💡 たとえ

道路にたとえると、**帯域**は車線の数（一度に通れる車の量）、**遅延**は目的地までの所要時間です。車線を増やしても、遠い場所へ行く時間は短くなりません。

帯域・遅延と、似た言葉の違い

！ たとえの限界

実際の遅延には途中の機器での待ち時間も含まれます。たとえば「量」と「時間」を分けるためだけのものです。

⚡ 似た言葉との違い

スループット（第2章）は実際に運べた量です。帯域は「利用できる容量」で、混雑などで実際はそれより少なくなることがあります。

第2章のレイテンシー（応答時間）も待ち時間を表す言葉で、通信では遅延とほぼ同じ意味です。

例：契約が「最大 1Gbps」でも、1秒に 100M ビットしか進まなければ、そのときのスループットは 100Mbps です。

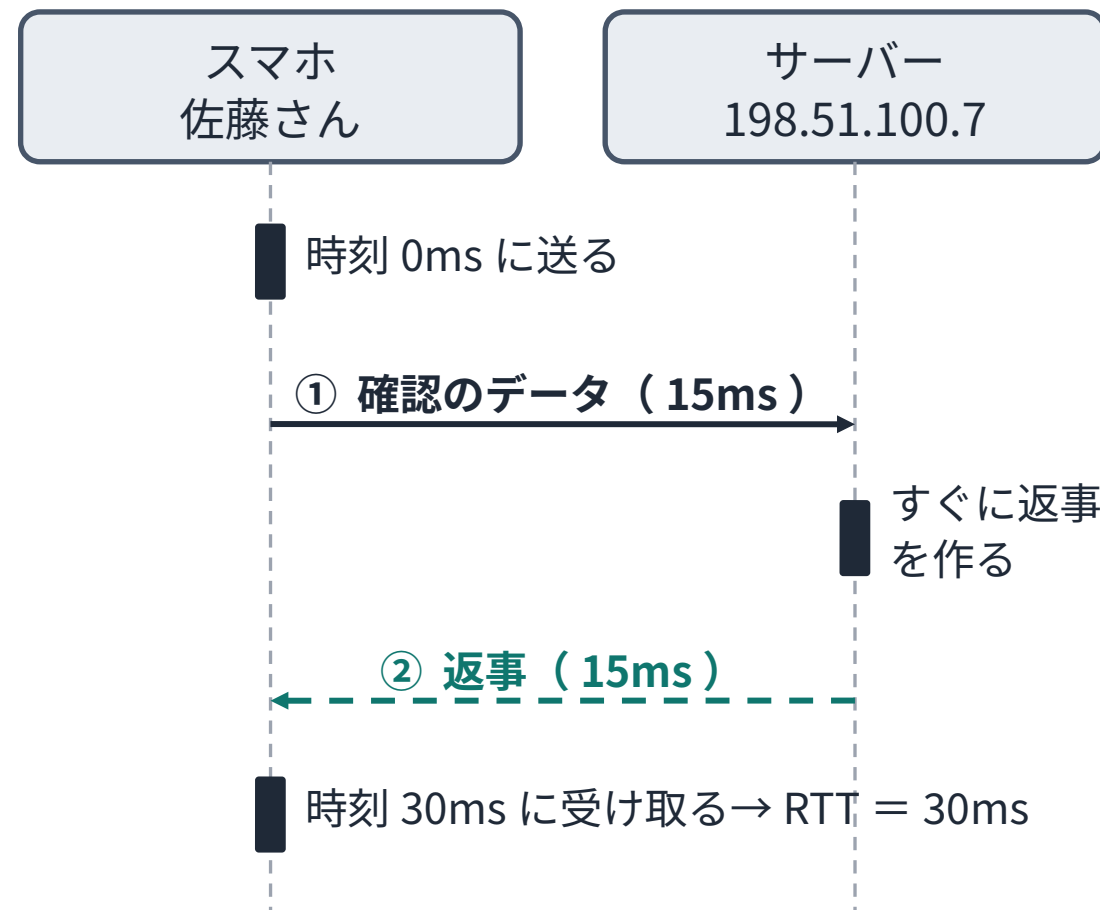
RTT：行きと帰りを合わせた時間

RTT は Round Trip Time の略で、日本語では往復時間です。データを送ってから、相手の返事が戻ってくるまでの時間を表します。

右の図は説明用の例です。行きに 15ms、帰りに 15ms かけると、RTT は 30ms です。

⚠ 注意

RTT を 2 で割っても、正確な片道の時間とは限りません。行きと帰りで通る経路や混み具合が違ふことがあるからです。



上から下へ時間が進む図（説明用の例）

ジッターとパケットロス：ばらつきと欠け

ジッターは遅延のばらつきです。下の表は、5つのパケットが届くまでの時間の説明用の例です。平均はどちらも約 50ms でも、回線 D では毎回の時間が大きく変わります。

パケットの番号	1	2	3	4	5
回線 C（ばらつき小）	50ms	51ms	49ms	50ms	50ms
回線 D（ばらつき大）	20ms	80ms	35ms	95ms	20ms

パケットロスは、送ったパケットの一部が途中で失われることです。100 個送って 2 個届かなければ 2 %です。TCP なら送り直すので遅れが生じ、UDP なら欠けたまま進むので音や映像が途切れます。

通信の品質を表す 6 つの指標

指標	意味	単位	悪いときの症状
帯域	利用できる通信の容量	Mbps	大きなファイル・動画が遅い
スループット	実際に運べた量	Mbps	契約の表示の速度より遅い
遅延	届くまでの時間	ms	操作してから反応までが遅い
RTT	往復にかかる時間	ms	ping の値が大きい（ p.313 ）
ジッター	遅延のばらつき	ms	動きがカクつく・ワープする
パケットロス	データの欠け	%	音が途切れる・再送で遅れる

🔑 ポイント

対戦ゲームの操作感は、**帯域**よりも、**遅延**とそのばらつき（**ジッター**）・欠け（**パケットロス**）に強く左右されます。

大容量の回線が、操作しやすいとは限らない

棒の長さ＝往復時間（RTT）。短いほど、操作してから結果が返るまでが早い（説明用の仮定の値）。

回線 A
(大容量・往復 200ms)

200ms

回線 B
(往復 30ms・安定)

30ms

説明用の例

回線 A は一度に運べる量（帯域）が大きくても、往復に 200ms かかります。大きなファイルのダウンロードには向いていても、押すたびに結果を待つ操作では、往復 30ms で安定した回線 B のほうが操作しやすくなります。

⚠ 注意

グラフの数値は特定の回線の実測ではなく、説明のための仮定です。

bps と B/s ：ビットとバイトで 8 倍違う

帯域はビット（b）、ファイルの大きさはバイト（B）で表すことが多いです。

表し方	意味	換算の例
100Mbps	1 秒に 1 億ビット（100 メガビット）運べる	÷8 → 1 秒に約 12.5MB（メガバイト）
12.5MB/s	1 秒に 12.5 メガバイト運べる	×8 → 100Mbps

📖 説明用の例（単純化した計算）

1GB（約 1000MB）を 100Mbps（1 秒に約 12.5MB）で運ぶと、 $1000 \div 12.5 = \text{約 } 80$ 秒。実際は手順のための余分なデータや混雑で、もっとかかります。

⚠️ 注意

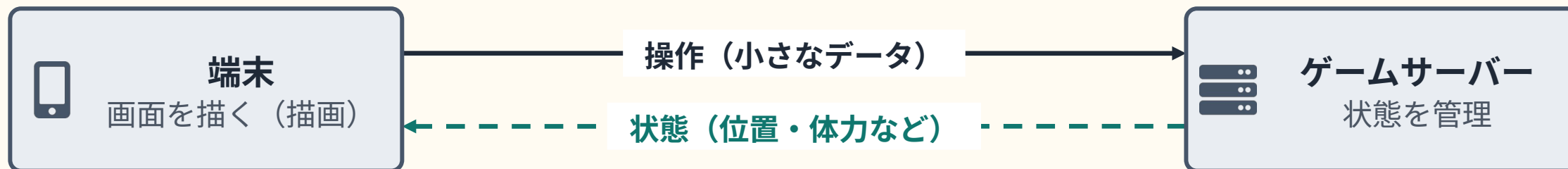
1 バイト = 8 ビット。「100M」とだけあれば、ビットかバイトかを確認めます。

対戦ゲームは操作を、クラウドゲームは映像を送る

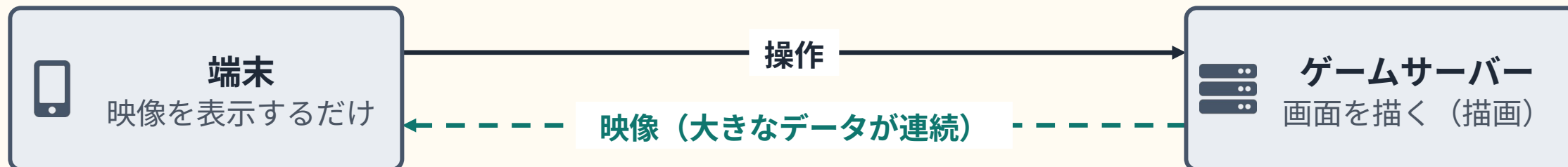
「描画」は、画面に映す絵をその都度作ることです。どこで描画するかで、送るものが変わります。

役割の関係図。黒い矢印＝端末から送る情報、緑の破線の矢印＝サーバーから返る情報。

通常の対戦ゲーム（多くのスマホ対戦ゲーム）



クラウドゲーム



学習用に簡略化した図

2つの方式で、必要な帯域と端末の性能が違う

	通常の大戦ゲーム	クラウドゲーム
送るもの	操作・状態の情報	操作（上り）と映像（下り）
必要な帯域	比較的小少なめ	映像を送り続けるため大きい
端末の性能	描画に必要	表示と映像の再生ができればよい
遅延の影響	操作の反映が遅れる	操作してから映像が変わるまでが遅れる

「上り」は端末からサーバーへ送る向き、「下り」はサーバーから端末へ届く向きです。クラウドゲームは高い性能の端末がなくても遊べるかわりに、映像を途切れずに受け取れるだけの帯域が必要です。どちらの方式でも、遅延が大きいと操作への反応が遅れます。

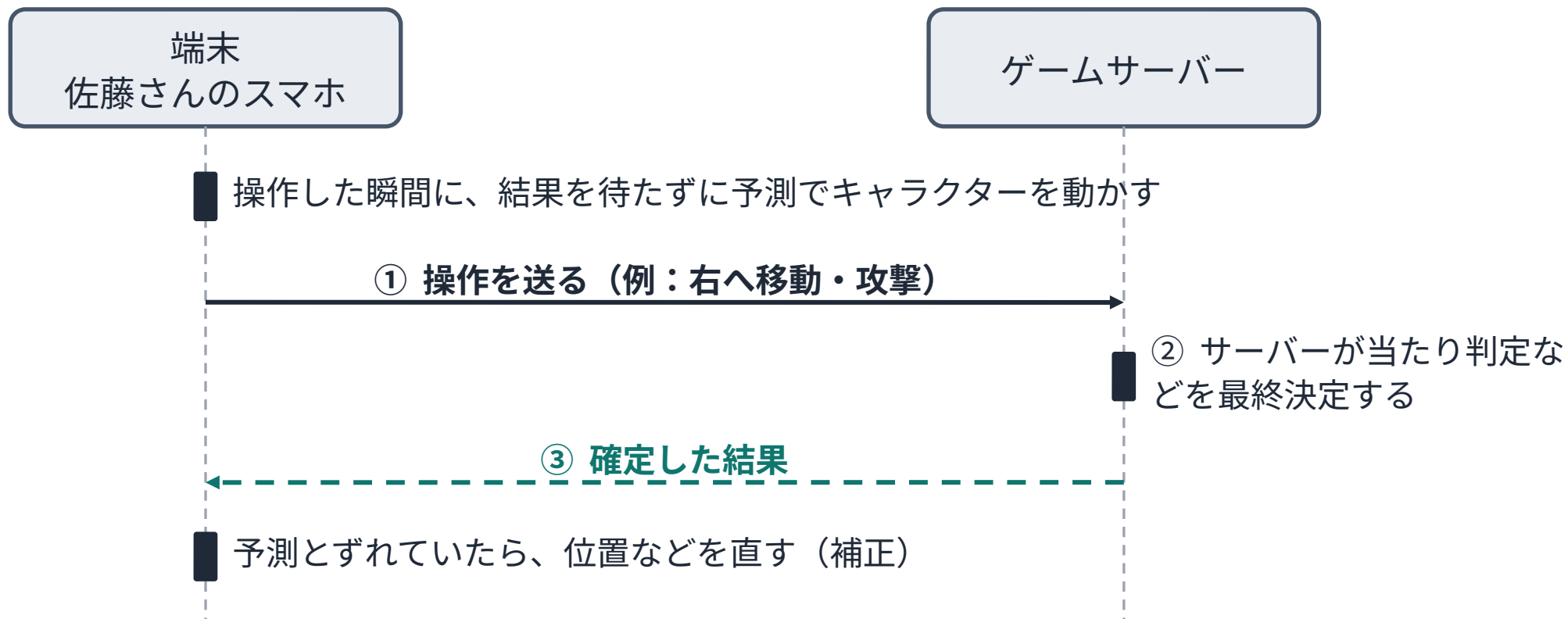
ゲームは場面ごとに別の機能を使う

ゲームを起動してから終わるまでに、通信の相手や中身が場面ごとに変わります。



たとえば更新ファイルの取得では大きなデータを運ぶので**帯域**が効き、対戦中は小さなデータを何度も往復させるので**遅延**が効きます。**マッチング**は、条件の合う対戦相手を探す機能です。1つのゲームでも、場面ごとに別の機能（別のサーバーのこともあります）が働いています。

対戦中はサーバーが判定し、端末は予測で動く



判定を1か所（サーバー）で決めると、対戦する全員の結果がそろいます。ただし結果が戻るまで往復の時間（**RTT**）がかかるので、端末は先に予測で動かし、遅れを感じにくくしています。

公式に確認できること・断定しないこと

前の2ページは、多くのオンラインゲームに見られる一般的な構成例です。個別のゲームについては、確認できた範囲だけを書きます。

会社の説明として確認できること

Brawl Stars（ブロスタ）は世界各地にサーバーを置き、地域ごとにマッチングを行う（開発会社 Supercell の説明。元資料に記載された出典で、今回は開き直していません）。

断定しないこと

ブロスタ・荒野行動の通信方式・使うポート番号・処理の周期・サーバーの構成。今回の資料では、公式の通信の仕様を確認できていません。

「相手が見つかるまで」と「試合中の遅れ」は別

どちらも「待たされる」と感じますが、原因の場所が違います。

	マッチングの待ち時間	試合中の遅延
何の時間か	条件の合う対戦相手が見つかるまでの時間	操作が相手やサーバーに届き、結果が返るまでの時間
主に何で決まるか	その時間帯・地域・条件で遊んでいる人の数と、相手の探し方	遅延・RTT・ジッター・パケットロス（通信の品質）
通信が速くなれば解決するか	しないことが多い	改善することがある

ポイント

症状を「何の時間か」に分けると、通信の問題なのか、それ以外（人数や仕組み）の問題なのかを区別できます。この考え方は、次のページの切り分けにもつながります。

「遅い・つながらない」の切り分け①

問題が起きている場所を、症状と簡単な確認で絞り込むことを切り分けといいます。

疑う場所	よくある症状	確認の方法
端末	その端末だけ遅い・つながらない	再起動する、他の端末と比べる、アプリを更新する
電波・Wi-Fi	部屋を移ると変わる	AP に近づく、5GHz と 2.4GHz を切り替える、有線で試す
混雑	夜や人の多い場所で遅い	時間帯・場所を変えて比べる
家の回線・ISP	家の全端末で遅い・つながらない	ルーター・ONU の表示、ISP の障害情報、契約内容を見る

「遅い・つながらない」の切り分け②

家の外の、名前解決やサービス側の問題もあります（前のページの表の続き）。

疑う場所	よくある症状	確認の方法
名前解決（DNS）	IP アドレスでは届くが、名前では届かない	nslookup で回答を確かめる
サービス側	特定のサービスだけ使えない	提供元の稼働状況、他のサービスは使えるか

ポイント

設定は変えず、まず観察します。表の上の行（自分の端末）から順に、1 つずつ疑う場所を外していくと、原因の場所に近づけます。

切り分けの例：注文履歴が開かない

佐藤さんが家の Wi-Fi でスマホから「注文履歴を見る」を押しても、ページが出ない場面を考えます（説明用の例）。

- ① 他の Web サイトやアプリは使えるか → 使えるなら、家の通信は動いている。サンプル商店のサービス側を疑い、お知らせや稼働状況を見る。
- ② 家の PC でも同じか → PC では開けるなら、スマホ（端末）やスマホの Wi-Fi を疑う。
- ③ Wi-Fi を切って 4G ・ 5G で試す → 開けるなら、家の Wi-Fi ・ 回線 ・ ISP のどこかを疑う。
- ④ PC で `nslookup shop.example.com` を実行 → 回答がなければ、名前解決（DNS）を疑う。

ポイント

まず設定は変えずに観察します。1 回に 1 つの条件だけを変えて比べると、原因の場所を絞り込めます。

コマンドで観察①：自分の設定と近くの機器

Windows のコマンドで、設定は変えずに「見るだけ」で通信の様子を確かめます。

コマンド	見るもの	関連
<code>ipconfig /all</code>	自分の IP アドレス・DNS サーバー・DHCP・デフォルトゲートウェイ	p.244
<code>route print</code>	OS が使う経路の一覧（どの宛先をどこへ渡すか）	前半の経路制御
<code>arp -a</code>	IPv4 で、近くの機器の IP アドレスと MAC アドレスの対応	p.227
<code>netsh wlan show interfaces</code>	Wi-Fi の接続の状態・リンク速度	p.209

⚠ 注意

Wi-Fi のリンク速度は端末と AP の間の速さで、インターネットまでの速さとは別です。

コマンドで観察②：相手までの通信

コマンド	見るもの	関連
nslookup example.com	DNS で調べた IP アドレス	p.247
ping example.com	相手に届くか・往復時間（RTT）	p.313
tracert -d example.com	経路の途中の中継点から返る応答	前半の経路制御
Test-NetConnection example.com -Port 443	TCP の 443 番への接続（PowerShell で実行）	p.258

ping は、相手に小さな確認のデータを送り、返事が戻るまでの往復時間を測るコマンドです。結果の読み方を次のページで見ます。

 注意

443 番に接続できても、サービスの全機能が正常とは限りません。

ping の結果を読む

下は ping の結果の説明用の例です（表示を一部簡略化しています）。

```
> ping shop.example.com
198.51.100.7 からの応答: 時間=28ms
198.51.100.7 からの応答: 時間=31ms
(あと2回、同じ形の行)
送信 = 4、受信 = 4、損失 = 0 (0% の損失)
最小 = 27ms、最大 = 31ms、平均 = 28ms
```

shop.example.com へ確認のデータを送る
1 回目の往復時間（RTT）は 28 ミリ秒
2 回目は 31 ミリ秒
既定では 4 回送る
4 個送って 4 個返ってきた＝パケットロスなし
最小と最大の差が小さい＝ジッターが小さい

⚠ 注意

返事をしない設定のサーバーもあるので、ping に返事がない＝止まっている、とは限りません。

第6章のまとめ（後半）

✓ ここまでで説明できること

- ・ 自宅からサービスまでの通信の順番と、DHCP・DNS・NAT・TLS・NTPの役割の違い（前半）
- ・ 外出先のスマホは基地局→バックホール→コア網を通して外へ出ること。SIMは契約を確かめる情報
- ・ 地下・混雑・周波数で、つながっていても遅くなる理由
- ・ 帯域（量）と遅延（時間）の違い、RTT・ジッター・パケットロスの意味
- ・ 症状から疑う場所を分ける切り分けの考え方

→ 次へのつながり

第7章では、第5章のデータベースとこの章の通信をつなぎ、佐藤さんの「注文履歴を見る」がサービス側でどう処理されるかを完成させます。

確認の問い（第 6 章後半）

自分の言葉で答えてから、次のページの解説を読みましょう。

- 1 「1Gbps の回線なのにゲームの反応が遅い」のはなぜか。
- 2 ping の結果から分かる 3 つのことは何か。
- 3 家のスマホでも PC でも開けず、4G ・ 5G では開けた。どこを疑うか。

確認の問い（第6章後半）：解説

前のページの問いへの答えと、考え方です。

✓ 問1 「1Gbpsの回線なのにゲームの反応が遅い」のはなぜか。

1Gbpsは帯域（1秒に運べる量）の値です。反応の速さは遅延・RTT（届くまで・往復の時間）で決まり、ジッターやパケットロスでも悪くなります。p.300

✓ 問2 pingの結果から分かる3つのことは何か。

往復時間（RTT）、返事が戻らなかった割合（パケットロス）、最小と最大の差から見るばらつき（ジッターの目安）です。p.313

✓ 問3 家のスマホでもPCでも開けず、4G・5Gでは開けた。どこを疑うか。

端末やサービス側ではなく、家のWi-Fi・ルーター・ONU・回線・ISPのどこかです。機器の表示やISPの障害情報を確認めます。p.310

第 7 章

Web サービスの仕組み

この章で学ぶこと

佐藤さんが「注文履歴を見る」を押してから一覧が出るまで、サンプル商店の側では Web サーバー・アプリケーションサーバー・DB サーバーが仕事を分担しています。それぞれの担当と、受け渡す情報（HTTP の要求・SQL・検索結果・HTML）を追います。

→ 前の章とのつながり

第 5 章でデータベースと DB サーバーを、第 6 章で要求と応答がどの道を通って届くか（IP アドレス・DNS・TLS など）を学びました。この章はその 2 つをつなぎ、注文履歴の全体像を完成させます。

Web とは：ブラウザーでページを見る仕組み

Web (う え ぶ)

分類：サービス

略語：WWW = World Wide Web

何か	ブラウザーで文書（Web ページ）を見たり、リンクをたどって別のページへ移ったりできる仕組み。
なぜ必要か	どの機器のブラウザーからでも、同じ書き方のページを同じ手順で取り寄せて表示できるようにするため。
どこにあるか	インターネットの上で動くサービスの1つ。ページを返すのはサービス側、表示するのは利用者のブラウザー。
身近な例	サンプル商店のサイト（shop.example.com）、ニュースサイト
似た言葉との違い	インターネットは機器どうしをつなぐ網そのもの（p.206）。Web はその上のサービスの1つで、メールなども別のサービス。

Web ページの中身：HTML（構造）とCSS（見た目）

Web ページは、文章の並びや見出しなどの構造を書いた **HTML** と、色や大きさなどの見た目を指定する **CSS** などできています。ブラウザはそれを受け取って画面に描きます。

HTML の例（説明用）

```
<h1> 注文履歴 </h1>
<ul>
  <li>101 ノート 3 3,000 円 </li>
  <li>103 ファイル 5 5,500 円 </li>
</ul>
```

大見出し「注文履歴」
ここから箇条書き
1 つ目の項目（注文 101）
2 つ目の項目（注文 103）
箇条書きの終わり

```
h1 { color: navy; }
```

CSS の例：大見出しの文字を紺色にする

HTML と CSS は書き方の約束で、注文の検索やデータの管理はしません（それはアプリと DB の仕事）。

URL ： どのサーバーの、どのページかを示す書き方

ブラウザーに「どのサーバーの、どのページがほしいか」を伝える書き方が **URL** です。
注文履歴のページを例に、部分ごとに読みます（説明用の例）。

https://shop.example.com/orders

部分	意味
https://	使う通信の約束。 HTTPS =次に学ぶ HTTP を TLS で保護したもの（ p.262 ）
shop.example.com	サーバーの名前（ ドメイン名 ）。 DNS で IP アドレスを調べる（ p.247 ）
/orders	そのサーバーの中の、どのページか（ここでは注文履歴）

HTTP：要求と応答の書き方の約束

ブラウザーと Web サーバーが要求と応答をやり取りするときの約束（**プロトコル**）が **HTTP** です。要求には「何をしたいか（GET = ください）」「どのページか」「どのサーバーあてか」などを書きます。

HTTP の要求の例（説明用。HTTP/1.1 の書き方）

```
GET /orders HTTP/1.1
Host: shop.example.com
Cookie: session=abc123
```

「/orders のページをください」
あて先のサーバーの名前
ログイン時に受け取った「ログイン済みの印」

補足

サーバーは「ログイン済みの印」（説明用の値）から、この要求が佐藤さん（顧客番号 1）からだと判断します。URL に顧客番号を書く作りにすると、他人の番号に書き換えて盗み見される危険があるため、本人の確認はサーバー側で行います。

HTTP の応答：結果の番号と、ページの本体

HTTP の応答の例（説明用）

```
HTTP/1.1 200 OK
Content-Type: text/html

<h1> 注文履歴 </h1> ...
```

「成功しました」（200 = 成功）
本体は HTML のページです
（空行の後が本体）
注文 101 と 103 を並べた HTML

応答の 1 行目には、結果を表す番号（ステータスコード）が入ります。200 は成功、404 は「そのページは見つからない」、500 は「サーバー側で処理に失敗した」という意味です。

⚠ 注意

HTTPS（p.262）では、この中身は TLS で暗号化されて運ばれ、途中では読めません。ここでは暗号化する前の中身を、人が読める形（HTTP/1.1 の書き方）で示しました。HTTP/2 や HTTP/3 は運ぶ形式が違いますが、要求と応答の意味は同じです。

Web サーバー：要求の受付と、決まったファイルの返却

Web サーバー（うぇぶさーばー）

分類：役割

何か	ブラウザからの HTTP の要求を受け付け、ページや画像を返す役割。
なぜ必要か	多数のブラウザからの要求を最初に受け止め、決まったファイルはすぐ返し、それ以外を担当へ渡すため。
受け取る→する→返す	HTTP の要求を受け取る → 画像・CSS などのファイルを返す、またはアプリケーションサーバーへ転送する → HTTP の応答を返す
身近な例	Web サーバーのソフトの例：Apache HTTP Server、nginx

決まったファイルを返す（静的）／その都度作る（動的）

静的コンテンツ

誰が見ても同じ中身のファイル。前もって置いておき、要求が来たらそのまま返す。

- ・ サンプル商店のロゴ画像
- ・ 見た目を決める CSS

動的コンテンツ

要求のたびにプログラムが中身を作るページ。見る人で中身が変わる。

- ・ 佐藤さんの注文履歴（101 と 103）
- ・ 鈴木さんが見れば鈴木さんの注文

ポイント

静的コンテンツは Web サーバーだけで返せます。動的コンテンツには、DB を調べてページを組み立てるプログラムが必要です。その仕事を受け持つのが、次のアプリケーションサーバーです。

アプリケーションサーバー：業務の処理を受け持つ

アプリケーションサーバー（あぷりけーしょんさーばー）

分類：役割

何か	業務の処理（ログイン確認・在庫確認・料金計算・検索の指示など）をするプログラムを動かし、動的なページを作る役割。
なぜ必要か	「誰が何をしてよいか」を判断し、DB への依頼を組み立てる仕事は、ファイルを返すだけでは足りないため。
受け取る→する→返す	転送された要求（誰から・何を）を受け取る → SQL で DB に問い合わせ、結果を HTML にまとめる → HTML を返す
似た言葉との違い	第5章の「注文を扱うアプリ」はこの役割のプログラム。「AP」は無線のアクセスポイントと紛らわしいため、この資料では使わない。

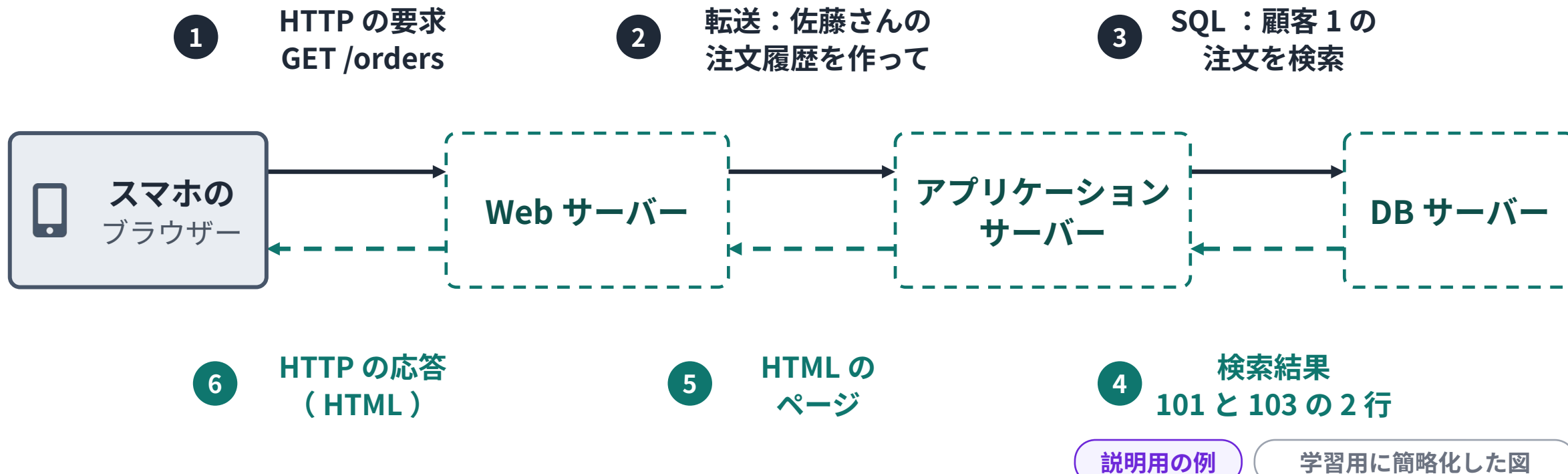
Web ・ アプリ ・ DB の担当と受け渡す情報

役割	担当する仕事	受け取るもの	渡す・返すもの
Web サーバー	要求の受付、画像・CSS の返却、それ以外の転送	ブラウザからの HTTP の要求（GET /orders）	HTML や画像（HTTP の応答）
アプリケーション サーバー	ログイン確認・検索の指 示・ページの組み立て	Web サーバーから転送 された要求	DB へ： SQL ／ Web へ： HTML
DB サーバー	データの検索・追加、整 合性と権限の管理	アプリからの SQL	検索結果（101 と 103 の 2 行）

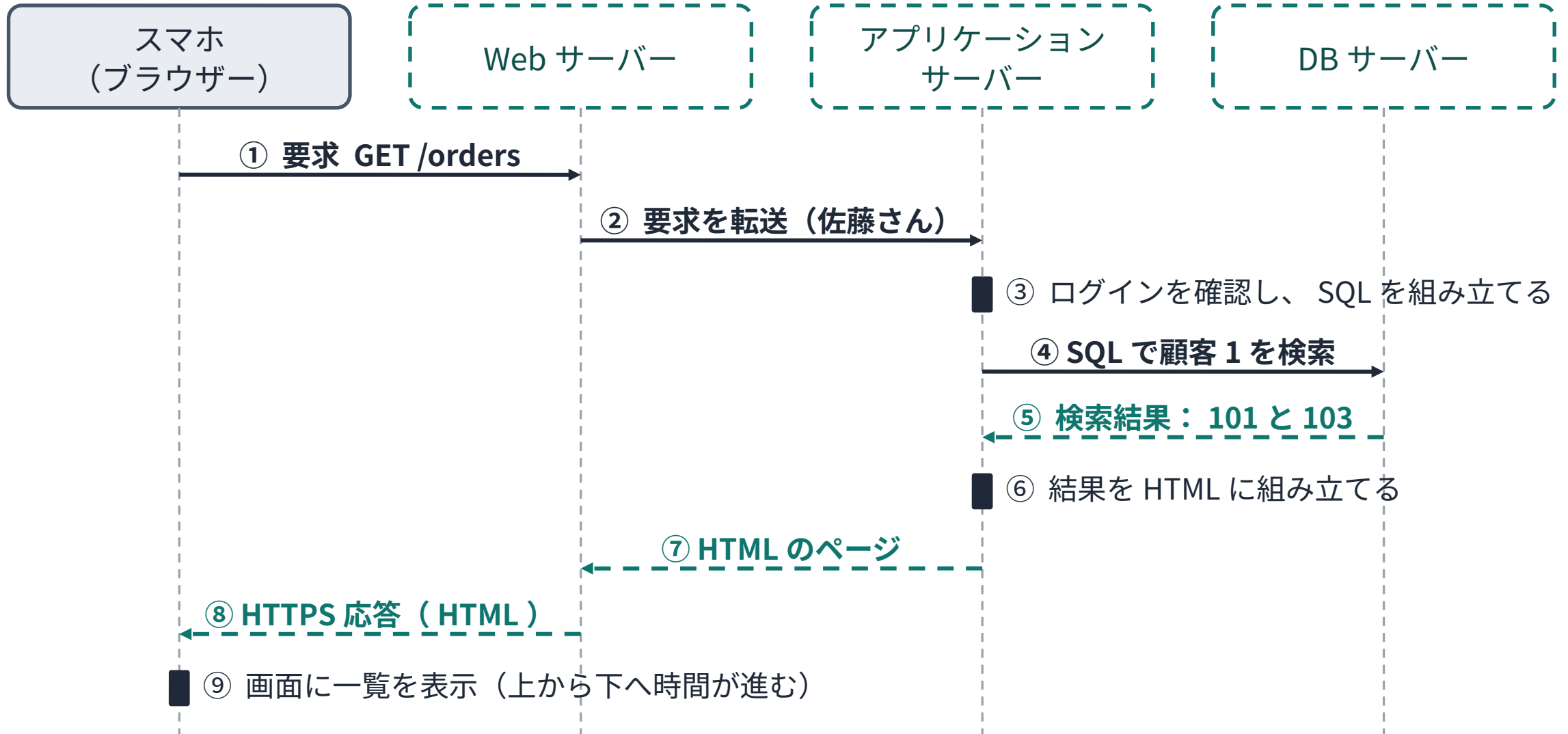
表示（ブラウザと Web サーバー） ・ 業務の処理（アプリケーションサーバー） ・ データ（DB
サーバー） の 3 つに役割を分ける形を、 **3 層クライアントサーバシステム**（3 層構造）と呼びます。

受け渡す情報を図で追う（役割の関係図）

役割の関係図。点線の箱＝役割、灰色の太枠＝機械。黒い矢印（上）＝要求・依頼、緑の破線（下）＝応答・結果。丸数字は順番、矢印の上下の文字が運ばれる中身です。

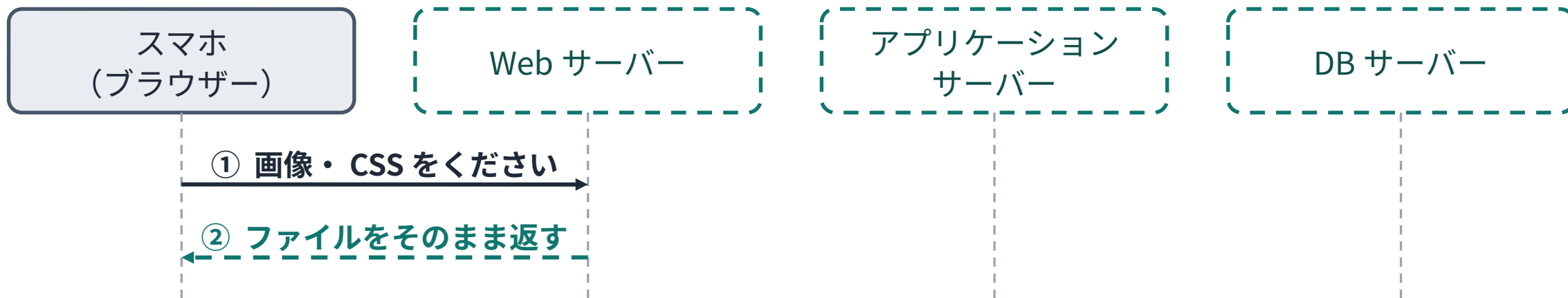


注文履歴を表示するまで（時間順のやり取り図）



すべての情報が全部のサーバーを通るわけではない

ページの中のロゴ画像や CSS（静的コンテンツ）は、Web サーバーが自分で返します。アプリケーションサーバーや DB サーバーまでは届きません。



⚠ 注意

サーバーの名前から IP アドレスを調べる DNS への問い合わせ（p.247）は、これより前に別の相手と行います。注文履歴の本文は DNS サーバーを通りません。

役割の数と機械の台数は一致しない

3つの役割は機能の名前で、機械の台数ではありません。同じ3役でも、規模によって機械の数は大きく変わります。

配置の模式図。灰色の太枠＝機械、紫＝その上で動くソフトウェア。

例1 小さなお店：1台で3役



例2 2台に分ける



例3 大きなサイト：多数台



同じ役割の機械を何台も並べる

説明用の例

学習用に簡略化した図

なぜ 1 台にまとめたり、多数台に分けたりするのか

形	よい点	気をつける点
1 台で 3 役	機械が少なく、費用と管理の手間が小さい	その 1 台が壊れると全部止まる。3 つの仕事が CPU やメモリを取り合う
役割ごとに分ける	役割に合う機械を選ぶ。DB を守りやすい	機械が増え、つなぐ設定や管理が増える
各役を多数台で	多数の要求を分担でき、1 台が壊れても続けられる	要求を振り分ける仕組み（ p.334 ）や、データの食い違いを防ぐ工夫が要る

i 補足

「機械」は物理的な機械とは限らず、仮想の機械やコンテナと呼ばれる区切り（第 10 章）のこともあります。大きなサービスでは、1 つの役割を数千台で分担することもあります。

確認の問い（Web ・ アプリ ・ DB の分担）

自分の言葉で答えてから、次のページの解説を読みましょう。

- 1 アプリケーションサーバーが受け取るもの・渡すものは？
- 2 1 台の機械が Web ・ アプリ ・ DB の 3 役を兼ねるとき、役割と台数はいくつですか。

確認の問い（Web・アプリ・DBの分担）：解説

前のページの問いへの答えと、考え方です。

✓ 問1 アプリケーションサーバーが受け取るもの・渡すものは？

Web サーバーから転送された要求（佐藤さんの注文履歴を）を受け取り、DB サーバーへ SQL を渡し、検索結果（101 と 103）を HTML にして Web サーバーへ返します。 p.326

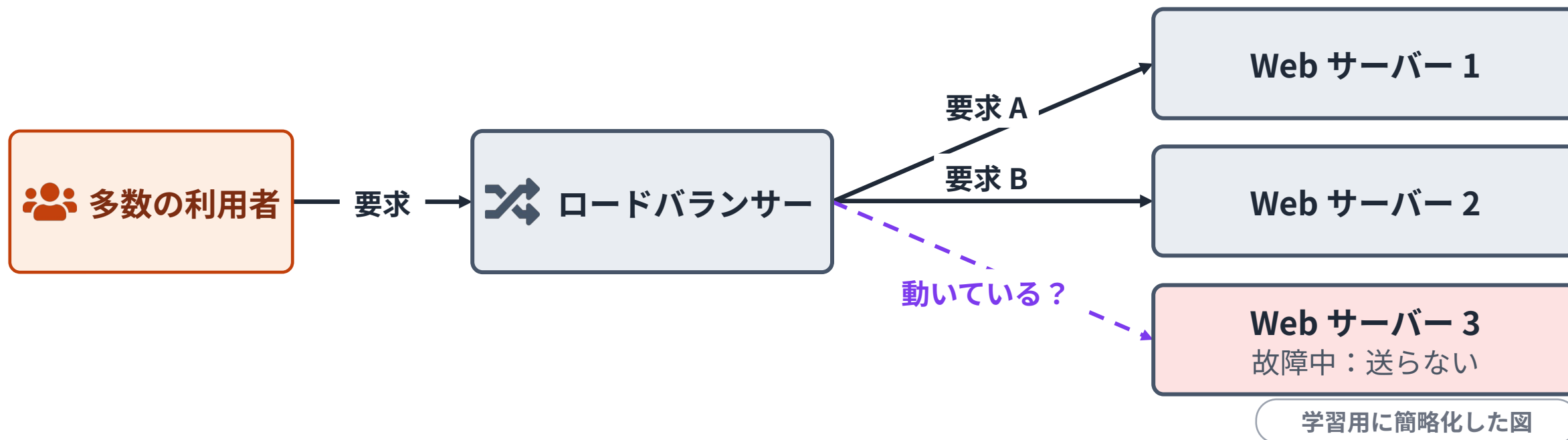
✓ 問2 1 台の機械が Web・アプリ・DB の 3 役を兼ねるとき、役割と台数はいくつですか。

役割は 3 つ、機械は 1 台です。役割は機能の名前で、台数とは一致しません。逆に 1 つの役割を多数台で分担することもあります。 p.330

負荷分散：要求を複数の機械に振り分ける

同じ仕事をする機械を何台も並べ、届いた要求を順番や混み具合で振り分けることを**負荷分散**、振り分ける装置やソフトウェアを**ロードバランサー**と呼びます。

黒い矢印＝要求の移動、紫の破線＝正常に動いているかの確認（補助の問い合わせ）



故障した機械には要求を送らないよう、各機械の状態を定期的に確かめます。

サービス側の主な機能（1/2）：入口と守り

大きなサービスには、Web・アプリ・DBのほかに、守る・振り分ける・確かめる機能があります。どれも機能の名前です（一般的な構成例）。

機能	何をするか
入口	外からの通信を受け付ける
ファイアウォール	決めた条件（送り元・あて先・ポート番号など）に合う通信だけを通し、それ以外を止める
WAF	Web アプリへの不正な要求を見分けて止める（Web Application Firewall の略）
負荷分散	要求を複数の処理先へ振り分ける（p.334）
認証	要求が誰からのものかを確かめる（ログインなど）

サービス側の主な機能（2/2）：処理と見守り

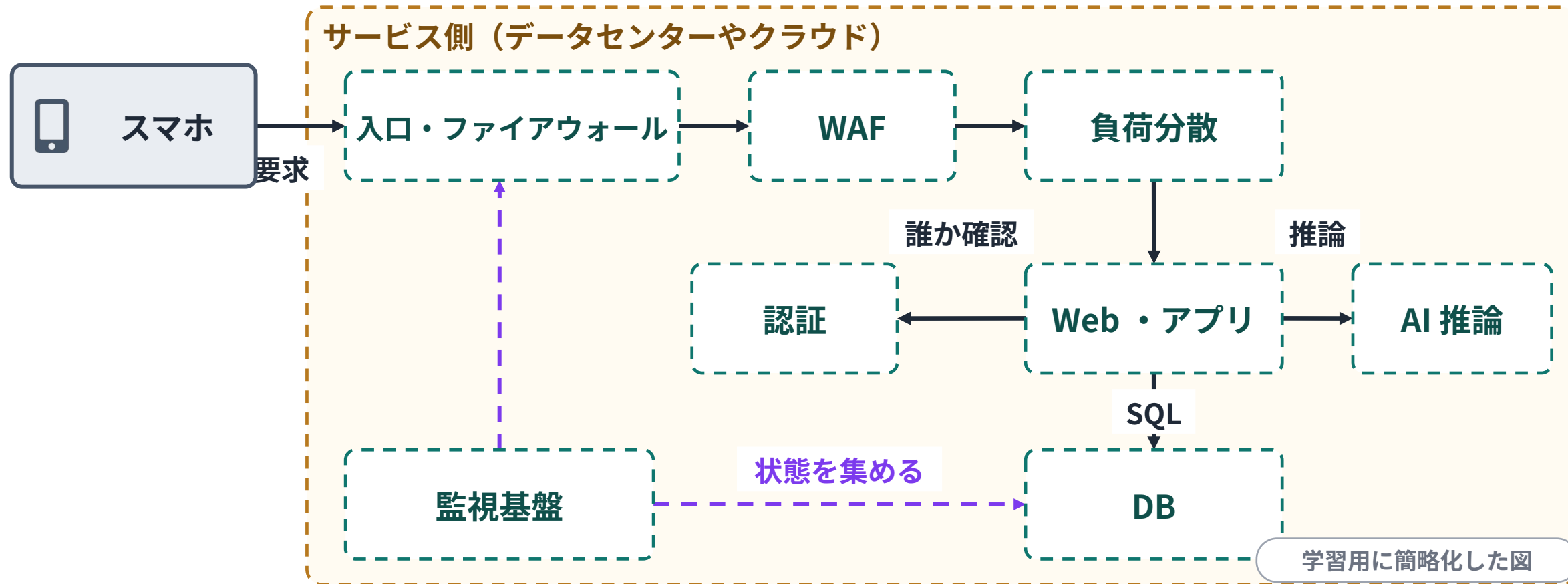
機能	何をするか
アプリ	業務の処理（アプリケーションサーバー）。AI サービスなら質問や会話を扱う
DB	データを整理して保存・検索する（DB サーバー）
AI 推論	学習済みの AI のモデルで答えを計算する（第 12 章）
監視基盤	遅延・エラー・GPU の温度や使用率などを観察し、異常を知らせる

補足

ファイアウォールと WAF の詳しい違いは第 11 章（p.475）で扱います。ここでは、要求が業務の処理に届く前に、通してよい通信かを確認める機能だと押さえれば十分です。

要求が通る機能の並び（一般的な構成例の図）

役割の関係図（一般的な構成例）。点線の箱＝機能。黒い矢印＝要求・依頼の移動（応答は逆向きに戻るが省略）、紫の破線＝監視基盤が状態を集める補助の通信。



構成図を読むときの注意

- 図の箱は機能の名前です。1 台がファイアウォールと負荷分散を兼ねることも、1 つの機能を数千台で分担することもあります。
- 特定のサービスが、どのクラウド・どの国の設備で動いているかは、公開情報だけでは断定できません。
- AI サービスでも骨組みは同じです：要求が届く→認証→アプリ→AI の推論→回答を返す。ただし ChatGPT など実際のサービスの内部構成は非公開の部分があり、ここで示したのは一般的な構成例です。
- AI の回答は、全部できあがってからではなく、生成した分から順に受け取って表示することがあります。

Java で作った Web アプリを動かす土台

Java は、Web のアプリを作るのによく使われるプログラミング言語（プログラムを書くための言葉）です。Java で作った Web アプリは、次の層の上で動きます（上ほど利用者に近い）。

自分の Web アプリ

サーブレット＝要求を受けて処理し、結果を返す Java の部品（例：注文履歴のページを作る）

サーブレットコンテナ

サーブレットを動かし、要求を渡す土台。Tomcat ・ Jetty （軽量）、WebLogic （企業向け）

Java 実行環境

JVM （Java Virtual Machine）：Java のプログラムを動かす仕組み

OS

Linux ・ Windows

Java の土台で混同しやすい言葉

⚠ 注意

「サーブレットコンテナ」のコンテナと、第 10 章で学ぶ Docker などの「コンテナ」は別の意味です。前者はサーブレットを入れて動かす土台、後者はアプリを動かす環境を区切る仕組みです。

🔄 似た言葉との違い

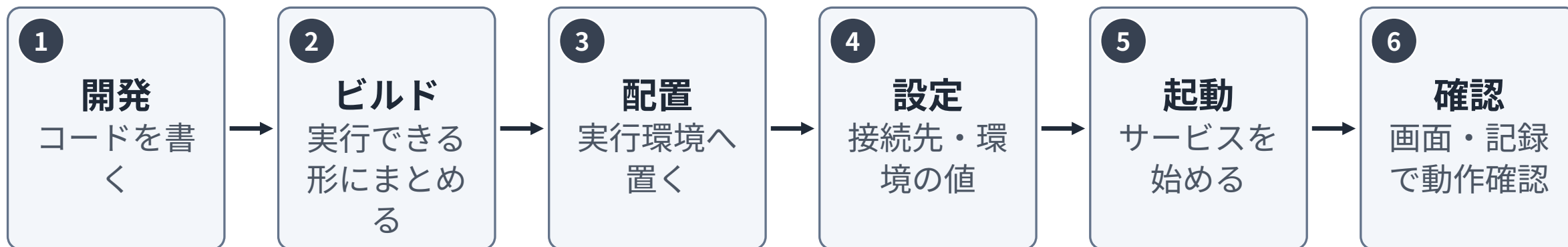
「アプリケーションサーバー」という言葉は、役割（p.325）の名前としても、WebLogic のようなアプリを動かす土台の製品の分類としても使われます。どちらの意味かは文脈で判断します。

📖 補足

製品が対応する仕様の範囲は版によって違うため、Tomcat と WebLogic など単純な優劣では比べません。

デプロイ：作ったアプリを使える状態にする

作ったアプリを実行環境（アプリを動かす機械・OS・土台のソフトのひとつ）に置き、使える状態にする工程を**デプロイ**と呼びます。注文履歴ページの表示を直したら、次の順で新しい版をデプロイします（説明用の例）。

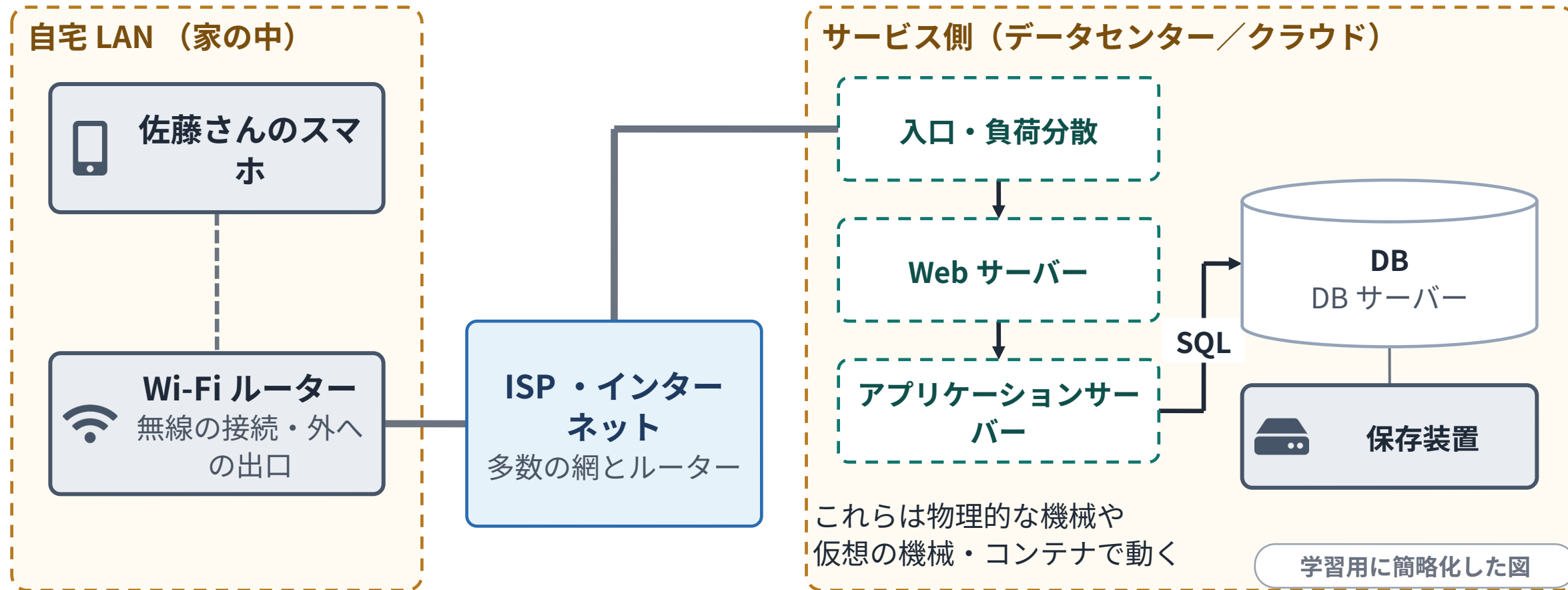


📖 具体例

設定の例：開発用の PC ではテスト用の DB に、本番の実行環境では本番の DB サーバーにつながよう、置く場所ごとに接続先の値を変えます。確認を省くと、表示の崩れや DB につながらない失敗に気づけません。

全体の地図：スマホから DB ・ 保存装置まで

学習用の地図。左と中央は機器のつながり（灰色の太線＝ケーブル、破線＝電波）、右のサービス側は機能の関係（黒い矢印＝要求の流れ、細線＝記録先）。



地図の区間ごとに起きること

区間	ここで起きること	くわしく
自宅 LAN	Wi-Fi への接続、IP アドレスなどの設定の取得、出口（ルーター）へ渡す	p.209 ・ p.244
インターネット	名前から IP アドレスを調べる（DNS）、経路の選択、暗号化された通信の運搬	p.247 ・ p.262
サービス側	受付・振り分け、業務の処理、データの検索・保存	p.326 ・ p.335
土台	データの保存方式、機械と設備、仮想の機械・コンテナ	第 8 章・第 9 章・第 10 章

i 補足

図の「入口」「Web」「アプリ」「DB」は機能の名前です。1 台が複数の機能を兼ねることも、1 つの機能を数千台で分担することもあります。

第 7 章のまとめ

✓ ここまでで説明できること

Web ・ HTML ・ CSS ・ URL : ブラウザーでページを見る仕組みと、ページ・住所の書き方。

HTTP : 要求 (GET /orders) と応答 (200 OK + HTML) の約束。HTTPS は TLS で保護。

Web サーバー : 受付と静的コンテンツの返却。**アプリケーションサーバー** : 業務の処理と動的なページ。**DB サーバー** : SQL で検索。

役割と台数は別 : 1 台で 3 役も、1 役を多数台も。負荷分散で振り分ける。

Java ・ デプロイ : アプリを動かす土台と、使える状態にする工程。

→ 次へのつながり

第 8 章では、DB などのデータを保存装置にどう保存し、故障や誤操作からどう守るかを学びます。

確認の問い（第 7 章）

自分の言葉で答えてから、次のページの解説を読みましょう。

- 1 HTTP の要求「GET /orders」と応答「200 OK」の意味は？
- 2 静的コンテンツと動的コンテンツの違いを例で説明してください。
- 3 デプロイとは何ですか。

確認の問い（第 7 章）：解説

前のページの問いへの答えと、考え方です。

✓ 問 1 HTTP の要求「GET /orders」と応答「200 OK」の意味は？

「/orders（注文履歴）のページをください」という要求と、「成功しました」という結果の番号です。応答の本体に HTML のページが入ります。 p.321

✓ 問 2 静的コンテンツと動的コンテンツの違いを例で説明してください。

静的はロゴ画像や CSS のように誰が見ても同じファイルで、Web サーバーがそのまま返します。動的は佐藤さんの注文履歴のように、要求のたびにアプリケーションサーバーが作ります。 p.324

✓ 問 3 デプロイとは何ですか。

作ったアプリを実行環境に配置し、設定・起動・確認して、利用者が使える状態にする工程です。 p.341

第 8 章

データの保存方式と守り方

この章で学ぶこと

- ・ 保存の 3 つの「見せ方」：ファイル・ブロック・オブジェクト
- ・ NAS ・ SAN ・ DAS ：保存装置のつなぎ方の違い
- ・ RAID ：複数のドライブで故障に備える方法と、パリティの意味
- ・ RAID とバックアップの違い：守れる場面の違い

→ 前の章とのつながり

第 2 章で SSD と HDD、第 3 章でファイルシステムを見ました。ここでは保存装置をネットワーク越しに使う方法と、壊れても失わない仕組みへ進みます。

保存装置を、ネットワーク越しに使う理由

PC の SSD は、その PC の中だけで使います。会社では、保存装置を別の機械に置き、ネットワーク越しに使うことがよくあります。

共有できる

部署の全員が、同じファイルを自分の PC から開ける。1 台の PC に置くと、その PC の持ち主しか使えない。

容量を増やしやすい

足りなくなったら、保存装置の側だけを増やせる。PC やサーバーを 1 台ずつ入れ替えなくてよい。

守りやすい

データが 1 か所に集まるので、まとめて故障に備えたり、コピーを取ったりしやすい。

この章では、保存装置が「使う側から何として見えるか」と「どうつながっているか」を順に見て、最後に「どう守るか」を学びます。

同じ「保存」でも、見せ方が 3 通りある

「見せ方」とは、使う側（人やプログラム）から、保存したデータが何として見え、どう指定して読み書きするかのことです。

フォルダとファイル

「営業部」フォルダの「見積.xlsx」のように、名前と場所で指定する。

番号付きの区画

同じ大きさに区切った区画を「1024 番から 8 個」のように番号で指定する。

キーとデータ

「photos/2026/10/a.jpg」のような名札（キー）で、データのまとまりを丸ごと出し入れする。

ポイント

中に入っているのは、どれも最終的には SSD や HDD に記録された 0 と 1 の並びです。違うのは、使う側への見せ方と、指定のしかたです。

ファイル・ブロック・オブジェクトの3方式

3つの方式で、使う側から見える形を並べた学習用の図（説明用の例）。緑の書類形＝データ、灰色の小さな箱＝区画、白い箱＝付け加える情報。

ファイルストレージ

[フォルダー] 営業部

見積.xlsx

提案.pptx

フォルダーとファイル名で扱う

ブロックストレージ

1024

+1

+2

+3

+4

+5

+6

+7

+8

+9

+10

+11

同じ大きさの区画を
番号で読み書きする

オブジェクトストレージ

キー： photos/2026/10/a.jpg

データ本体（画像）

メタデータ

種類・作成日など

キーで丸ごと出し入れする

区画の箱の数字は、先頭を 1024 番とした説明用の番号です（+1 は 1025 番）。次のページから、1 つずつ具体的に見ます。

ファイルストレージ：名前と場所で指定する

役割の関係図。黒い矢印＝依頼、緑の破線の矢印＝返事。



説明用の例

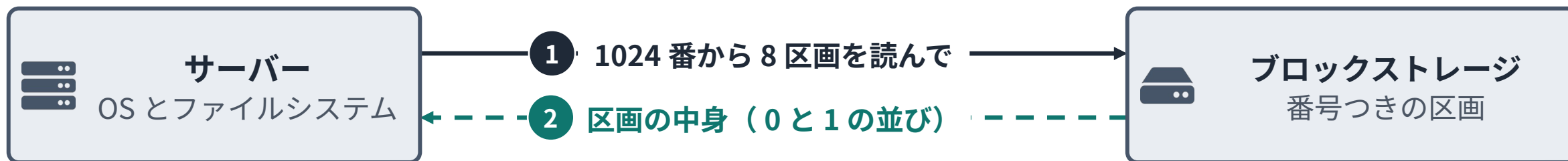
使う人には、自分の PC のフォルダーと同じように見えます。PC は「どのフォルダーの、どの名前のファイルか」で依頼し、ファイルストレージの側が、そのファイルが保存装置のどこにあるかを調べて中身を返します。

具体例

部署の共有フォルダー。複数の人が同じファイルを開け、フォルダーごとに「営業部の人だけ読み書きできる」のような権限を決められます。

ブロックストレージ：番号付きの区画で指定する

役割の関係図。黒い矢印＝依頼、緑の破線の矢印＝返事。



説明用の例

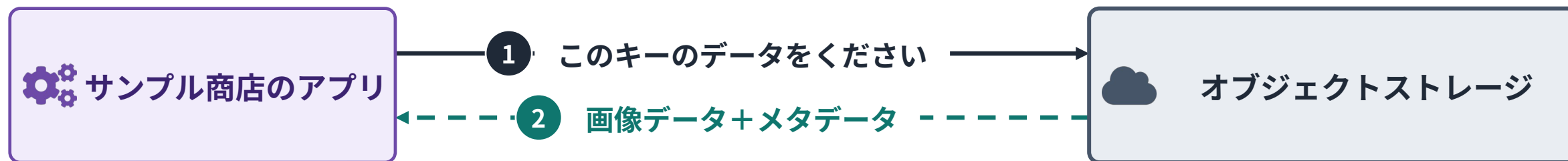
ブロックストレージは、ファイルの名前もフォルダーも知りません。「見積.xlsx は 1024 ～ 1031 番の区画にある」と覚えているのは、使う側のサーバーの OS のファイルシステムです。使う側からは、自分につながったディスクのように見えます。

具体例

OS を入れるディスク、DB のデータを置くディスク、仮想の機械（第 10 章）のディスク。細かい読み書きを速く何度も行う用途に向きます。

オブジェクトストレージ：キーで丸ごと出し入れする

役割の関係図。黒い矢印＝依頼、緑の破線の矢印＝返事。依頼は Web と同じ HTTP で送ります。



説明用の例

データ本体と、種類・作成日などの付け加える情報（メタデータ）を1つのまとまりにし、キー（例：photos/2026/10/a.jpg）という名札で保存・取得します。依頼は **HTTP** で送ります。

⚠ 注意

キーの「/」はフォルダーのように見えますが、多くの場合は名札の文字の一部です。一部だけ書き換えるより、丸ごと出し入れする使い方に向きます。

3 方式の比較

	ファイル	ブロック	オブジェクト
見える形	フォルダー・ファイル	ディスクのような領域	キー・データ・メタデータ
使い方	共有フォルダー（SMB・NFS）	OS・DB・仮想の機械のディスク	HTTP で保存・取得
向く用途	部署のファイル共有	速さが必要な DB など	大量の画像・ログ・バックアップ
例	NAS、ファイルサーバー	SAN、内蔵 SSD	Amazon S3 など

補足

Amazon S3 は、AWS（Amazon のクラウドの事業）のサービス名で、方式の名前ではありません。SMB・NFS は共有フォルダー用の通信の約束です（p.363）。

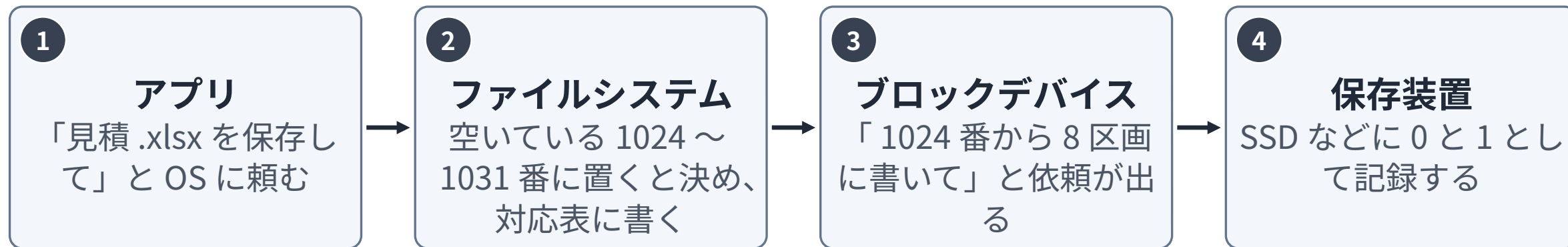
ブロックの上にファイルシステムが乗る

ファイルとして見えるものも、下の層では番号付きの区画として記録されています。上から順に役割を見ます。

アプリ	「ファイルを開く」「保存する」と依頼する
ファイルシステム	名前・フォルダー・権限・空き領域を管理する（NTFS・ext4・APFS など）
ブロックデバイス	番号付きの区画として読み書きされる「ディスク」
保存装置	SSD／HDD／SAN 上のディスク。実際に 0 と 1 を記録する

ファイルシステムが「どのファイルが、どの番号の区画にあるか」の対応表を持っているので、アプリは区画の番号を知らなくてもファイルを使えます。

例：「保存」を押すと、区画に書かれるまで



ファイルストレージを使うときは、この「ファイルシステム」の仕事を保存装置の側が行います。ブロックストレージを使うときは、使う側のサーバーの OS が行います。この違いが、次の NAS ・ SAN の違いにつながります。

NAS ・ SAN ・ DAS は、つなぎ方の違い

物理的な配置図（学習用に簡略化）。灰色の太線＝ケーブル。右端は、使う側から保存装置がどう見えるか。

DAS：直接つなぐ



NAS：LAN 越しにファイルを提供



SAN：保存専用の網



DAS : その 1 台に直接つなぐ

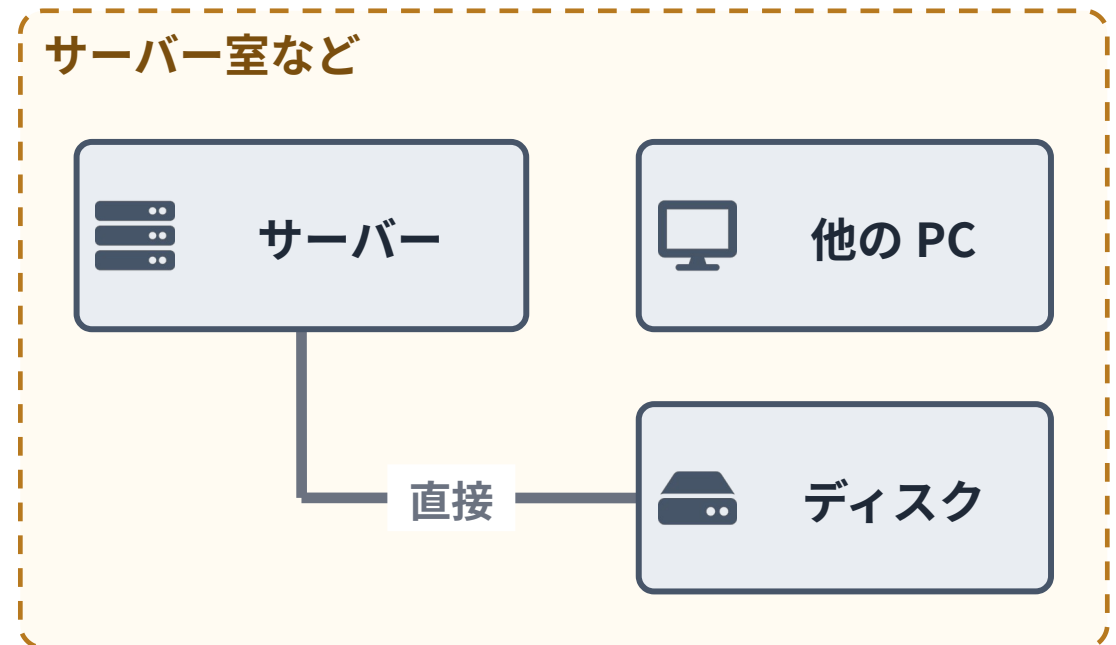
DAS (Direct Attached Storage : 直接つないだ保存装置) は、保存装置を、ネットワークを通さずにサーバーや PC へケーブルで直接つなぐ方式です。

つないだ 1 台からは、自分のディスクとして見えます。基本的には、そのつないだ 1 台だけが使います。

📋 具体例

PC の中の SSD、PC に USB でつないだ外付け SSD、サーバーの中のディスク。

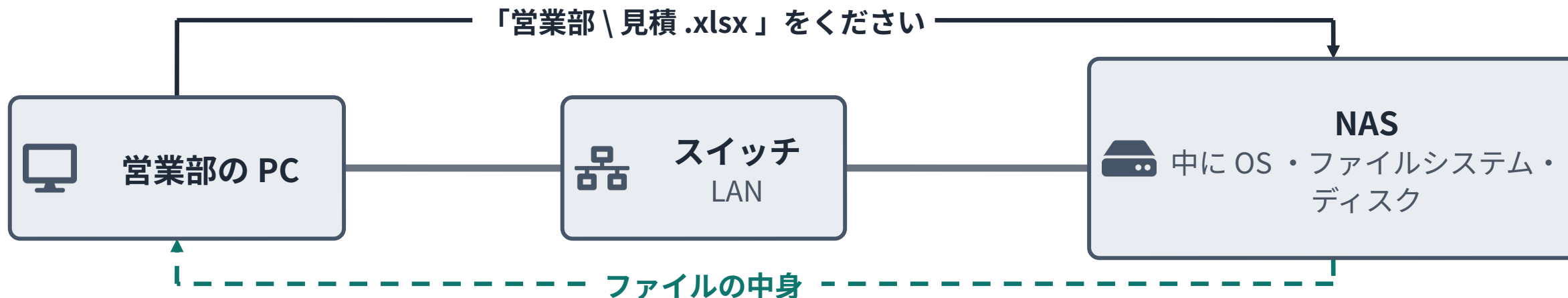
物理的な配置図 (簡略化)。灰色の太線 = ケーブル。



他の PC からは、このディスクを直接は使えません。

NAS：LAN 越しにファイルを提供する

配置とやり取りを重ねた図。太線＝ケーブル、黒い矢印＝依頼、緑の破線＝返事。



NAS（Network Attached Storage）は、ファイルを提供する小さなコンピューターのような装置です。中で OS とファイルシステムが動き、LAN の PC に共有フォルダーを見せます。

📖 具体例

家庭用の NAS、部署の共有フォルダー。多数の利用者で共有しやすい。

SAN：保存装置のための専用の網

配置とやり取りを重ねた図。太線＝ケーブル、黒い矢印＝依頼、緑の破線＝返事。



SAN（Storage Area Network）は、サーバーと保存装置の間だけを結ぶ保存のための網で、利用者の PC がつながる LAN とは分けます。サーバーからはディスクに見えます。

📖 具体例

企業の DB、仮想の機械を多数動かす基盤（第 10 章）など。

「装置の種類」ではないことを確かめる

NAS ・ SAN ・ DAS は「保存装置の種類の名前」ではなく、「どうつながぎ、何として見せるか」の違いです。

- SAN は装置の名前ではなく、網（ネットワーク）の名前です。SAN につながる保存装置は、SAN の先にある「保存装置」です。
- NAS の中にもディスクがあり、それは NAS の中で直接つながっています。外から見れば NAS、中を見ればディスクが直接つながった形です。
- 製品によっては、1 台の保存装置がファイルを提供（NAS として）しつつ、別のサーバーには区画を提供（SAN の先の保存装置として）できるものもあります。

ポイント

図を読むときは、①保存装置まで何でつながっているか（直接・LAN・専用の網）、②使う側から何として見えるか（ディスク・共有フォルダー）の 2 点を確認めます。

NAS ・ SAN ・ DAS の比較

	NAS	SAN	DAS
何か	ファイルを提供する装置・システム	保存装置へアクセスするための網	サーバーへ直接つなぐ方式
見え方	共有フォルダー	サーバーからはディスク	サーバーからはディスク
共有	多数の利用者で共有しやすい	複数のサーバーで共有できる構成も	基本はつないだ 1 台
身近な例	家庭用 NAS ・ 部署の共有	企業の DB ・ 仮想化の基盤	PC の外付け SSD ・ 内蔵ディスク

「仮想化の基盤」は、1 台の機械を複数の仮想の機械に分けて動かす仕組み（第 10 章）のことです。

保存に使う通信の約束

ファイル用とブロック用で、別の通信の約束（プロトコル）を使います。

約束	方式	運ぶもの・特徴
SMB	ファイル	Windows で広く使う、共有フォルダーの約束
NFS	ファイル	Linux ・ UNIX で広く使う、共有の約束
iSCSI	ブロック	SCSI の命令を IP ネットワークで運ぶ。通常の Ethernet で使える
ファイバチャネル （ FC ）	ブロック	保存専用設計された、高速な網と約束
FCoE	ブロック	FC の形のデータを Ethernet の上で運ぶ。 iSCSI （ IP の上）とは別

SCSI（スカジー）は、保存装置に「この区画を読め・書け」と命じる命令の決まりです。Ethernet（イーサネット）は、LAN で広く使われる有線の通信の規格です。

iSCSI ・ FC ・ FCoE は、何に乗せて運ぶかが違う

iSCSI

- ・ 運ぶもの：SCSI の命令
- ・ 乗せる道：IP （と TCP ）
- ・ つなぐ機器：普通の LAN のスイッチと NIC

ファイバチャネル

- ・ 運ぶもの：SCSI の命令など
- ・ 乗せる道：FC 専用の網
- ・ つなぐ機器：FC スイッチと HBA （第 9 章）

FCoE

- ・ 運ぶもの：FC の形のデータ
- ・ 乗せる道：Ethernet （IP は使わない）
- ・ つなぐ機器：対応した Ethernet の機器

⚡ 似た言葉との違い

iSCSI と FCoE は、どちらも Ethernet のケーブルを使えますが別物です。iSCSI は IP の上で運び、FCoE は IP を使わずに FC の形のまま運びます。

フォーマットは、ファイルシステムを作り直す作業

フォーマット（初期化）は、保存装置の上にファイルシステムを新しく作る作業です。前のページで見た「どのファイルがどの区画にあるか」の対応表が作り直されるので、基本的に中のデータは使えなくなります。

⚠ 注意

外付け SSD や SAN のディスクをつないだとき「フォーマットしますか」と表示されても、中に必要なデータがあるなら実行しないでください。

表示される理由は、その OS が読めないファイルシステムの種類（NTFS ・ ext4 ・ APFS など）だったり、まだ何も作られていなかったりすることです。

i 補足

ブロックストレージを新しくサーバーにつないだときは、ふつう、中身が空であることを確かめてからフォーマットして使い始めます。

RAID：複数のドライブを組み合わせる

「ドライブ」は、SSD や HDD の 1 台 1 台のことです。ドライブはいつか壊れる部品です。

RAID（れいと）

分類：仕組み

略語： RAID = Redundant Arrays of Independent Disks（独立したディスクを冗長に並べたもの）

何か	複数のドライブにデータを分けたり複製したりして、速さや故障への強さを得る技術。
なぜ必要か	1 台だけだと、壊れたときにデータを失い、サービスも止まる。
どこにあるか	サーバーや保存装置の中。専用の部品（ RAID コントローラー）か、OS の機能で行う。
受け取る→する→返す	OS から書き込みを受け取る → 決まった置き方で複数のドライブに書く → OS からは 1 台の大きなディスクに見える。
似た言葉との違い	バックアップの代わりにはならない（ p.379 ）。

冗長化：予備を持って、1 つ壊れても止めない

冗長化は、予備を用意して、一部が壊れても全体が止まらない・データを失わないようにすることです。「冗長」は「余分がある」という意味で、ここでは「わざと余分を持つ」ことを指します。

ドライブ

RAID で、1 台が壊れても他のドライブからデータを読める。

電源

電源の部品を 2 つ持ち、片方が壊れても動き続ける（第 9 章）。

機器・経路

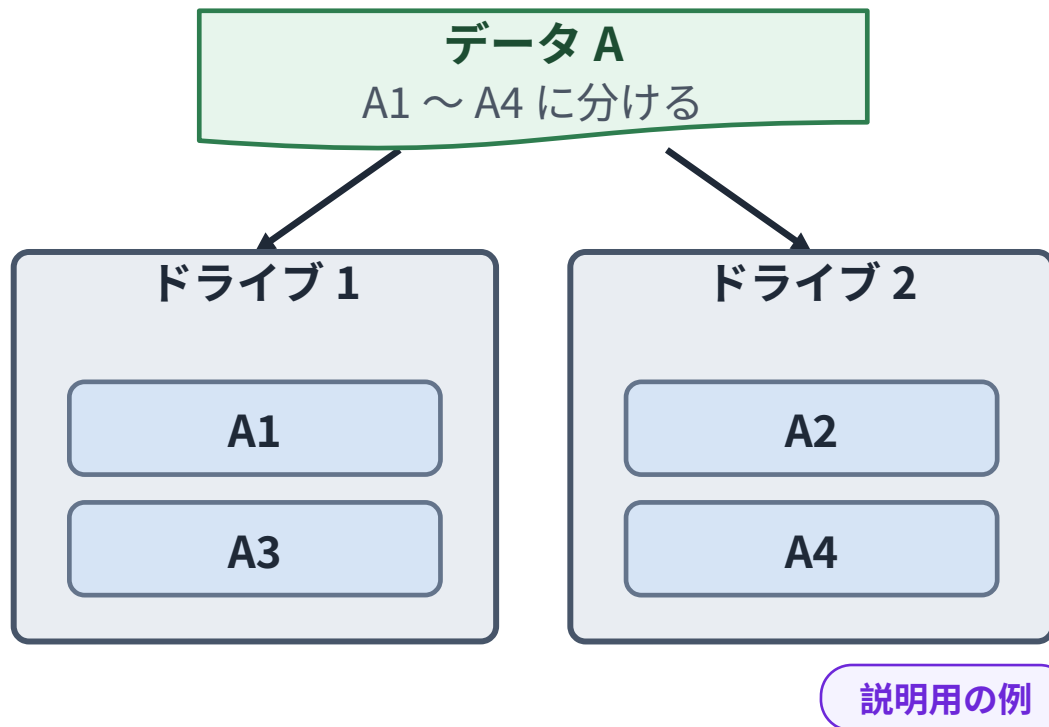
スイッチやケーブルを 2 系統用意し、片方が切れても通信できる。

補足

次のページから、RAID の置き方を 1 つずつ見ます。容量は「同じ容量 D のドライブ N 台」という単純なモデルで示します（D は 1 台の容量、N は台数）。

RAID 0：分けて並べる（ストライピング）

置き方の図。灰色の太枠＝ドライブ、中の小箱＝分けたデータ。



RAID 0 は、データを小さく分けて、複数のドライブに順番に並べて書きます。これを**ストライピング**といいます（縞模様のように並べる、という意味）。

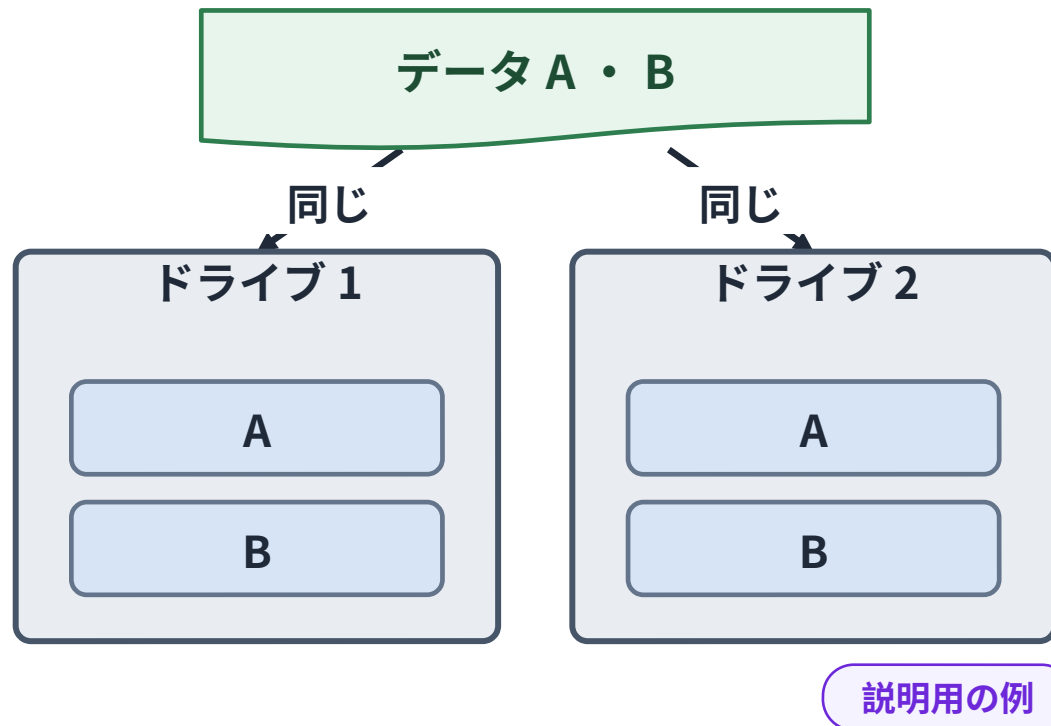
2 台に同時に読み書きできるので速くなり、容量は $N \times D$ （全部の合計）を使えます。

⚠ 注意

1 台でも壊れると、どのデータも半分ずつ欠けて、全部を失います。耐えられる故障：なし。

RAID 1：同じデータを 2 台に（ミラーリング）

置き方の図。2 台に同じ中身を書きます。



RAID 1 は、同じデータを 2 台のドライブに同時に書きます。鏡に映したように同じ中身になるので、**ミラーリング**といいます。

1 台が壊れても、もう 1 台に同じデータが残っているので、そのまま使い続けられます。

🔑 ポイント

使える容量：D（2 台構成でも 1 台分）。
耐えられる故障：2 台のうち 1 台。

パリティ：失ったデータを計算で戻すための情報

RAID 1 は丸ごと複製するので、容量が半分になります。そこで、複製の代わりに「計算で戻すための情報」を持つ方法があります。この情報を**パリティ**といいます。小さな例で作り方を見ます。

	1 桁目	2 桁目	3 桁目	4 桁目
データ 1	1	0	1	1
データ 2	0	1	1	0
パリティ	1	1	0	1

作り方（説明用の例）

作り方の規則：各桁で、データ 1 とデータ 2 の「1」の数が奇数なら 1、偶数なら 0 にします。例：1 桁目は 1 と 0 で「1」が 1 個（奇数）→ 1。3 桁目は 1 と 1 で 2 個（偶数）→ 0。

1 台が壊れても、残りから計算で戻せる

前のページの例で、データ 2 を記録していたドライブが壊れたとします。

	1 桁目	2 桁目	3 桁目	4 桁目
データ 1（残っている）	1	0	1	1
データ 2（壊れた）	?	?	?	?
パリティ（残っている）	1	1	0	1
計算で戻したデータ 2	0	1	1	0

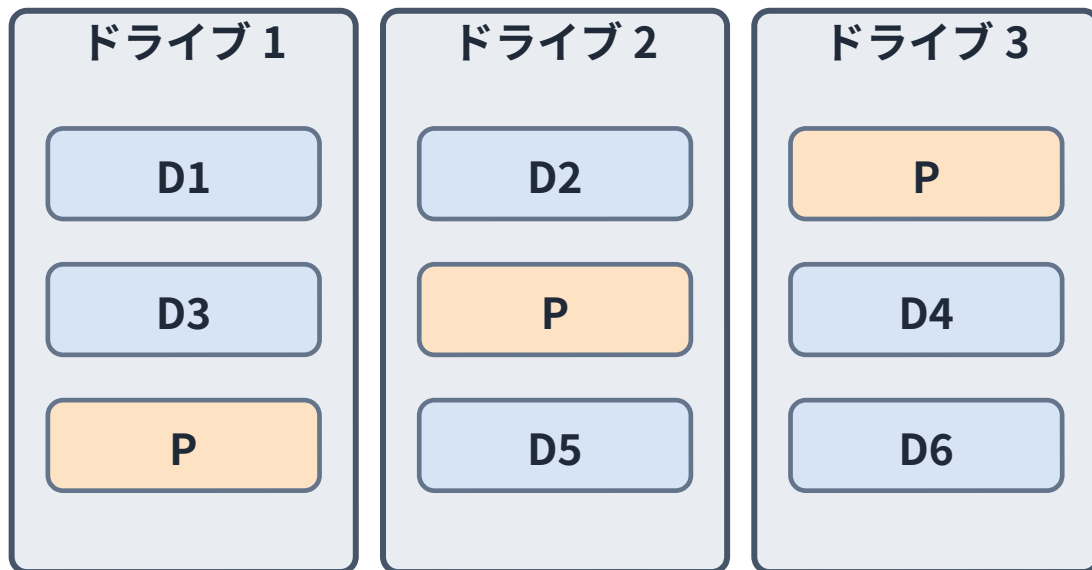
残ったデータ 1 とパリティに同じ規則（「1」の数が奇数なら 1、偶数なら 0）を使うと、データ 2 の「0110」が戻ります。元のデータ 2 と同じです。

❗ この例の限界

最小の形の例です。実際は多くのドライブと大量のデータで行います。2 つ同時に失うと、1 種類のパリティでは戻せません。

RAID 5：パリティを全ドライブに分散する

置き方の図（3 台の例）。D = データ、P = パリティ。



説明用の例

RAID 5 は、データとパリティを、3 台以上のドライブに少しずつずらして置きます。各段の横の並び（例：D1・D2・P）が、前のページの「データ 1・データ 2・パリティ」に当たります。

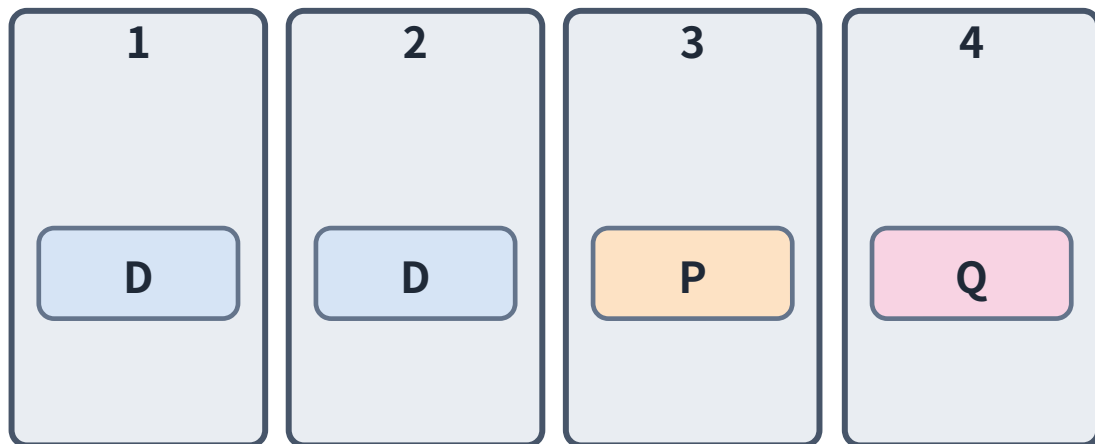
どの 1 台が壊れても、残りの台から計算で戻せます。

🔑 ポイント

使える容量： $(N-1) \times D$ （1 台分をパリティに使う）。耐えられる故障：通常 1 台。

RAID 6 : 2 種類のパリティで 2 台の故障に備える

置き方の図（4 台・1 段だけの例）。P・Q
= 2 種類のパリティ。



説明用の例

箱の上の数字はドライブの番号です。実際は RAID 5 と同じように、段ごとに P・Q の位置をずらします。

RAID 6 は、計算のしかたが違う 2 種類のパリティ（P と Q）を持ちます。2 台が同時に壊れても、残りから戻せます。

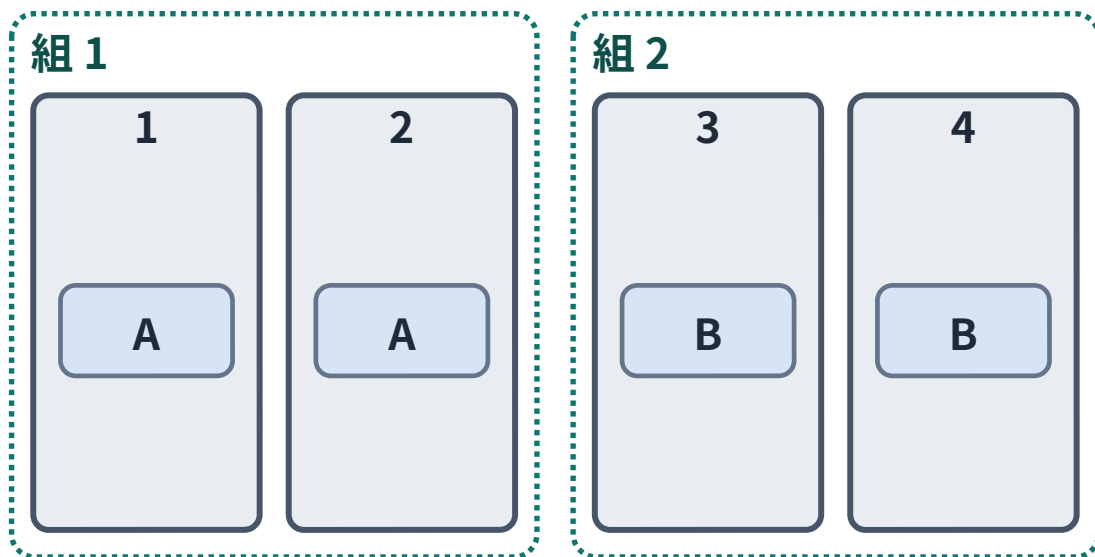
ドライブの台数が多いと、1 台を交換して戻している途中に、別の 1 台が壊れることもあります。その備えになります。

🔑 ポイント

使える容量： $(N-2) \times D$ 。耐えられる故障：通常 2 台。

RAID 10：ミラーの組を並べる

置き方の図（4 台の例）。点線の枠＝ミラーの組。



説明用の例

RAID 10 は、2 台ずつの**ミラーリング**の組を作り、データを組ごとに分けて並べます（**ストライピング**）。上の図では、A は組 1 に、B は組 2 に、それぞれ 2 台ずつ同じ中身で書かれます。

🔑 ポイント

使える容量： $N \times D \div 2$ 。耐えられる故障：壊れる位置による。違う組なら 1 台ずつ壊れても大丈夫ですが、同じ組の 2 台が壊れると失います。

RAID の方式ごとの容量と、耐えられる故障

方式	データの置き方	使える容量（理論）	耐えられる故障
RAID 0	分けて並べる	$N \times D$	なし（1 台でも壊れると失う）
RAID 1	2 台に同じデータを複製	D （2 台構成）	2 台のうち 1 台
RAID 5	データとパリティを分散	$(N-1) \times D$	通常 1 台
RAID 6	パリティを 2 種類持つ	$(N-2) \times D$	通常 2 台
RAID 10	ミラーの組を並べる	$N \times D \div 2$	位置による（同じ組の 2 台は不可）

⚠ 注意

容量 D のドライブ N 台という単純なモデルの理論値です。管理のための領域・予備のドライブ・製品ごとの作りは除いています。

計算例：4TB のドライブ 4 台なら

D = 4TB、N = 4 台として、前のページの式に当てはめます（RAID 1 だけは 2 台構成）。

方式	式	使える容量	耐えられる故障
RAID 0	$4 \times 4\text{TB}$	16TB	なし
RAID 1（2 台）	4TB	4TB	1 台
RAID 5	$(4-1) \times 4\text{TB}$	12TB	通常 1 台
RAID 6	$(4-2) \times 4\text{TB}$	8TB	通常 2 台
RAID 10	$4 \times 4\text{TB} \div 2$	8TB	位置による

容量を多く使える方式ほど、故障への強さは下がる傾向があります。用途に合わせて、速さ・容量・故障への強さのつり合いで選びます。

データを失う場面は、ドライブの故障だけではない

サンプル商店の注文表を例に、守りたい 4 つの場면을具体的に考えます（説明用の例）。

ドライブ故障

注文表を記録していた SSD の 1 台が、寿命で壊れる。

誤削除

担当者が、条件を間違えて注文 103 の行を消してしまう。

攻撃・破壊

不正なプログラムが、データを勝手に暗号化して読めなくする。

建物の災害

火災や地震で、機械を置いた建物ごと使えなくなる。

RAID が備えるのは、主に 1 つ目の「ドライブの故障」です。残りの 3 つには、別の仕組みが必要です。

スナップショットとレプリケーション

スナップショット

ある時点の状態を記録します。例：毎日 0 時の状態を記録しておけば、昼に注文 103 を誤って消しても、その日の 0 時の状態に戻せます。

- ・弱点：同じ装置の中に記録することが多く、装置ごと壊れると一緒に失う

レプリケーション（複製）

別の装置や別の建物へ、データを写し続けます。例：東京の建物が使えなくなっても、大阪の写しで続けられます。

- ・弱点：誤削除や破壊も、そのまま写ってしまう

RAID・冗長化・バックアップは、守れる場面が違う

仕組み	何をするか	ドライブ故障	誤削除	攻撃・破壊	建物の災害
RAID	複数ドライブで冗長化	○	×	×	×
冗長化（機器・経路）	予備の機器・経路	○	×	×	離れた場所なら○
スナップショット	ある時点の状態を記録	装置による	○	同じ装置だと弱い	×
レプリケーション（複製）	別の場所へ写す	○	誤りも写る	誤りも写る	○
バックアップ	独立した場所にコピーと履歴	○	○	隔離していれば○	離れた場所なら○

○× は一般的な傾向です。同じ装置ごと巻き込まれる構成なら、独立したバックアップと同じ守りにはなりません。

なぜ RAID では誤削除を戻せないのか

役割の関係図。黒い矢印＝書き込みの依頼。RAID は「書け」と言われたことを全部のドライブに正しく書きます。



誤削除は「故障」ではなく、正しく受け付けられた操作です。RAID は両方のドライブに同じように「消した後の状態」を書くので、どちらにも注文 103 は残りません。前日の状態を別の場所に持つバックアップがあれば、そこから戻せます。

バックアップは「戻せる」ことまで確かめる

バックアップは「取れている」だけでは不十分です。別の場所に置いたコピーから、実際に「戻せる」ことを、ときどき試して確かめます。いざというときに、コピーが壊れていたり、戻す手順が分からなかったりすることがあるからです。

i 一般的な考え方（この再構築で追加）

よく知られた目安に「3-2-1」があります：データのコピーを 3 つ持ち、2 種類の保存先に置き、そのうち 1 つは離れた場所に置く。攻撃に備えるには、普段の操作では書き換えられない場所（隔離）に置くことも大切です。

→ 次へのつながり

どの時点まで戻せればよいか（RPO）、どれだけの時間で戻すか（RTO）という目標の決め方は、第 13 章 p.556 で扱います。

フル・差分・増分バックアップ

日曜に**フルバックアップ**（全部をコピー）を取り、月～水は変わった部分だけを取る例です（説明用の例）。

曜日	差分バックアップで取るもの	増分バックアップで取るもの
日	全部（フル）	全部（フル）
月	月曜の変更	月曜の変更
火	月・火曜の変更	火曜の変更だけ
水	月～水曜の変更	水曜の変更だけ
水曜の状態に戻すには	日曜のフル＋水曜の差分（2 つ）	日曜のフル＋月＋火＋水（4 つ）

差分バックアップは日がたつほど 1 回の量が増えますが、戻すのは簡単です。**増分バックアップ**は 1 回の量が少ないかわりに、戻すときに全部を順に重ねる必要があります。

第8章のまとめ

✓ ここまでで説明できること

- ・ 保存の見せ方は3つ：ファイル（名前と場所）、ブロック（番号付きの区画）、オブジェクト（キーとデータ）
- ・ NAS・SAN・DASは、装置の種類ではなく、つなぎ方と見せ方の違い
- ・ RAIDは複数のドライブで速さや故障への強さを得る技術。パリティは計算で戻すための情報
- ・ RAIDはドライブの故障に強いが、誤削除・攻撃・災害には、独立したバックアップが必要。バックアップは戻せることまで確かめる

→ 次へのつながり

第9章では、保存装置やサーバーそのものの外観と中身、つながる線（データ・保存・管理・電力）、そして建物の電源と冷却を見ます。

確認の問い（第 8 章）

自分の言葉で答えてから、次のページの解説を読みましょう。

- 1 ブロックストレージは、ファイル名を知っているか。
- 2 NAS と SAN の違いを「何でつなぎ、何として見えるか」で説明せよ。
- 3 RAID 1 を使っていれば、誤って消した注文を戻せるか。

確認の問い（第 8 章）：解説

前のページの問いへの答えと、考え方です。

✓ 問 1 ブロックストレージは、ファイル名を知っているか。

知りません。区画の番号で読み書きされるだけで、どのファイルがどの区画にあるかは、使う側の OS のファイルシステムが管理します。 p.352

✓ 問 2 NAS と SAN の違いを「何でつながり、何として見えるか」で説明せよ。

NAS は LAN につながり、PC からは共有フォルダーに見えます。SAN はサーバーと保存装置を結ぶ専用の網で、サーバーからはディスクに見えます。 p.357

✓ 問 3 RAID 1 を使っていれば、誤って消した注文を戻せるか。

戻せません。削除の操作も両方のドライブに正しく書かれるからです。前の状態を別の場所に持つバックアップ（またはスナップショット）が必要です。 p.380

第 9 章

サーバー機器とデータセンター設備

この章で学ぶこと

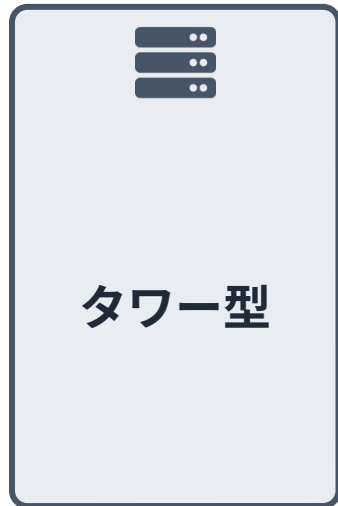
- ・サーバーの外観の 3 つの型と、ラックの高さの単位 U
- ・サーバーの中身と、外へつながる 4 種類の線
- ・ NIC ・ HBA ・ BMC の違い
- ・ 電源（ UPS ・ PDU ・ 二重電源）と冷却の物理的な配置

→ 前の章とのつながり

第 2 章で PC の部品、第 4 章でデータセンター、第 8 章で保存装置のつながり方を見ました。ここでは、サーバーと建物の設備を見ます。

サーバーの外観：タワー型・ラック型・ブレード型

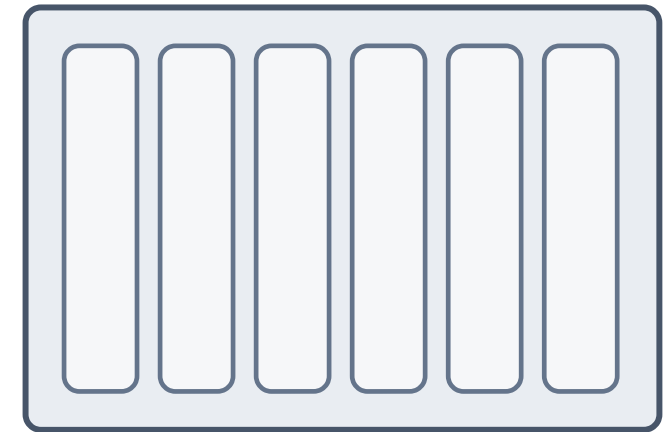
正面から見た外形の模式図。灰色の太枠＝機械、薄い灰色の箱＝その中の 1 台分。



タワー型：机の横に置ける縦長の箱。1 台で完結する。



ラック型：**ラック**（棚）に薄い箱を積み重ねる。

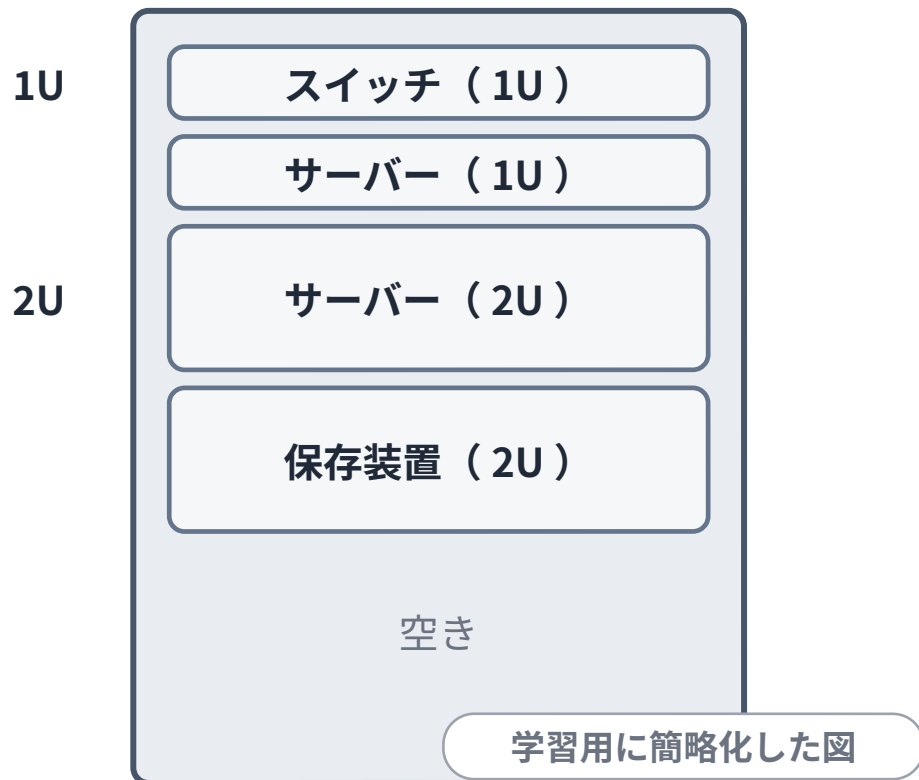


学習用に簡略化した図

ブレード型（**ブレードサーバ**）：共用の箱に薄い板状の計算機を差す。

ラックと、高さの単位 U

正面から見たラックの模式図（高さは目安）。



ラックは、機器を上下に積み重ねて載せる金属の棚です。データセンターでは、ラックが何列も並びます。

機器の高さは「U（ユー）」という単位で数えます。1Uは44.45mmで、2Uはその2倍です。ラックの高さも「42U」のようにUで表します。

⚠ 注意

高さ（U）が同じでも、消費電力や載せられるGPUの数は機種で違います。

外観の違いは、置き方と管理のしかたの違い

種類	向いている場面	考えること
タワー型	小さな事務所・机の横	設置は簡単。多数を並べるには場所をとる
ラック型	サーバー室・データセンター	高さはU単位。電源・冷却・配線の計画が必要
ブレード型	多数の計算機を高密度に置く	電源・通信・管理を共用の箱（筐体）で分け合う

ポイント

外観は、設置場所や台数の増やし方に関係します。外観だけで、そのサーバーの役割（WebサーバーかDBサーバーか）や性能は決まりません。

サーバーの中身：まずは部品だけを見る

上から見た配置の模式図（簡略化）。黄色の太枠＝このページの部品。右側に、次のページから線を足します。

サーバー（1台）の中

CPU

メモリ

GPU・拡張
カード

SSD / HDD

ファン

部品はマザーボードの上でつながる

電源ユニット 1

電源ユニット 2

中身は PC とよく似ています（第2章）。違うのは、電源ユニットを2つ持つことが多い点と、外へつながる口が多い点です。

中の部品の役割をおさらいする

部品	役割
CPU	汎用の計算をする。プログラムの命令を実行する
メモリ（RAM）	作業中のデータを置く。電源を切ると消える
GPU・拡張カード	並列の計算などを受け持つ追加の部品
SSD／HDD	データを長く保存する
ファン	空気を流して、部品の熱を外へ運ぶ
電源ユニット ×2	コンセントからの交流の電気を、部品が使う直流に変える

交流は、電気の流れる向きが周期的に入れ替わる電気（建物のコンセントの電気）、直流は、電池のように向きが一定の電気です。部品は直流で動くので、電源ユニットで変換します。

外へつながる線は 4 種類

サーバーは、目的の違う 4 種類の線で外とつながります。次のページから 1 種類ずつ見ます。

① 業務データ

Web の依頼や業務のデータを、利用者や他のサーバーとやり取りする。

② 保存装置

SAN を通じて、保存装置へ区画の読み書きをする（第 8 章）。

③ 管理

管理者が、離れた場所からサーバーの状態を見て操作する。

④ 電力

建物の電源設備から電気を受け取る。

補足

もう 1 つ、線ではありませんが、部品が出す熱を外へ運ぶ「冷却」も欠かせません（p.402）。

① 業務データの線：NIC からスイッチへ

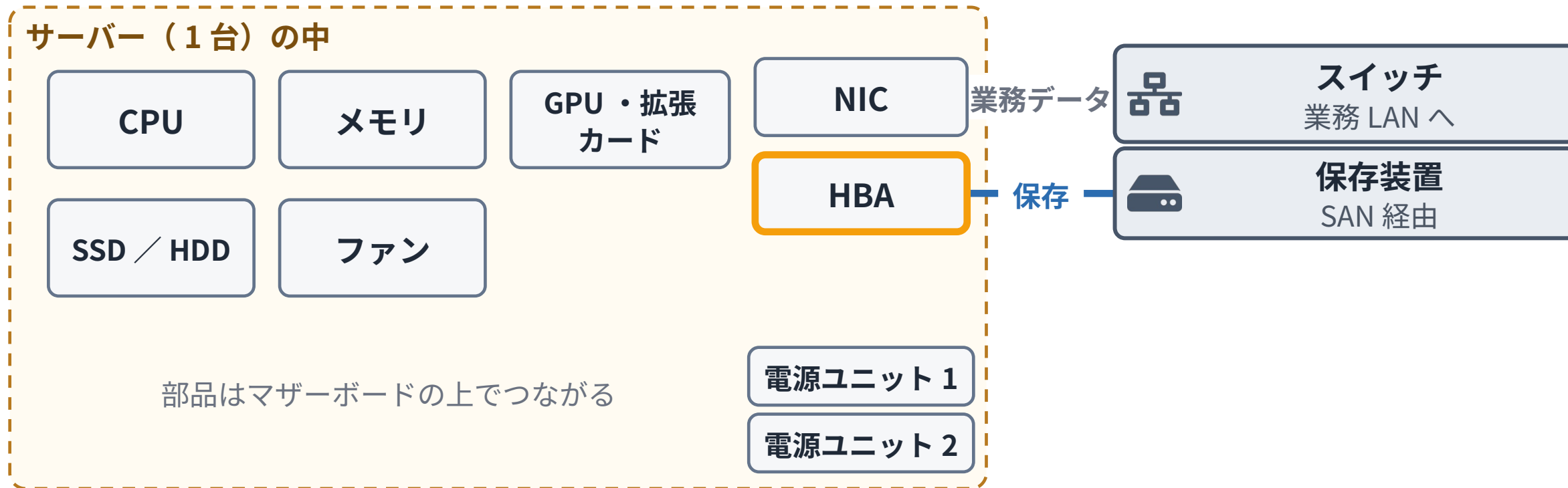
上から見た配置の模式図（簡略化）。黄色の太枠＝このページの部品。灰色の太線＝ケーブル。



NIC（ネットワークの口になる部品。第6章）からスイッチへつながり、業務 LAN で利用者や他のサーバーとデータをやり取りします。佐藤さんの「注文履歴を見せて」もこの線で届きます。

② 保存装置への線：HBA から SAN へ

上から見た配置の模式図（簡略化）。黄色の太枠＝このページの部品。青い太線＝保存装置へのケーブル。



HBA（Host Bus Adapter）は、SAN（保存装置のための専用の網。第8章）を通じて保存装置へつなぐ部品です。「区画を読み・書け」という読み書きが通ります。

NIC と HBA は、つなぐ先と運ぶものが違う

	NIC （ Network Interface Card ／ Controller ）	HBA （ Host Bus Adapter ）
つなぐ先	LAN ・ インターネット ・ 他のサーバー	SAN の保存装置など
主な約束	Ethernet ・ IP ・ TCP/UDP	ファイバチャネル、 SAS など
運ぶもの	Web の依頼 ・ 業務データなど	ディスクとしての読み書き（ブロック）
つながる機器	Ethernet のスイッチ	FC スイッチ（ SAN ）

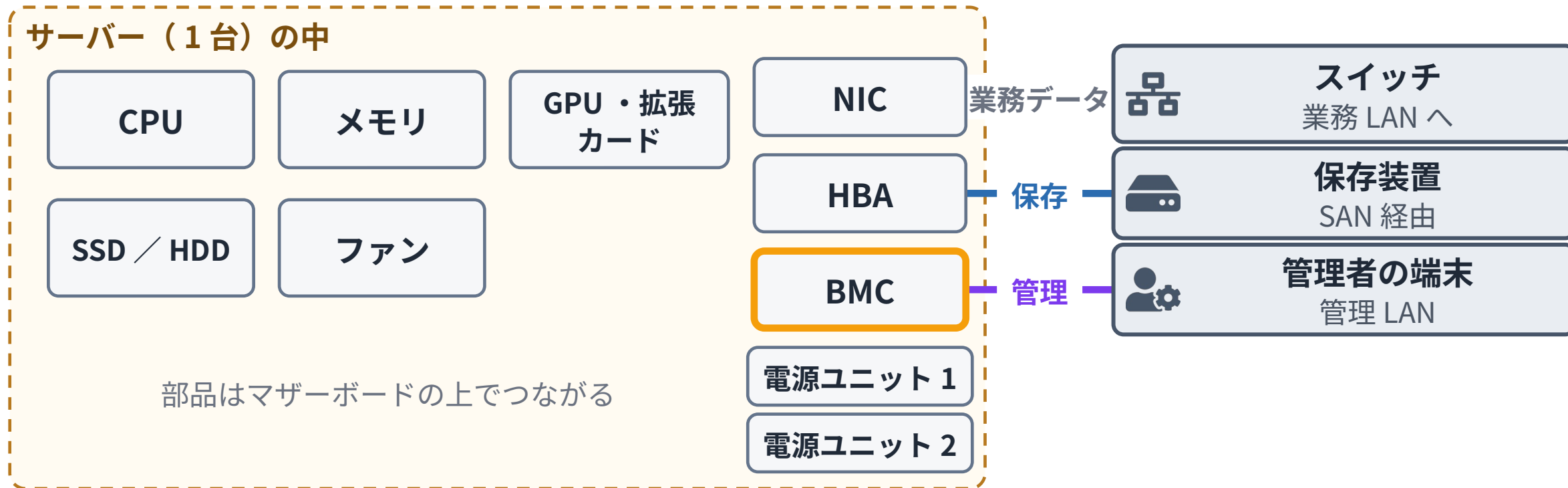
i 補足

SAS は、サーバーと保存装置をつなぐ接続の規格の 1 つです。

iSCSI のように、NIC を通して保存装置へアクセスする方式もあります（ p.363 ）。そのときは、保存の読み書きも NIC と LAN の機器を通ります。

③ 管理の線：BMC から管理 LAN へ

上から見た配置の模式図（簡略化）。黄色の太枠＝このページの部品。紫の太線＝管理用のケーブル。



BMC は、業務用の OS とは別に動く管理専用の部品です。管理 LAN につながり、管理者が離れた場所から状態を見たり電源を入れ直したりできます（次のページ）。

BMC：OSから独立した管理の仕組み

BMC（びーえむしー）

分類：部品

略語：BMC = Baseboard Management Controller（基板の管理用コントローラー）

何か	マザーボードに載った、業務用のOSから独立して動く管理用の小さなコンピューター。専用の口から管理LANにつながる。
なぜ必要か	OSが止まっても、離れた場所から状態を見て操作するため。
受け取る→する→返す	管理者の操作を受け取る → 電源の入・切、画面の遠隔表示、温度や故障の記録の確認 → 結果を返す。
身近な例	HPEのサーバーでは「iLO」という名前（p.603）。
似た言葉との違い	OSの中の監視ソフトとは別物。BMCにも独自のファームウェアがあり、OSとは別に更新が必要なことがある。

なぜ、管理の線を業務の線と分けるのか

夜中に、データセンターにあるサンプル商店の DB サーバーの OS が固まった場面を考えます（説明用の例）。

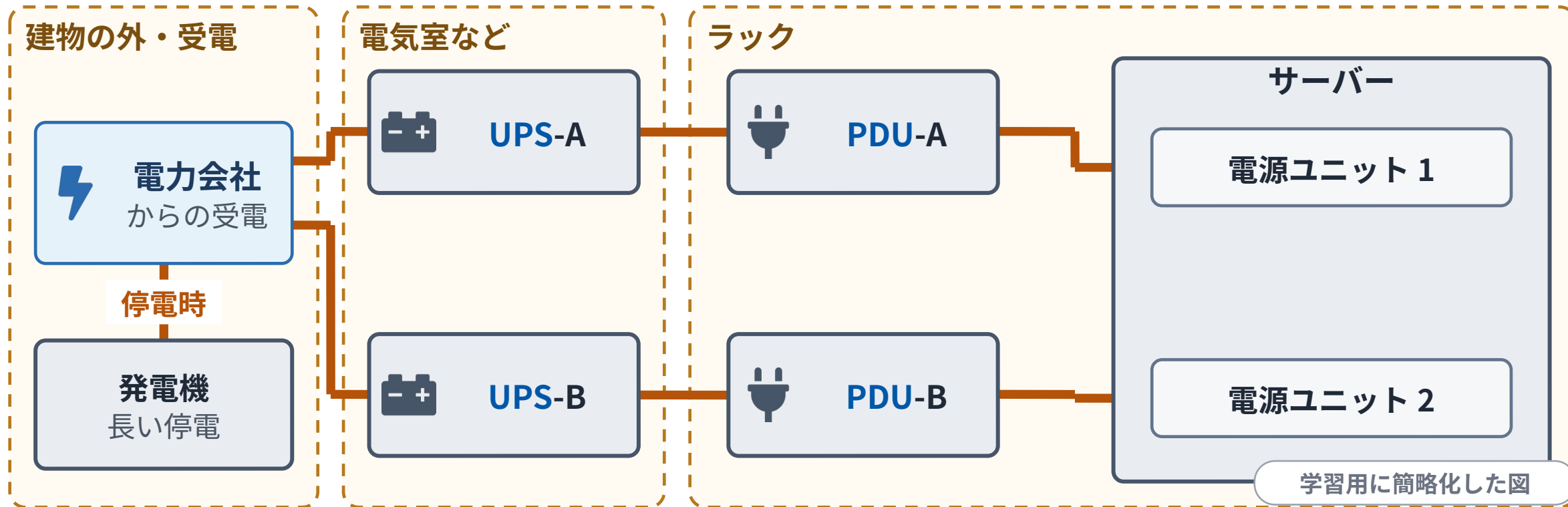
- ① OS が固まっているので、業務 LAN（NIC）からは操作できない。
- ② 管理者は自宅から管理 LAN を通じて BMC につなぐ。BMC は OS とは別に動いているので応答する。
- ③ BMC で画面を遠隔表示し、温度や故障の記録を確かめてから、電源を入れ直す。

ポイント

業務の線とは別の経路があるので、OS や業務のネットワークに問題があっても、建物へ行かずに状態を確かめて対処できます。管理の線は大事な操作ができるので、関係者だけが使えるように守ります。

④ 電力の線：受電から UPS ・ PDU を通ってサーバーへ

物理的な配置図（学習用に簡略化）。茶色の極太線＝電力。A と B の 2 つの経路に分けています。



電気は、受電→UPS→PDU→サーバーの電源ユニットの順に届きます。電源ユニットを2つ持ち、AとBの別の経路につなぐ（二重電源）と、片方が止まっても動き続けます。

電源の設備は、それぞれ何をするか

UPS

無停電電源装置。停電したとき、電池にためた電気で一定時間、給電を続ける。

発電機

長い停電のとき、燃料で電気をつくる。動き出すまでの間はUPSでつなぐ。

PDU

電源分配装置。ラックの中の各機器へ電気を配る、コンセントがたくさん付いた装置。

二重電源

サーバーが電源ユニットを2つ持ち、AとBの別の経路から電気を受ける。

ポイント

どれも**冗長化**（予備を持つこと。第8章）の考え方です。1つの装置や経路が止まっても、サーバーへの電気が途切れないようにします。

電源の備えにも、できないことがある

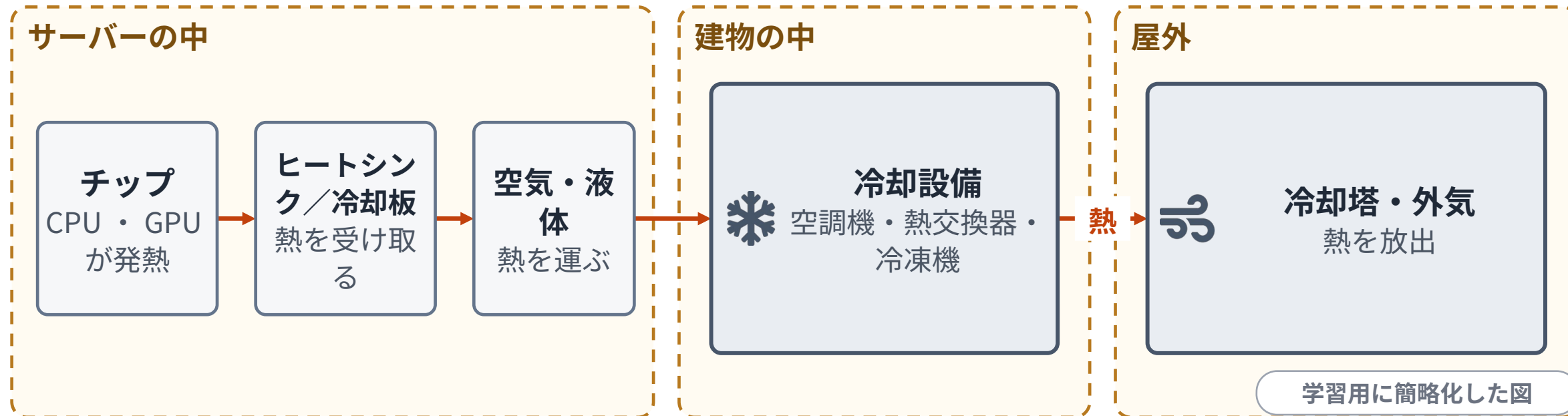
装置	役割	備えられないこと
UPS（無停電電源装置）	停電時に一定時間、電池で給電を続ける	電池の時間を超える停電
発電機	長い停電で電気をつくる	起動までの数十秒（UPS でつなぐ）
PDU（電源分配装置）	ラックの中の機器へ電気を配る	上流（電気が来る側）の装置の停止
二重電源	片方の電源ユニット・経路の故障に耐える	両方の経路が同時に止まる場合

⚠ 注意

A と B の経路をつくっても、途中で同じ装置を通っていたり、両方のケーブルを同じ PDU に差していたりすると、そこが止まったときに両方とも止まります。

冷却：計算で生まれた熱を、建物の外へ運ぶ

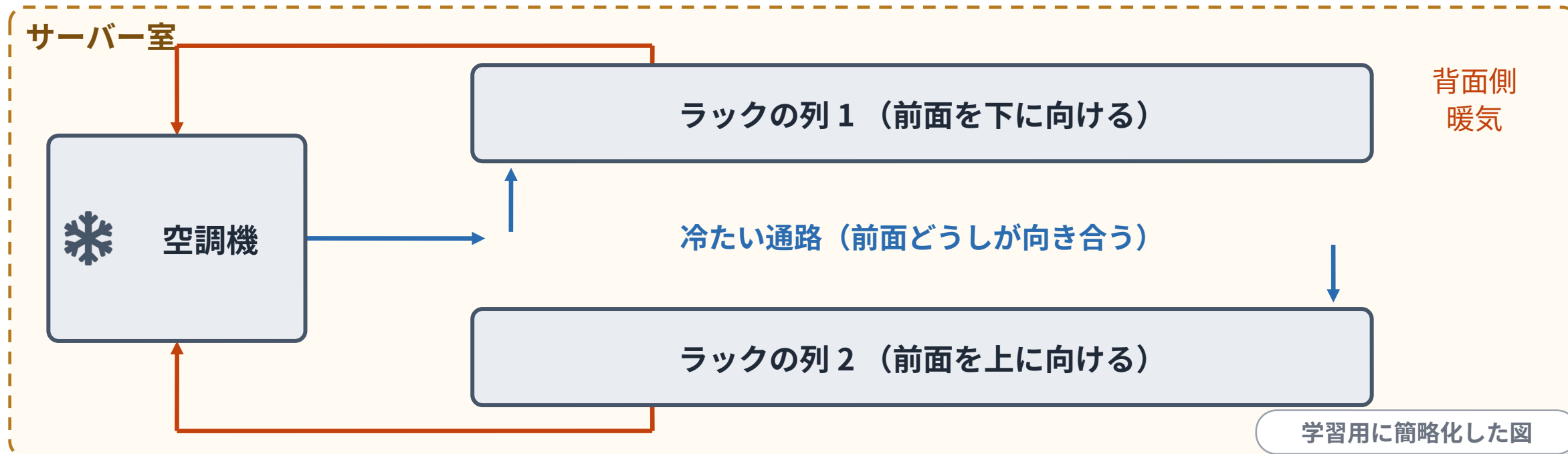
熱の流れの物理的な模式図。赤い矢印＝熱が運ばれる向き。



サーバーが使った電力の多くは、最終的に熱になります。熱がたまると部品が壊れたり、性能を落として動いたりするので、部品→空気や液体→冷却設備→屋外の順に熱を運んで捨てます。これが**冷却**です。

ラックの前から冷気を吸い、後ろへ暖気を出す

サーバー室を上から見た模式図（一般的な構成例）。青い矢印＝冷たい空気、赤い矢印＝暖まった空気。



多くのサーバーは前面から空気を吸い、背面から暖まった空気を出します。前面どうしを向かい合わせて冷たい通路をつくり、暖かい空気と混ぜないと、効率よく冷やせます。

空冷・直接液冷・液浸冷却

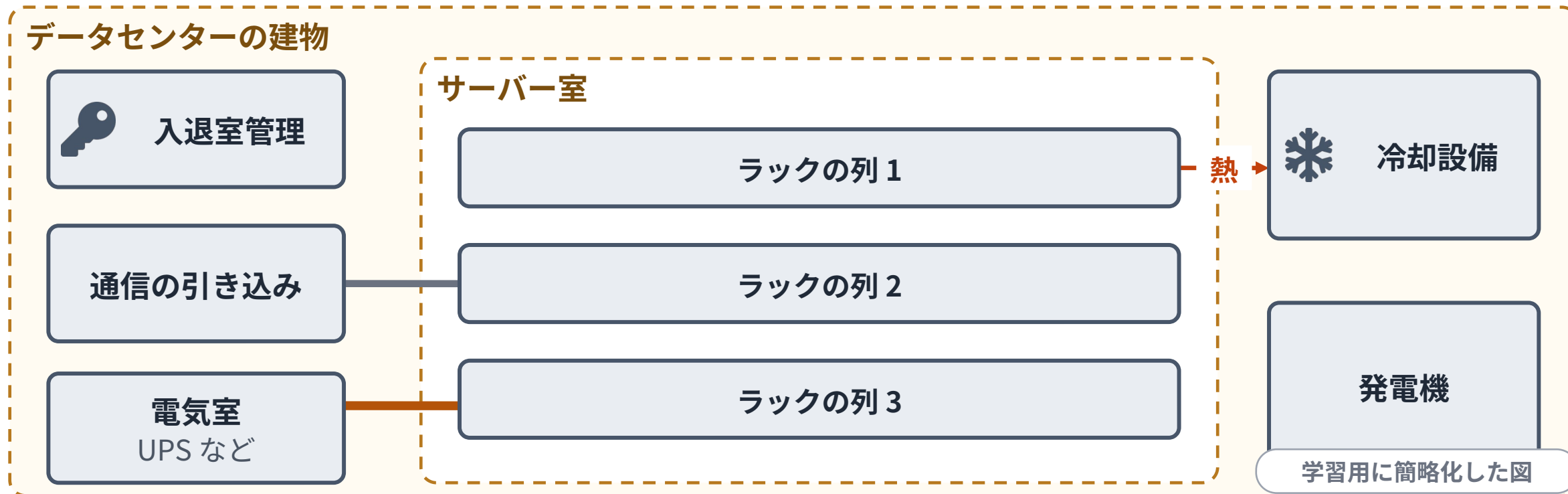
方式	仕組み	向く場面・注意
空冷	ファンで空気を流し、熱を運ぶ	多くのサーバー。高密度になると、空気だけでは運びきれないことがある
直接液冷	チップに当てた冷却板へ液体を流す	発熱の大きい GPU の基盤など。配管と漏れへの対策が必要
液浸冷却	電気を通さない液体に機器を浸す	特殊な設備が必要。水とは別の液体を使う方式もある

⚠ 注意

液体を循環させて熱を運ぶことと、蒸発などで水を消費することは別です。データセンターの電力と水の統計は第12章で扱います。

データセンターの中の配置

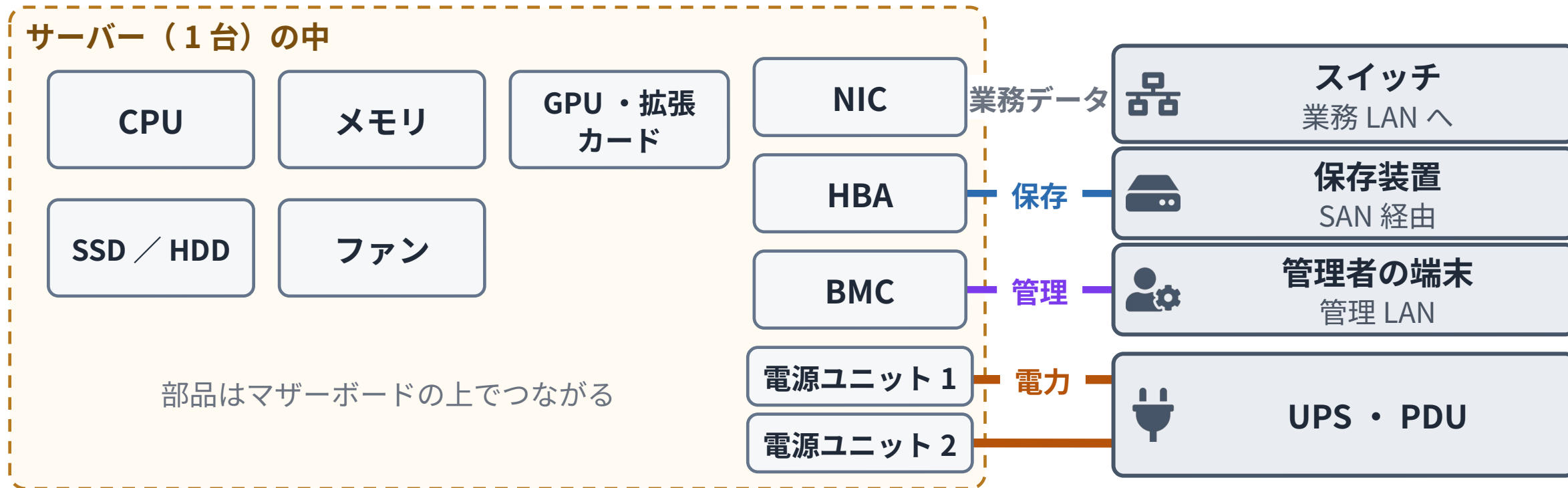
建物を上から見た模式図（一般的な構成例。配置は施設ごとに違う）。灰色の太線＝通信の回線、茶色の極太線＝電力。



サーバー室のほか、電気室（UPS など）・冷却設備・発電機・回線の引き込み・入退室の管理があります。第4章で見た建物の中身です。

全体をまとめると：サーバーと4種類の線と熱

上から見た配置の模式図（学習用に簡略化）。灰色の太線＝業務データ、青＝保存装置、紫＝管理、茶色の極太線＝電力。



足してきた線を1枚にまとめた図です。ファンが運んだ熱は冷却設備を通して外へ捨てられます。BMCはOSから独立した管理機能で、HPEではiLOと呼びます。

第9章のまとめ

✓ ここまでで説明できること

- ・ 外観の3つの型は、置き方と管理の違い。高さは $1U = 44.45\text{mm}$ で数える
- ・ 外への線は4種類：業務データ・保存装置・管理・電力
- ・ 電気は受電→UPS→PDU→二重電源の順。AとBの経路で備える
- ・ 熱はチップ→空気や液体→冷却設備→屋外へ運ぶ

→ 次へのつながり

第10章では、1台のサーバーの上で複数の仮想の機械を動かす「仮想化」と、それを使って提供される「クラウド」を学びます。

確認の問い（第9章）

自分の言葉で答えてから、次のページの解説を読みましょう。

- 1 NIC と HBA は、それぞれどこへつなぎ、何を運ぶか。
- 2 OS が固まったサーバーを、離れた場所から入れ直せるのはなぜか。
- 3 UPS があれば、どんなに長い停電でも止まらないか。

確認の問い（第9章）：解説

前のページの問いへの答えと、考え方です。

✓ 問1 NIC と HBA は、それぞれどこへつなぎ、何を運ぶか。

NIC は LAN へつなぎ、Web の依頼や業務データを運びます。HBA は SAN を通じて保存装置へつなぎ、ディスクとしての読み書き（ブロック）を運びます。 p.395

✓ 問2 OS が固まったサーバーを、離れた場所から入れ直せるのはなぜか。

BMC が OS から独立して動き、別の管理 LAN につながっているからです。管理者は BMC を通じて電源の入れ直しなどができます。 p.398

✓ 問3 UPS があれば、どんなに長い停電でも止まらないか。

いいえ。UPS が給電できるのは電池の時間までです。長い停電には発電機が必要で、発電機が動き出すまでの数十秒を UPS でつなぎます。 p.401

第 10 章

仮想化・コンテナ・クラウド

この章で学ぶこと

1 台の機械をソフトの力で複数の「仮想の機械」に分ける仕組み、アプリを部品ごと箱詰めするコンテナ、機械を買わずに借りるクラウドを順番に学びます。最後に、事業者と利用者の担当の分かれ目を確認めます。

→ 前の章とのつながり

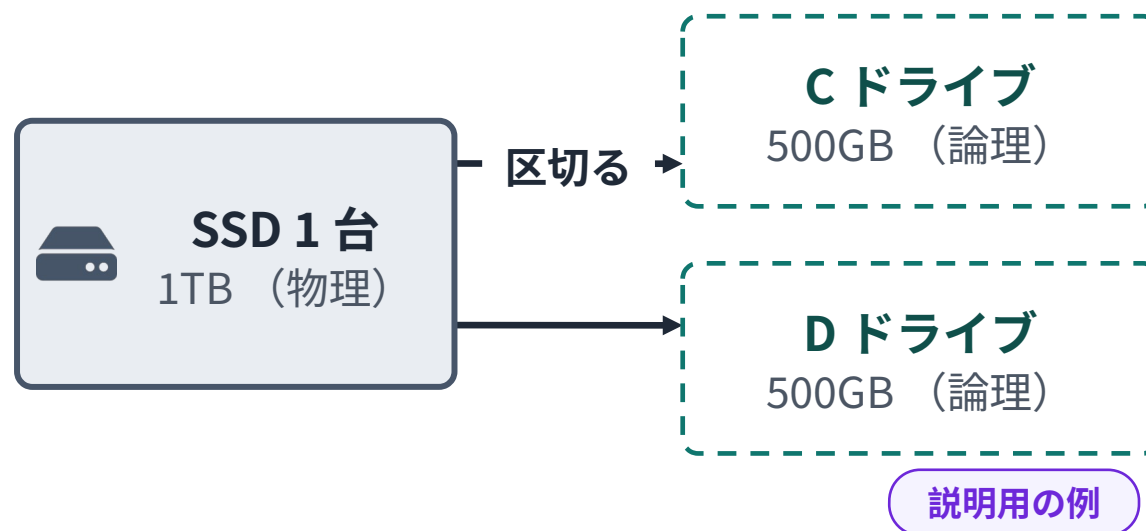
第 9 章では、サーバーがラックに収まり、電源と冷却に支えられて動く様子を見ました。この章では、その機械を「どう分けて、どう貸し出すか」を見ます。

「物理」と「論理」：SSD を 2 つに分ける例

PC の中に SSD が 1 台（1TB）入っているとします。これは手で触れられる実体なので**物理**的な装置です。

この 1 台を、OS の設定で「C ドライブ 500GB」と「D ドライブ 500GB」に分けると、画面には 2 つの保存場所が見えます。このようにソフトや設定で作った区切りを**論理**的な区切りと呼びます。

左＝実体（物理）、右＝ソフトで作った区切り（論理）。矢印は「設定で区切って見せる」という変化を表します。



⚠ 日常語との違い

日常語の「論理的（筋道が通っている）」とは意味が違います。IT の「論理」は「ソフトで作った見え方」のことです。

物理と論理を比べる：例・増やし方・限界

サーバーやネットワークでも同じ区別をします。下の表の「論理」の列が、この章で学ぶ仮想化の世界です。

比べる点	物理（実体）	論理（ソフトで作った区切り）
例	サーバー、ディスク、ケーブル	仮想の機械（VM）、仮想ディスク、仮想ネットワーク
増やし方	買って、運んで、設置する（届くまで日数がかかる）	ソフトの操作で作る・複製する・消す（数分で終わることも多い）
限界	その機械の性能まで	下にある物理資源の合計を超えて使い続けることはできない

ポイント

論理の区切りは、必ず物理の実体の上に作られます。区切りをいくら増やしても、下にある機械の CPU ・メモリ ・ディスクの量が増えるわけではありません。

仮想化とは：1 台の機械を複数の環境に分ける

仮想化（かそうか）

分類：仕組み

何か	ソフトの力で、1 台の物理的な機械を、複数の独立した論理的な環境に分けて使う技術。「仮想」は「実物ではないが、使う側からは本物と同じように見える」という意味。
なぜ必要か	1 台に 1 つの役割だけを載せると、ほとんど使われずに待っている機械が増える。分けて使えば機械が少なくて済み、新しい環境もすぐ作れる。
どこにあるか	サーバーの OS より下、機械のすぐ上で動くソフトが行う（次のページの図）。
身近な例	1 台のサーバーの中で、Web 用・データベース用・業務用の 3 つの環境を同時に動かす。

仮想化しない場合と、する場合

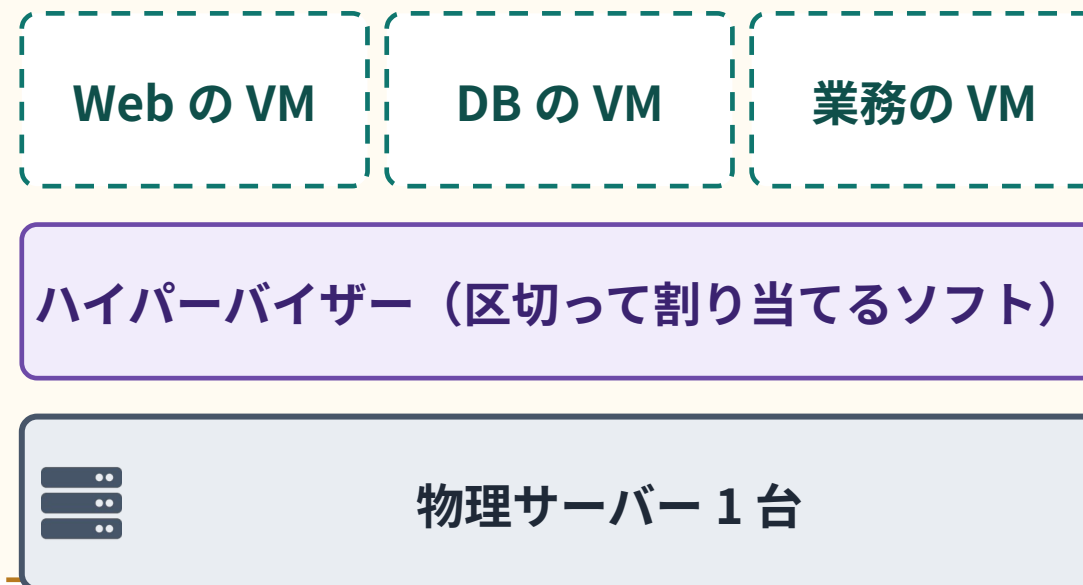
役割と層の関係図です。灰色の太枠＝物理的な機械、紫＝ソフトウェア、点線の枠＝ソフトで作った仮想の機械（VM）です。

仮想化しない場合



1 台に 1 つの役割。
どの機械も、使われていない時間や余った力が多くなりやすい。

仮想化する場合



学習用に簡略化した図

右の「VM」は、ソフトで作った仮想の機械です。「ハイパーバイザー」は、1 台の機械の力を区切って VM に配るソフトです。どちらも次のページから 1 つずつ説明します。

なぜ仮想化するか、何が限界か

❓ 良い点

機械を有効に使える：3 台分の仕事を 1 台にまとめれば、買う機械・置き場所・電気が減ります。

環境を速く作れる：新しい環境が必要になっても、機械を買って届くのを待たず、ソフトの操作で作ったり複製したりできます。

❗ 限界

資源は分け合う：VM が使う CPU ・メモリ ・ディスクは、下にある 1 台の機械のものです。

設定上は物理より多く割り当てることもできますが、全部の VM が同時にたくさん使うと足りなくなり、全体が遅くなります。

🔑 ポイント

仮想化は「機械を増やす魔法」ではなく、「1 台の力を区切って配る仕組み」です。

VM（仮想マシン）とは

仮想マシン（かそうましん）

分類：ソフトウェア

略語：VM = Virtual Machine

何か	ソフトで作った、独立したコンピューター環境。仮想的な CPU ・ メモリ ・ ディスク ・ ネットワークの口を持ち、中で OS とアプリが動く。
なぜ必要か	1 台の機械の中に、互いに影響しにくい別々の「機械」を作れる。OS を入れ直したり壊したりしても、ほかの VM には及ばない。
どこにあるか	物理サーバーの上の、ハイパーバイザー（次のページ）の上。クラウドで借りる「仮想サーバー」も VM の一種。

ポイント

中の OS から見ると、普通の機械と同じように「CPU ・ メモリ ・ ディスクがある」と見えます。実際には、下のソフトが物理の資源を割り当てています。

ハイパーバイザーとは

ハイパーバイザー（はいぱーばいざー）

分類: ソフトウェア

何か	物理的な機械の CPU ・ メモリ ・ ディスク ・ ネットワークの口を区切って、複数の VM に割り当てるソフト。
なぜ必要か	VM どうしが同じ機械の資源を取り合わないように、誰にどれだけ使わせるかを決めて見張る係が必要だから。
どこにあるか	物理サーバーと VM の間。例：ESXi、KVM、Hyper-V（製品の分類は数ページ後の表）。

1

受け取る

物理の CPU ・ メモリ ・ ディスク



2

する

区切って各 VM に割り当て、使い方を見張る



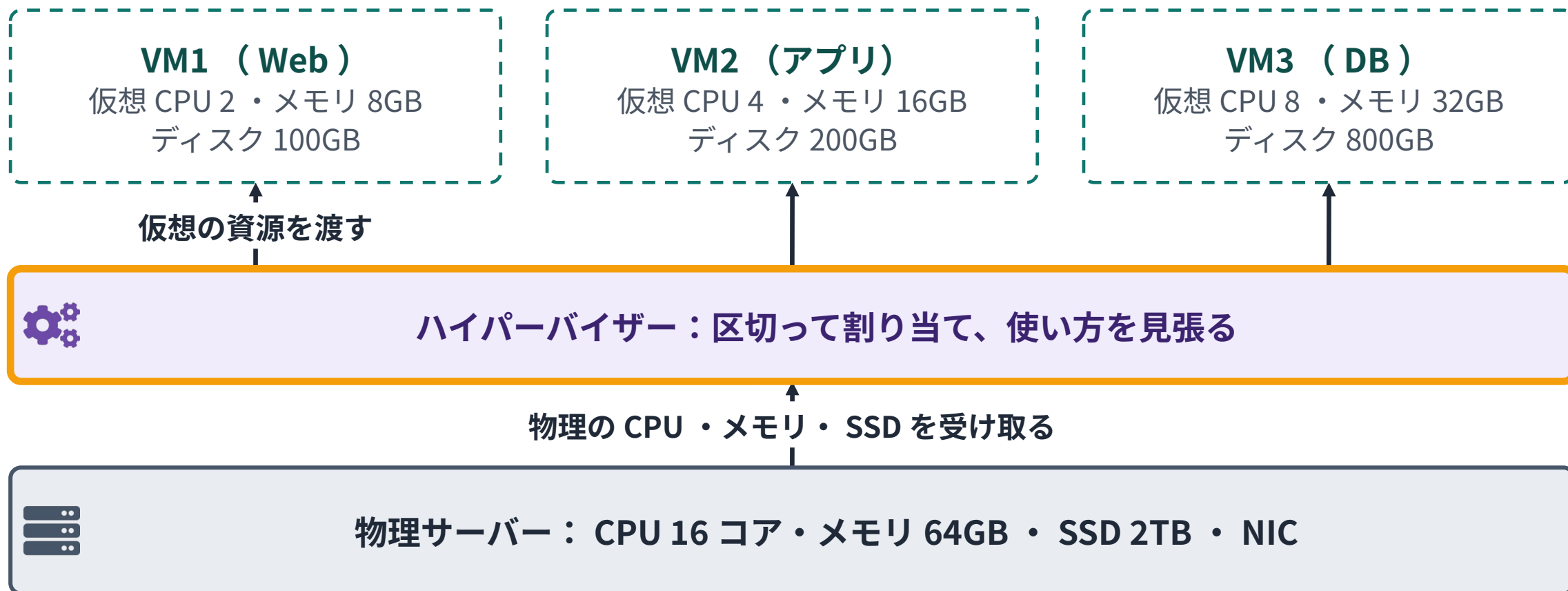
3

返す

各 VM にとっての仮想の CPU ・ メモリ ・ ディスク

ハイパーバイザーが受け取り、して、返すもの

下から上へ読む層の図です。黒い矢印は「資源を渡す」流れ。数字は説明用の例で、3つのVMの合計（CPU 14・メモリ 56GB・ディスク 1.1TB）は物理の量の範囲に収めています。



説明用の例

学習用に簡略化した図

層の図で見る：物理機に直接 OS と、仮想化

物理機に直接 OS

アプリ	利用者が使う機能
ミドルウェア	アプリを支える共通の機能
OS	機械の資源を管理
ハードウェア	物理の機械

仮想化を使う構成

アプリ	利用者が使う機能
ミドルウェア	アプリを支える共通の機能
ゲスト OS	VM の中の OS
ハイパーバイザー	追加される層
ハードウェア	物理の機械

VM の中で動く OS を**ゲスト OS**と呼びます。仮想化は構成によって加わる層で、いつも必要なわけではありません。

ハイパーバイザーの 2 つの置き方とホスト OS

層の図です（下が機械）。点線の枠＝VM、紫＝ソフトウェア、灰色＝物理の機械。

機械に直接入れる型（例：ESXi）

VM（ゲスト OS）

VM（ゲスト OS）

ハイパーバイザー

物理サーバー

OS の上で動く型（PC 用の仮想化ソフトなど）

VM（ゲスト OS）

VM（ゲスト OS）

仮想化ソフト（アプリの 1 つ）

ホスト OS（例：Windows や Linux）

PC や物理サーバー

学習用に簡略化した図

物理的な機械の上で直接動く OS を**ホスト OS**と呼びます。サーバーでは左の型が多く使われます。なお KVM は、ホスト OS である Linux の**カーネル**自体が仮想化の役を担う仕組みです。

VM でできること、注意すること

できること

VM ごとに別の OS を動かせる（例：1 台の機械で Windows と Linux）。

VM は中身がファイルとして扱えるので、作成・複製・別の機械への移動が速い。

スナップショット（ある時点の状態の記録）を取り、変更に失敗したらその時点に戻せる。

注意

VM に資源を割り当てすぎると、同じ機械の上の全 VM が遅くなる。

物理の機械が故障すると、その上の VM はすべて止まる。

スナップショットは同じ機械やストレージの中にあることが多く、**バックアップ**の代わりにはならない。

例：物理の機械が 1 台壊れると、上の VM は全部止まる

役割と層の関係図です。黄色の太枠＝故障した機械、点線の枠＝VM。

A：1 台に集める



物理サーバー 1：故障

サービス全体が止まる

B：2 台に分けて同じ役割を 2 つずつ



物理 1：故障

物理 2：正常

物理 2 の上の VM で続けられる

説明用の例

学習用に簡略化した図

B のように予備を用意して止まりにくくすることが、第 8 章で学んだ冗長化の考え方です。ただし予備を置く分、機械や設定は増えます。

VMware ・ ESXi ・ KVM ・ Hyper-V を分類で整理する

会社の名前・製品の名前・OS の機能が混ざりやすいので、「分類」をそろえて比べます。

名前	分類	何か・代表的な使い方
仮想化	一般的な技術	1 台を複数の環境に分ける考え方と仕組み
VMware	企業・ブランド名	企業向け仮想化製品群のブランド（現在は Broadcom 傘下）
ESXi	製品（ハイパーバイザー）	VMware の、物理サーバーに直接入れるハイパーバイザー。管理は vCenter など
KVM	Linux の機能	Linux のカーネルに組み込まれた仮想化。OpenStack や多くのクラウドの土台

表は次のページに続きます。

Hyper-V ・ Morpheus と、ライセンスの注意

名前	分類	何か・代表的な使い方
Hyper-V	製品・機能	Windows Server と、 Windows の一部のエディション（ Pro など）に入っているハイパーバイザー
HPE Morpheus / VM Essentials	管理ソフト・仮想 化製品	VM やクラウドの資源をまとめて管理する HPE の製品 （第 14 章）

ポイント

ESXi ・ KVM ・ Hyper-V は、名前は違っても、どれも物理の資源を区切って VM に配る「ハイパーバイザー」の役をします。

注意

ライセンスや提供条件は、買収や方針の変更で変わることがあります。導入を検討する時点で各社の公式情報を確認してください。

コンテナとは

コンテナ (こんてな)

分類: ソフトウェア

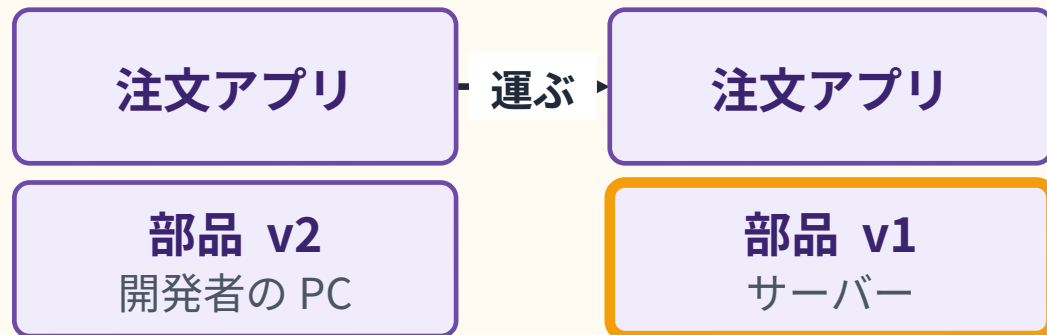
何か	アプリと、それが動くのに必要な部品をひとまとめにし、ほかのアプリと分けて動かす実行の環境。ホスト OS のカーネル（OS の中心部分）を共有し、プロセスや資源を分けて動かす。
なぜ必要か	「開発者の PC では動いたのに、サーバーでは動かない」という食い違いを減らし、同じアプリを素早くたくさん起動できる。
どこにあるか	物理の機械や VM のホスト OS の上。コンテナごとに OS を丸ごと持たない点が VM と違う。
身近な例	注文履歴を表示するアプリを、必要な部品ごと 1 つのコンテナにして、本番のサーバーで 3 つ同時に動かす。

なぜコンテナにするのか：部品の食い違いを防ぐ

アプリは単独では動かず、「ライブラリ」（日付の計算や DB への接続など、多くのアプリが使う共通の部品のプログラム）に頼っています。

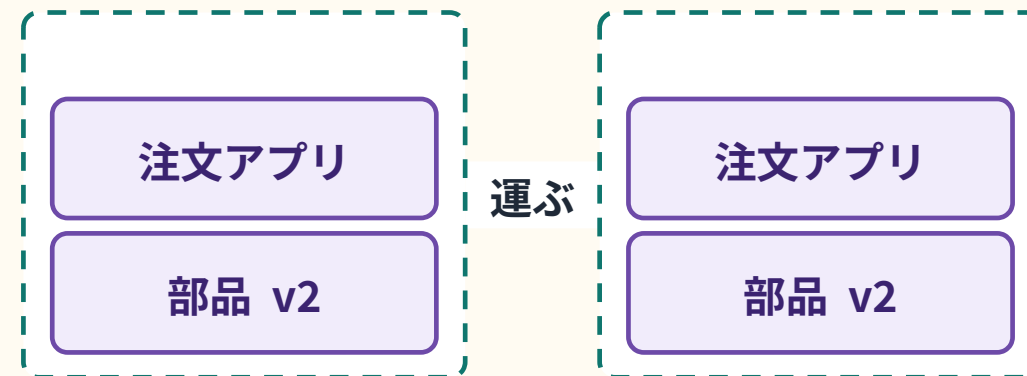
関係図です。左：アプリだけを運ぶと、サーバー側の部品の版（v1）が違って動かない。右：部品ごとコンテナに入れて運ぶので、同じ組み合わせのまま動く。

アプリだけを運ぶ



× 版が違って動かない

コンテナごと運ぶ



○ 同じ組み合わせで動く

説明用の例

学習用に簡略化した図

イメージ：コンテナを作るための雛形

イメージ (いめーじ)

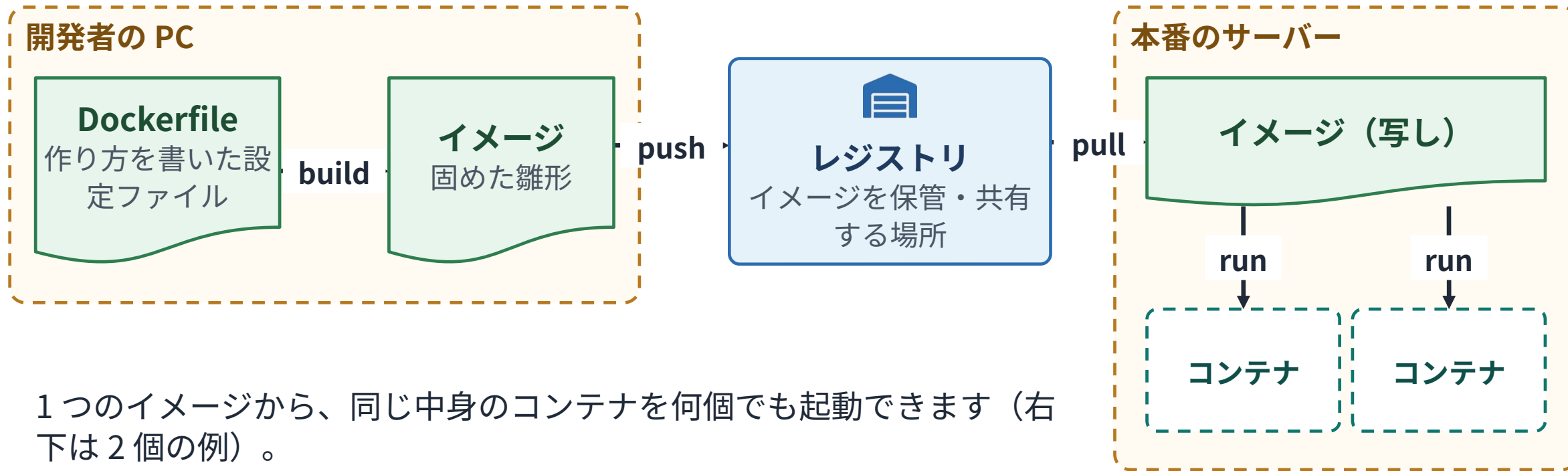
分類: データ

何か	アプリと必要な部品・設定を固めた、変更されない雛形（ファイルのまとまり）。コンテナイメージとも呼ぶ。
なぜ必要か	同じ雛形からなら、どのサーバーでも同じ中身のコンテナを何度でも起動できる。
どこにあるか	イメージを指定して起動すると、そこから実行中のコンテナが作られる。1つのイメージから多数のコンテナを同時に動かせる。
似た言葉との違い	「画像」の意味のイメージとは別。ディスクの中身を丸ごと写したファイルも「イメージ」と呼ぶことがある。

イメージを作り、配り、コンテナとして動かすための代表的な道具が **Docker** です。

Dockerfile→ イメージ→レジストリ→コンテナ

関係図です。破線の大きな枠＝場所、緑の書類形＝ファイル（データ）、点線の枠＝実行中のコンテナ。黒い矢印に、Docker の操作の名前（build・push・pull・run）を書いています。



1つのイメージから、同じ中身のコンテナを何個でも起動できます（右下は2個の例）。

学習用に簡略化した図

イメージとコンテナの違い

	イメージ	コンテナ
正体	変更されない雛形	雛形から起動した、実行中の環境
たとえ	料理のレシピと材料のセット	そのレシピで作っている最中の料理
たとえの限界	—	同じイメージから多数のコンテナを同時に動かせる。コンテナを消すと中で変えた内容は失われる（残したいデータの保存先は別に設計する）

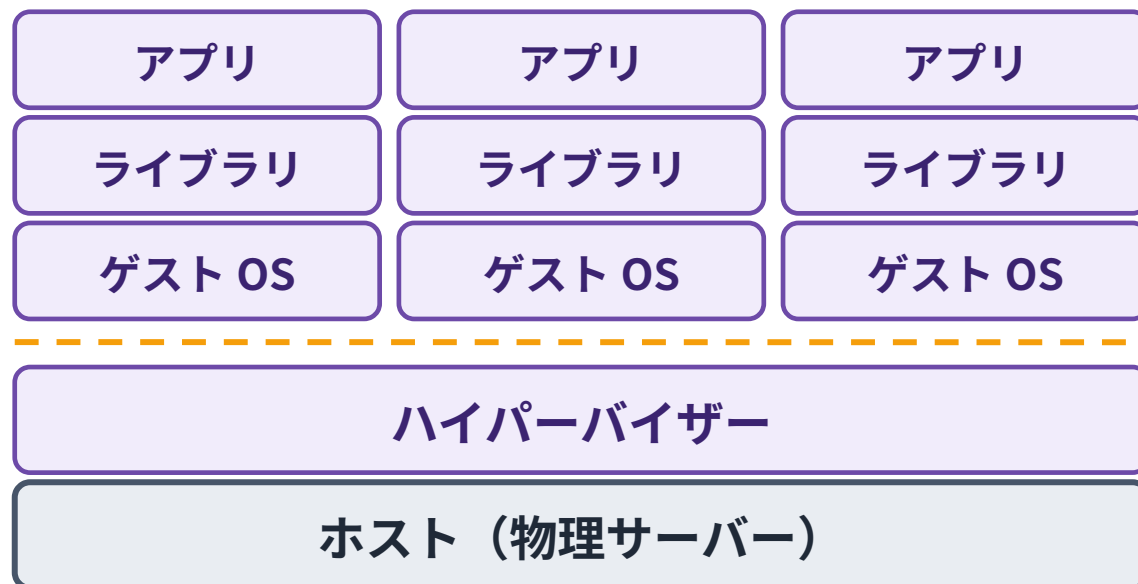
⚠ 注意

Docker はコンテナを扱う代表的な道具の名前で、コンテナという考え方そのものや、OS の名前ではありません。

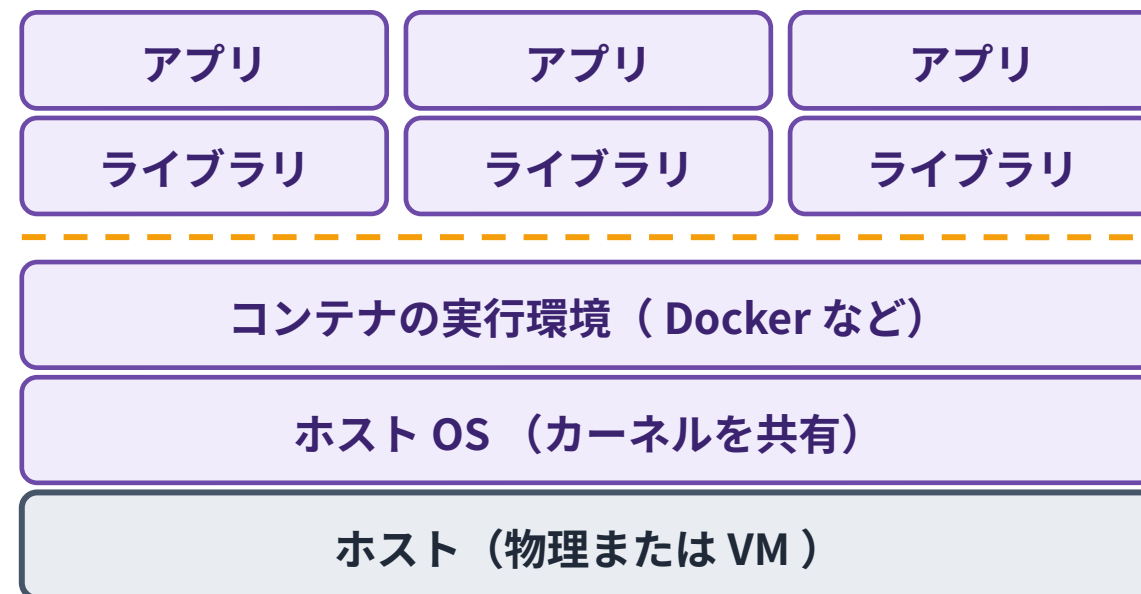
VM とコンテナは、どこから上を分けるかが違う

層の図です（下が機械）。黄色の破線は「ここから上を、利用単位ごとに分ける」という境目です。VM はゲスト OS から上を、コンテナはアプリと部品（ライブラリ）だけを分けます。

VM（仮想マシン）



コンテナ



学習用に簡略化した図

VM とコンテナの比べ方と、組み合わせ

	VM	コンテナ
分ける単位	OS ごと	アプリ（プロセス）ごと
起動	OS の起動が必要（数十秒～数分の場合も）	プロセスの起動に近く、短いことが多い
別の OS	Windows と Linux を同じ機械で動かせる	ホストのカーネルに合うものが基本
向く用途	OS ごとに分けたい、既存のシステムの移し替え	アプリの配布、頻繁な更新、多数の複製

補足

どちらが優れているかではなく、目的で使い分けます。コンテナを VM の上で動かすことも多く、両者は排他ではありません。分離の強さは VM の方が一般に高いとされますが、設定や実装によって変わります。

同じ「コンテナ」でも、別のものを指すことがある

第 7 章の「サーブレットコンテナ」（p.339）は、この章のコンテナとは別物です。

	この章のコンテナ（ Docker など）	サーブレットコンテナ（ Tomcat など）
正体	アプリと部品を分けて動かす、 OS の上の実行の環境	Java で書かれた Web アプリの部品を動かすミドルウェア
中に入るもの	どんな言語のアプリでも（ Java に限らない）	Java のサーブレット（ Web の要求に応える部品）
関係	サーブレットコンテナごとコンテナに入れて動かすこともある	—

ポイント

「コンテナ」は「入れ物」という普通の英語なので、分野ごとに別の物を指します。何を入れる入れ物なのかを文脈で確かめます。

Kubernetes とは：多数のコンテナを保ち続ける基盤

Kubernetes (くーばねていす)

分類：ソフトウェア

何か	コンテナにしたアプリの配置・数の増減・故障したときの再起動を、自動で行う基盤のソフトウェア。
なぜ必要か	コンテナが数十～数百になると、人が 1 つずつ起動したり、止まっていないか見張って直したりするのは追いつかない。
どこにあるか	複数の機械（物理または VM）をまとめた上で動く。クラウドでもサービスとして提供されている。

1

受け取る

「こうあってほしい状態」の宣言



2

する

実際の状態と比べ、差があれば直す



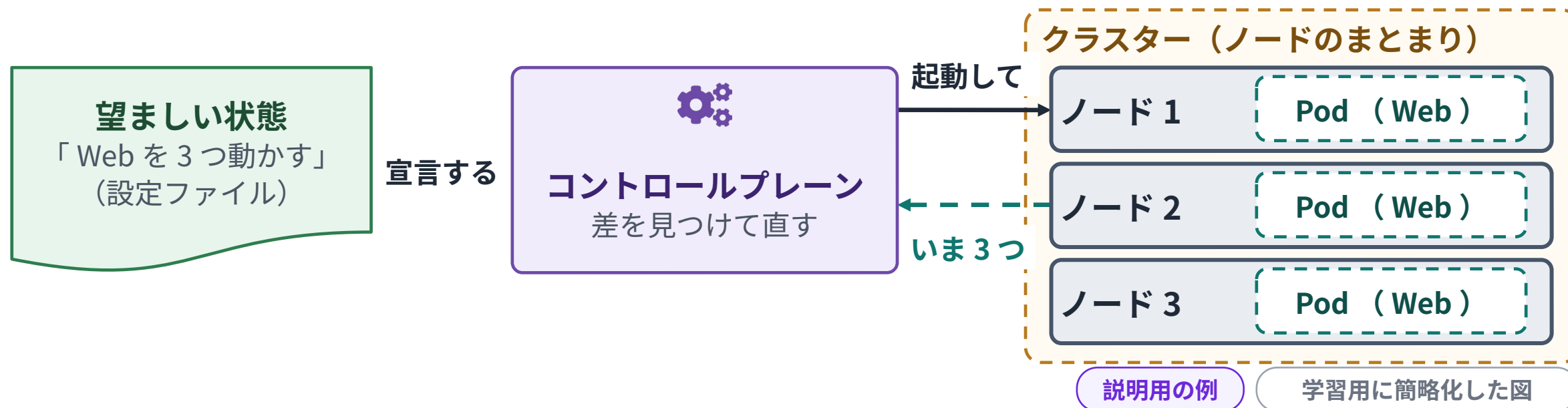
3

返す

宣言どおりに動き続けるアプリ

「望ましい状態」を宣言する：Web を 3 つ動かす

関係図です。黒い矢印＝宣言や指示、緑の破線＝報告。「Pod」はコンテナを入れて配置する単位、「ノード」はコンテナを実行する機械です。



💡 たとえ

エアコンに 25℃ と設定すると、温度を測って冷やす・弱めるを繰り返します。

❗ たとえの限界

直せるのは宣言した範囲だけです。アプリの不具合や機械全体の不足は直せません。

クラスター・ノード・Pod の関係

複数の機器や環境をまとめて 1 つのように扱う単位を **クラスター** と呼びます。

用語	意味	混同しやすい点
クラスター	複数の機器・環境をまとめて扱う単位	まとめただけで、必ず止まりにくくなるとは限らない
ノード	コンテナを実行する機器・VM	物理の機械でも VM でもよい
Pod	コンテナをまとめた、最小の配置の単位	コンテナ 1 つとは限らない（密接に協力する複数を入れることがある）

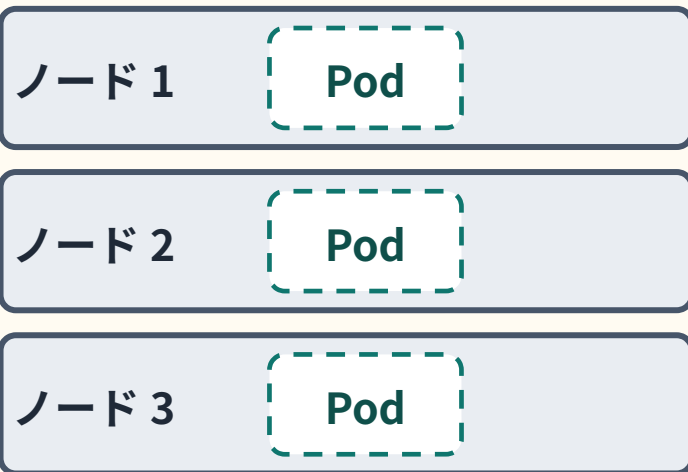
⚠ 誤解しやすい点

Kubernetes は Docker と同じ道具でも、OS そのものでも、アプリのプログラムを自動で作る製品でもありません。コンテナを動かす道具（Docker など）の上で、配置と数を管理する係です。

ノードが 1 台故障したとき（連続した 3 つの場面）

左→右へ時間が進む連続図。灰色＝ノード（機械）、点線＝Pod（Web のコンテナ）、黄色＝変わった所。

① ふだん：3 つ



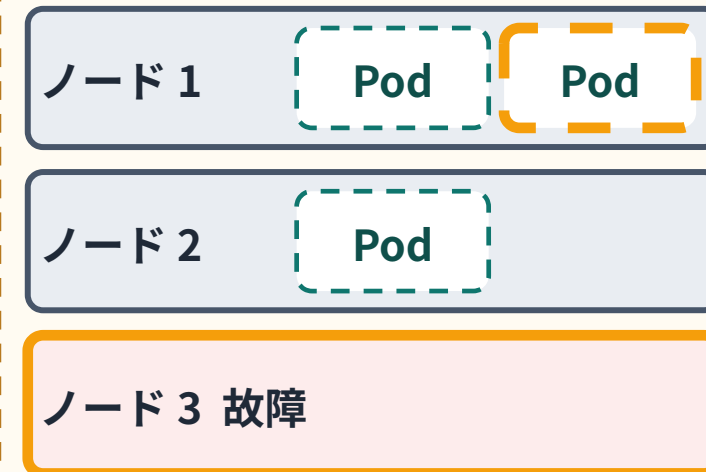
実際 3 = 望ましい 3

② ノード 3 が故障



実際 2 ≠ 望ましい 3

③ 別のノードで起動



実際 3 = 望ましい 3

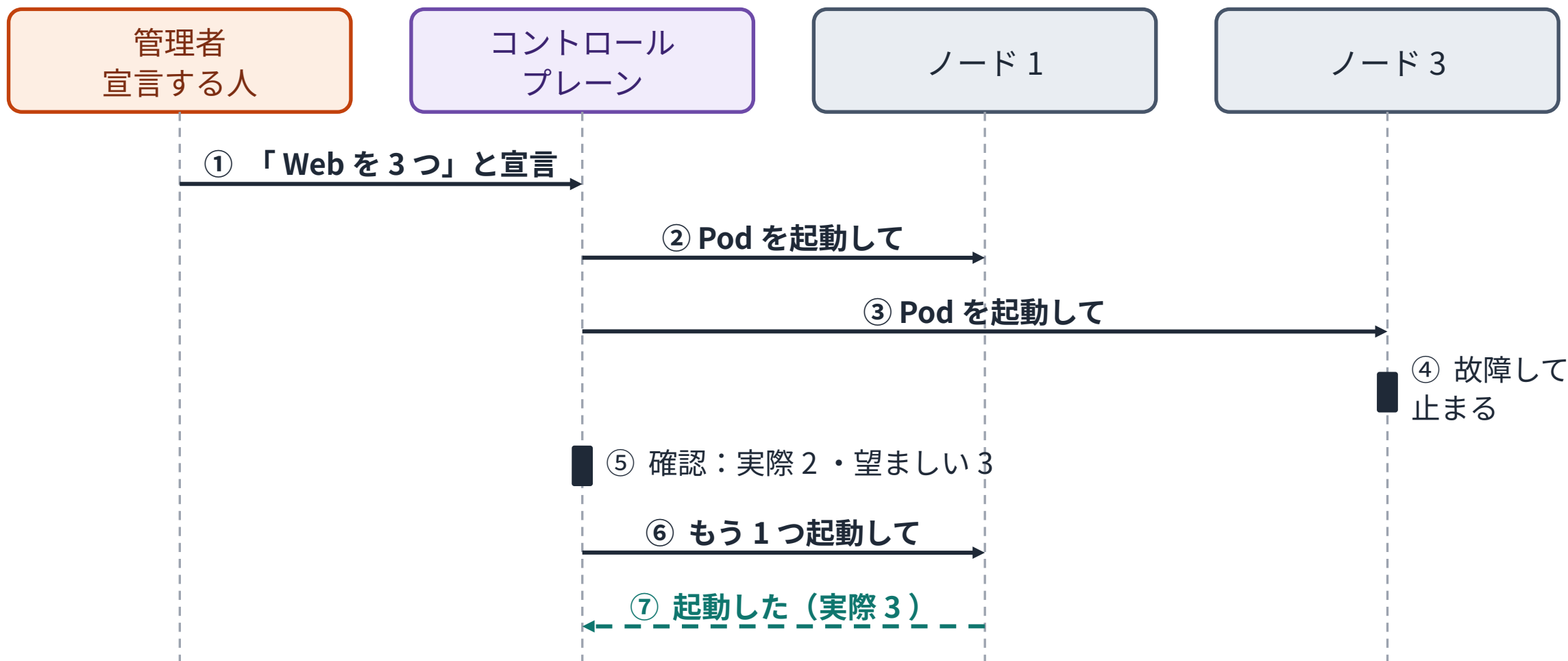
説明用の例

学習用に簡略化した図

人が操作しなくても基盤が 3 つに戻します。ただし検知と起動の間は 2 つで処理します。

同じ場面を時間順のやり取りで見る

上から下へ時間が進みます。黒＝指示、緑の破線＝報告。ノード 2 は省略。



確認の問い（仮想化とコンテナ）

自分の言葉で答えてから、次のページの解説を読みましょう。

- 1 物理サーバーが 1 台故障すると、その上の 3 つの VM はどうなりますか。
- 2 VM とコンテナは、それぞれどこから上を分けますか。
- 3 「Web を 3 つ」と宣言した後、ノードが 1 台壊れると何が起きますか。

確認の問い（仮想化とコンテナ）：解説

前のページの問いへの答えと、考え方です。

✓ 問 1 物理サーバーが 1 台故障すると、その上の 3 つの VM はどうなりますか。

3 つとも止まります。VM は同じ物理の機械の資源を区切って使っているからです。止めたくない場合は、別の物理サーバーにも同じ役割の VM を置きます（冗長化）。 p.422

✓ 問 2 VM とコンテナは、それぞれどこから上を分けますか。

VM はゲスト OS から上を丸ごと分けます。コンテナはホスト OS のカーネルを共有し、アプリと部品（ライブラリ）だけを分けます。 p.430

✓ 問 3 「Web を 3 つ」と宣言した後、ノードが 1 台壊れると何が起きますか。

実際の数（2 つ）と望ましい数（3 つ）の差を基盤が見つけ、別のノードで Pod を起動して 3 つに戻します。 p.436

置き場所・管理する人・提供方式は別の軸

「クラウドかオンプレミスか」は、場所だけの話ではありません。3つの軸に分けると混乱しません。

軸	選択肢の例	説明
置き場所	自社のサーバー室／借りたデータセンター／事業者のデータセンター	機器が物理的にある建物
管理する人	自社／委託先／クラウド事業者	機器・OS・設定を誰が面倒を見るか
提供方式	自分で所有・構築する／サービスとして利用する	必要な分を申し込み、使った量で払うか

📖 具体例（説明用の例）

サンプル商店が「借りたデータセンターに自社の機械を置き、自社の社員が管理する」なら、置き場所は外部でも、管理と所有は自社です。

オンプレミスとデータセンター

オンプレミス (おんぷれみす)

分類: 考え方

何か	組織が自分で管理する環境に設備を置いて運用する考え方。英語の「構内で (on premises)」から。
なぜ必要か	機械・OS・設定を自分で細かく決められる。その代わり、購入・設置・故障の対応・更新も自分の仕事になる。
どこにあるか	自社ビルのサーバー室だけでなく、借りたデータセンターに自社の機械を置いて自社で管理する形も含む。
似た言葉との違い	データセンターは建物（施設）の名前。そこでクラウドが提供されることもあるが、施設とクラウドは別物。

クラウドとは：機械を買わずに、必要な分を借りる

クラウドコンピューティング

分類：サービス

何か	計算・保存・通信などの資源を、ネットワーク経由で、必要なときに必要な分だけ申し込んで使い、要らなくなれば解約できる提供方式。略してクラウド。
なぜ必要か	機械を買うと、届くまで日数がかかり、混む日に合わせて買えば普段は余る。借りる方式なら、使う量に合わせて増減できる。
どこにあるか	事業者のデータセンターにある大量の機械（多くは VM として貸し出される）を、利用者がインターネットなどを通して使う。
似た言葉との違い	「遠くにサーバーを置いただけ」ではクラウドとは限らない。次のページの 5 つの特徴で判断する。

クラウドの 5 つの特徴（NIST の定義）

NIST（米国の標準化機関）が挙げる 5 つの特徴です。右の列は説明用の例です。

特徴	意味	サンプル商店の例（説明用）
① セルフサービス	画面や API（プログラムから操作する窓口）で、必要なときに自分で資源を用意できる	管理画面で仮想サーバーを申し込むと、事業者の人の手続きを待たずに使える
② ネット経由で利用	ネットワーク経由で、さまざまな端末から使える	社員が会社の PC から、出張先のノート PC から管理画面を開ける
③ 資源の共有	事業者の資源を多数の利用者で分け合い、割り当てる	サンプル商店と別の会社の VM が、同じ建物の機械を分け合うことがある
④ 伸び縮み	負荷に合わせて増やしたり減らしたりできる	セールの日だけ Web サーバーを 3 台から 6 台に増やし、翌日戻す
⑤ 利用量の計測	使った量を測り、見える形にする（料金・管理に使う）	何台を何時間使ったかが記録され、月ごとの請求に表れる

電気・水道のたとえと、その限界

💡 たとえ：電気・水道

家庭は発電所を持たず、電気を使った分だけ払います。クラウドも、機械を持たずに使った分だけ払う点が似ています。

❗ たとえの限界

設定・権限・データの管理は利用者の仕事として残ります。誰が見られるかの設定を誤れば、利用者の事故になります。

止めても料金がかかる資源があります。仮想サーバーを止めても、ディスクが残れば課金が続くことがあります。

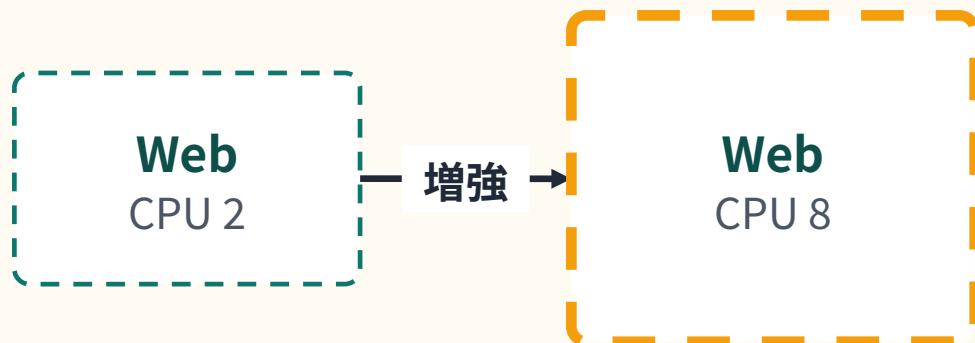
⚠️ 練習するときの注意

練習では、使い終わった資源を削除し、予算の通知（設定した金額を超えそうなら知らせる機能）を設定しておく、と思わぬ請求を防げます。

「伸び縮み」の 2 つの方向：スケールアップとアウト

関係図です。左は 1 台を大きくする、右は同じ役割の台数を増やす。右では、要求を振り分ける負荷分散の仕組みが前に必要です。

スケールアップ（1 台を強く）



スケールアップ：1 台の CPU やメモリを増やします。アプリを作り替えずに済みますが、1 台の上限があり、増やすときに止める必要がある場合もあります。

スケールアウト（台数を増やす）



説明用の例

学習用に簡略化した図

スケールアウト：同じ役割の台数を増やし、**負荷分散**で仕事を分けます。台数を戻すことも簡単ですが、アプリが複数台で動ける作りである必要があります。

プライベートクラウドとパブリッククラウド

クラウドは「誰が使うか」でも分けられます。

プライベートクラウド

プライベートクラウドは、1つの組織だけが使うクラウドです。申し込み・自動化・計測などクラウドの特徴を持ちながら、使う人を自社に限ります。

- 置き場所：自社または外部のデータセンター
- 管理：自社または委託先
- 外部の施設や委託管理でも成り立つ

パブリッククラウド

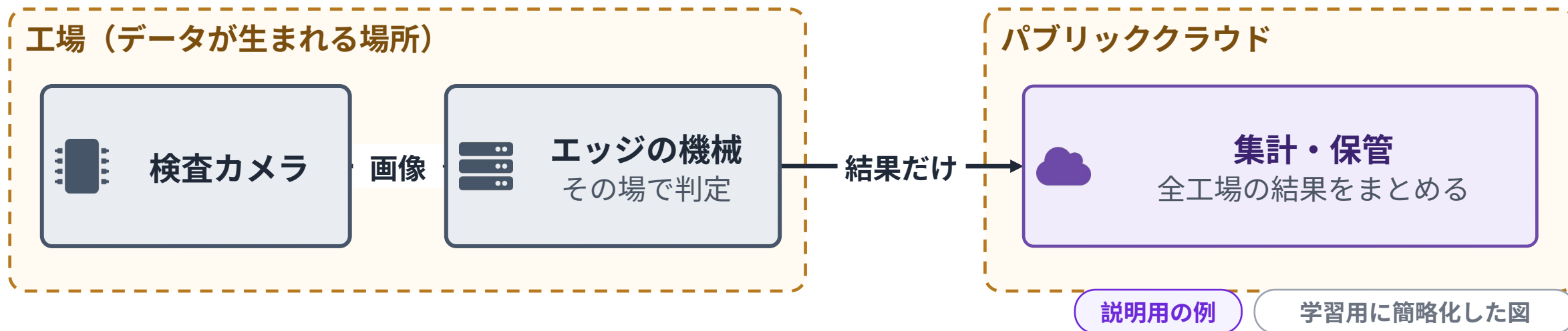
パブリッククラウドは、事業者の資源を広く多数の顧客で分け合って使うクラウドです。

- 置き場所：事業者のデータセンター
- 機械の管理：事業者
- 中の設定やデータの管理：利用者

ハイブリッドクラウドとエッジ

複数の環境を組み合わせる使う形が**ハイブリッドクラウド**、データが生まれる場所の近くで処理する考え方が**エッジコンピューティング**（エッジ）です。

配置と情報の移動を合わせた図です。黒い矢印＝データの移動。



エッジとクラウドは対立する方式ではなく、組み合わせて使います。工場ですぐ判定し、集計はクラウドで行う、という分担がその例です。

5つの形態を、場所・使う人・管理する人で比べる

形態	設置場所の例	使う人	管理する人の例	ひとことで
オンプレミス	自社のサーバー室・借りたデータセンター	自社	自社（委託も）	自分で所有・管理する設備
プライベートクラウド	自社・外部のデータセンター	特定の組織だけ	自社または委託先	1つの組織専用のクラウド
パブリッククラウド	事業者のデータセンター	広く多数の顧客	事業者（設定は利用者）	多数で共有するクラウド
ハイブリッド	上の組み合わせ	—	—	複数の環境を組み合わせる
エッジ	工場・店舗・基地局の近く	その場の機器・利用者	利用者・事業者	発生場所の近くで処理

HPE の GreenLake は、オンプレミスとクラウドの資源をまとめて利用・管理する基盤の例です（製品は第 14 章）。

IaaS ・ PaaS ・ SaaS ：どこまでを借りるか

クラウドは「どの層まで事業者が用意してくれるか」でも分けます。サンプル商店に当てはめると次のようになります。

IaaS

Infrastructure（基盤） as a Service。仮想サーバーや保存領域を借ります。

- ・ 例：仮想サーバーを借り、OS を選び、注文アプリと DB ソフトを自分で入れる
- ・ OS の更新も利用者

PaaS

Platform（実行の基盤） as a Service。アプリを動かす場所を借ります。

- ・ 例：注文アプリのプログラムを置くだけで動く
- ・ OS やミドルウェアの更新は事業者

SaaS

Software（ソフト） as a Service。完成したソフトを使います。

- ・ 例：社員がブラウザでメールや表計算のサービスを使う
- ・ 利用者は使い方・データ・権限を管理

3 つとも「as a Service」＝「サービスとして（買わずに借りて）使う」という意味です。

どの層を誰が担当するか（責任の分かれ目）

下ほど機械に近い層です。オレンジ＝利用者が担当、青＝事業者が担当。

層	オンプレミス	IaaS	PaaS	SaaS
データ・利用者の権限	利用者	利用者	利用者	利用者
アプリ	利用者	利用者	利用者	事業者
ミドルウェア・実行環境	利用者	利用者	事業者	事業者
OS	利用者	利用者	事業者	事業者
仮想化	利用者	事業者	事業者	事業者
サーバー・保存・通信	利用者	事業者	事業者	事業者
施設（電力・冷却）	利用者	事業者	事業者	事業者
例（分類の後に製品名）	自社の設備	AWS EC2、Azure VM	Azure App Service など	Microsoft 365、Gmail

IaaS→PaaS→SaaS の順に、事業者の担当が上の層へ広がります。

データと権限の設定は、どの形態でも利用者の責任

前のページの表で、一番上の「データ・利用者の権限」はどの列でも利用者の担当でした。事業者が機械やソフトをどれだけ管理してくれても、「どのデータを置くか」「誰に見せるか」を決めるのは利用者です。

📖 具体例（説明用の例）

SaaS のメールを使っている、注文データの表を「リンクを知っている全員が見られる」設定で共有してしまえば、それは利用者の設定の誤りです。弱いパスワードの放置も同じです。

クラウドで、どこまでを事業者が守り、どこからを利用者が守るかの分担を示した考え方を**責任共有モデル**と呼びます。境界は製品・契約ごとに違うので、提供元の資料で確かめます。

AWS ・ Azure ・ Google Cloud のサービスの対応

主要なパブリッククラウドは、似た種類のサービスをそろえています。名前を覚えるより、種類と対応を押さえます。

種類	AWS	Microsoft Azure	Google Cloud	何をするか
仮想サーバー (IaaS)	Amazon EC2	Azure Virtual Machines	Compute Engine	VM を借りる
オブジェクトストレージ	Amazon S3	Azure Blob Storage	Cloud Storage	大量のファイル・データを保存
ブロックストレージ	Amazon EBS	Azure Managed Disks	Persistent Disk	VM のディスク
マネージド RDB	Amazon RDS ・ Aurora	Azure SQL Database	Cloud SQL	DB の運用の一部を任せる
コンテナの管理	Amazon EKS	Azure Kubernetes Service	Google Kubernetes Engine	Kubernetes を提供

マネージドサービスと、バケットの公開設定

🔑 マネージドサービス

マネージドサービスは、運用の一部（ソフトの更新・バックアップなど）を事業者が担うサービスです。

任せられるのは運用の一部で、利用者の設定やデータの責任がなくなるわけではありません。

⚠️ バケット（S3 の例）

バケットは、オブジェクトストレージ（Amazon S3 など）でデータを入れる入れ物です。

公開の設定を誤ると、見せたくないデータが外部から誰でも読める事故につながります。

📖 具体例（説明用の例）

サンプル商店が、商品画像を公開用のバケットに置いているとします。同じバケットに注文データのファイルを入れてしまうと、画像と同じように外部から読めてしまいます。公開するものと、しないものは入れ物を分けて設定します。

VM を作れることと、クラウドとして提供することは違う

ハイパーバイザーで VM を作れても、申し込みの画面・利用量の計測・権限の分離がなければ、クラウドとしては足りません。

利用者の画面・API 申し込むと VM や保存領域ができる
(セルフサービス)

クラウド管理 OpenStack・Morpheus など：利用者・権限・割当・計測・自動化

仮想化 KVM・ESXi など：VM を作って動かす

物理資源 サーバー・保存・通信の実際の機械

OpenStack

OpenStack は、計算・保存・ネットワークなどの資源を、クラウドとして提供・管理するオープンソースのソフト群です。KVM や物理の機器とは分類が違います。

API は、人が画面を操作する代わりに、プログラムから申し込みや操作をするための窓口です。

仮想化基盤だけと、プライベートクラウドの違い

	仮想化基盤だけ	プライベートクラウド
VM の作成	管理者が依頼を受けて作る	利用者が自分で申し込める
利用量	把握しにくい	計測し、部門ごとに見える
権限	管理者が中心	組織・プロジェクトごとに分ける

📖 具体例（説明用の例）

サンプル商店の開発部が試験用の VM を欲しいとき、仮想化基盤だけなら管理者にお願いして待ちます。プライベートクラウドなら、自分の部の割り当ての範囲で自分で作れ、使った量は部ごとに集計されます。

第 10 章のまとめ

✓ ここまでで説明できること

物理（実体）と論理（ソフトで作った区切り）の区別、仮想化は 1 台の力を区切って VM に配る仕組み。

VM はゲスト OS から上を、コンテナはアプリと部品だけを分ける。Kubernetes は望ましい状態を保ち続ける。

置き場所・管理する人・提供方式は別の軸。クラウドの 5 つの特徴と、プライベート／パブリック／ハイブリッド／エッジ。

IaaS・PaaS・SaaS で担当の層が変わっても、データと権限の設定は利用者の責任。

→ 次へのつながり

「誰に何を見せるか」を決める仕組みが大切だと分かりました。次の第 11 章では、認証・認可・暗号化・ファイアウォールなど、セキュリティの基本を 1 つずつ学びます。

確認の問い（第 10 章）

自分の言葉で答えてから、次のページの解説を読みましょう。

- 1 SSD を C ドライブと D ドライブに分けるとき、物理と論理はどれですか。
- 2 遠くのデータセンターにサーバーを置けば、クラウドと言えますか。
- 3 SaaS を使えば、データの公開設定は事業者の責任になりますか。

確認の問い（第 10 章）：解説

前のページの問いへの答えと、考え方です。

✓ 問 1 SSD を C ドライブと D ドライブに分けるときの、物理と論理はどれですか。

SSD 1 台が物理（手で触れられる実体）、C ドライブと D ドライブが論理（設定で作った区切り）です。 p.411

✓ 問 2 遠くのデータセンターにサーバーを置けば、クラウドと言えますか。

言えるとは限りません。自分で申し込める、ネット経由で使える、資源を共有する、伸び縮みする、利用量を計測する、という 5 つの特徴で判断します。 p.443

✓ 問 3 SaaS を使えば、データの公開設定は事業者の責任になりますか。

いいえ。どの形態でも、データと利用者の権限の設定は利用者の責任です。境界は提供元の責任共有モデルで確かめます。 p.451

第 11 章

セキュリティの基本

この章で学ぶこと

佐藤さんが「注文履歴を見る」とき、本人だけが自分の注文を見られ、他人の注文は見られないのはなぜか。認証・認可・多要素認証・最小権限・暗号化・ファイアウォール・WAF を 1 つずつ、「何を守るか」「何をしてくれないか」と具体例で学びます。

→ 前の章とのつながり

第 10 章の最後で、クラウドでもデータと権限の設定は利用者の責任だと分かりました。第 6 章の TLS、第 7 章のファイアウォール・WAF・認証も、ここで詳しく見直します。

セキュリティで守る 3 つのこと

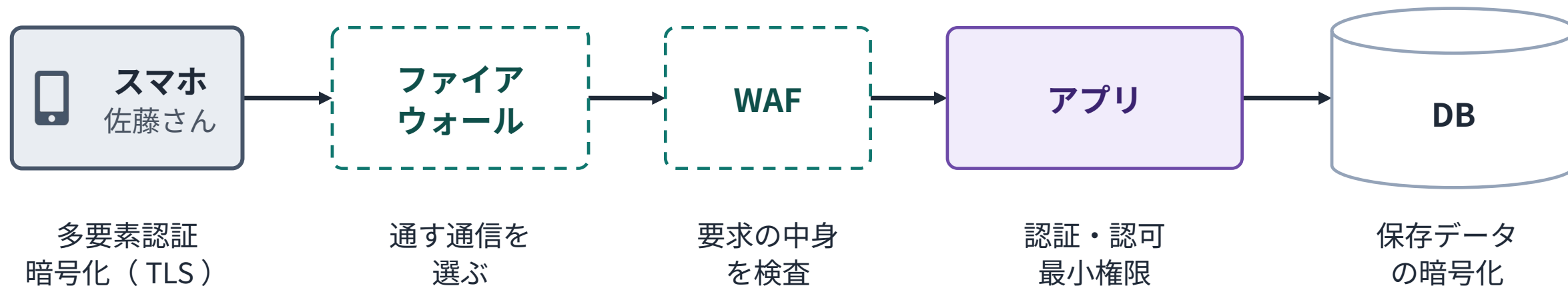
セキュリティは「情報を、見てよい人が、正しい内容で、使いたいときに使える」状態を保つことです。

要素	意味	注文履歴の例	崩れると
機密性	見てよい人だけが見られる	佐藤さんの注文は佐藤さんと店の担当者だけが見られる	他人に住所や買った物を知られる
完全性	内容が正しく、勝手に書き換えられない	注文 101 の数量は 3 のまま	数量や金額が書き換えられる
可用性	使いたいときに使える	夜中でも注文履歴を開ける	サイトが止まって見られない

この 3 つをまとめて**情報セキュリティの 3 要素**と呼びます（可用性は第 13 章で詳しく扱います）。

守りの仕組みは、通り道の別々の場所にある

役割の関係図です。黒い矢印＝要求の流れ。各箱の下文字が、そこで働く守りの仕組みです（1つの役割＝1台とは限りません）。



学習用に簡略化した図

使う権限のない人がシステムやデータに入り込むことを**不正アクセス**と呼びます。1つの仕組みですべては防げないので、通り道の別々の場所で、別々のものを守ります。

認証と認可：「誰か」と「何をしてよいか」

認証：誰であるかを確かめる

認証は、ログインのときに ID とパスワードなどで「この人は佐藤さん（顧客番号 1）本人か」を確かめることです。

- ・受け取る：ID とパスワード
- ・返す：「佐藤さんと確認した」という結果

認可：何をしてよいかを決める

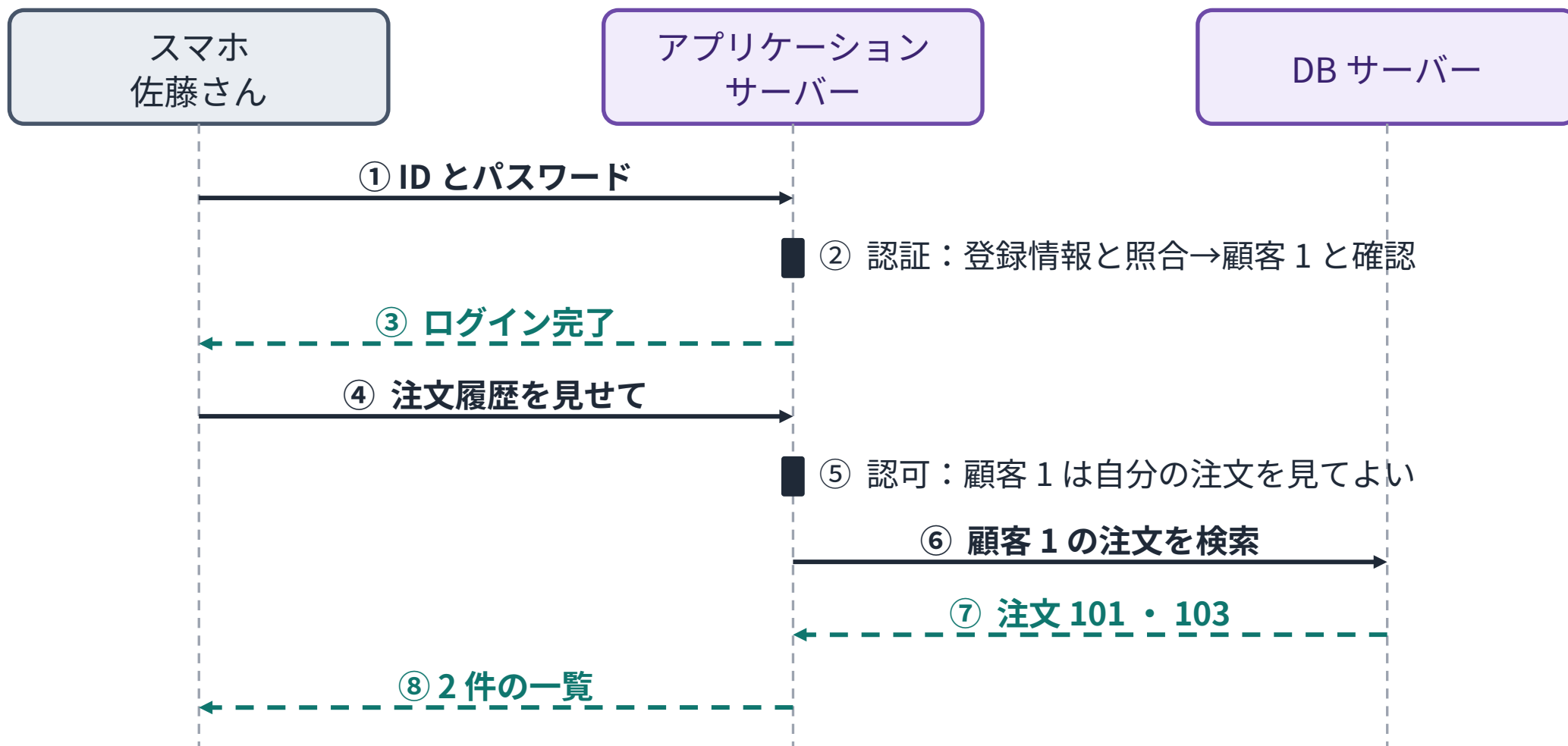
認可は、確かめた相手に、どの操作や情報を許すかを判断することです。第 3 章の**権限**を実際に当てはめる仕事です。

- ・佐藤さん：自分の注文 101 ・ 103 は見てよい
- ・高橋さんの注文 102 は見てはいけない

ログインできた（認証できた）からといって、何でもできるわけではありません。毎回の要求ごとに認可で確かめます。

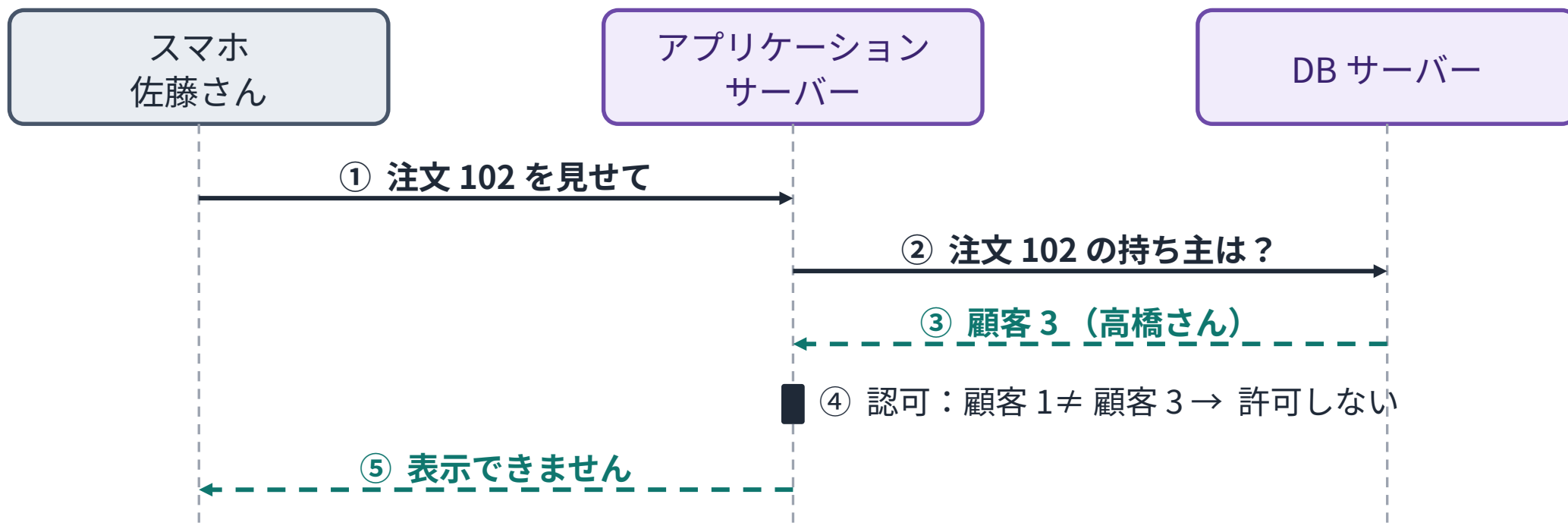
佐藤さんが自分の注文履歴を見るまで

時間は上から下へ。黒＝要求、緑の破線＝応答（通信は HTTPS で暗号化）。



他人の注文 102 を見ようとする

佐藤さんが画面のアドレスの番号を 103 から 102 に書き換えて、要求を送った場合です。



アプリが④の確認を忘れていれば、正しくログインしただけで高橋さんの注文が見えてしまいます。認証が正しくても、認可の誤りは防げません。

認証と認可が守るもの、してくれないこと

認証

守るもの

なりすまし（他人のふりをしたログイン）を防ぎ、「誰の要求か」をはっきりさせる。

してくれないこと

認証できた人が何をしてよいかは決めない。
パスワードが漏れれば本人と区別できない。

認可

守るもの

ログインした人でも、見てよい情報・してよい操作だけに限る。

してくれないこと

相手が本人かどうかは確かめない（認証に頼る）。設定や作りを誤れば素通りになる。

多要素認証（MFA）：種類の違う証拠を組み合わせる

本人を確かめる証拠（要素）には 3 つの種類があります。種類の違う要素を 2 つ以上組み合わせる方法を**多要素認証**（MFA：Multi-Factor Authentication）と呼びます。

知っているもの

- ・パスワード
- ・暗証番号
- ・秘密の質問の答え

持っているもの

- ・スマホのアプリに出る 6 桁の数字
- ・SMS で届く確認コード
- ・IC カード

本人の特徴

- ・指紋
- ・顔
- ・静脈

具体例（説明用の例）

佐藤さんはパスワードを入れたあと、スマホに届いた確認に「承認」を押してログインします（知っているもの＋持っているもの）。

同じ種類を 2 回使っても多要素ではない

組み合わせ	種類	多要素か
パスワード＋スマホへの確認	知っている＋持っている	○ 多要素
指紋＋暗証番号	本人の特徴＋知っている	○ 多要素
パスワード＋秘密の質問	知っている＋知っている	✕ 多要素ではない

守るもの

パスワードが漏れても、スマホなど別の種類の要素がなければログインできない。

してはいけないこと

ログインした後に何ができるか（認可）は決めない。確認を何も考えずに承認すれば効果が薄れる。

最小権限：必要な範囲だけ権限を与える

人やプログラムに、仕事に必要な範囲だけの権限を与える考え方を**最小権限**と呼びます。サンプル商店の社員なら次のようにします（説明用の例）。

立場	与える権限	与えない権限
配送担当	届け先の住所と商品を見る	金額の変更、注文の削除
経理担当	金額を見る	届け先の住所を書き換える
システム管理者	必要な作業のときだけ管理者の権限を使う	管理者の権限で普段の作業をする

守るもの

ID が乗っ取られても、できることが限られるので被害が小さく済む。

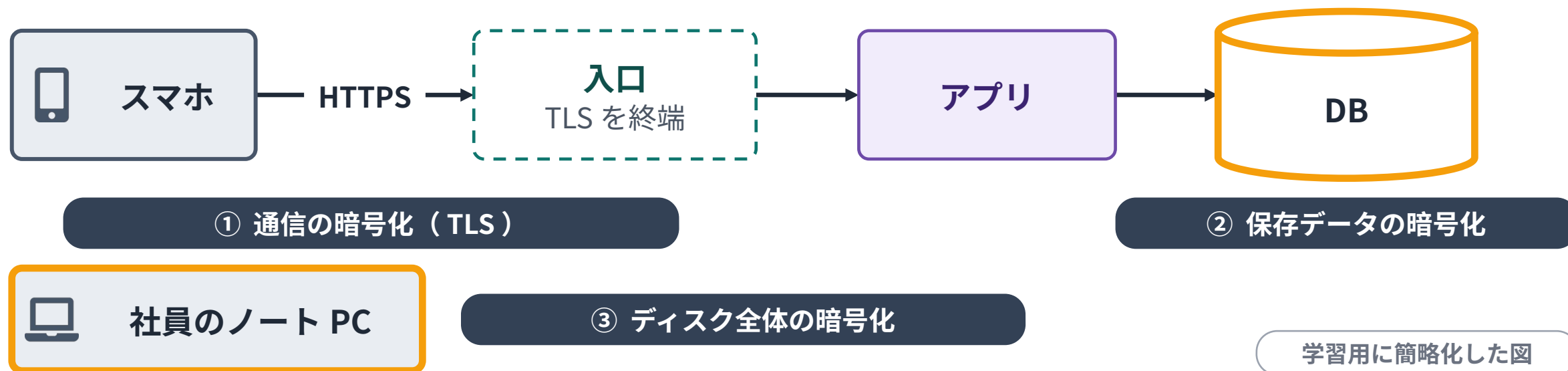
してはいけないこと

乗っ取り自体は防がない。異動などで要らなくなった権限は、定期的に見直して外す必要がある。

暗号化は 3 つの場所で使う

暗号化は、データを鍵がないと読めない形に変えることです（第 6 章）。使う場所で守るものが違います。

関係図です。黒い矢印＝通信の流れ。黄色の太枠＝暗号化される場所。TLS で守られるのは端末～入口の区間で、入口から先を暗号化するかは設計によります。



暗号化が守るもの、してくれないこと

守るもの

盗聴されたり、ディスクや PC を盗まれたりしても、中身を読まれない（機密性）。TLS は途中での書き換えの検知もする（完全性）。

してくれないこと

正しくログインした人や乗っ取られた ID には、復号された中身が見える。鍵の管理を誤ると効果がない。

鍵マークの意味

ブラウザの鍵マークは「通信路が TLS で保護されている」という意味で、サービスそのものが安全・正しいことの保証ではありません。家庭のルーターの NAT（宛先の書き換え）も TLS とは別の仕事です。

TLS で守られるのは端末から入口（TLS を終える機械）までです。その先を暗号化するかは設計で決めます。

鍵の 2 つの使い方：共通鍵と公開鍵

共通鍵暗号方式

共通鍵暗号方式は、閉める（暗号化）のも開ける（復号）のも同じ鍵を使います。

- ・ 良い点：処理が速く、大きなデータに向く
- ・ 課題：鍵を相手にどう安全に渡すか

公開鍵暗号方式

公開鍵暗号方式は、公開してよい鍵と、本人だけが持つ秘密の鍵を組で使います。

- ・ 公開鍵で暗号化したものは、組の秘密鍵でしか開けられない
- ・ 課題：共通鍵より処理が重い

補足

TLS は両方を組み合わせます。公開鍵の技術（鍵の交換と、証明書による相手の確認）で、通信ごとに共通鍵を安全に決め、本文はその共通鍵で速く暗号化します。

鍵の管理：回復キーをなくすと自分も読めない

暗号化の強さは鍵の管理で決まります。鍵が盗まれれば誰でも読めてしまい、逆に鍵をなくせば、正しい持ち主でもデータを取り出せなくなります。

📖 具体例：BitLocker の回復キー

Windows の BitLocker は、ディスク全体を暗号化する機能です。部品の交換などで本人確認が通らなくなると、「回復キー」（長い数字の列）の入力を求められます。回復キーを保管していなければ、自分のデータでも読めなくなります。

⚠️ 先に準備すること

学習用の PC でも、暗号化を有効にするなら、回復キーを別の安全な場所（例：自分のアカウントや印刷した紙）に保管してから始めます。

ファイアウォール：条件で通信を通す・止める

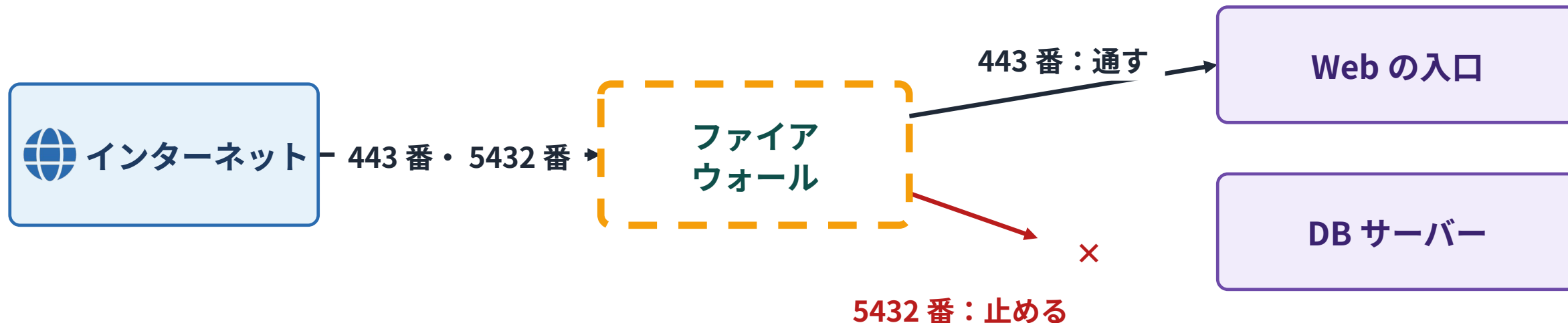
ファイアウォールは、宛先やポート番号などの条件を決めた表（ルール）に従って通信を選びます。

順	送り元	宛先と番号	動作
1	どこからでも	Web の入口 198.51.100.7 の 443 番 （ HTTPS ）	通す
2	管理用のネットワークだけ	各サーバーの 22 番（遠隔操作）	通す
3	アプリケーションサーバー だけ	DB サーバーの 5432 番（ DB への接続）	通す
4	それ以外すべて	すべて	止める

上から順に当てはめ、どれにも当たらない通信は 4 番で止めます（説明用の例。5432 番は DB ソフト PostgreSQL の標準の番号）。

ファイアウォールが守るもの、してくれないこと

関係図です。黒い矢印＝通された通信、赤い矢印＝止められた通信。



説明用の例

学習用に簡略化した図

🔑 守るもの

使わない番号（ポート）を閉じて、外から入れる口を減らす。DB のような大事な機械に、外から直接つながないようにする。

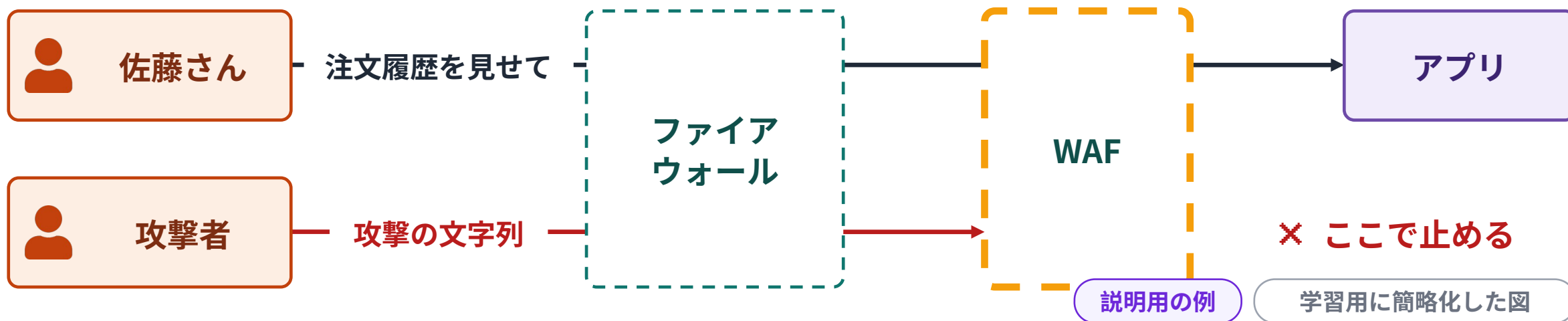
⚠️ してくれないこと

通した 443 番の中身が攻撃かどうかは見ない。設定・更新・記録（ログ）を続ける運用が要る。

WAF : Web への要求の中身を検査する

ファイアウォールが封筒の宛名（宛先・番号）を見るなら、WAF は手紙の本文（Web への要求の中身）を見ます。

関係図です。黒い矢印＝普通の要求、赤い矢印＝攻撃を含む要求。どちらも 443 番なのでファイアウォールは通します。



攻撃の例：検索欄に、文字の代わりに DB への命令（SQL）の一部を混ぜた文字列を入れる。アプリがそれをそのまま命令に組み込む作りだと、他人の注文まで取り出されます（SQL インジェクション）。WAF は既知の攻撃の形を見つけて止めます。

脆弱性と、WAF が直してくれないもの

前のページの攻撃が成り立つのは、アプリに「入力をそのまま命令に組み込む」という弱点があるからです。ソフトや設定にある、攻撃に悪用できる弱点を**脆弱性**と呼びます。

守るもの

既知の攻撃の形（不正な入力のパターン）を含む要求を、アプリに届く前に止める。直すまでの時間稼ぎにもなる。

してくれないこと

アプリの脆弱性そのものは直さない。新しい形や言い換えた攻撃はすり抜けることがある。

ポイント

根本の対策は、アプリの作りを直すこと（入力を命令として扱わない）と、ソフトの更新です。WAF はその手前の追加の守りです。

7つの仕組みを並べて比べる（1/2）

それぞれ別のものを守るので、組み合わせて使います。

仕組み	役割	守る対象・例	注意（してくれないこと）
認証	誰であるかを確かめる	ログイン（ID・パスワード等）	認証できても何でもできるわけではない
認可	何の操作・情報にアクセスしてよいか判断	閲覧のみ・編集可などの権限	AI が文書を探して答える仕組みでも、文書ごとに確認が必要
多要素認証	種類の違う要素を組み合わせる	パスワード＋スマホの確認	同じ種類を 2 回は多要素ではない

7つの仕組みを並べて比べる（2/2）

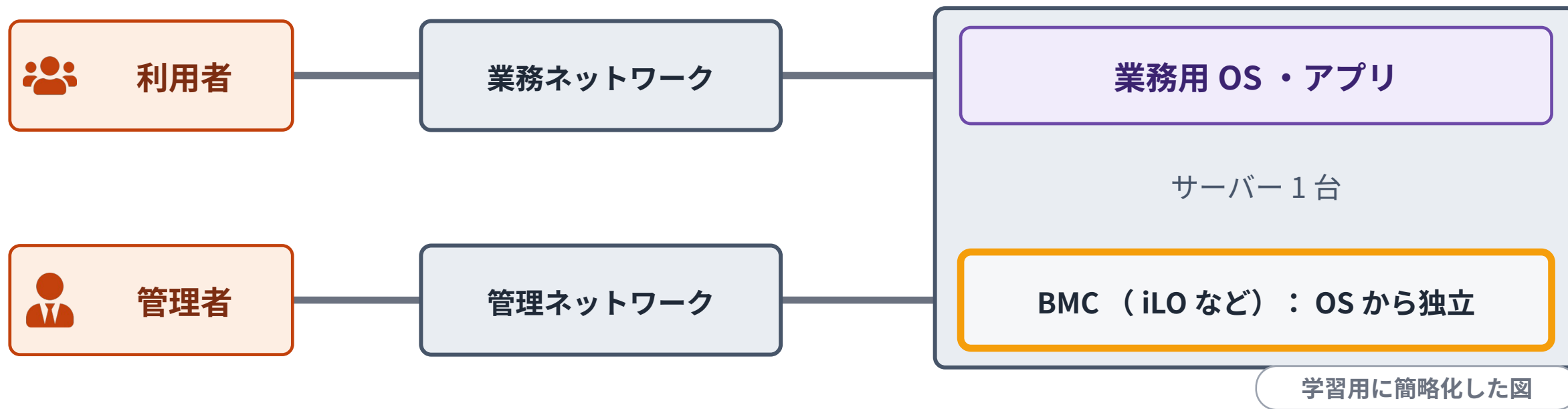
それぞれ別のものを守るので、組み合わせて使います。

仕組み	役割	守る対象・例	注意（してくれないこと）
最小権限	必要な範囲だけ権限を与える	管理者の権限を普段使わない	定期的な見直しが必要
暗号化	読まれても内容が分からないようにする	通信（TLS）・保存データ・ディスク	鍵の管理が要（回復キーの保管）
ファイアウォール	条件に応じて通信を通す・止める	使わないポートを閉じる	設定・更新・記録の運用が必要
WAF	Web アプリへの攻撃的な要求を検査	不正な入力・既知の攻撃パターン	アプリの脆弱性を直す代わりではない

管理の通信は、業務の通信と分ける

サーバーを遠くから操作する管理の入口は強い力を持つので、使える人と経路を絞ります。

物理的な接続の図です。灰色の太線＝ケーブル。管理の網には管理者だけが入れます。



BMC は電源の操作や画面の遠隔操作までできるため、管理の網を分け、**認証**情報を厳しく管理し、権限を最小にします（p.396）。

ログ：何が起きたかを後から確かめる記録

システムで起きた出来事を順に書き残した記録を**ログ**と呼びます。下はアプリのログの例です（説明用の例）。

```
10:15:02 login ok    user=1
10:15:09 GET /orders user=1 200
10:16:30 GET /orders/102 user=1 403
10:20:41 login fail user=2 (5 回目)
```

顧客 1 がログインに成功

顧客 1 が注文履歴を表示（200 = 成功）

顧客 1 が注文 102 を要求→拒否（403 = 権限なし）

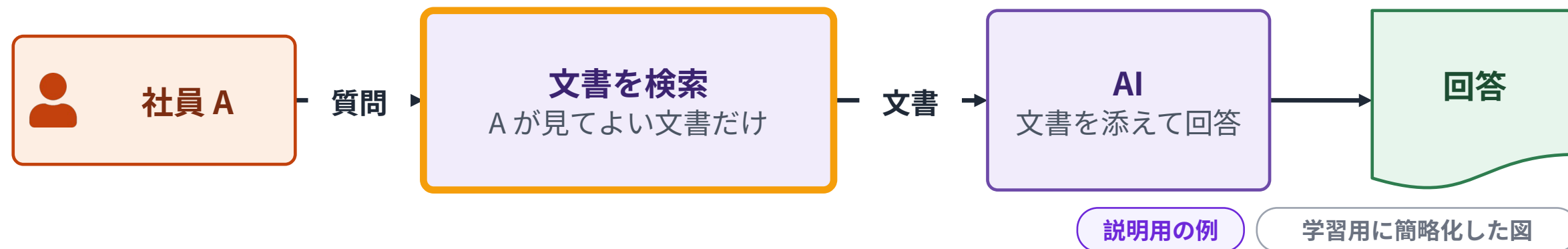
顧客 2 の ID でパスワード誤りが 5 回続いた

ログがあれば、不正アクセスの試み（最後の行）に気づき、後から何が起きたかを調べられます。順番を正しく比べるため、機器の時計を **NTP** で合わせておきます。

AI が社内文書を使って答える仕組みでも、権限を確かめる

質問に関係する社内文書を探し、その文書を添えて AI（文章で答えるプログラム）に答えさせる方法があります（RAG、第 12 章で詳しく扱います）。

関係図です。黒い矢印＝要求と文書の流れ。黄色の太枠＝権限を確かめる場所。



⚠ 注意

社員 A が見てはいけない人事評価の文書を検索で拾うと、その内容が回答に混ざって漏れます。文書ごとの閲覧権限を、検索の段階で質問した人に合わせて確かめます（認可）。

第 11 章のまとめ

✓ ここまでで説明できること

守るのは機密性・完全性・可用性。1つの仕組みでは全部を守れないので、通りの道の別々の場所で組み合わせる。

認証＝誰か、認可＝何をしてよいか。多要素認証は種類の違う要素を組み合わせ、最小権限は被害を小さくする。

暗号化は通信・保存データ・ディスクで使い、鍵の管理が要。ファイアウォールは通信を選び、WAF は要求の中身を検査するが、脆弱性は直さない。

→ 次へのつながり

次の第 12 章では、AI とそれを動かす基盤を学びます。この章の「文書ごとの権限確認」が、AI の仕組みでも必要になる理由がそこで具体的に分かります。

確認の問い（第 11 章）

自分の言葉で答えてから、次のページの解説を読みましょう。

- 1 ログインできた佐藤さんが、注文 102 を見られないのはなぜですか。
- 2 パスワードと秘密の質問の組み合わせは多要素認証ですか。
- 3 WAF を入れれば、アプリの脆弱性は直さなくてよいですか。

確認の問い（第 11 章）：解説

前のページの問いへの答えと、考え方です。

✓ 問 1 ログインできた佐藤さんが、注文 102 を見られないのはなぜですか。

認証で佐藤さん（顧客 1）と確かめたうえで、認可で「注文 102 は顧客 3 の注文なので顧客 1 には許可しない」と判断するからです。認証と認可は別の仕事です。 p.464

✓ 問 2 パスワードと秘密の質問の組み合わせは多要素認証ですか。

いいえ。どちらも「知っているもの」なので、同じ種類を 2 回使っているだけです。スマホへの確認など、別の種類を組み合わせます。 p.467

✓ 問 3 WAF を入れれば、アプリの脆弱性は直さなくてよいですか。

いいえ。WAF は既知の攻撃の形を手前で止める追加の守りで、新しい形はすり抜けることがあります。アプリの作りを直し、ソフトを更新するのが根本の対策です。 p.476

第 12 章

AI と AI 基盤

この章で学ぶこと

AI が中で何をしているのかを、迷惑メールの判定と「明日の天気は」の続きを予測する例で確かめます。基本の言葉を押さえてから、AI を動かす設備（AI 基盤）と、社内文書を使って答えさせる方法へ進みます。

→ 前の章とのつながり

CPU ・ GPU ・ メモリ（第 2 章）、サーバーとデータセンター（第 4 章・第 9 章）、クラウド（第 10 章）、認証と認可（第 11 章）を使います。AI 基盤は、これらの組み合わせです。

AI とは：認識・予測・生成を機械で行う分野

AI（えーあい）

分類：考え方

略語：AI = Artificial Intelligence 日本語：人工知能

何か	人が頭を使ってしてきた「見分ける（認識）」「先を読む（予測）」「段取りを考える（計画）」「文章や画像を作る（生成）」などを、機械で行う技術の広い分野の名前。
なぜ必要か	大量の写真の仕分けや問い合わせへの回答など、人手では時間がかかる仕事を、速く大量にこなすため。
身近な例	写真の中の顔を見分ける（認識）、来月の売れ行きを見積もる（予測）、質問に文章で答える（生成）
似た言葉との違い	特定の製品やサービスの名前ではなく分野の名前。人が書いた決まり（ルール）だけで動く仕組みも、AI に含めることがある。

人がルールを書く方法と、データから学ぶ方法

迷惑メールを自動で振り分ける仕組みを例に、AIの作り方を2つ比べます。

方法1：人がルールを書く

「件名に『当選』があれば迷惑メール」のような条件を、人が1つずつ書きます。新しい手口が出るたびに、人が条件を書き足す必要があります。

方法2：データから学ぶ

「迷惑」「普通」の印を付けたメールを1万通集め、どんな特徴があると迷惑メールらしいかを、コンピューターに数値として見つけさせます。

ポイント

方法2のように、データからパターン（くり返し現れる規則性）をコンピューターに見つけさせる方法を**機械学習**と呼びます。ルールを人が全部書かなくてよいのが特徴です。方法1も、AIという分野には含まれます。

モデル：入力から出力を計算する仕組み

モデル（もでる）

分類：ソフトウェア

何か	入力（例：届いたメール 1 通）を受け取り、決まった計算をして出力（例：「迷惑メール」という判定）を返す仕組み。計算の手順（構造）と、計算に使う数値の組でできている。
なぜ必要か	データから見つけた特徴を、新しい入力にくり返し当てはめるため。一度できれば、何通のメールにも同じ計算で答えを出せる。
どこにあるか	ファイルとして保存され、使うときにサーバーのメモリや GPU のメモリに読み込まれる。
似た言葉との違い	「AI モデル」はこの計算の仕組みのこと。チャットの画面・会員管理・会話の保存まで含めた「AI サービス」全体とは別。

迷惑メール判定のモデルを図で見る

役割の関係図。緑の書類形＝データ、紫＝モデル（ソフトウェアとして動く計算の仕組み）。黒い矢印は情報の移動です。



説明用の例

学習用に簡略化した図

モデルの中には「どう計算するか」という手順と、その計算に使う数値が入っています。この数値を**パラメータ**と呼びます（次のページで詳しく）。

パラメータ：モデルの中の、調整される数値

パラメータ (ぱらめーた)		分類: データ
何か	モデルの計算に使う数値。迷惑メールの例なら「『当選』という語があれば +3.0 点」の 3.0 のような値。「重み」とも呼ぶ。	
なぜ必要か	この数値を変えると、同じ入力でも出力が変わる。正しく答えられるかどうかは、パラメータの値で決まる。	
どこにあるか	モデルのファイルの中。推論（使う工程）のときは GPU のメモリなどに読み込まれる。	
似た言葉との違い	実際の文章のモデルでは、パラメータが数十億を超えることも珍しくなく、人が 1 つずつ決めることはできない（決め方は「学習」）。設計書の「パラメーター表」の設定値とは別の意味。	

パラメータを使って計算してみる

件名に含まれる語ごとに点数（パラメータ）を足し、合計 2.0 点以上なら「迷惑メール」と判定する、小さなモデルです。

件名に含まれる語	パラメータ（点数）
「当選」	+3.0
「無料」	+2.0
「至急」	+1.0
「会議」	-1.5

↑ 説明用の例。数値は架空です。

計算の例

件名「当選！無料で受け取り」
→ $3.0 + 2.0 = 5.0$ 点 → 迷惑メール

件名「明日の会議の資料」
→ -1.5 点 → 普通のメール

数値が違えば、判定も変わります。では、この数値はどう決めるのでしょうか。

学習：例題を解かせて、数値を少しずつ直す

学習（がくしゅう）		分類：活動
何か	例題（入力と正解の組）をモデルに次々に解かせ、答えが正解に近づくようにパラメータを少しずつ直していく工程。	
なぜ必要か	パラメータは数が多く、人が 1 つずつ決めるのは無理。例題（印付きのメール 1 万通など）から、コンピューターに数値を決めさせる。	
身近な例	最初は全部 0 点。「当選…」（正解は迷惑）を「普通」と誤判定 → 「当選」の点数を少し上げる。これを大量の例題でくり返す。	
似た言葉との違い	人の勉強のように意味を理解するのではなく、数値の調整のくり返し。	

推論：学習済みのモデルで答えを出す

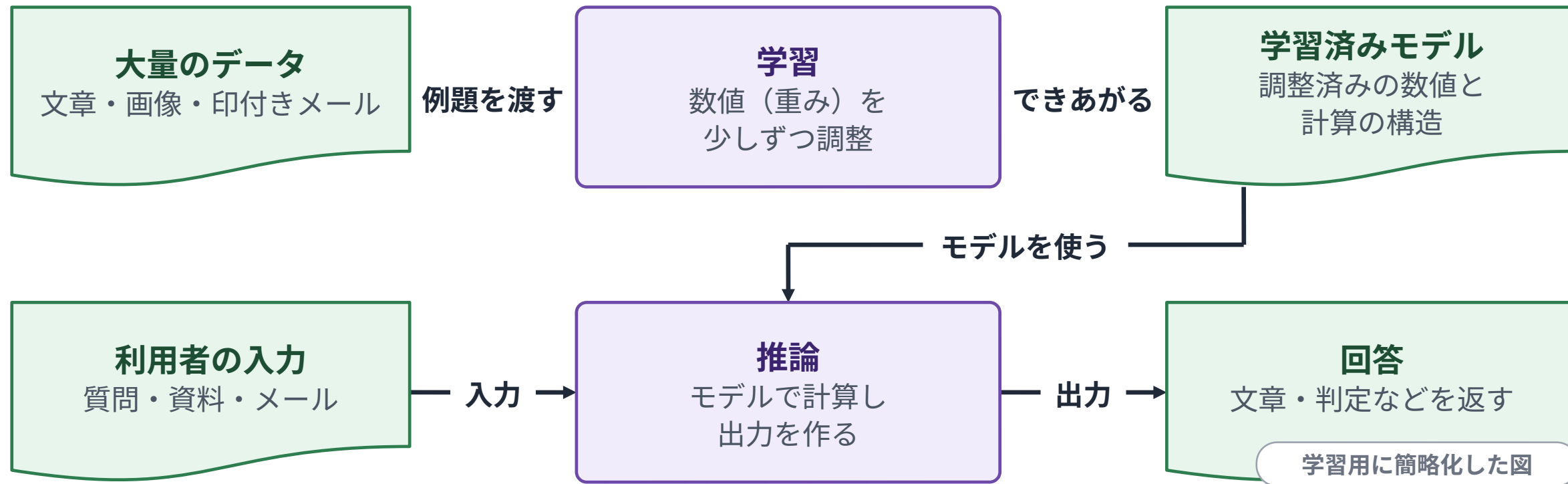
推論（すいろん）

分類：活動

何か	学習が終わったモデルに新しい入力を渡し、計算して出力を得る工程。パラメータは通常変えない。
身近な例	新しく届いたメール1通に、学習で決まった点数を当てはめて計算し、「迷惑メール」と判定する。AIサービスに質問して回答の文章をもらうのも推論。
似た言葉との違い	日常語の「推論（筋道を立てて考える）」とは違い、ここでは「学習済みモデルで計算して出力を作ること」を指す。英語ではinference。

学習はモデルを作る工程、推論はモデルを使う工程

役割の関係図。上の段が**学習**、下の段が**推論**です。緑の書類形＝データ、紫＝コンピューターが行う計算、黒い矢印＝データの移動。



AI サービスへの質問の中心は推論です。通常の問い合わせのたびに、モデル全体を学習し直すわけではありません。

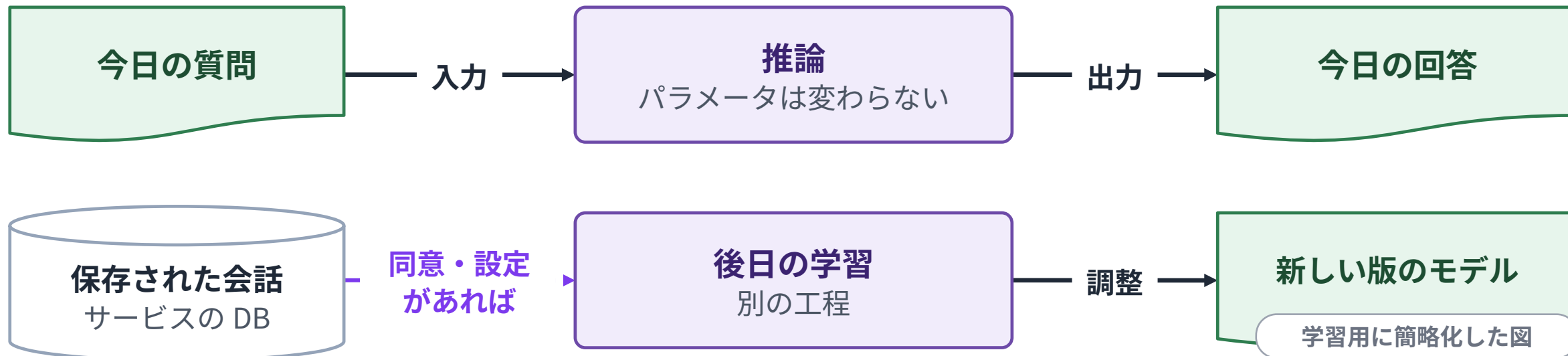
学習と推論を 4 つの観点で比べる

観点	学習	推論
目的	モデルを作る・改良する	モデルを使って答えを出す
頻度	まとまった期間に行う	利用のたびに行う
必要な資源	非常に多くの GPU ・データ・時間	利用者数・応答の速さに応じた GPU など
重み（パラメータ）の変化	変わる	通常は変わらない

「資源」は、計算に使う GPU やメモリ、時間のことです。学習は一度に大量の資源を使い、推論は 1 回ごとには小さくても、利用者の数だけくり返されます。

会話を入力に使うことと、重みを変えることは別

上の段＝今の回答（推論）、下の段＝将来の学習。黒い矢印＝データの移動、紫の破線＝条件つきで後から使われる道。



i 補足

サービスによっては、利用者の同意や設定に基づき、会話データが将来の学習に使われることがあります。それは今の回答の計算（推論）とは別の工程です。質問した直後にモデルが変わるわけではありません。

ニューラルネットワーク：計算を層に重ねたモデル

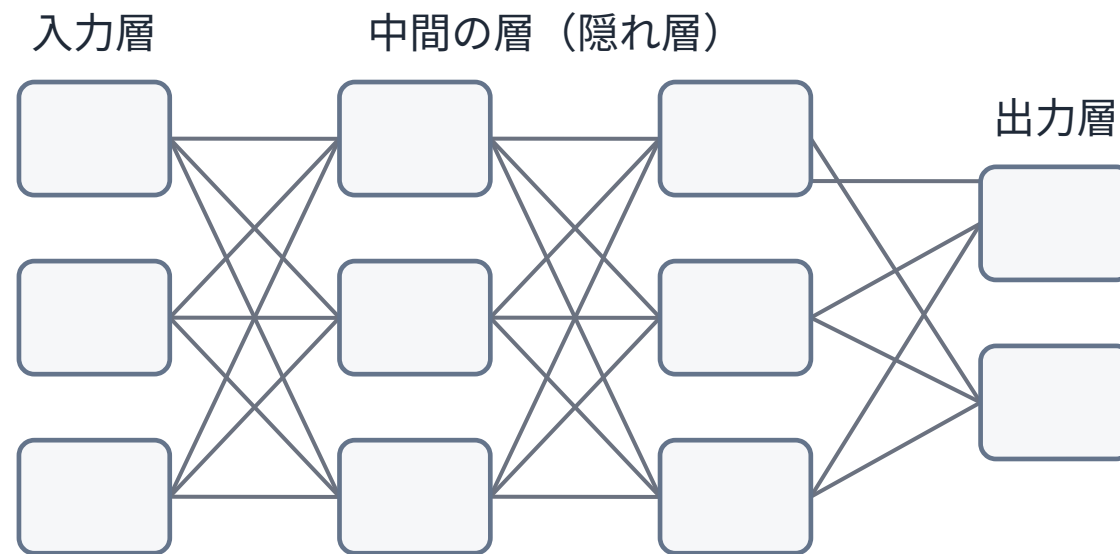
ニューラルネットワークはモデルの形の1つです。小さな計算（入力に重みを掛けて足し、次へ渡す）を多数並べ、層にして順につながります。

左から入力（例：メールの特徴）を入れると、層ごとに計算され、右から出力（例：迷惑かどうか）が出ます。線1本ごとに重み（パラメータ）があります。

！ 名前の注意

脳の神経細胞にヒントを得た名前ですが、脳と同じように考えるわけではありません。中身は掛け算と足し算のくり返しです。

丸い箱＝小さな計算、灰色の細い線＝計算の結果を次の層へ渡すつながり（左から右へ）。



学習用に簡略化した図

深層学習：層を深く重ねて学習する方法

深層学習 (しんそうがくしゅう)		分類: 考え方
何か	中間の層を何段も深く重ねたニューラルネットワークを使う機械学習。英語で deep learning (ディープラーニング)。	
なぜ必要か	層を重ねると、画像・音声・文章のような複雑なデータの特徴をとらえやすくなる。	
身近な例	写真に写った物の認識、音声の文字起こし、文章を作るモデル（この後の LLM）	
似た言葉との違い	機械学習の方法の一部。機械学習には、決定木（質問の分かれ道で判定する方法）など深層学習以外の方法も多い。深層学習は大量の計算とデータが必要。	

生成 AI：「作る」という用途で分けた呼び名

生成 AI（せいせいえーあい）

分類：考え方

何か	文章・画像・音声・プログラムなどを新しく作り出す AI。「何をするか（用途）」で分けた呼び名。
身近な例	質問に文章で答える、説明文から画像を作る、会議の録音から議事録の下書きを作る
似た言葉との違い	機械学習・深層学習のような「方法」の名前ではない。現在は深層学習を使う方式が多い。認識や判定だけをする AI（迷惑メールの判定など）は、ふつう生成 AI とは呼ばない。

LLM ：大量の文章で学習した言語のモデル

LLM （える・える・えむ）

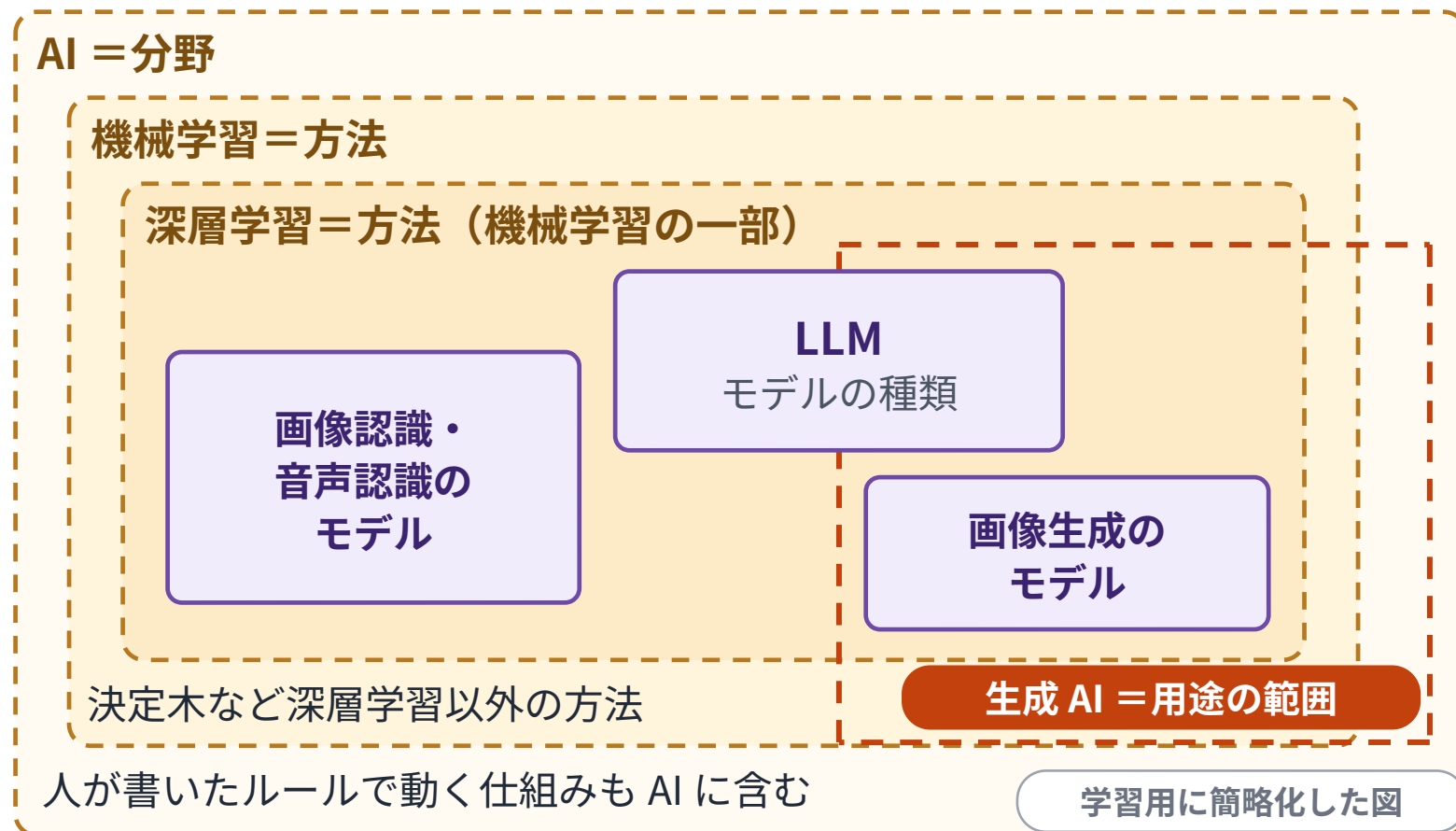
分類：ソフトウェア

略語： LLM = Large Language Model 日本語：大規模言語モデル

何か	大量の文章で学習した、非常に多くのパラメータを持つ、言葉（文章）を扱うモデル。「モデルの種類」の名前。
身近な例	文章の生成・要約・分類（問い合わせを種類別に振り分ける）・抽出（文書から日付や金額を抜き出す）
似た言葉との違い	ChatGPT などのサービス全体とは別。サービスは、LLM に画面・会話の保存・検索などを組み合わせたもの。

AI・機械学習・深層学習・生成AI・LLMの関係

AI ⊃ 機械学習 ⊃ 深層学習は「方法」の入れ子です。生成AIは用途、LLMはモデルの種類で、同じ軸の入れ子ではありません。



茶色の破線の枠は言葉が指す範囲で、外ほど広い入れ子です。

赤い破線は、別のものさし（用途）で引いた「生成AI」の範囲です。深層学習の外にもはみ出します。

紫の箱はモデルの種類です。LLMは生成だけでなく分類・抽出にも使うので、赤い枠をまたいでいます。

5 つの言葉は「何の分類か」が違う

言葉	何の分類か	注意
AI	分野	ルールベース（人が書いた条件で動く）のシステムも含む
機械学習	方法	決定木など深層学習以外の方法も多い
深層学習	方法（機械学習の一部）	大量の計算とデータが必要
生成 AI	用途（文章・画像・音声を作る）	現在は深層学習の方式が多い
LLM	モデルの種類	生成だけでなく分類・抽出にも使う。ChatGPT などのサービス全体とは別

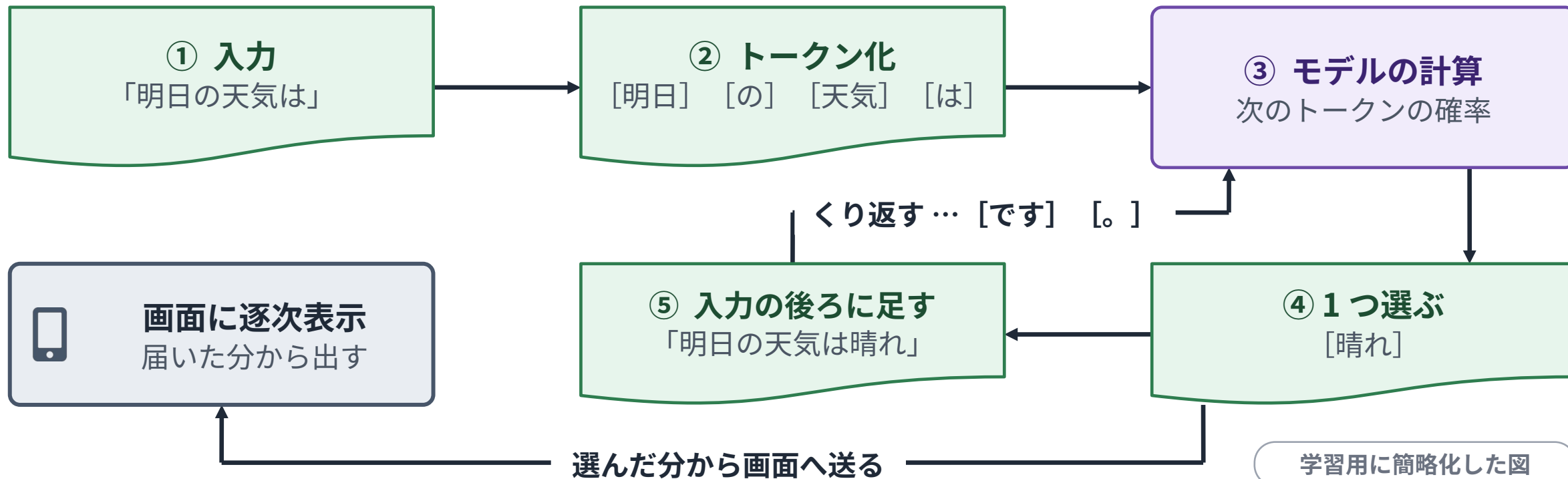
生成 AI は用途、LLM はモデルの種類です。文章を作る生成 AI の多くは LLM を使いますが、画像を作る生成 AI もあります。

トークン：LLM が文章を扱う単位

トークン (とーくん)		分類: データ
何か	LLM が文章を扱うときの区切りの単位。入力された文章は、まずトークンの並びに分けられる（トークン化）。	
なぜ必要か	モデルは文字のままでは計算できない。決まった単位に区切り、それぞれを番号（数）に置き換えてから計算する。	
身近な例	「明日の天気は」→ [明日] [の] [天気] [は] の 4 つのトークン（説明用の分け方）	
似た言葉との違い	日本語の 1 文字や英語の 1 語と、固定的には一致しない。分け方はモデルごとに違う。AI サービスの料金や入力の上限は、トークン数で数えることが多い。	

LLM は次のトークンを1つずつ予測して文章を作る

処理の順番（①→⑤）。緑＝データ、紫＝計算、黒い矢印＝データの移動。⑤から③へ戻ってくり返します。



画面に文字が少しずつ出るのは、この1つずつの生成と、できた分から送る配信のためです。

「次のトークンの確率」とはということか

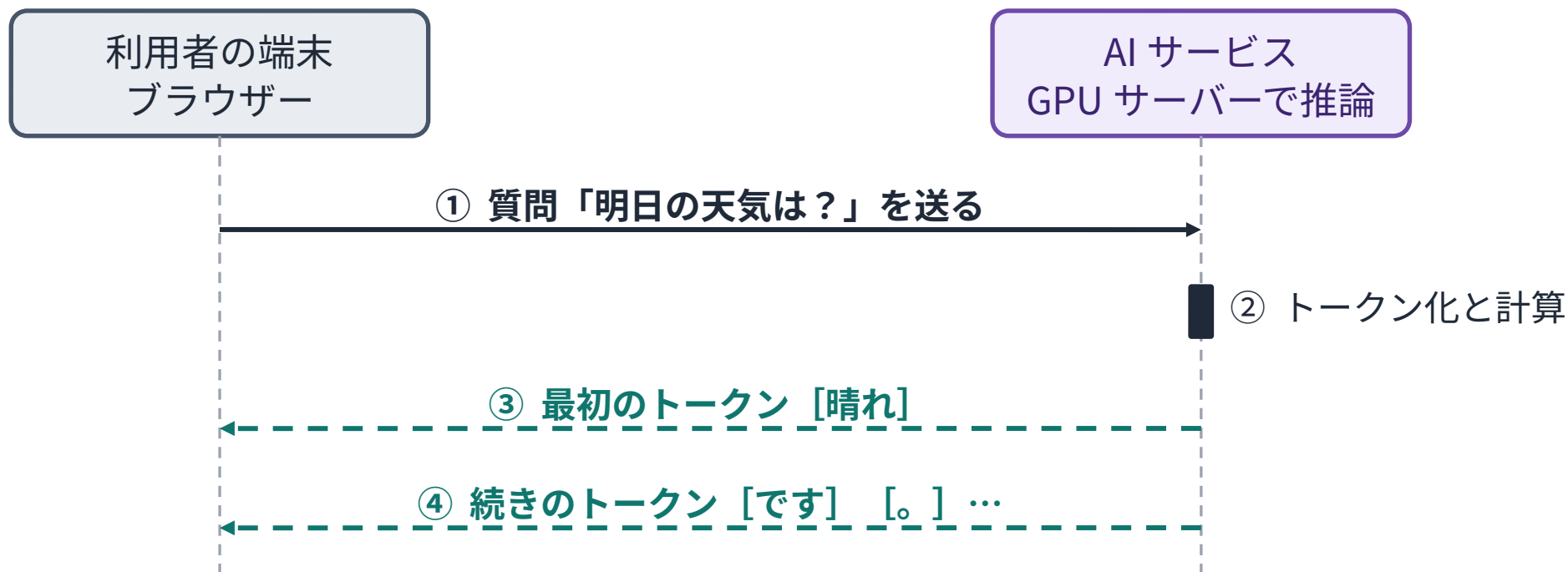
次のトークンの候補	確率（説明用）
[晴れ]	40%
[雨]	25%
[くもり]	20%
[よく]	5%
その他	10%

モデルは「明日の天気は」の次に来そうなトークンを、候補ごとに確率として計算します。この計算に、学習で決まったパラメータが使われます。

確率の高いものを選ぶのが基本ですが、設定によっては確率に応じて少しばらつかせて選びます。同じ質問でも答えが毎回少し違うことがあるのは、このためです。

↑ 説明用の架空の数値です。

最初の出力までの時間（TTFT）と生成速度



TTFT（①～③）

Time to First Token。最初の出力が現れるまでの時間。通信の往復時間（RTT）や、全体の完了時間とは別の指標です。

生成速度（④）

最初の出力のあと、1秒あたりに出せるトークンの量。モデル・GPU・混雑で変わります。

もっともらしい誤り：ハルシネーション

ハルシネーション（はるしねーしょん）

分類：考え方

何か	AI が、もっともらしいが事実と違う内容を作ってしまうこと。英語で「幻覚」の意味。
なぜ必要か	LLM は「次に来そうなトークン」を確率で選んで文章を作る。事実を調べて確かめる仕組みではないため、自然だが誤った文章ができることがある。
身近な例	存在しない論文名や、実際とは違う数値を、自信がありそうな文で答える。

⚠ 注意

重要な内容は、出典（元の資料）で確かめます。この後の RAG（文書を探して渡す方法）でも、必ず正しくなるわけではありません。

AI の計算で GPU が使われる理由（第 2 章の復習）

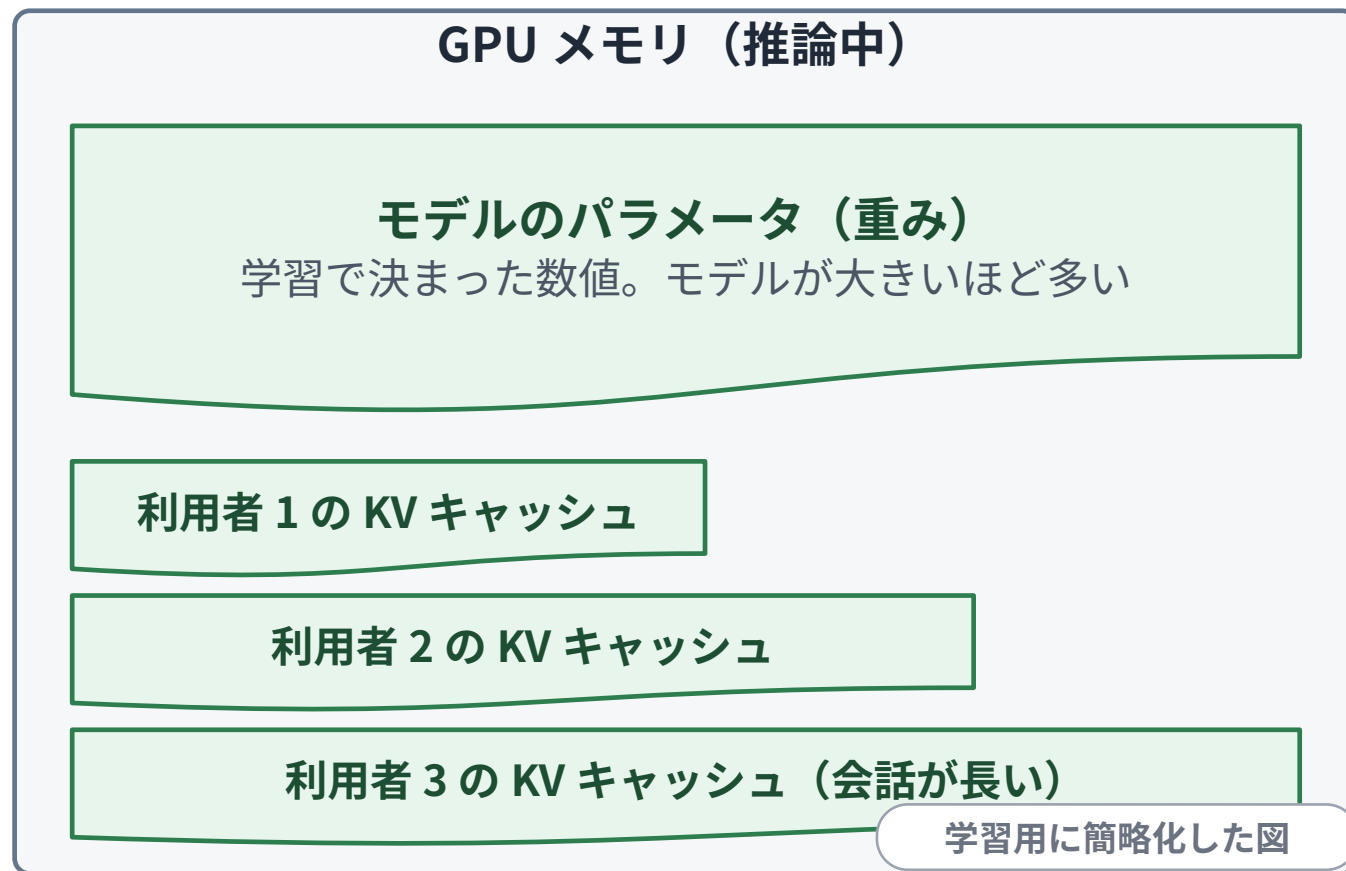
モデルの計算は、入力の数値とパラメータを掛けて足す、同じ形の計算の大量のくり返しです（行列計算と呼びます）。同じ計算を多数同時に進めるのが得意な GPU が、ここで活躍します。

	CPU	GPU
得意なこと	多様な処理・順番に依存する処理	大量の同じ計算の並列処理
AI での役割	データの準備・ GPU への指示・通信の管理	学習・推論の行列計算
向かない例	膨大な行列計算だけを延々とする	分岐が多く並列化できない処理

GPU だけで動くのではなく、CPU と協力して働きます（ p.89 ）。

推論中の GPU メモリには何が置かれているか

模式図。灰色の枠＝GPUのメモリ、緑の箱＝置かれるデータ。横幅は量のイメージ。



推論では、GPU のメモリ（GPU 専用の記憶、p.93）に、モデルのパラメータと、会話ごとの途中計算（KV キャッシュ、次のページ）が置かれます。

🔑 ポイント

会話が長いほど、同時に使う人が多いほど、必要な GPU メモリが増えます。

コンテキストと KV キャッシュ

コンテキスト

1回の処理でモデルが参照する入力範囲です。今回の質問だけでなく、それまでの会話や添付した資料も含まれます。量に上限があり、永続的な記憶ではありません。

KV キャッシュ

生成中の途中計算を保持し、再計算を減らすデータです。[晴れ]の次を計算するとき、「明日の天気は」の分を最初から計算し直さずに済みます。

🔄 似た言葉との違い

KV キャッシュは「キャッシュ（よく使うものを近くに写して置く）」という考え方は同じですが、CPUのキャッシュメモリ（p.70）やDNSのキャッシュとは用途が違う、別のものです。

会話履歴・コンテキスト・パラメータは別のもの

名前	何か	どこにあるか
保存された会話履歴	過去のやり取りの記録	サービスのデータベース（DB）
コンテキスト	今回の処理で参照する入力。上限がある	処理中のメモリ（今回の入力として渡される）
パラメータ	モデルの数値。推論中は通常変わらない	モデルのファイル／推論中は GPU メモリ

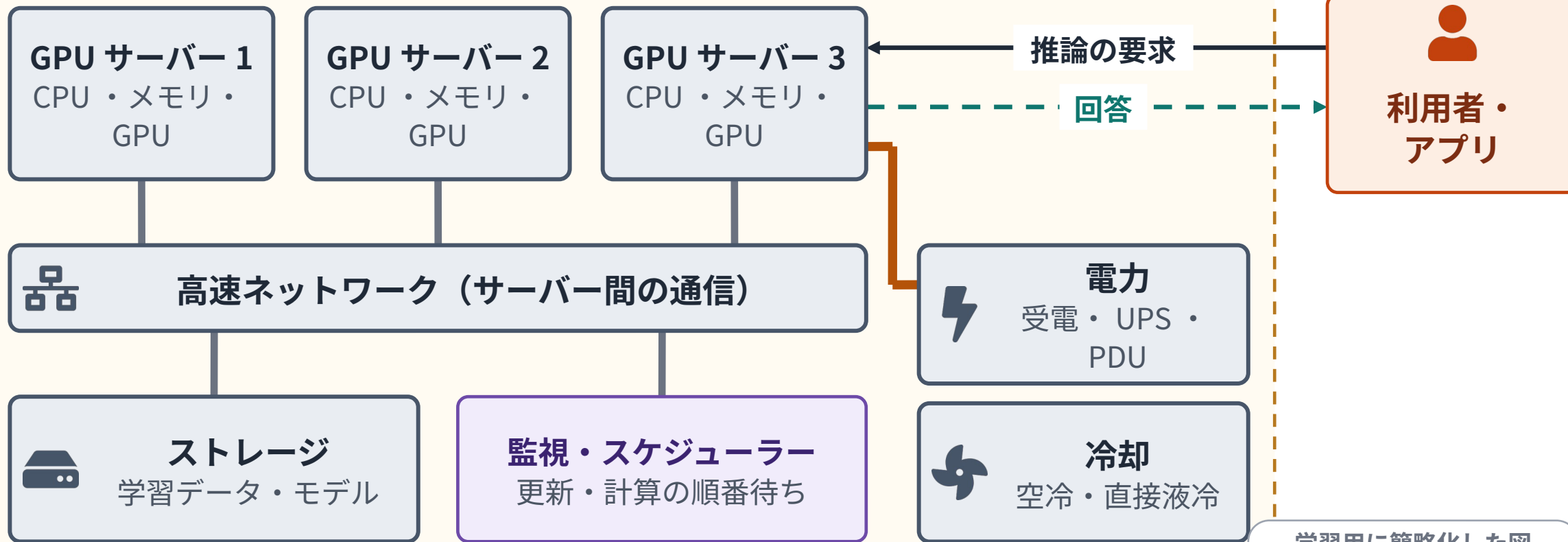
SE にとっての意味

SE にとっての意味：同時利用者数と会話の長さが、必要な GPU メモリの量、つまり GPU の台数や機種を選び方に影響します（p.530）。

AI 基盤は、GPU だけでなく保存・通信・電力・冷却で動く

物理的な配置図（簡略）。茶色の破線＝データセンターの中、灰色の太線＝ケーブル、茶色の極太線＝電力、黒い矢印＝推論の要求、緑の破線＝回答。

AI 基盤（データセンター内）



学習用に簡略化した図

AI 基盤の構成要素は、それぞれ何をするか

構成要素	何をするか	足りないと
GPU サーバー	CPU が段取りをし、GPU が行列計算をする	計算が追いつかず、応答が遅い
高速ネットワーク	サーバー間で計算の途中結果を渡す	他のサーバーの結果待ちが増える
ストレージ	学習データやモデルのファイルを置き、速く渡す	GPU がデータの読み込みを待つ
監視・スケジューラー	状態を見張り、計算の順番と割り当てを決め、更新する	故障や混雑に気づけない
電力・冷却	大きな電力を送り、出た熱を屋外へ運ぶ	機器を置いても動かせない

スケジューラーは、多くの人の計算の依頼を受け、どの GPU でいつ動かすかを決めるソフトウェアです。電力と冷却の仕組みは第 9 章（ p.399 ・ p.402 ）で見ました。

GPUを増やすだけでは性能が出ない理由

全体の速さは、いちばん待たされる部分で決まります（[ボトルネック](#)、p.98）。GPUの枚数が多くても、次のどれかが足りなければ、GPUは仕事を待つ時間が長くなります。

データの読み込み

ストレージから学習データが届かないと、GPUの[使用率](#)が低いのに遅い状態になる。

機器間の通信

他のサーバーの途中結果を待つ。帯域と遅延が待ち時間になる。

電力・冷却

電力が来なければ動かず、熱を逃がせなければ性能を落として守る。

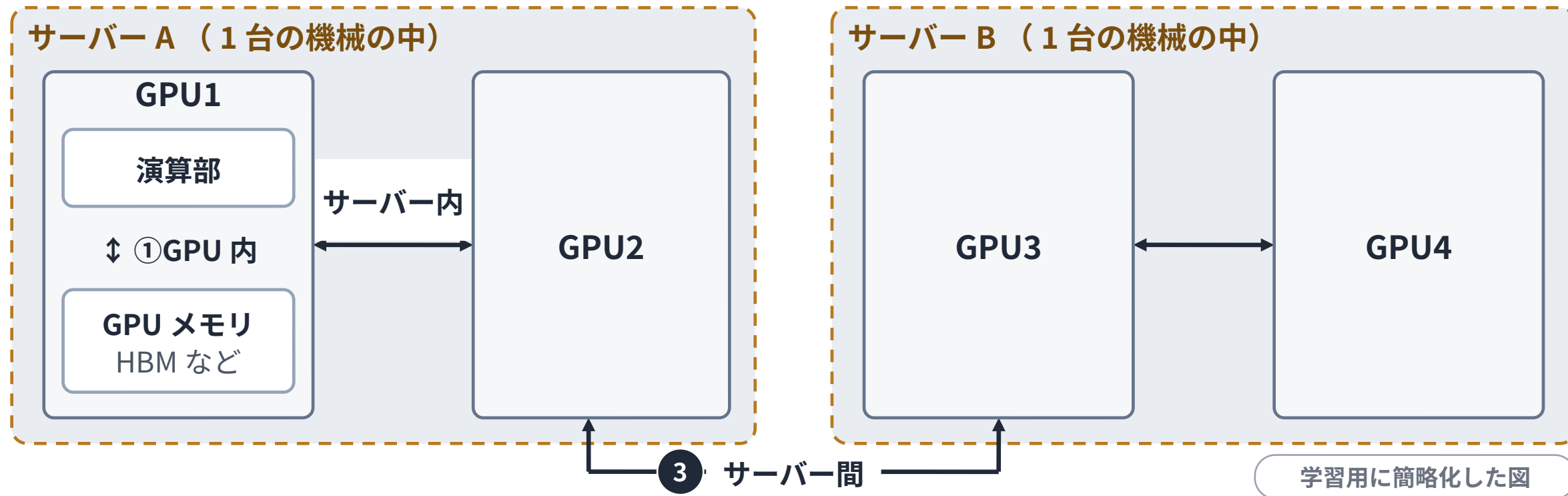
注意

GPUの枚数から決めず、データ・通信・電力・冷却を合わせて設計します。

複数 GPU の連携：3つの場所で通信が違う

大きなモデルは1枚のGPUのメモリに収まらず、複数のGPUで分担します。そのとき、データの通り道が3種類あります。

物理的な配置図（簡略）。破線の枠＝1台のサーバーの中、薄い灰色＝GPU、黒の両矢印＝データのやり取り。番号は次のページの表の段階。



3 つの通り道の接続と注意点

段階	何が通るか	接続の例	注意
① GPU の内部	演算部と GPU メモリの間	HBM ・ GDDR など	メモリの帯域が不足すると演算部が待つ
② 同じサーバー内	GPU どうし ・ CPU と GPU	PCIe 、 NVLink など	機種ごとに対応 ・ 帯域が違う
③ サーバー間	別の機械の GPU どうし	InfiniBand 、 高速 Ethernet など	遅延と帯域が、分担した計算の待ち時間に直結

NVLink は GPU どうしを直接つなぐ接続、InfiniBand ・ Ethernet はサーバー間のネットワークの方式です。

 注意

AI 基盤がすべて同じ接続方式ではありません。規格名 ・ 接続範囲 ・ 対応製品を分けて確認します。

半導体は広い分類：演算用・作業記憶・保存用

半導体は、電気の通し方を制御できる材料と、それで作ったチップの総称です。CPU も GPU もメモリも半導体ですが、働きは違います。

ロジック（演算）

CPU ・ GPU ・ AI 用チップ。
計算や制御をする。

DRAM ・ HBM （作業記憶）

計算中のデータを置く。電源を切ると消える。

NAND （保存用記憶）

電源を切っても残る。SSD の中心部品。

SSD は、NAND と、それを制御する部品を組み合わせた製品です。

HBM と NAND ・ SSD の比較

	HBM	NAND ・ SSD
主な仕事	計算中のデータを高速に供給	データを大容量で保存
電源を切ると	データは保持されない	データを保持する
重視される点	帯域・容量・電力効率	容量・読み書き性能・耐久性・価格
企業の例	SK hynix ・ Samsung ・ Micron	キオクシア・ Sandisk など

どちらも「メモリ」と呼ばれることがありますが、HBM は計算の作業場所、NAND は長く残す保存場所で、役割が違います。

AI用チップは、複数の会社が分担してつくる



- 設計：NVIDIA・AMD など／記憶：HBMはSK hynixなど、NANDはキオクシアなど
- 製造装置：ASML（露光＝回路を焼き付ける）・東京エレクトロン（成膜・洗浄など）

NVIDIAはGPUの設計会社で、製造は**ファウンドリー**（他社が設計した半導体の製造を請け負う会社）のTSMCなどが行います。HBMは別のメモリ会社が供給します。設計・製造・装置の会社は、役割が違います。

半導体の会社の立ち位置と AI 需要

企業	主な立ち位置	AI 需要とのつながり
NVIDIA	GPU などの設計・システム提供	AI 計算設備の需要
キオクシア	NAND ・ SSD などの供給	保存容量・高速ストレージの需要
TSMC	他社が設計した半導体を製造（ファウンドリー）	GPU などの製造需要
東京エレクトロン	半導体の製造装置（成膜・洗浄など）	メーカーの設備投資
ASML	微細な回路を作る露光装置（EUV）	先端半導体の製造設備

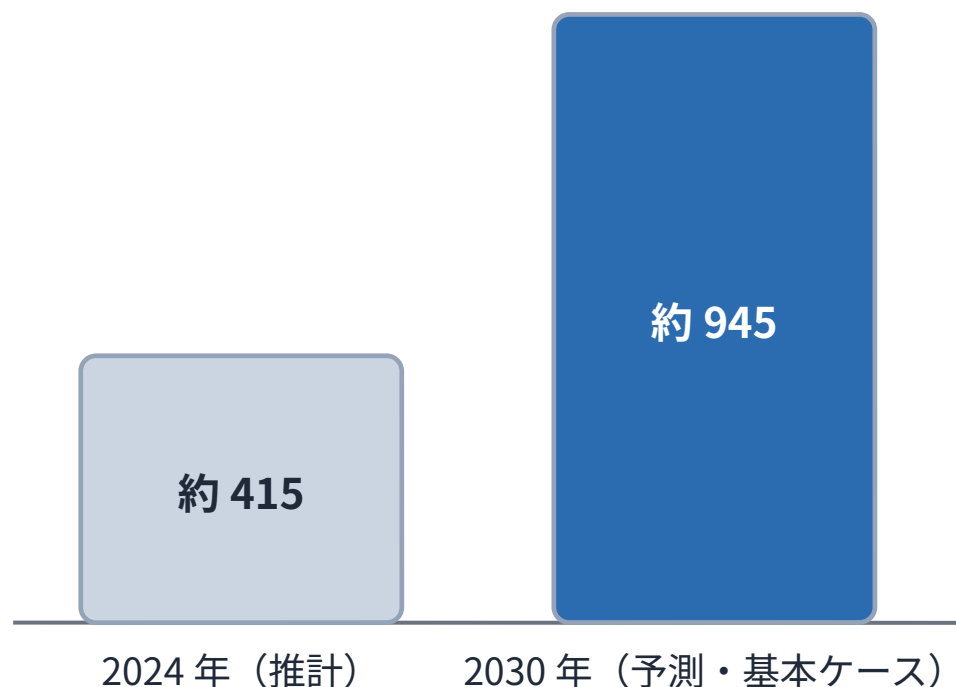
i 補足

原資料の記載：キオクシアは 2026 年 7 月 30 日、AI 推論向けの PCIe 6.0 ・直接液冷対応の企業向け SSD を発表し、限定顧客へのサンプル出荷を開始（今回未再確認）。AI 需要の影響の現れ方は、時期・価格・規制で変わります。

データセンターの電力は 2030 年に約 945TWh と予測

棒の高さ＝世界のデータセンター全体の年間電力消費（IEA 2025 年報告）。

単位：TWh／年



IEA（国際エネルギー機関）の 2025 年報告は、世界のデータセンター全体の電力消費を、2024 年に約 415TWh（推計）、2030 年に約 945TWh（基本ケースの予測）としています。

$945 \div 415 \approx 2.28$ 倍。TWh（テラワット時）は電力量の単位で、1TWh = 10 億 kWh です。

⚠ 注意

AI 専用の電力ではありません。2026 年の実測値でもありません。

AI と水の関係：冷却・発電・半導体製造

AI と水は、3 つの経路で関係します：①データセンターの冷却、②発電所での水利用、③半導体の製造。冷却の方式によって、水の使い方が違います。

冷却方式	水との関係
空冷	建物側の冷却方式によっては水も使う
直接液冷・閉回路	液体を循環させて熱を運ぶ（循環≠消費）
蒸発冷却	水の蒸発で熱を奪うため、蒸発分の補充が必要

出典： S21 ・ S34 。確認日： 2026 年 10 月 3 日（依頼者の追加調査）。冷却の仕組みは p.402 。

RAG：関係する文書を探して、モデルに渡す

RAG（らぐ）

分類：仕組み

略語：RAG = Retrieval-Augmented Generation 日本語：検索拡張生成

何か	質問に関係する文書を検索して探し、モデルへの入力（コンテキスト）に添えてから回答を作らせる方法。
なぜ必要か	モデルは社内規程や最新の情報を学習していない。文書を渡せば、それを根拠に答えさせられ、出典も示せる。
身近な例	社員の質問「出張旅費の上限は？」→ 社内規程から該当部分を探す → 質問＋規程をモデルへ → 規程を根拠に出典つきで回答

ファインチューニング：追加のデータで学習し直す

ファインチューニング (ふあいんちゅーにんぐ)

分類：活動

日本語：追加学習

何か	学習済みのモデルに追加のデータで学習をさせ、パラメータを調整すること。
なぜ必要か	回答の形式・口調や、特定の作業での振る舞いを、目的に合わせて変えるため。
身近な例	問い合わせへの回答を、自社の決まった書式と言葉づかいで返すようにする。
似た言葉との違い	RAG はモデルの数値を変えずに入力へ文書を添える。ファインチューニングはモデルの数値そのものを変える。両方を組み合わせることもある。

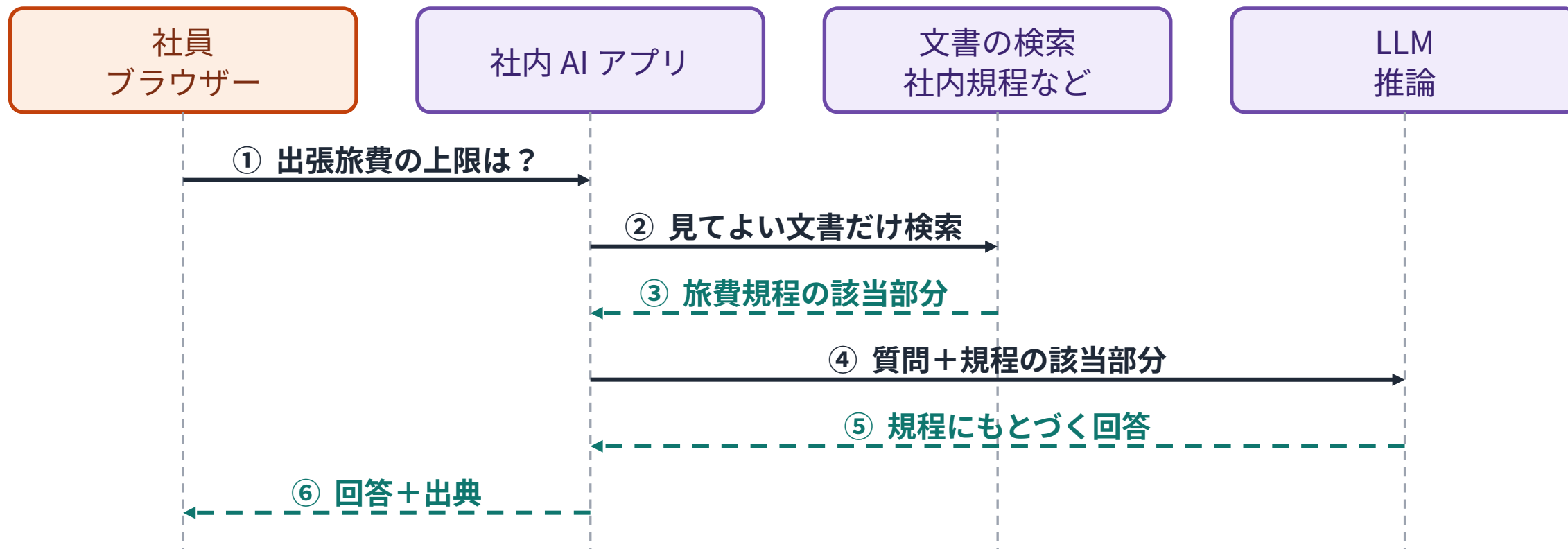
RAG とファインチューニングの使い分け

	RAG	ファインチューニング（追加学習）
何をするか	文書を検索して入力に添える	追加データでモデルの数値を調整
文書の更新	検索対象を差し替えれば反映しやすい	反映には再学習が必要
向く用途	社内文書・最新情報への質問、出典の提示	回答の形式・口調・特定の作業の振る舞い
必要なこと	検索の精度、閲覧権限、出典、誤答の評価	学習データの準備、評価、再学習の費用

 注意

社内文書への質問で、追加学習が必ず必要とは限りません。

RAGの流れと、権限の確認



注意： 渡す文書を、質問した本人が見てよいかを②の検索の段階で確かめます（認可）。⑥では根拠の規程名を出典として示します。検索しても必ず正答になるとは限りません。

米国・中国・欧州・日本の役割は、比べる軸で変わる

モデル・研究・資金・計算設備・半導体と装置・産業での利用など、軸によって評価が変わります。1つの順位にまとめません。

地域	目立つ強み	課題・制約	例
米国	最先端モデル、大規模な民間投資、巨大クラウド、GPU 設計	電力・設備の建設、費用	OpenAI ・ Google ・ Anthropic ・ NVIDIA
中国	大きな国内市場、研究、公開重みのモデル、産業導入	先端半導体・装置へのアクセス制約（輸出管理）	Alibaba ・ DeepSeek ・ Huawei
欧州	研究人材、産業技術、半導体の製造装置、企業向け AI	大型の資金調達、計算基盤、市場の分断	Mistral AI ・ ASML
日本	半導体の材料・装置、製造業、産業データ	巨大モデル開発の資金・計算基盤	キオクシア・東京エレクトロン

地域の比較を読むときの注意

「公開重みのモデル」は、パラメータ（重み）のファイルが公開され、誰でも手元で動かせるモデルのことです。

補足

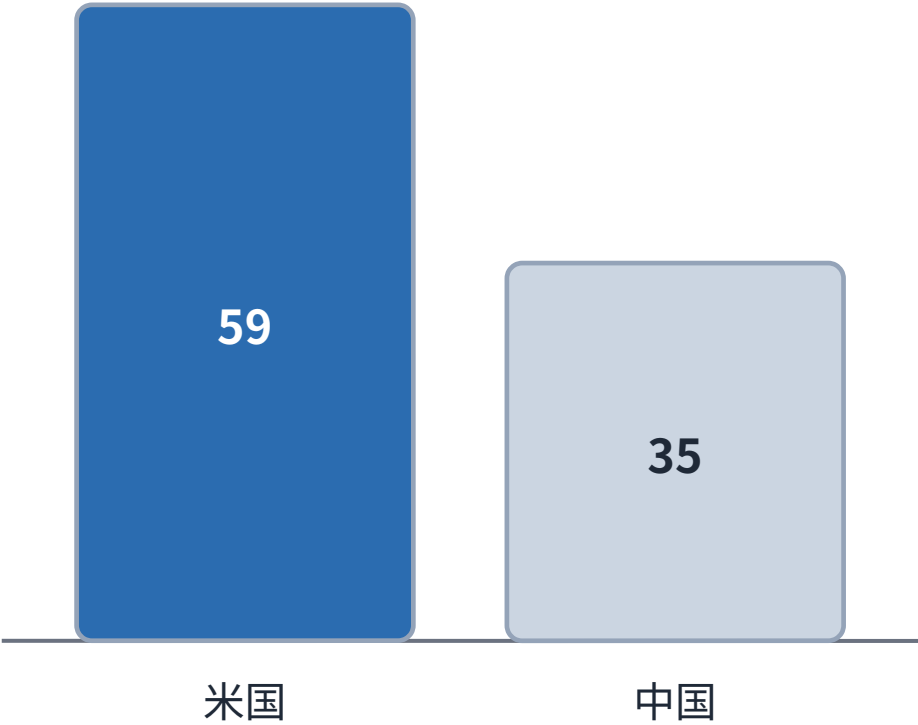
欧州が最先端モデルで目立たない理由は複数あり（ベンチャー資金の浅さ・分断、計算基盤など）、「規制のせい」だけでは説明できません。EUは2026年7月にAIギガファクトリー（大規模なAI計算設備）の公募を開始しました（計画であり稼働済みではない）。

注意

地域の特徴は傾向の整理であり、個々の企業・研究機関の能力を示すものではありません。

「注目モデル数」は、定義と集計範囲を確かめて読む

棒の高さ＝ 2025 年に開発された注目モデルの数（ Stanford AI Index 2026 ）。



何を数えたか	AI Index が定めた基準で「注目すべき」と選んだモデルの、開発元の所在地別の数（ 2025 年）
読めないこと	モデルの品質・国の総合力・利用者数・企業での導入状況
比べるときは	同じ報告・同じ定義・同じ年の数値どうしで比べる。論文・投資など他の指標と並べて見る

確認日： 2026 年 10 月 3 日（依頼者の追加調査）

企業の AI 基盤は、用途と運用から決める（1/2）

GPU の枚数から決めず、何に使い、誰が、どれだけの速さで使うかを先に確かめます。

SE が確かめる問い	なぜ必要か
何を AI で改善するか	成果の測り方を決める
データはどこにあり、誰が見てよいか	権限と情報漏えいの条件を決める
学習か推論か、どのモデルか	計算・メモリの必要量が変わる
同時利用者数と会話の長さ	GPU メモリ・KV キャッシュの必要量に影響 （ p.509 ）

企業の AI 基盤は、用途と運用から決める（2/2）

GPU の枚数から決めず、何に使い、誰が、どれだけの速さで使うかを先に確かめます。

SE が確かめる問い	なぜ必要か
最初の応答まで何秒を許すか	応答速度（TTFT）の目標を決める
外部のクラウドを使えるか	設置先・接続・契約を決める
どう監視・更新・停止・復旧するか	運用の責任を明確にする（MLOps / LLMOps）
電力と冷却を確保できるか	機器を実際に置いて動かせるかを判断

 **MLOps / LLMOps**

モデルを継続して運用するためのデータ・評価・配備・監視・更新の仕組み。GPU を設置するだけでは成り立ちません。

第 12 章のまとめ

✓ ここまでで説明できること

AI ・ 機械学習 ・ 深層学習は方法の入れ子、生成 AI は用途、LLM はモデルの種類。
モデル＝計算の仕組み＋パラメータ。学習でパラメータを調整し、推論でそのモデルを使う。
LLM は次のトークンを 1 つずつ予測する。もっともらしい誤りがある。
AI 基盤は、GPU だけでなく、メモリ・保存・GPU 間通信・電力・冷却・運用でできている。
RAG は文書を探して渡す方法、ファインチューニングはモデルの数値を調整する学習。

→ 次へのつながり

次の第 13 章では、こうした基盤を顧客の要求に合わせて設計・構築・運用する SE の仕事を見ます。

確認の問い（１）

自分の言葉で答えてから、次のページの解説を読みましょう。

- ① 生成 AI と LLM は、それぞれ何の分類ですか。
- ② 迷惑メールの例で、学習と推論は何をする工程ですか。
- ③ 推論中の GPU メモリに置かれる 2 種類のデータは何ですか。

確認の問い（1）：解説

前のページの問いへの答えと、考え方です。

✓ 問1 生成AIとLLMは、それぞれ何の分類ですか。

生成AIは「作る」という用途の分類、LLMは大量の文章で学習した言語のモデルという「モデルの種類」です。LLMは要約・分類にも使います（p.501）。

✓ 問2 迷惑メールの例で、学習と推論は何をする工程ですか。

学習は、印付きのメールで判定が正解に近づくよう点数（パラメータ）を調整する工程。推論は、新しいメールに学習済みの点数を当てはめて判定する工程です（p.494）。

✓ 問3 推論中のGPUメモリに置かれる2種類のデータは何ですか。

モデルのパラメータと、会話ごとの途中計算であるKVキャッシュです。会話が長く、同時利用者が多いほどKVキャッシュが増えます（p.509）。

確認の問い（2）

自分の言葉で答えてから、次のページの解説を読みましょう。

- 1 IEAの「2030年に約945TWh」を読むときの条件は？
- 2 RAGで、検索の段階で確かめるべきことは何ですか。
- 3 GPUを増やしても速くならないのは、どんなときですか。

確認の問い（２）：解説

前のページの問いへの答えと、考え方です。

✓ 問１ IEAの「2030年に約945TWh」を読むときの条件は？

AI専用ではなく世界のデータセンター全体の電力であること、基本ケースの予測で実測値ではないこと（2024年の約415TWhも推計）です（p.521）。

✓ 問２ RAGで、検索の段階で確かめるべきことは何ですか。

渡す文書を、質問した人が見てよいか（認可）です。検索しても必ず正答になるとは限りません（p.526）。

✓ 問３ GPUを増やしても速くならないのは、どんなときですか。

データの読み込み、サーバー間の通信、電力・冷却のどれかが足りず、GPUが待たされているときです（p.514）。

第 13 章

SE の仕事：設計・構築・運用

この章で学ぶこと

システムが「何を、どの条件で」すべきかを決め（要件定義）、どう作るかを決め（設計）、止まりにくくし、動き始めた後は見張って直す（運用）—— SE（システムエンジニア）の仕事の流れと、そこで使う言葉を学びます。

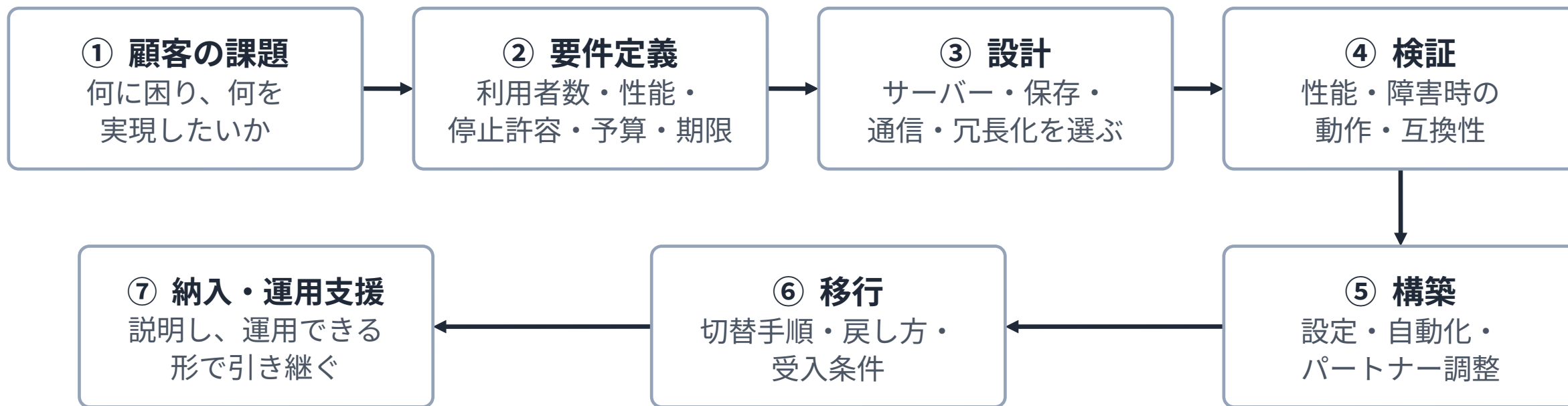
→ 前の章とのつながり

これまでの章で学んだサーバー・DB・ネットワーク・保存方式・クラウド・セキュリティ・AI 基盤を、顧客の目的に合わせて組み合わせるのが SE の仕事です。

システムができるまでの工程

顧客の課題を聞くところから、動き始めたシステムを引き継ぐところまで、順に進みます。

仕事の順番を表す図。箱＝工程、黒い矢印＝次の工程へ進む順番（①→⑦）。



各工程で設計書や試験結果などの成果物を残し、「なぜこの構成で要件を満たせるか」を説明します。

各工程で残す成果物

工程	残す成果物の例
要件定義	要件一覧・前提と制約
設計	構成図・設計書
検証	検証計画・結果
構築	設定値・手順・変更記録
移行	試験結果・切戻し（元に戻す）計画
納入・運用支援	運用設計書・完了記録

補足

SE の 1 日は打合せ・設計・設定・確認・報告の組み合わせです。割合は部署・顧客・工程で変わり、常にコードを書く仕事でも、常に会議だけの仕事でもありません。

要件定義：何を、どの条件で実現するかを決める

要件定義（ようけんていぎ）

分類：活動

何か	システムに「何を、どの条件で」実現させるかを顧客と合意し、文書にする工程。必要な機能・性能・制約・責任・完成の条件を決める。
なぜ必要か	頼む側と作る側の思い違いを防ぎ、完成したかどうかを判断する基準にするため。
身近な例	サンプル商店の「注文履歴を見る」なら、「誰が何を見られるか」「何秒以内に表示するか」「何人が同時に使うか」を決める。
似た言葉との違い	設計は「どう実現するか」を決める工程。要件定義は「何を実現するか」を決める工程で、設計より前に行う。

機要件と非機要件

機要件＝何をするか

システムができることです。

例：佐藤さんが、自分の注文履歴（注文 101 と 103）を一覧で見られる。

非機要件＝どの条件で動くか

性能・止まりにくさ・安全・運用・拡張のしやすさです。

例：一覧を 3 秒以内に出す。夜中も止めない。他人の注文は見せない。

ポイント

機能が同じでも、非機要件が違えば構成が大きく変わります。例：「年に数時間なら止まってよい」ならサーバー 1 台で足りても、「ほぼ止めない」なら予備の機械と切替の仕組みが要ります。

要件の例：買い物サイトと社内 AI （1/2）

要件の種類	買い物サイトの例	社内 AI の例
機能要件	商品を検索できる、カートに入れて注文・決済できる	社内文書について質問すると、出典付きで回答する
性能	セール時に毎秒○件の注文を受けられる	最初の応答を○秒以内に返す
止まりにくさ（可用性）	年間の停止は○時間以内	業務時間中は使える

要件の例：買い物サイトと社内 AI （2/2）

要件の種類	買い物サイトの例	社内 AI の例
セキュリティ	カード情報を保存しない、管理画面は MFA	閲覧権限のない文書を回答に使わない
運用性	障害を 5 分以内に検知する	モデルの更新手順と評価方法がある
拡張性	会員数が 2 倍になっても対応できる	利用部署を段階的に広げられる

○ は顧客と合意して決める数値です。セキュリティの仕組み（認証・認可・MFA など）は第 11 章（p.462）で学びました。

非機能要件があいまいだと、何に困るか

「速く表示してほしい」だけでは、何秒なら合格なのか分かりません。設計が正しいかを確かめられず、完成したかどうか判断できません。

あいまいな言い方	確かめられる言い方（説明用の例）
速く表示する	混雑時も、注文履歴の 95% を 3 秒以内に表示する
なるべく止めない	年間の停止は合計 9 時間以内（計画停止を除く）
たくさんの人が使える	同時に 1,000 人が使っても上の速さを保つ

⚠ 注意

数値は、顧客と合意して決めます。SE が勝手に決めたり、根拠なく「大丈夫です」と言ったりしません。

基本設計と詳細設計：大きな方式から設定値へ

基本設計

全体の方式・構成をどうするか、なぜそれか。

成果物：構成図・方式設計書

例：Web サーバー 2 台・アプリサーバー 2 台・DB サーバー 2 台（予備つき）、入口に負荷分散

詳細設計

具体的な設定値・手順は何か。

成果物：パラメーター表・手順書

例：Web サーバー 1 の IP アドレス、OS の設定、DB のバックアップは毎日午前 2 時

ソフトウェア開発では、基本設計を**外部設計**、詳細設計を**内部設計**と呼ぶこともあります（呼び方と範囲は会社によって違います）。

PoC ・サイジング・テスト

PoC（実現性の検証）

Proof of Concept。その方式で本当にできそうかを小さく試す。

例：規程 100 件だけで、社内 AI が出典付きで答えられるか試す。

サイジング

必要な容量・処理能力・台数を見積もる。

例：同時利用者数と会話の長さから、必要な GPU メモリと台数を出す（第 12 章）。

テスト（試験）

設計どおり動くか、障害時にどうなるかを確かめる。

例：サーバーを 1 台止めても、注文履歴が見られるか。

工程ごとに、答える問いと成果物が違う

工程	答える問い	主な成果物
要件定義	何を、どの条件で実現するか	要件一覧・前提と制約
PoC（実現性の検証）	その方式で本当にできそうか	検証計画・結果・課題
サイジング	どれだけの容量・処理能力・台数が要るか	見積もりの根拠と前提
基本設計	全体の方式・構成をどうするか、なぜそれか	構成図・方式設計書
詳細設計	具体的な設定値・手順は何か	パラメーター表・手順書
試験	設計どおり動くか、障害時にどうなるか	試験仕様・試験結果

PoC の限界と、設計書に残す「選ばなかった案」

⚠ 注意

PoC が成功しても、本番の利用者数・データ量・障害時の動きまで保証されたわけではありません。PoC と本番で前提がどう違うかを記録します。

i 設計書に残すこと

設計書には「選んだ理由」と「選ばなかった案」を残すと、後で変更するときの判断材料になります。

🔧 設計書の書き方の例（説明用）

採用：DB サーバー 2 台で、片方が壊れたらもう片方が引き継ぐ構成。理由：年間の停止を○時間以内にする要件のため。

不採用：DB サーバー 1 台＋毎日のバックアップ。理由：故障時、戻すまでに数時間かかり要件を満たせない。

障害：システムが本来の働きをできなくなること

障害（しょうがい）

分類：考え方

何か	システムや機器が、本来の働きをできなくなること、またはその状態。機器の故障・設定の誤り・ソフトウェアの不具合・回線の切断など、原因はさまざま。
なぜ必要か	障害は必ず起こるものとして備える。サンプル商店なら、注文が止まれば売上を失い、利用者の信頼も下がる。
身近な例	DB サーバーの SSD が壊れて、佐藤さんが注文履歴を見られない。
似た言葉との違い	「故障」は機器が壊れることで、障害の原因の 1 つ。障害は、設定ミスや混雑で使えない場合も含む広い言葉。

可用性と稼働率：どれだけ使える状態を保てるか

可用性は、使いたいときに使える状態を保つ性質です。速さ（性能）とは別の性質です。

可用性を数で表す代表が**稼働率**です。
稼働率＝実際に使えた時間 ÷ 使えるはずの時間

1 年は 365 日 × 24 時間＝ 8,760 時間です。

稼働率	1 年で止まってよい時間
99%	87.6 時間
99.9%	8.76 時間
99.99%	約 53 分
99.999%	約 5 分

↑ 8,760 時間 × （1－稼働率）で計算した説明用の値です。「9」が 1 つ増えるごとに、止まってよい時間は 10 分の 1 になります。

機械を 2 台使うと稼働率はどう変わるか

稼働率 99%（0.99）の機械を 2 台使う場合を、つなぎ方で比べます（故障はそれぞれ独立に起きると仮定）。



直列： A と B の両方が動かないと使えない
（例：Web サーバーと DB サーバー）。
 $0.99 \times 0.99 = 0.9801$ （98.01%）→ 下がる



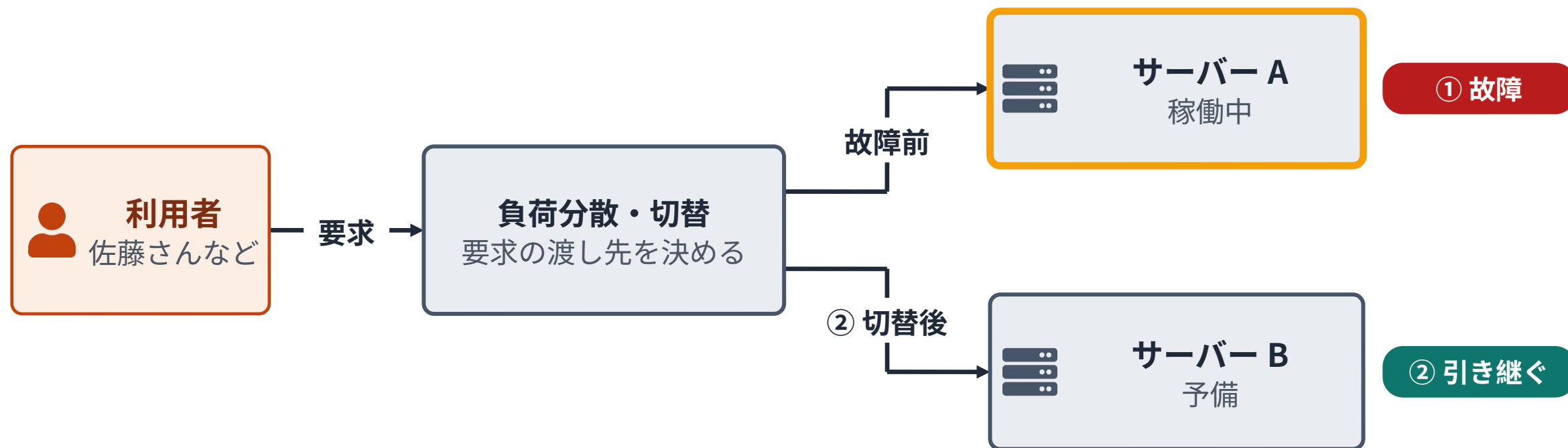
並列： A か B のどちらか 1 台が動けば使える
（予備あり）。
 $1 - (1 - 0.99) \times (1 - 0.99) = 0.9999$ （99.99%）→ 上がる

i 別の求め方

平均の故障間隔（MTBF）と平均の修理時間（MTTR）から、稼働率 = $\text{MTBF} \div (\text{MTBF} + \text{MTTR})$ と求められます。例：MTBF 999 時間、MTTR 1 時間 → 99.9%

冗長化と、予備への切り替え（フェイルオーバー）

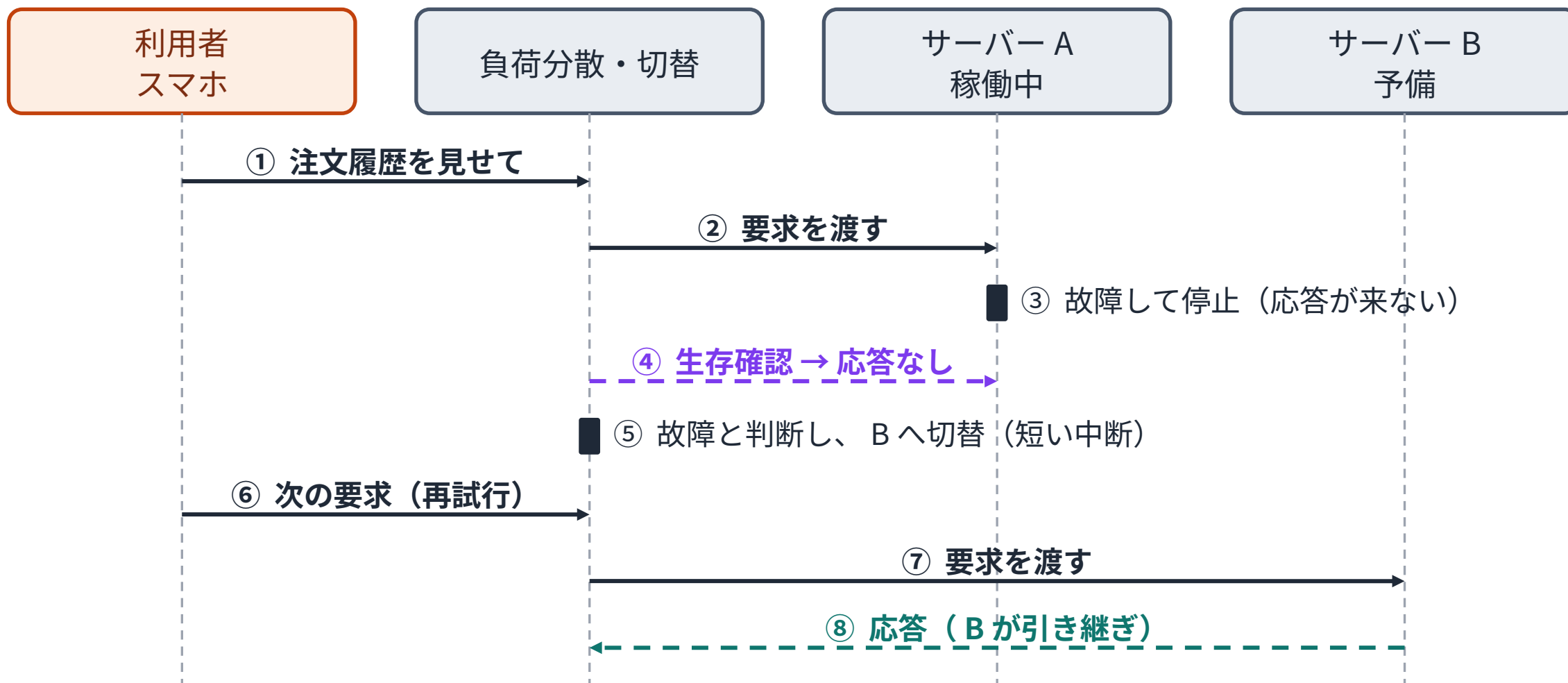
役割の関係図（一般的な構成例）。黒い矢印＝要求の移動。①サーバー A が故障すると、②要求の渡し先がサーバー B に切り替わります。



冗長化（予備を用意すること）だけでは足りず、故障に気づいて予備へ切り替える仕組み＝**フェイルオーバー**が必要です。複数の機器をまとめて1つのように動かす構成を**クラスタリング**と呼びます。

障害が起きたときの時間の流れ

上から下へ時間が進みます。紫の破線は生存確認（補助の問い合わせ）。



可用性・高性能・冗長化・クラスター・切替の違い

言葉	意味	混同しやすい点
可用性（HA）	使える状態を保つ性質・構成	速さとは別。稼働率などで表す
高性能	処理能力が高いこと	速くても、1 台なら故障で止まる
冗長化	予備の機器・経路を用意すること	予備があっても切替の仕組みが必要
クラスター	複数の機器をまとまりとして使う構成	目的は性能・管理・継続稼働などさまざま
フェイルオーバー	障害時に処理を予備へ切り替えること	切替中の短い中断や、データの扱いに注意

HA は High Availability（高い可用性）の略です。

予備があっても止まる場合と、障害の分離

⚠ 注意

冗長化しても、切替の仕組みと試験がなければ止まります。切替時の短い中断や、切り替えた後のデータの整合性（食い違いがないこと）も設計で考えます。

障害の分離

1 つの部品の故障が全体に広がらないように区切る考え方。止めにくい取引の基盤（第 14 章の NonStop など）が使う。

冗長化とバックアップは別

冗長化は止まらないための備え、バックアップは戻すための備え（p.379）。

止まりにくく設計しても「絶対に止まらない」わけではありません。計画停止・運用ミス・災害への備えは別に必要です。

RPO と RTO ：どの時点まで戻し、何分で復旧するか

RPO（目標復旧時点）は「どの時点までのデータを戻せばよいか」、**RTO**（目標復旧時間）は「どれだけの時間で使えるようにするか」の目標です。

時刻を左から右へ並べた図（説明用の例）。黒い横線＝時間の流れ、縦の札＝出来事の時刻。



目標と実績を分けて考える・DR（災害復旧）

	目標（事前に決める）	実績（前のページの例）
RPO：どの時点まで戻すか	例：最大 5 分前まで戻せばよい	9:55 まで戻せた → 損失 5 分（目標内）
RTO：どれだけで復旧するか	例：20 分以内に再開	10:20 に再開 → 20 分（目標ちょうど）

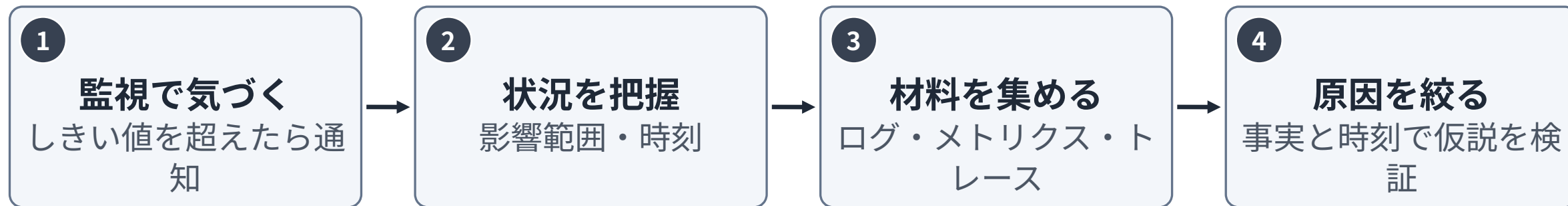
RPO と RTO は「目標」で、実際の損失・停止時間（実績）とは区別します。時刻と目標値は説明用の例です。

ポイント

DR（災害復旧）では、建物ごと使えなくなる場合を想定し、離れた場所のバックアップや予備の設備を用意します。RPO を短くするほど頻繁な複製が、RTO を短くするほど待機した予備が必要になり、費用が増えます。

監視で異常に気づき、記録から原因を調べる

監視は、状態や異常に気づく活動です。**可観測性**は、得られる情報から内部の状態を調べられる性質です。気づいた後、原因を調べる材料が要ります。



しきい値は「ここを超えたら知らせる」と決めた境目の値です。例：注文履歴の表示に 3 秒以上かかる要求が 5 分間で 1 割を超えたら、担当者に通知する（説明用の例）。

ログ・メトリクス・トレース：原因を調べる3つの材料

ログ

出来事の記録。いつ何が起きたか。

例：「10:02 ログイン失敗
user=xx」

メトリクス

定期的に測った数値。グラフで傾向を見る。

例：CPU 使用率・応答時間・エラー率

トレース

1つの要求が、複数の処理をどの順に通り、どこで時間がかかったかの記録。

3つを組み合わせると、「何時から（メトリクス）」「どの要求が（トレース）」「何のエラーで（ログ）」遅くなったかが分かります。

トレースの例：注文履歴の表示はどこで遅いか

時間を左から右へとった図。帯の長さ＝その処理にかかった時間。1つの要求「注文履歴を見せて」の記録です（数値は説明用）。



この例では、DB サーバーでの検索に時間の大半がかかっています。調べる場所を DB に絞れます。

ログの時刻をそろえる・秘密をログに書かない

ログの時刻がずれていると、機器をまたいだ出来事の順番が分からなくなります。機器の時計は **NTP** で合わせます（p.265）。

ログに書かれた時刻	記録した機器	実際の順番
10:02:05 要求を受けた	アプリサーバー（時計が正しい）	1 番目
09:59:10 検索を開始	DB サーバー（時計が 3 分遅れ）	2 番目（実際は 10:02:10）

↑ 説明用の例。時計がずれると、DB の処理が先に起きたように見えてしまいます。

⚠ 注意

パスワード・カード番号・個人情報などをログに書かない設計も必要です。ログは多くの人や仕組みが読むためです。

障害の切り分け：手前から順に確かめる

例：佐藤さんが注文履歴を見られない。端末から順に範囲を絞ります。

段階	確認すること	確認の例
① 端末	その端末だけか、他でも起きるか	別の端末で試す
② LAN	IP アドレス・出口を取得できているか	ipconfig /all
③ 名前解決	名前から IP アドレスを引けるか	nslookup （DNS）
④ 到達性	相手の機器・ポートに届くか	ping、Test-NetConnection - Port 443
⑤ TLS ・ 認証	証明書・ログインで失敗していないか	エラーメッセージ・証明書の期限
⑥ アプリ	アプリや DB でエラーが出ていないか	アプリのログ・監視画面

切り分けの考え方は p.308、コマンドの使い方は p.311 で見ました。

復旧を優先し、原因究明は別に進める

障害が起きたら、まず利用者が使える状態に戻すこと（復旧）を優先します。根本原因の特定と再発防止は別に進めます。両者が同じ順で終わるとは限りません。

📋 流れの例（説明用）

10:00 注文履歴が見られないと通知

10:15 DB サーバーを予備に切り替えて再開（復旧）

翌日以降 なぜ故障したか（SSD の劣化か、設定か）を調べ、再発防止策を実施

原因が分かるまで待っていると、その間ずっとサービスが止まってしまうためです。

インシデントと問題管理

インシデント

サービスの中断・品質低下という出来事。まず影響を抑え、復旧させる。

例：10時から注文履歴が見られない。

問題管理

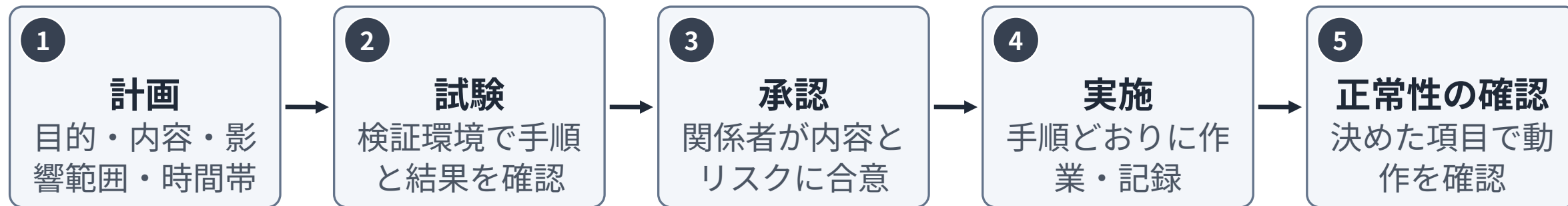
根本原因を特定し、再発防止策を実施する。時間がかかることもある。

例：SSDの劣化を早めに見つける監視を足す。

インシデントへの対応、問題管理、変更管理などをまとめて、ITサービスを決めた品質で提供し続けるための管理を**サービスマネジメント**と呼びます。

変更は、試験・影響範囲・切戻しを決めてから

本番環境（利用者が実際に使っているシステム）の変更は、事前に試験し、影響範囲と、失敗したときに元に戻る方法を決めてから行います（[変更管理](#)）。



⚠ 注意

⑤ の確認に失敗したら → **切戻し**（変更前の状態へ戻す）。戻せる範囲は変更の内容で違います。

変更の前に考えること

考えること	例
影響範囲	どのサービス・利用者・他システムが影響を受けるか
切戻しの条件	何が起きたら戻すか、誰が判断するか、何分で決めるか
戻せる範囲	設定は戻せても、変換後のデータは戻せないことがある → 事前のバックアップ

📖 具体例（説明用）

サンプル商店で DB のソフトを新しい版にする。夜 2 時から作業し、注文履歴の表示を確認できなければ 3 時までに切り戻すと決めておく。事前に DB のバックアップを取る。

移行：古いシステムから新しいシステムへ

移行は、旧環境から新環境へ、データや利用者の使い先を切り替えることです。一度に全部切り替える方法のほかに、次のような方法があります。

並行稼働

しばらく旧と新を同時に動かし、結果が同じかを比べてから旧を止める。安全だが手間と費用がかかる。

段階的な切替

一部の利用者や機能から順に新へ移す。問題が出ても影響が一部で済む。

どの方法でも、切替手順・戻し方・受入条件（何が確認できたら完了とするか）を事前に決めます。

プロジェクトマネジメント（PM）と責任分界

プロジェクトマネジメント

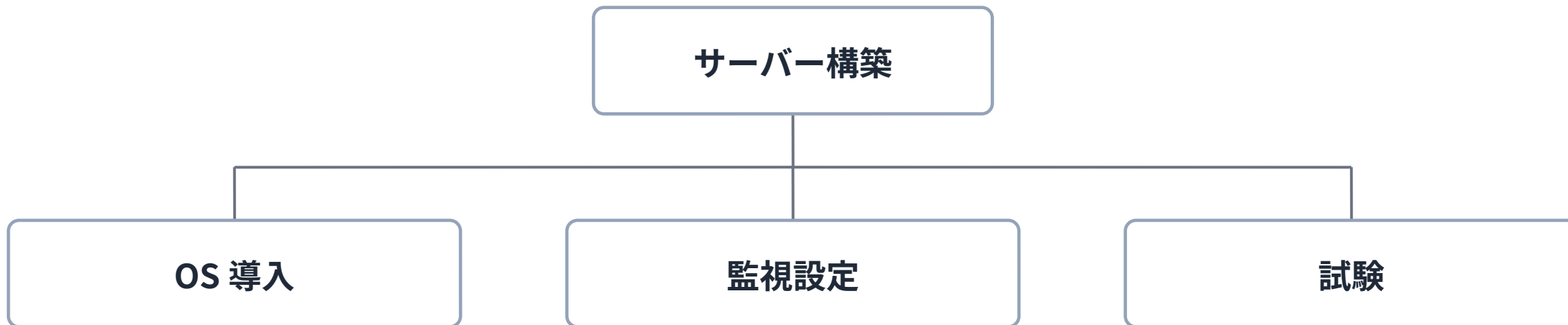
分類：活動

略語：PM = Project Management

何か	期限のある仕事のまとまり（プロジェクト）を目的どおりに完了させる管理。範囲・期限・品質・費用・リスク・関係者を管理する。
なぜ必要か	多くの人・会社・機器が関わり、1 か所の遅れが全体の遅れになるため。
身近な例	サーバーの納期が 2 週間遅れる → 試験と切替の日程を組み直し、顧客と合意する。
似た言葉との違い	「PM」は管理する人（プロジェクトマネージャー）の略でも使う。運用中のサービスの管理（サービスマネジメント）とは対象が違う。

WBS：作業を細かく分けて抜けを防ぐ

層の図。上の箱の作業を、下の箱の作業に分けています。灰色の線＝「分けるようになる」という関係。



説明用の例

WBS（Work Breakdown Structure：作業分解構成）は、作業を細かく分けて整理したものです。分けると、担当・期間・抜けている作業が見えるようになります。

RACI：だれが実行し、だれが責任を持つか

RACI は、作業ごとに 4 つの関わり方を決める担当表です。

作業	R：実行	A：最終責任	C：相談	I：共有
試験	若手 SE	リーダー	顧客	営業
監視設定	若手 SE	リーダー	運用担当	顧客

R = 手を動かす人、A = 結果に最終的な責任を持つ人（1 人に決める）、C = 事前に意見を聞く人、I = 結果を知らせる人。表は説明用の例です。

責任分界・エスカレーション・SLA と SLO

用語	意味	具体例
責任分界	誰がどこまで担当するか	OS までは HPE、アプリは顧客の開発会社
エスカレーション	必要な支援や判断を上位・専門担当に求める	30 分で原因不明なら専門チームへ
SLA / SLO	契約で合意した水準／運用上の目標	稼働率・応答時間・問い合わせへの回答時間

補足

回線の開通や機器の納期は、試験や切替の前提になります。技術を理解しているほど、遅れの影響や依存関係を正しく判断できます。

職種によって、答える問いと成果物が違う（1/2）

下の表は典型例です。職種名が同じでも会社や部署で範囲は変わるため、実際は募集要項と配属面談で確認します。

職種	主に答える問い	主な成果物	主に使う知識
SE（システムエンジニア）	要件を満たす構成は何か。どう作り、どう確かめるか	構成図・設計書・試験結果	サーバー、ストレージ、通信、仮想化、運用
アプリ開発者	利用者に必要な機能をどう実装するか	プログラム・試験コード	言語、DB、Web、設計手法

職種によって、答える問いと成果物が違う（2/2）

下の表は典型例です。職種名が同じでも会社や部署で範囲は変わるため、実際は募集要項と配属面談で確認します。

職種	主に答える問い	主な成果物	主に使う知識
運用担当	止まらず安全に動き続けているか	監視設定・手順書・障害記録	監視、ログ、バックアップ、変更管理
技術提案職（プリセールス）	顧客の課題に、どの製品・構成が合うか	提案書・概算構成・検証	製品知識、構成、説明力
PM（プロジェクトマネージャー）	範囲・期限・品質・費用・リスクを守れるか	計画・WBS・課題管理	管理手法、技術の影響理解

補足

HPE の公開 SE 募集は、サーバー・ストレージ・ネットワーク・スーパーコンピューター・クラウドを用いた設計・構築・納入を説明しています。

第 13 章のまとめ

✓ ここまでで説明できること

要件定義で「何を（機能要件）」「どの条件で（非機能要件）」を数値で合意し、基本設計→詳細設計へ進む。

可用性＝使える状態を保つ性質。冗長化に加え、切替の仕組みと試験が要る。

RPO は戻す時点、RTO は復旧までの時間の目標。実績とは区別する。

監視で気づき、ログ・メトリクス・トレースで原因を絞る。復旧と原因究明は別に進める。

変更は試験・影響範囲・切戻しを決めてから。PM は WBS・RACI・SLA などを使う。

→ 次へのつながり

次の第 14 章では、こうした仕事を担う IT 業界の会社と、HPE の製品を見ます。

確認の問い（１）

自分の言葉で答えてから、次のページの解説を読みましょう。

- ① 機能要件と非機能要件の違いを、例で説明してください。
- ② 稼働率 99% の機械 2 台を並列にすると、稼働率は？
- ③ 冗長化しているのに止まるのは、どんな場合ですか。

確認の問い（１）：解説

前のページの問いへの答えと、考え方です。

✓ 問 1 機能要件と非機能要件の違いを、例で説明してください。

機能要件は「何をするか」（例：注文履歴を一覧で見られる）、非機能要件は「どの条件で動くか」（例：3 秒以内に表示、他人の注文は見せない）です（p.541）。

✓ 問 2 稼働率 99% の機械 2 台を並列にすると、稼働率は？

$1 - (1 - 0.99) \times (1 - 0.99) = 0.9999$ で 99.99% です。直列なら $0.99 \times 0.99 = 98.01\%$ に下がります（p.551）。

✓ 問 3 冗長化しているのに止まるのは、どんな場合ですか。

故障に気づいて予備へ切り替える仕組み（フェイルオーバー）がない、または試験していなくて切替が動かない場合です（p.555）。

確認の問い（２）

自分の言葉で答えてから、次のページの解説を読みましょう。

- ① RPO5 分・RTO20 分とは、それぞれ何の目標ですか。
- ② ログ・メトリクス・トレースは、それぞれ何の記録ですか。
- ③ 本番の変更の前に決めておくことを 3 つ挙げてください。

確認の問い（２）：解説

前のページの問いへの答えと、考え方です。

✓ 問 1 RPO5 分・RTO20 分とは、それぞれ何の目標ですか。

RPO5 分は「障害の 5 分前までのデータを戻せばよい」、RTO20 分は「20 分以内に使えるようにする」という目標です（p.556）。

✓ 問 2 ログ・メトリクス・トレースは、それぞれ何の記録ですか。

ログは出来事の記録、メトリクスは定期的に測った数値、トレースは 1 つの要求が通った処理と時間の記録です（p.559）。

✓ 問 3 本番の変更の前に決めておくことを 3 つ挙げてください。

影響範囲、切戻しの条件（何が起きたら誰が戻すと判断するか）、戻せる範囲（データのバックアップ）などです（p.566）。

第 14 章

IT 業界の会社と HPE ・ HPE 製品

この章で学ぶこと

機器を作る会社、組み合わせて作り上げる会社、サービスとして貸す会社など、立場の違う会社が 1 つのシステムにどう関わるかを学びます。

そのうえで、HPE が扱う分野と製品を、「何をする製品か」と「一緒に学ぶ基礎技術」の組み合わせで読みます。

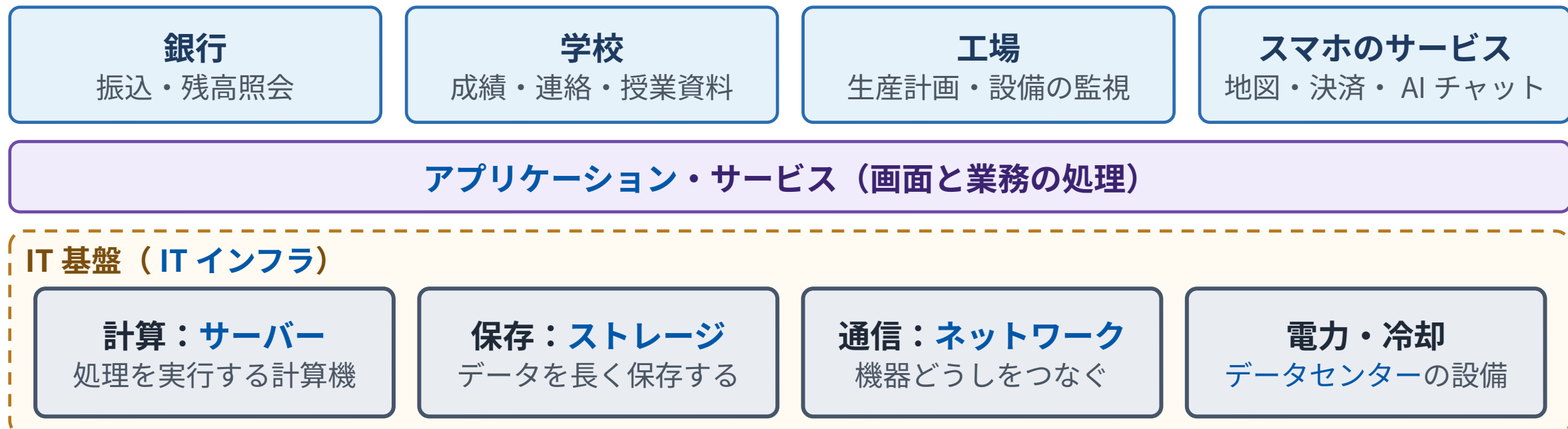
→ 前の章とのつながり

第 1 ～ 13 章で学んだ技術の知識で、会社や製品の説明が読めます。製品の説明には、基礎のページ番号を添えます。

IT 企業は、社会の仕組みを動かす土台を支える

銀行の振込、学校の連絡、工場の生産管理、スマホのサービスは、どれも見えない場所にある計算機・保存装置・通信で動いています。それらを作り、組み合わせ、動かし続けるのが IT 企業の仕事です。

上の段ほど利用者に近く、下ほど土台に近いことを表す図です（線や矢印はありません）。



学習用に簡略化した図

元資料が先に使っていた 6 つの言葉を復習する

元資料は次の 6 語を「詳しい説明は後の章」として先に使っていました。この資料では説明済みです。

言葉	この章で使う意味	説明したページ
サーバー	サービスを提供する役割の計算機やソフト	p.124
ストレージ	データを長く保存する装置・資源	p.82 ・ p.350
ネットワーク	機器をつないでデータを運ぶ仕組み	p.191
クラウド	ネット経由で必要な資源を利用する方式	p.443
AI	認識・予測・生成などを機械で行う分野	p.486
SE	要件を聞き、仕組みを設計・構築する技術者	p.540

意味が思い出せない言葉は、右の列のページに戻ってから先へ進みましょう。SE は「システムエンジニア」の略です。

1 つのシステムに関わる 5 つの立場

1 つのシステムには、立場の違う複数の会社に関わります（例は元資料が挙げた会社）。

立場	何をするか	例
利用企業（顧客）	業務を持ち、最終的な責任を負う	銀行・工場・自治体など
メーカー	機器・ソフトを開発し、提供・保守する	HPE、Dell、Cisco
Sler	複数の製品を組み合わせ、設計・構築する	CTC、NTT データ
クラウド事業者	計算・保存などをサービスで提供する	AWS、Azure、Google Cloud
通信・DC 事業者	回線、設置場所、電力・冷却を提供する	KDDI、IIJ、Equinix

DC はデータセンターの略です。Sler は次のページで説明します。

Sler（システムインテグレーター）とは

Sler は「エスアイヤー」と読みます。「SI」は System Integration（システムの統合）の略で、別々の会社の製品を組み合わせ、1つの仕組みとして動くようにまとめることです。「er」は「それをする人・会社」という意味です。

❓ なぜこの役割が必要か

サーバー、データベースのソフト、ネットワーク機器、クラウドは、それぞれ別の会社を作っています。どれを選び、どうつなぎ、どう設定すれば顧客の要件を満たせるかを決めて作り上げる役が必要です。

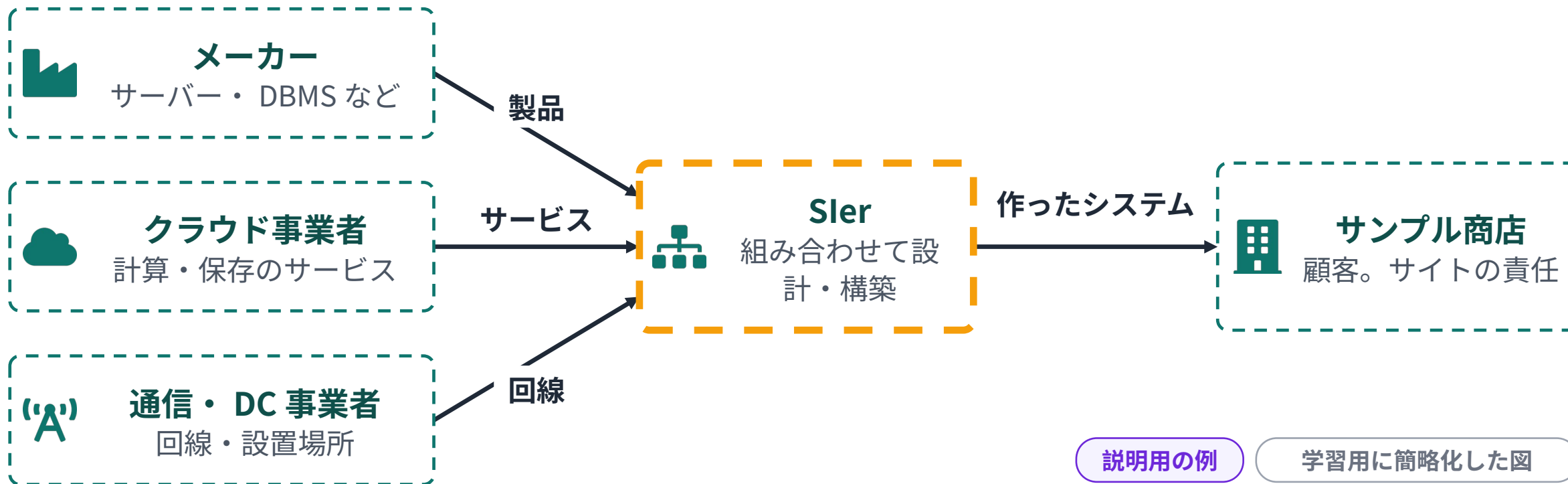
⚖️ メーカーとの違い

メーカーは自社の製品を作って提供します。Sler は、メーカーを問わず製品を組み合わせ、顧客ごとのシステムを作ります。

Sler は何を組み合わせるのか（説明用の例）

サンプル商店（架空の会社）が注文サイトを作り直す場面で考えます。

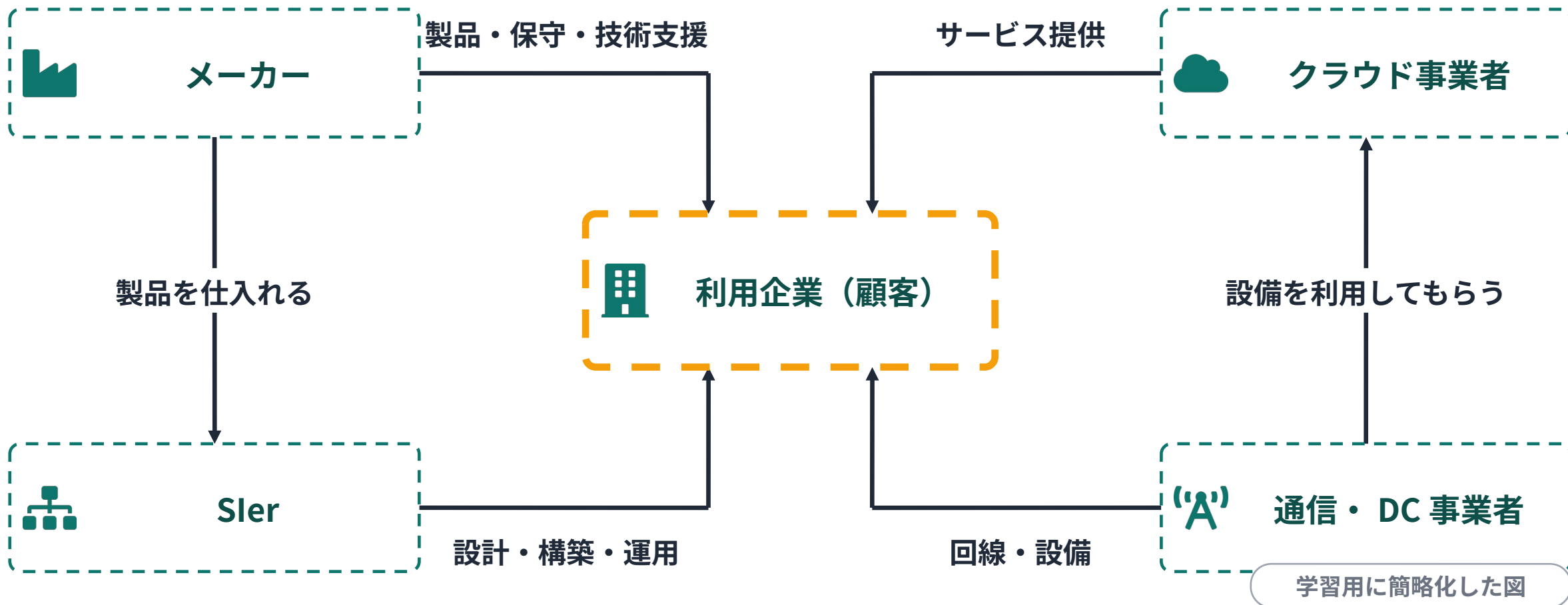
点線の箱は会社の立場、矢印は製品・サービス・作業の成果が提供される向きです（お金の流れではありません）。



実際には、顧客が直接クラウドを契約するなど、つながり方は案件ごとに違います。

会社どうしは、何を提供し合っているか

役割の関係図。点線の箱は会社の立場、矢印は製品・サービス・設備が提供される向きです（実際の配置ではありません）。



1 社が複数の役割を持つことも多い

会社名だけで役割は 1 つに決まりません。「この案件で、この会社はどの立場か」を考えます。

メーカーが構築も

構築を手がけたり、使った量に応じて払う「従量課金」の利用モデルを出したりする。
例：GreenLake（p.611）

Sler が自社クラウドを

自社のデータセンターでクラウドのサービスを運営し、構築とあわせて提供する。

通信会社が DC も

回線を提供する通信事業者が、データセンターやクラウドも運営する。

前のページの図でも、顧客はメーカーから直接買うことも、Sler に任せることも、クラウドを借りることもあります。

HPE と日本ヒューレット・パッカートの位置づけ

HPE（Hewlett Packard Enterprise）は企業向けの技術を扱う会社です。PC やプリンターの HP Inc. とは、2015 年の分社（1 つの会社を複数に分けること）で別会社になりました。

	HPE（Hewlett Packard Enterprise）	HP Inc.
主な領域	企業向けのサーバー、ストレージ、ネットワーク、クラウド管理、AI・HPC 基盤	PC、プリンター、周辺機器など
主な顧客	企業・官公庁・研究機関など	個人・企業の利用者
成り立ち	2015 年の分社で、企業向け事業を担う会社として独立	同じ分社で、PC・印刷事業を担う会社に
日本の法人	日本ヒューレット・パッカート合同会社 （Hewlett Packard Japan, G.K.）	（日本 HP は別の法人）

HPC は高性能計算のことです（p.590）。

「HPE」が何を指すかと、年の扱いの注意

「HPE」はどこを指すか

「法人」は、法律上ひとつの会社として扱われる単位です。日本では「日本ヒューレット・パカード合同会社」という法人が事業をしています。

この資料の「HPE」は、文脈により米国本社を含むグループ全体、または日本法人を指します。採用・給与は日本法人の情報です。

年を混ぜない

グローバル企業の分社（2015 年）と、日本法人の設立・名称変更の年は別の出来事です。この資料では日本法人の設立年を扱いません。

HP Inc. の日本法人（日本 HP）は、日本ヒューレット・パカード（HPE 側）とは別の会社です。名前が似ているので、採用情報を調べるときは会社名を正確に確認します。

HPE が提供する 6 つの分野

製品名を覚える前に、何のための分野かを押さえます。→のページで詳しく見ます。

計算（サーバー）

業務や AI の処理をする計算機。例：ProLiant →p.601

保存（ストレージ）

業務データやバックアップを保存。例：Alletra →p.606

通信

社内・DC・拠点間の網。
例：Aruba →p.608

利用・管理

自社設備とクラウドの資源を管理。例：GreenLake →p.611

AI ・ HPC

GPU を使う AI 基盤やスパコン。例：Private Cloud AI →p.613

継続稼働

止めにくい取引処理の基盤。
例：NonStop →p.615

HPC とは：たくさんの計算機で大きな計算をする

HPC（High Performance Computing、高性能計算）は、多数の計算機や **GPU** を高速な **ネットワーク** でつなぎ、1 台では何日もかかる大規模な計算を、分担して短時間で行うことです。スーパーコンピューターが代表例です。AI の **学習** も、多数の GPU を連携させる点で近い技術を使います（p.515）。

⚖ 業務用サーバーとの違い

普通の業務用サーバーは、多数の利用者からの小さな依頼を次々に処理します。HPC は、1 つの大きな計算を多数の計算機で分け合って、速く終わらせることを重視します。

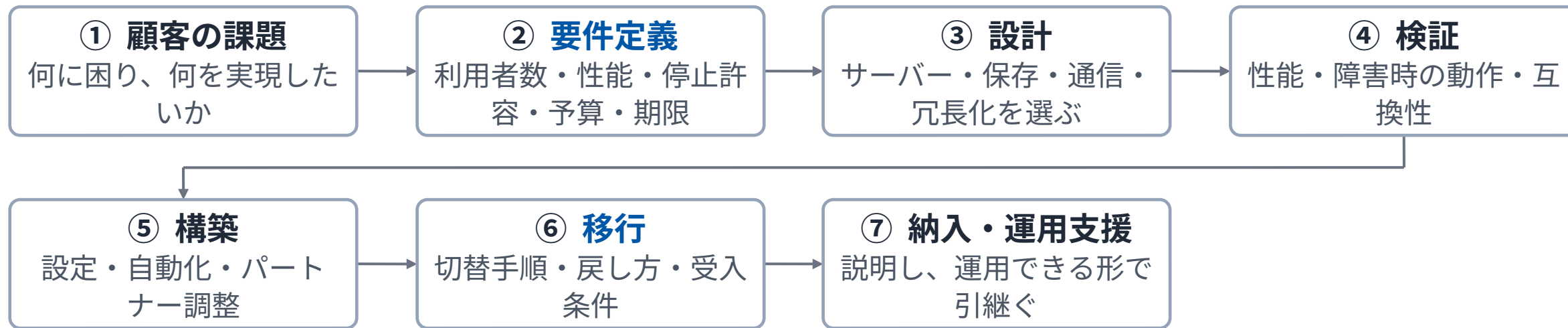
i 通信分野の補足

Juniper（Juniper Networks、ネットワーク機器の会社）は、2025 年 7 月に HPE による買収完了が発表されました（企業公表）。配属は、買収の事実だけから推測しません（p.617）。

HPE の SE の仕事は、課題の確認から運用支援までの流れ

各工程で設計書や試験結果などの成果物を残し、「なぜこの構成で要件を満たせるか」を説明します。

灰色の矢印は工程の順番です。検証の結果で設計に戻るなど、前の工程へ戻ることもあります。



i 補足

HPE の公開 SE 募集（S03）は、サーバー・ストレージ・ネットワーク・スーパーコンピューター・クラウドを用いた設計・構築・納入を説明しています（工程は p.540）。

工程ごとに残す成果物

工程	残す成果物の例
① 顧客の課題	— （次の要件定義で文書にまとめる）
② 要件定義	要件一覧 ・ 前提と制約
③ 設計	構成図 ・ 設計書
④ 検証	検証計画 ・ 結果
⑤ 構築	設定値 ・ 手順 ・ 変更記録
⑥ 移行	試験結果 ・ 切戻し計画
⑦ 納入 ・ 運用支援	運用設計書 ・ 完了記録

i 補足

右の列は元資料の「残す成果物の例」を工程順に対応させたもの。SE の 1 日は打合せ ・ 設計 ・ 設定 ・ 確認 ・ 報告の組み合わせで、割合は部署 ・ 顧客 ・ 工程で変わります。

顧客の業界と、担当する技術分野は別の軸

「金融担当」は業界、「サーバー担当」は技術の分野。組み合わせは配属で決まります。

業界\技術	サーバー・仮想化	ネットワーク	クラウド	セキュリティ	AI ・ HPC 基盤
金融	取引系基盤の更改	拠点間の冗長な回線	社内システムの移行	監査・権限管理	社内文書の検索 AI
通信	大規模な仮想化基盤	データセンター網	コンテナ基盤	不正通信の対策	網の分析・予測
製造	工場・本社の基盤	工場の無線・拠点網	部門システムの移行	工場設備の保護	設計の計算・画像検査

⚠ 注意

典型的な案件の例です。金融担当でもクラウドや AI に関わり、通信担当の全員が**基地局**を設計するわけではありません。

前のページの表の言葉

言葉	やさしい意味
更改	古い仕組みを新しく入れ替えること
拠点	本社・支店・工場・店舗などの場所
基盤	アプリを動かすためのサーバー・保存・通信などの土台（IT インフラ）
冗長な回線	1 本が切れても通信できるよう、複数用意した回線（冗長化）
監査	決まりどおりに運用・管理されているかを、記録で確かめること

同じ「金融」の担当でも、技術の分野が違えば、毎日扱う機器や設計の中身は大きく違います。逆に、同じ「ネットワーク」の担当でも、顧客の業界によって求められる止めにくさが違います。

技術分野の全体（8 分野）と、学んだページ

技術分野（元資料の 8 分野）	中身（ひとことで）	この資料のページ
ネットワーク	機器をつなぎ、データを運ぶ	p.191 ・ p.269
サーバー・仮想化	計算機と、それを分けて使う技術	p.390 ・ p.413
ストレージ・バックアップ	データの保存と守り方	p.350 ・ p.379
パブリッククラウド	事業者のクラウドを使う設計	p.443 ・ p.449
セキュリティ	認証・権限・通信の制御など	p.462 ・ p.475
SRE ・ 運用自動化	監視と自動化で安定稼働を保つ	p.559 ・ p.670
AI ・ HPC 基盤	GPU を使う計算の基盤	p.512 ・ p.590
設備（電源・冷却）	データセンターの電力と冷却	p.399 ・ p.402

SRE（Site Reliability Engineering）は、サービスを安定して動かし続けることを、監視と自動化を中心に技術的に進める考え方です。

金融・通信・製造で、求められやすい条件が違う

業界ごとに「止めにくさ」「規模」「現場の制約」が違い、経験しやすい技術にも差が出ます。

業界	典型的な要求	経験しやすい技術	将来つながりやすい方向
金融	止めにくい業務、厳しい変更管理、監査	高可用性、災害対策、大規模移行、権限管理	金融基盤、企業クラウド、アーキテクト、大規模 PM
通信	大規模で 24 時間の通信、多数の機器	大規模ネットワーク、分散システム、仮想化・コンテナ、自動化	ネットワーク、クラウド、SRE、技術リード
製造	工場や拠点の停止・安全、長い機器寿命	IT と OT（工場設備の制御）、エッジ、産業データ、HPC	産業 AI、エッジ、OT セキュリティ、アーキテクト

OT は次のページ、表のほかの言葉はその次のページで説明します。

OT とは：工場の設備を動かす技術

OT（おーていー）

分類：考え方

略語：OT = Operational Technology 日本語：運用技術・制御技術

何か	工場の生産設備や制御装置（機械を決まった動きで動かす装置）を動かし、監視するための技術。
なぜ必要か	生産ラインを正しく、安全に動かし続けるため。止まると生産そのものが止まる。
どこにあるか	工場の現場、発電所や水道などの設備の中。
身近な例	生産ラインの制御装置、設備の温度や振動を測る装置
似た言葉との違い	IT（事務や業務の情報を扱う技術）より、機器を長く使い、止めにくく、更新の頻度が低い。IT の網とつながるときは安全の配慮が要る。

業界の表に出てくる、そのほかの言葉

言葉	やさしい意味	ページ
高可用性	可用性（使える状態を保つ性質）が高いこと	p.552
災害対策	災害時にも業務を続ける・戻す備え（DR）	p.556
分散システム	多数の機械で分担して動かす仕組み	p.433
エッジ	データが生まれる場所の近くで処理すること	p.447
アーキテクト	システム全体の構成を決める役割の技術者	—
技術リード	チームの技術面の判断をまとめる役割	—

⚠ 注意

業界の表は、典型的な案件から得やすい経験の例です。HPE の業界別配属の内容を示すものではなく、配属や働き方を保証するものでもありません。

HPE の製品地図：何をする製品か

製品名の暗記ではなく、「何をする製品か」を押さえます。

分野	製品の例	何をするか	詳しく
利用・管理	GreenLake ／ Morpheus ／ VM Essentials	ハイブリッド環境・仮想化の管理	p.611
AI 基盤	Private Cloud AI	企業が AI を導入・運用する統合基盤	p.613
継続稼働	NonStop	止めにくい取引処理の基盤	p.615
計算	ProLiant （ iLO で管理）	企業向けサーバー	p.601
保存	Alletra など	企業向けストレージ	p.606
通信	Aruba ／ Juniper	LAN ・ 無線 ・ DC ・ 広域ネットワーク	p.608

下の行ほど物理的な機器、上ほど利用・管理のソフトです。ハイブリッド環境は p.447 。

製品と一緒に学ぶ基礎技術

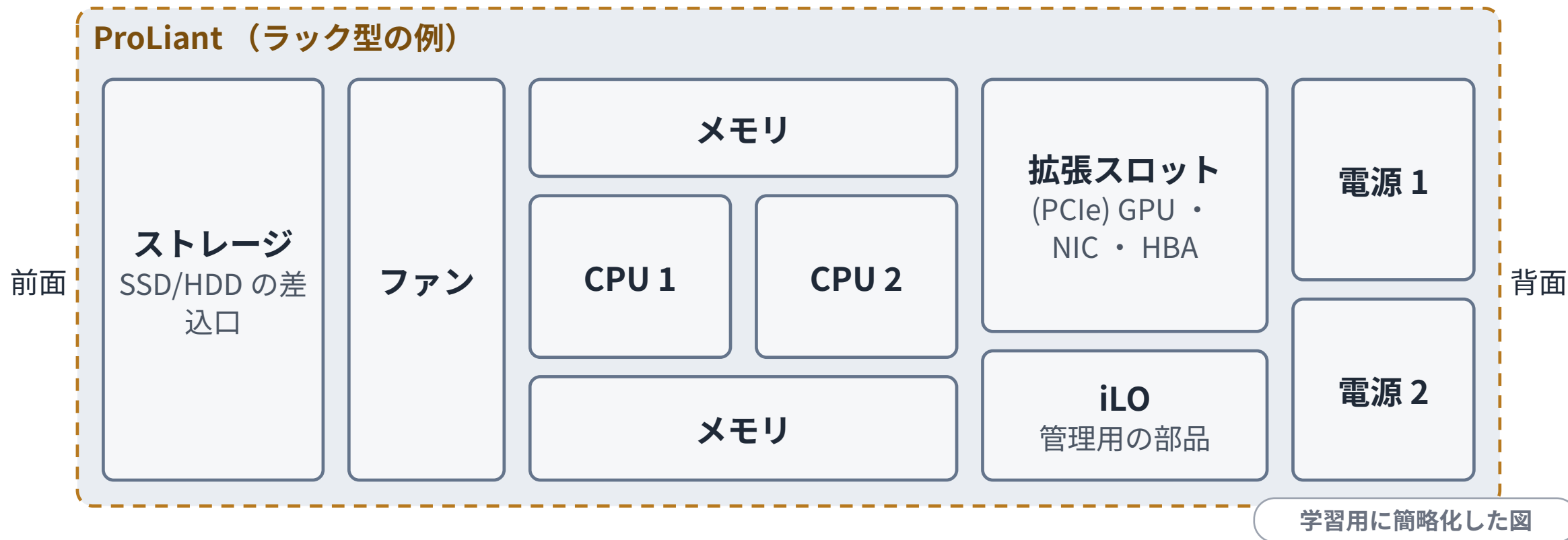
製品の説明を読むときは、右の列の基礎のページを思い出すと理解しやすくなります。

製品	一緒に学ぶ基礎技術（この資料のページ）
GreenLake ／ Morpheus ／ VM Essentials	クラウド p.443 ・ VM p.416 ・ 自動化 p.670 ・ 契約（利用の条件）
Private Cloud AI	GPU p.89 ・ 保存 p.350 ・ 通信 p.515 ・ 推論 p.494 ・ 権限 p.462
NonStop	可用性 p.552 ・ 障害の分離 p.615 ・ トランザクション p.174
ProLiant	CPU p.46 ・ メモリ p.44 ・ PCIe p.87 ・ 電源 p.399 ・ OS p.106
Alletra など	ブロック・ファイル p.350 ・ 性能 p.95 ・ 冗長化 p.366
Aruba ／ Juniper	VLAN p.609 ・ 経路 p.256 ・ 冗長化 p.552 ・ 監視 p.559

ProLiant ： HPE の企業向けサーバー

ProLiant （プロライアント）は HPE の企業向けサーバーの製品群です。

ラック型を上から見た配置の模式図。左が前面、右が背面。部品の数・位置は機種と世代で異なります。



ProLiant の構成表は、部品ごとに読む

構成表の項目	読むポイント	基礎のページ
CPU	ソケット数（ CPU の数） ・ コア数 ・ 世代	p.46 ・ p.73
メモリ	容量 ・ 枚数 ・ 対応する規格	p.44 ・ p.80
ストレージ	SSD/HDD の種類 ・ 台数 ・ RAID の構成	p.82 ・ p.366
拡張スロット（ PCIe ）	GPU ・ NIC ・ HBA を何枚載せられるか	p.87 ・ p.390
ネットワーク	NIC の速度 ・ ポート数（差込口の数）	p.390
電源 ・ 冷却	電源ユニットの二重化 ・ 容量 ・ 冷却方式	p.399 ・ p.402
管理	iLO による遠隔管理	p.603

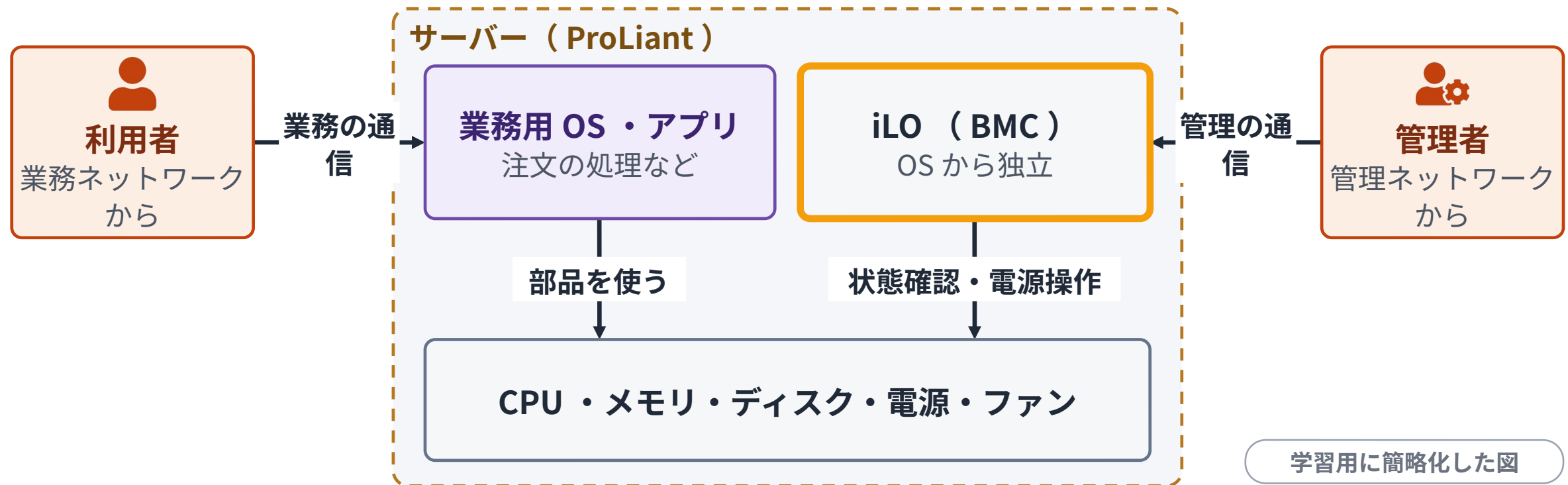
⚠ 注意

同じ製品名でも、世代 ・ 型番 ・ 構成で性能や対応部品が変わります。仕様は、型番 ・ 世代 ・ 根拠の資料をそろえて述べます。

iLO : 業務の通信とは別の経路でサーバーを管理する

iLO (アイロー、Integrated Lights-Out) は、HPE の対応サーバーに入った管理機能 (BMC) です。業務用 OS が止まっても、電源操作・状態確認・遠隔操作ができます。

関係図。破線の枠がサーバー本体、黄色の太枠が iLO 。黒い矢印は依頼や指示の移動です。



iLO でできることと、その前提

管理の通信を業務の通信と分けておくと、OS が止まっても、離れた場所から原因を調べ、復旧の操作ができます。

できること	前提 ・ 注意
電源のオン ・ オフ、再起動	サーバーに給電されていること（完全な無給電では操作できない）
温度 ・ ファン ・ 部品の状態とログの確認	管理ネットワークに到達できること
遠隔コンソール（画面を遠くから操作）	誰が接続できるかを設計で絞る（業務の通信と分ける）
Redfish API と Python で状態を自動取得	認証情報の管理 ・ 権限の最小化

Redfish はサーバー管理用の API の標準的な決まりで、API は次のページで説明します。Python はプログラミング言語の 1 つです（第 16 章）。iLO は強い操作ができるので、[最小権限](#)と[認証](#)（p.462）が大切です。

API とは：プログラムが使う窓口

API（えーぴーあい）

分類：仕組み

略語：API = Application Programming Interface

何か	プログラムが、別のソフトやサービスの機能呼び出すための、決まった窓口と呼び出し方。
なぜ必要か	画面を操作しなくても、プログラムで同じ操作を正確に繰り返せる（例：100 台の温度を自動で集める）。
どこにあるか	機器の管理機能（iLO の Redfish API など）、クラウドや AI のサービスが公開している。
受け取る→する→返す	「サーバーの状態を教えて」という要求を受け取り → iLO が調べ → 電源や温度のデータを返す（説明用の例）。
似た言葉との違い	画面は人が使う窓口、API はプログラムが使う窓口。インターフェースの一種。

Alletra などのストレージ：何を、どうつなぎ、どう守るか

企業向けストレージは、業務データ・VM のディスク・バックアップなどを保存します。対応する方式は機種で異なり、一括りにしません。

何を保存するか

DB ・ **VM** のディスク（ブロック）、共有ファイル（ファイル）、バックアップや大量データ（オブジェクト）など → p.350

性能

IOPS ・ 転送速度 ・ 応答時間。DB や VM の数、読み書きの割合で必要量が変わる → p.95

どうつなぐか

SAN （ **Fibre Channel** ・ **iSCSI** ）、ネットワーク上のファイル共有、NVMe over Fabrics など。機種による → p.357

守り方

RAID などの冗長化、**スナップショット**、複製、別装置への**バックアップ**を組み合わせる → p.379

ストレージの説明に出てくる言葉

NVMe over Fabrics は、SSD を速く読み書きするための手順（[NVMe](#)）を、ネットワーク越しに使う方式です。サーバーの外にあるストレージの中の SSD を、手元の SSD に近い速さで使うことを目指します。

ポイント

企業向けストレージを選ぶときは、「何を保存するか」で方式（ブロック・ファイル・オブジェクト）が決まり、「どれだけ速く、どれだけ安全に」で性能と守り方が決まります。製品名より先に、この 4 つの問いに答えられるかを確かめます。

Aruba ・ Juniper ： 社内 LAN ・ 無線 ・ DC ・ 広域の網

HPE のネットワーク製品は、オフィスの LAN からデータセンターや拠点間の網まで扱います。HPE は 2025 年 7 月に Juniper Networks の買収完了を発表しました。

オフィス LAN ・ 無線

スイッチ・アクセス
ポイント（AP）・
認証。社員の端末を
安全に接続する。

データセン ター網

サーバー・保存装置
をつなぐ、高速で冗
長な網。

拠点間・広域 （WAN）

本社・工場・店舗・
クラウドをつなぐ。

管理・監視

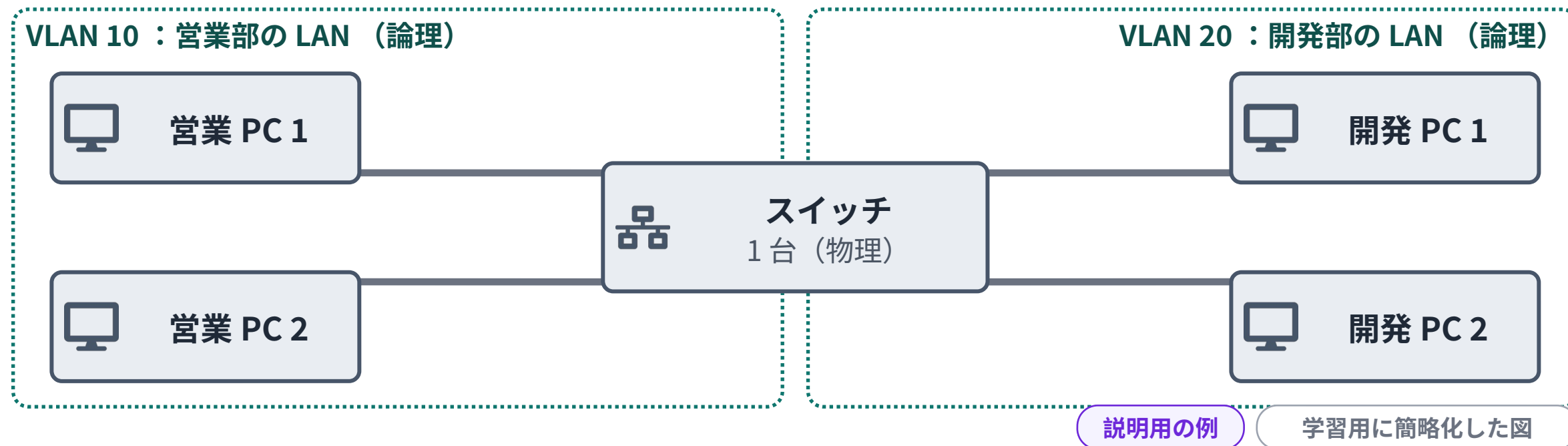
設定の一括管理、状
態の見える化、異常
の検知。

どれも第 6 章の家庭の網と同じ考え方（スイッチ・アクセスポイント・ルーター）でできています。違いは、台数が多く、止めにくく、まとめて管理する必要がある点です。

VLAN とは：1 つの網を、設定で複数の LAN に分ける

VLAN（Virtual LAN）は、1 台のスイッチという物理的な網を、設定で論理的に複数の LAN に分ける仕組みです。部署ごとに網を分けたいとき、スイッチを部署の数だけ買わずに済みます。

灰色の太線はケーブル、点線の枠は設定で作った論理的な LAN です。枠が違う PC どうしは直接は通信できず、ルーターなどを経由します。



ネットワーク製品と一緒に学ぶ基礎

一緒に学ぶ基礎	意味	基礎のページ
VLAN	1 つの物理的な網を、論理的に複数の LAN に分ける	p.609
経路制御	どの網へ渡すかを決める	p.256
冗長化	機器や回線を二重にして止まりにくくする	p.552
監視	通信量 ・ エラー ・ 機器の状態を観察する	p.559

 注意

製品の正式名称 ・ 区分は変わることがあります。買収の事実だけから、新卒の配属先や担当製品を推測しません。

GreenLake ・ Morpheus ・ VM Essentials の役割

GreenLake は利用 ・ 管理の基盤、Morpheus は資源を管理するソフト、VM Essentials は仮想化の製品です。

製品	役割	一緒に学ぶ基礎
HPE GreenLake	オンプレミスとクラウドなどの資源を利用 ・ 管理する基盤。従量的な利用モデル（使った量に応じた支払い）も	オンプレミスとクラウド（ p.447 ）、運用、契約
HPE Morpheus	VM ・ クラウドの資源を 1 つの画面で管理し、作成を自動化	VM （ p.416 ）、セルフサービス、自動化（ p.454 ）
HPE VM Essentials	仮想化の製品（ VM を動かし管理する）	ハイパーバイザー（ p.417 ）、VM

「セルフサービス」は、利用者が管理者に頼まずに、画面や API から自分で VM などを作れることです。

管理・仮想化・機器は層になっている

利用者の画面・API

利用者が VM を頼む・状態を見る窓口

GreenLake / Morpheus

複数の場所の資源をまとめて管理し、作成を自動化する

VM Essentials などの仮想化

1 台の物理サーバーの上で複数の VM を動かす（p.413）

サーバー・保存・通信

ProLiant ・ Alletra ・ Aruba などの物理的な機器

層の図。上ほど利用者に近いソフト、下ほど物理的な機器で、上の層は下の層の機能を使って動きます。

⚠ 注意

提供時期・機能は製品の版や契約で変わります。「発表済み」「利用可能」「将来予定」を区別して読みます。

Private Cloud AI ： AI 基盤を組み合わせて提供

HPE Private Cloud AI は、企業が自社の管理下で AI を導入・運用するための統合基盤です。NVIDIA との協業による製品です。

構成要素の層の概念図。上ほど AI を使う側、下ほど物理的な土台です（線はありません）。

AI アプリ・モデルの提供（推論・RAG など）

AI 用ソフト・管理（モデルの配備・監視・権限）



GPU サーバー



ストレージ



ネットワーク

電力・冷却・設置場所（顧客のデータセンターなど）

学習用に簡略化した図

Private Cloud AI ：製品と一般概念を分けて読む

製品として

GPU ・ 保存 ・ 通信 ・ ソフト ・ 管理を組み合わせ提供
する。 NVIDIA との協業
（ 2026 年 3 月の発表あり ）。

一般概念として

AI 基盤は計算 ・ メモリ ・ 保存 ・ 通信 ・ 電力 ・ 冷却 ・ 運用
で成り立つ （ p.512 ）。

SE が確認すること

用途 ・ データの場所と権限 ・
同時に使う人数 ・ 応答時間 ・
電力と冷却 （ p.530 ）。

「自社の管理下」とは、データや機器を、顧客が自分で管理する場所（自社のデータセンターなど）に置くという意味です。

注意

宣伝の性能値や効果は、条件付きの値として扱います。導入の前提 ・ 試験条件を確かめます。

NonStop : 障害を分離し、取引処理を続ける基盤

HPE NonStop は、金融取引など止めにくい業務のための基盤です。部品の故障を分離し、処理を続ける設計をとります。「絶対に停止しない」という意味ではありません。

高可用性

可用性（使える状態を保つ性質）が高いこと。予備の部品・経路を用意し、**障害**時に切り替える（p.552）。

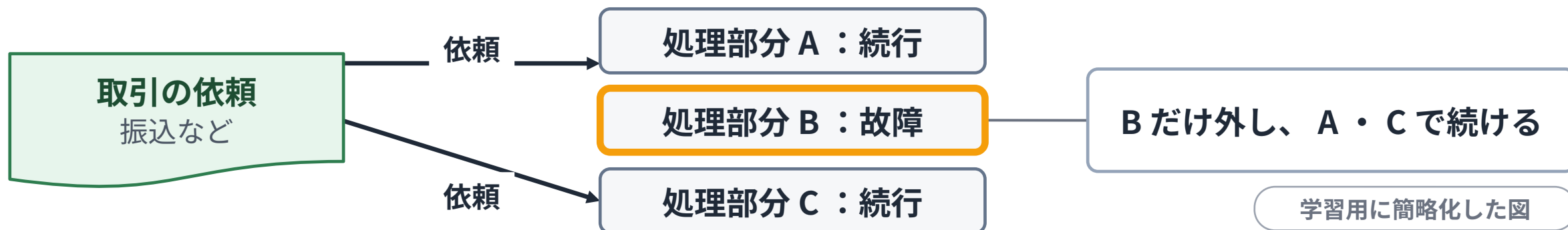
障害の分離

1 つの部品の故障が全体に広がらないように区切る。故障した部分を外し、残りで処理を続ける。

トランザクション

振込の引き落としと入金のように、まとまった処理を途中で中途半端にしない（p.174）。

障害の分離の一般的な考え方（NonStop の実際の内部構造ではない）。矢印は処理の依頼。



学習用に簡略化した図

NonStop についてのよくある誤解

よくある誤解	正しい理解
絶対に止まらない	止まりにくく設計されているが、計画停止 ・ 運用ミス ・ 災害などへの備えは別に必要
冗長化 = バックアップ	冗長化は止まらないための備え、バックアップは戻すための備え（ p.379 ）

📖 具体例

冗長化した 2 台に、誤って同じ間違ったデータを書き込むと、2 台とも同じ間違いを持ちます。止まらないことと、間違う前の状態に戻せることは別の備えです。

製品情報を読むときの注意

- **時期を区別する**：製品ごとに「発表済み」「利用可能」「将来予定」の時期が違います。発表と、今使えることは別です。
- **名称・区分は変わる**：正式名称や分類は、買収や再編で変わることがあります。
- **配属は推測しない**：担当製品は、製品の存在や買収の事実（例：Juniper）だけからは推測できません。
- **性能値は条件つき**：宣伝の性能値は、導入の前提と試験条件を確かめてから使います。
- **型番・世代・根拠をそろえる**：同じ製品名でも、世代・型番・構成で仕様が変わります。

第 14 章のまとめ

✓ ここまでで説明できること

- ・メーカー・ Sler ・クラウド事業者・通信 /DC 事業者が顧客のシステムにどう関わるか。1 社が複数の役割を持つこと。
- ・ HPE の SE の仕事の流れ（課題→要件定義→設計→検証→構築→移行→納入・運用支援）と成果物。
- ・ HPE の製品を、計算・保存・通信・管理・ AI ・継続稼働に分けること。iLO の役割と前提。
- ・製品（ProLiant ・ NonStop など）と、一般的な技術の考え方の区別。

→ 次へのつながり

第 15 章では、働き方と給与の情報を、出所と条件に分けて読む方法を学びます。

確認の問い（１）

自分の言葉で答えてから、次のページの解説を読みましょう。

- ① メーカーと Sler の違いを、サンプル商店の例で説明してください。
- ② iLO が業務と別の経路を使う理由と、操作できない場合は？

確認の問い（１）：解説

前のページの問いへの答えと、考え方です。

✓ 問 1 メーカーと Sler の違いを、サンプル商店の例で説明してください。

メーカーはサーバーやソフトなどの製品を開発し、提供・保守します。Sler は、メーカーの製品・クラウド・回線などを組み合わせて、サンプル商店のシステムを設計・構築します。1 社が両方の役割を持つこともあります。 p.584

✓ 問 2 iLO が業務と別の経路を使う理由と、操作できない場合は？

業務用 OS が止まっても、管理ネットワークから電源操作や状態確認をして復旧するためです。強い操作ができるので、接続できる人を絞る意味もあります。給電されていないときや、管理ネットワークに届かないときは操作できません。 p.604

確認の問い（２）

自分の言葉で答えてから、次のページの解説を読みましょう。

- ① 「NonStop は絶対止まらないのでバックアップ不要」の誤りは？
- ② 「Juniper 買収で新卒はネットワーク担当になりやすい」は適切？

確認の問い（２）：解説

前のページの問いへの答えと、考え方です。

✓ 問 1 「NonStop は絶対止まらないのでバックアップ不要」の誤りは？

NonStop は止まりにくく設計されていますが、計画停止・運用ミス・災害への備えは別に必要です。また冗長化は止まらないための備え、バックアップは戻すための備えで、役割が違います。 p.616

✓ 問 2 「Juniper 買収で新卒はネットワーク担当になりやすい」は適切？

適切ではありません。配属や担当製品は、買収や製品の存在といった事実だけでは推測できません。配属は募集要項や面談で確認します。 p.617

第 15 章

働き方・給与・企業比較の読み方

この章で学ぶこと

働き方を決める条件、給与の用語、HPE の給与情報の 4 種類、投稿された年収の読み方、企業比較の表の読み方を学びます。

数字は必ず、年度・学歴・対象職種・時点・確認状態・情報の区分とセットで読みます。

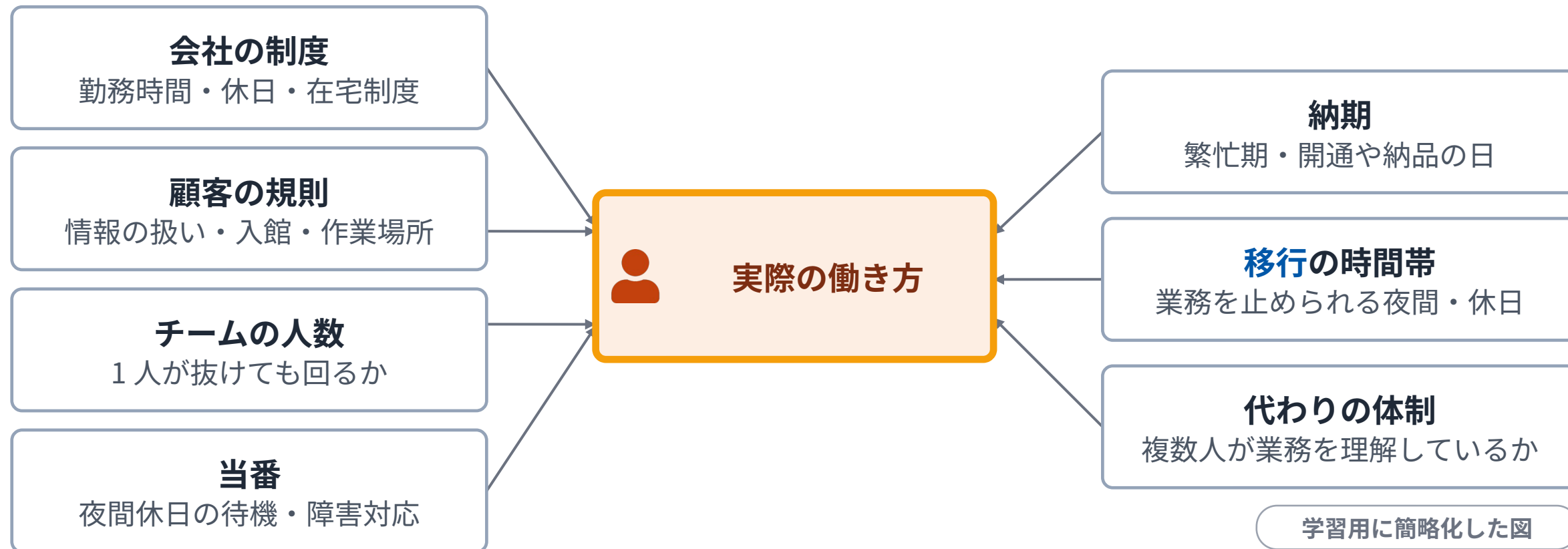
→ 前の章とのつながり

第 14 章で会社の立場と HPE の仕事・製品を見ました。この章では、働き方・給与と、他社との比べ方を扱います。

働き方は、制度だけでなく顧客・人数・当番で決まる

休みやすさや在宅勤務の可否は、業界名より、次のような条件の組み合わせで決まります。

灰色の矢印は「この条件が実際の働き方に影響する」という関係を表します。



「当番」「移行の時間帯」とは何か

システムは夜も休日も動いているので、**障害**が起きたときにすぐ対応できるように、交代で連絡を待つ担当を決めることがあります。これが「当番」（待機）です。

また、古いシステムから新しいシステムへの**移行**や設定の変更は、利用者が少ない夜間・休日に行うことが多くあります（p.565）。そのため、担当する顧客や工程によって、働く時間帯が変わります。

ポイント

チームの人数が少なく、業務を理解している人が 1 人しかいないと、その人は休みにくくなります。「代わりの体制」があるかは、制度と同じくらい働き方に影響します。

HPE の公開制度と、OB 訪問で聞いた個別例

HPE の公開 SE 募集の制度（企業公表）

実働 7.5 時間、コアタイムなし、年間休日 130 日、会社掲載の残業は月 17 時間（2024 年度）。
出典：マイナビ 2027 の HPE SE 募集（S03、依頼者が 2026 年 10 月 3 日に確認）。構築・納入時の忙しさや顧客先対応は配属先で確認します。

OB 訪問で聞いた個別例（個別の体験談）

金融（あるチーム）：人数が多く代わりが見つかり休みやすいが、機密性でリモートに制限。
通信（あるチーム）：リモートは可能だが、少人数で代わりが利かず休みにくい。

注意

個別例は貴重ですが、「金融は休める、通信は休めない」と一般化しません。本人の部署・契約・制度は未確認です。

制度の言葉：実働・コアタイム・OB 訪問

言葉	意味
実働	休憩時間を除いた、実際に働く時間
コアタイム	働く時間を各自が調整できる制度（フレックスタイム制など）で、全員が必ず勤務する時間帯。「なし」はその時間帯を設けていないこと
年間休日	1 年のうち、会社が定めた休みの日数（土日・祝日・年末年始など）
会社掲載の残業	会社が採用情報に載せた、1 か月あたりの平均的な残業時間
OB 訪問	その会社で働く卒業生などに会って、仕事の話を聞くこと

会社掲載の残業時間は会社全体の平均などで、部署や時期で変わります。

給与は、基本給・手当・賞与・総年収を分けて読む（1/2）

月給 33 万円と月給 26 万円をそのまま比べると、住宅手当・固定残業代・賞与月数の違いを見落とします（元資料の例）。

要素	意味	比較するときの注意
基本給	毎月の基本となる賃金	賞与の計算基準になることが多い
固定残業代	一定時間分の残業代を先に月給へ含める	含まれる時間数と、超えた分の扱いを確認
手当	住宅・通勤・家族など条件付きの支給	一律か、条件付きか。基本給とは別物
固定賞与	基本給の○か月分など、あらかじめ決めた賞与	初年度は在籍期間で調整されることがある

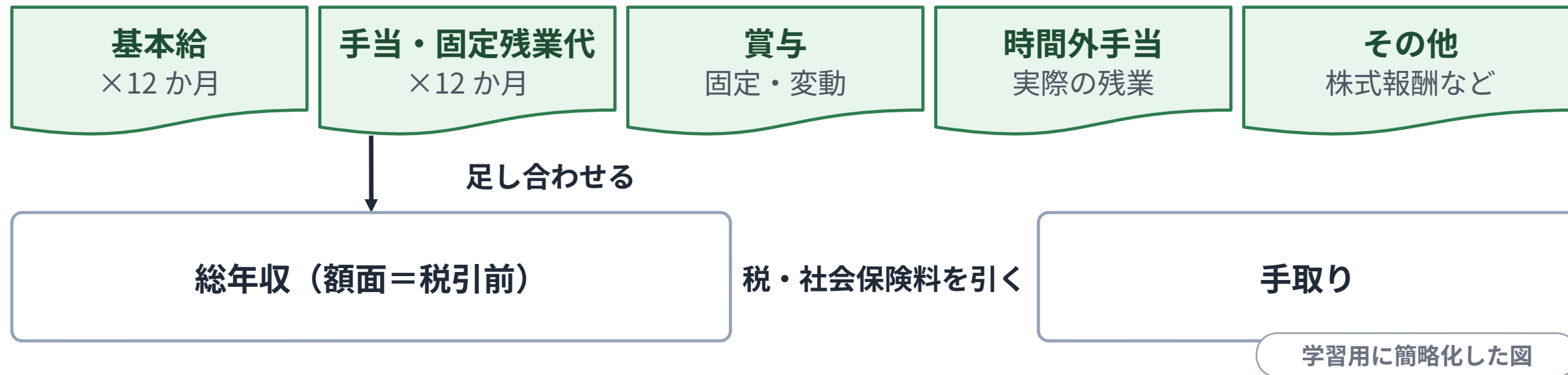
給与は、基本給・手当・賞与・総年収を分けて読む（2/2）

月給 33 万円と月給 26 万円をそのまま比べると、住宅手当・固定残業代・賞与月数の違いを見落とします（元資料の例）。

要素	意味	比較するときの注意
変動賞与・業績報酬	会社や個人の業績で変わる支給	保証額ではない。過去実績は将来を約束しない
時間外手当	実際に働いた残業時間に応じた支給	部署・時期で大きく変わる
株式報酬	株式やその権利で受け取る報酬	株価で価値が変わる。外資で比重が大きいことも

総年収・額面・手取りの関係

箱の長さは説明用で、実際の比率ではありません。矢印は「足し合わせる」「差し引く」という計算の順です。



- **総年収**は、1年間に受け取る給与の合計です。月給だけでは分かりません。
- この章で扱う金額はすべて税引前（額面）です。手取りは所得税・住民税・社会保険料を差し引いた後の金額で、家族構成などで変わります。

給与の数字を比べる時の 3 つの確認

- ① **中身を分ける**：月給に固定残業代や手当が含まれていないか。月給が高く見えても、基本給は低いことがあります。
- ② **期間をそろえる**：月の額か、年の額か。年なら、賞与（何か月分か）や残業代を含むかどうか。
- ③ **条件をそろえる**：年度・学歴（大卒・院卒）・対象職種・時点が同じ数字どうしで比べます。

具体例

「月給 33 万円」と「月給 26 万円」の会社でも、前者に固定残業代が含まれ、後者に住宅手当や多い賞与月数があれば、年間の総額の差は月給の差と同じになりません（元資料の例を使った説明）。

HPE の給与情報は、出所と条件の違う 4 種類がある

同じ「HPE の給与」でも、意味が違う 4 種類の数字があります。1 つの数字に混ぜません。

① 公開の実績

企業公表。採用情報に載った、2025 年 4 月の実際の基本月額 → p.633

② 原資料の提示条件

原資料 A が「本人に提示された 27 卒条件」として扱った額。今回は未確認 → p.634

③ 会社のモデル例

条件を置いた年収の例（原資料 C の記載） → p.635

④ 投稿の平均

社員・元社員の投稿を集計した平均。企業の公式値ではない → p.635

① 公開の実績（企業公表）

マイナビ 2027 の HPE SE 募集（S03：給与は 2025 年 4 月実績、最終更新 2026 年 8 月 6 日。依頼者が 2026 年 10 月 3 日に確認）。

学歴	基本月額	条件
大卒	256,300 円	2025 年 4 月実績
院卒	274,300 円	2025 年 4 月実績

読み方

固定賞与：基本月額の 6 か月分。初年度の夏季は在籍期間で調整されます。

「実績」は、その時点で実際に支払われた額です。2027 年入社の確定額ではありません。

「基本月額」は毎月の基本給のことです。

② 原資料の提示条件：約 473 万円の計算

$$\begin{array}{|c|} \hline \text{27 卒 基本月額} \\ \hline 262,720 \text{ 円} \\ \hline \end{array} \times \begin{array}{|c|} \hline \text{18 か月} \\ \hline 12 \text{ か月} + \text{固定賞与 6 か月} \\ \hline \end{array} = \begin{array}{|c|} \hline \text{4,728,960 円} \\ \hline \text{≒ 約 473 万円} \\ \hline \end{array}$$

原資料 A の 4,728,600 円とは 360 円の差があります（上の計算が正しい値）。

⚠ 注意

原資料 A は 262,720 円を「本人に提示された 27 卒条件」として扱っています。今回の公開ページで確認した値ではなく、正式な通知も未確認です。

約 473 万円は、満額の固定賞与を仮定した年換算です。初年度の実際の支給額でも、残業代・変動報酬を含む総年収でもありません。

③ 会社のモデル例と ④ 投稿の平均

③ 会社のモデル例

院卒 2 年目以降の例：約 530 万円。

賞与・残業手当・業績報酬などを含む、条件付きの例です（原資料 C の記載）。

④ 投稿の平均（公開投稿の集計）

OpenMoney 「IT エンジニア」：平均 759 万円・中央値 717 万円、平均年齢 31.1 歳・78 人（原資料 A の集計、2026 年 10 月 3 日時点、S66）。

OpenWork（会社全体）：801 万円／820 万円。取得した時点が違う 2 つの値です（次のページ）。

OpenMoney・OpenWork は、社員・元社員が投稿した情報を集計して公開するサイトです。企業の公式値ではありません。

OpenWork の 801 万円と 820 万円は、取得時点が違う

同じサイトの「会社全体の平均」でも、いつ取得したかで値が変わります（元資料 p.15 のノートの補足）。

値	取得した時点	一緒に示された条件
801 万円	原資料 A の取得時点	月間残業 29.4 時間
820 万円	原資料 C の取得時点	平均年齢 37 歳・正社員 453 人

⚠ 注意

取得時点が違うので、2 つを平均したり、「上がった」と読んだりしません。

どちらも会社全体（中途入社・様々な年齢と職種を含む）の値なので、新卒 SE の年収予測には使いません。

平均と中央値の違い（説明用の例）

平均は、全員の値を足して人数で割った値です。**中央値**は、値を小さい順に並べたときの真ん中の値です。少数の大きな値が混ざると、平均は引き上げられますが、中央値はあまり動きません。

架空の 5 人	A さん	B さん	C さん	D さん	E さん
年収（万円）	400	450	500	550	1,500

平均 = $(400 + 450 + 500 + 550 + 1,500) \div 5 = 680$ 万円。中央値 = 真ん中（3 番目）の 500 万

ポイント

投稿の平均が中央値より大きいときは、高い年収の投稿が平均を引き上げている可能性があります。OpenMoney の HPE（平均 759 万円・中央値 717 万円）でも、平均と中央値は違います。

投稿の平均を読むときの注意

- 投稿の平均には、中途入社・様々な年齢と職種が混ざります。新卒 SE の年収や「入社〇年目の平均」と読み替えません。
- 人数が少ない会社ほど、数人の投稿で平均が大きく動きます。
- 年齢帯ごとの平均は、勤続年数ごとの平均と同じではありません。
- 在籍している人の勤続年数は、新卒が辞めるまでの期間とは別のものです。

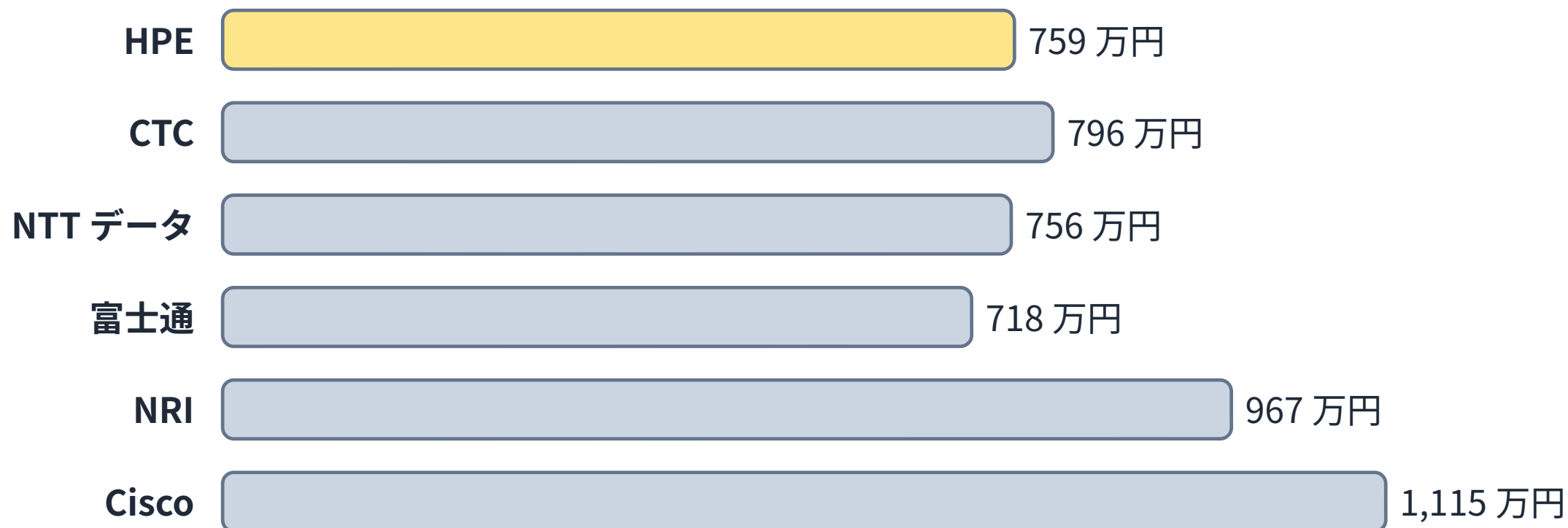
⚠ 注意

約 473 万円 (②) と、投稿の平均 759 万円 (④) は、出所も条件も違う数字です。差を「何年で上がる」と読むことはできません。

投稿年収の比較：代表 6 社の平均（OpenMoney）

OpenMoney 「IT エンジニア」投稿の平均。原資料 A の集計、確認日 2026 年 10 月 3 日（S66）。

横棒の長さは投稿の平均（万円）。年齢・役職・入社経路がそろわず、順位表としては読めません。



平均だけでなく、中央値・年齢・人数と見る

同じ 6 社の、中央値・平均年齢・人数です（OpenMoney 「IT エンジニア」、2026 年 10 月 3 日時点の原資料 A の集計）。

企業	平均	中央値	平均年齢	人数
HPE	759 万円	717 万円	31.1 歳	78 人
CTC	796 万円	780 万円	30.6 歳	219 人
NTT データ	756 万円	750 万円	29.5 歳	684 人
富士通	718 万円	680 万円	29.8 歳	669 人
NRI	967 万円	940 万円	30.1 歳	619 人
Cisco	1,115 万円	1,027 万円	34.1 歳	84 人

HPE（78 人）や Cisco（84 人）は人数が少なく、数人の投稿で平均が動きやすいことに注意します。全 24 社の表は付録にあります（p.695）。

企業比較の表を読む前に：列と言葉の意味

次の 3 ページの表は、原資料作成者による学習の提案としての整理で、企業の公式な分類ではありません。

列・言葉	意味
対象職種	会社全体ではなく、その職種で比べる（同じ会社でも職種で仕事が違う）
立場	メーカー・Sler・基盤サービスなど、第 14 章の立場（p.582）
インフラ寄り	アプリより、IT インフラ（サーバー・保存・通信など）の設計・構築・運用が中心
プリセールス	技術提案職。導入前に、顧客の課題に合う製品・構成を提案する
基盤系 SE	IT インフラ（基盤）を担当する部門・職種の SE

企業比較①：製品・基盤を持つ会社（1/2）

会社全体を「アプリだけ」「インフラだけ」と固定せず、職種・部門で変わる点を残します。

企業・対象職種	立場	仕事の傾向	経験しやすい技術
HPE ・ システムエンジニア	メーカー	強くインフラ寄り。顧客の基盤を設計・構築・納入	サーバー、ストレージ、ネットワーク、HPC、基盤設計
Dell ・ 設計／プリセールス※	メーカー	強くインフラ寄り。技術提案の比重が大きい	サーバー、ストレージ、仮想化、提案

企業比較①：製品・基盤を持つ会社（2/2）

会社全体を「アプリだけ」「インフラだけ」と固定せず、職種・部門で変わる点を残します。

企業・対象職種	立場	仕事の傾向	経験しやすい技術
Cisco ・ 技術職	メーカー	強くインフラ寄り。技術提案・顧客対応	ネットワーク、セキュリティ、DC ネットワーク
IBM ・ IT スペシャリスト	メーカー兼 SI	アプリと基盤の両方。配属による	ハイブリッドクラウド、大規模案件の設計
キンドリル・技術職	基盤サービス	強くインフラ寄り。運用・移行が多い	大規模基盤、クラウド移行、運用設計

※Dell は設計・プリセールス職を想定。テクニカルサポート職などとは別に比較します。23 社の一覧は付録（p.710）。

企業比較②：Sler・大手 IT の基盤系職種（1/2）

基盤系の職種・部門を選べるかが、インフラ経験の積みやすさに影響します。「全社では両方」＝アプリ開発と基盤の両方の部門がある意味です。

企業・対象職種	仕事の傾向	経験しやすい技術
CTC・エンジニア	インフラに強い。アプリもある	複数ベンダーの組合せ、クラウド、基盤全体の統合
NRI・テクニカルエンジニア	対象職種はインフラ寄り	金融等の大規模基盤、クラウド、上流設計
日鉄ソリューションズ （NSSOL）	両方。基盤部門は強い	大規模基盤、クラウド、金融・産業、設計／PM
NTT データ・基盤系 SE	全社では両方	大規模クラウド、金融・公共基盤、設計／PM

企業比較②：Sier・大手 IT の基盤系職種（2/2）

基盤系の職種・部門を選べるかが、インフラ経験の積みやすさに影響します。「全社では両方」＝アプリ開発と基盤の両方の部門がある意味です。

企業・対象職種	仕事の傾向	経験しやすい技術
日立製作所・IT 基盤系 SE	全社では両方	大規模基盤、クラウド、社会システム
NEC・基盤系 SE	全社では両方	クラウド、ネットワーク、セキュリティ、公共基盤
富士通・基盤系 SE	全社では両方	クラウド、HPC、基盤アーキテクチャ

企業比較③：通信・クラウド専門・業務システム系（1/2）

業務システム寄りの会社では、基盤部門の有無を確認します。

分野	企業・対象職種	経験しやすい技術・注意
通信・DC・自社サービス	IIJ・技術職、さくらインターネット・エンジニア	ネットワーク、クラウド、GPU 基盤。 24 時間運用・シフト勤務の有無を確認
通信キャリア	KDDI・ソフトバンクの基盤系	通信網、クラウド、DC、AI 関連基盤。 担当領域が広いので具体化が必要
クラウド専門	クラスメソッド・専門エンジニア	AWS、IaC（p.670）、自動化、クラウド設計。コードを書く経験も重視

企業比較③：通信・クラウド専門・業務システム系（2/2）

業務システム寄りの会社では、基盤部門の有無を確認します。

分野	企業・対象職種	経験しやすい技術・注意
総合コンサル・SI	アクセンチュア・SE	クラウド移行、設計、推進。コンサル職と SE 職を分けて比べる
業務システム寄り	SCSK、日立ソリューションズ、TISI、BIPROGY、電通総研	部門次第。基盤設計を担当できるかを確認

企業比較の表の言葉と、合併の注意

言葉	意味
ベンダー	製品を売る会社（メーカーや販売会社）。「複数ベンダーの組合せ」は、いろいろな会社の製品を組み合わせること
上流設計	工程の初め（要件定義・基本設計）を担うこと（ p.540 ）
公共基盤	国や自治体などのシステムの土台
基盤アーキテクチャ	基盤全体の構成の考え方・設計
シフト勤務	交代制で、日によって勤務する時間帯が変わる働き方

⚠ 注意

TIS とインテックは 2026 年 7 月 1 日に合併し TISI になりました（ S05 ）。合併前の口コミが、統合後の全員に同じ条件で当てはまる証拠にはなりません。

会社は、身につく経験と働き方の条件で比べる

比較の軸をそろえると、自分に合う会社と、確認すべき点が見えます。

比較の軸	確かめること
担当する技術	サーバー・通信・クラウド・AI 基盤のどれを、どの深さで扱うか
設計・検証の経験	若手が設計・検証・移行の一部を、責任を持って担当できるか
育成と代わりの体制	教える先輩、レビュー、休暇時の代わりがあるか
働き方	当番・顧客先勤務・移行の時間帯・在宅の可否
報酬	基本給・賞与・手当・残業の内訳と、昇格の仕組み

推定値（難易度・年収レンジ・内定確率）の扱い

⚠ 推定は実測統計ではない

就職難易度・2～20年目の年収レンジ・転職時の内定確率は、いずれも原資料作成者の想定です。統計的に検証された値ではありません。条件とともに付録で扱います（p.708）。

i この章に関係する、今回確認できていないこと

- ・ 27 卒の提示額 262,720 円の正式通知
- ・ 本人の配属・チーム別の制度

数字を使うときは、「企業公表」「原資料の提示条件」「公開投稿の集計」「個別の体験談」「原資料の想定」のどれかを、必ず一緒に書きます。

第 15 章のまとめ

✓ ここまでで説明できること

- ・ 働き方は、制度だけでなく顧客の規則・チームの人数・当番・移行の時間帯・代わりの体制で決まること。
- ・ 給与の数字を、基本給・手当・賞与・総年収、額面と手取りに分けて読むこと。
- ・ HPE の給与情報 4 種類（公開実績・原資料の提示条件・モデル例・投稿平均）を混ぜないこと。
- ・ 企業比較は、対象職種と比較の軸をそろえて読むこと。

→ 次へのつながり

第 16 章では、ここまでの技術を自分で確かめる学び方と、キャリアの考え方を扱います。

確認の問い（１）

自分の言葉で答えてから、次のページの解説を読みましょう。

- ① 月給 33 万円が月給 26 万円より年収が高いと言えない理由は？
- ② 約 473 万円は何を仮定した値で、何ではありませんか。

確認の問い（１）：解説

前のページの問いへの答えと、考え方です。

✓ 問 1 月給 33 万円が月給 26 万円より年収が高いと言えない理由は？

月給に固定残業代や手当が含まれているか、賞与が何か月分かが分からないからです。年間の総額（総年収）を、同じ条件（年度・学歴・職種）でそろえて比べる必要があります。 p.631

✓ 問 2 約 473 万円は何を仮定した値で、何ではありませんか。

27 卒の基本月額 262,720 円に、満額の固定賞与 6 か月分を加えた 18 か月分の年換算です。初年度の実際の支給額でも、残業代・変動報酬を含む総年収でもありません。262,720 円自体も今回は未確認です。 p.634

確認の問い（２）

自分の言葉で答えてから、次のページの解説を読みましょう。

- ① OpenWork の 801 万円と 820 万円を新卒の予測に使わない理由は？
- ② 平均と中央値の違いを、高い年収が混ざった例で説明すると？

確認の問い（２）：解説

前のページの問いへの答えと、考え方です。

✓ 問 1 OpenWork の 801 万円と 820 万円を新卒の予測に使わない理由は？

取得した時点が違う会社全体の平均で、中途入社や様々な年齢・職種が混ざっているからです。新卒 SE の年収や入社〇年目の平均とは別のものです。 p.636

✓ 問 2 平均と中央値の違いを、高い年収が混ざった例で説明すると？

400 ・ 450 ・ 500 ・ 550 ・ 1,500 万円の 5 人なら、平均は 680 万円、中央値は 500 万円です。少数の高い値は平均を引き上げますが、中央値はあまり動きません。 p.637

第 16 章

学習・実習・キャリア

この章で学ぶこと

学ぶ順番、自分の PC で通信を観察するコマンド、シェル・Python・SQL などの道具の違い、設定の自動化、安全な実習の進め方、資格、時期ごとの目標とキャリアの方向を学びます。

→ 前の章とのつながり

第 1 ～ 15 章で、技術の基礎と、会社・働き方の読み方を学びました。この章では、それを自分の手で確かめ、仕事につなげる方法を扱います。

学ぶ順番：前の段階が、次の段階の前提になる

機器と OS から始め、通信、Web・DB、仮想化、運用、自動化、コンテナ・AI 基盤へと広がります。

灰色の矢印は学ぶ順番です。前の段階を説明できると、次の段階が理解しやすくなります。



9つの段階と、この資料で学んだページ（1/2）

段階	学ぶこと	この資料のページ
1 機器と OS	CPU ・ メモリ ・ SSD ・ OS ・ プロセス	p.50 ・ p.82 ・ p.106 ・ p.110
2 IP と経路	IP ・ サブネット ・ ゲートウェイ ・ ARP	p.225 ・ p.234 ・ p.256
3 接続の流れ	DHCP ・ DNS ・ NAT ・ TCP/UDP ・ TLS	p.244 ・ p.247 ・ p.253 ・ p.258 ・ p.262
4 サーバーの操作	Linux ・ Windows のサービス ・ 権限 ・ ログ	p.106 ・ p.112 ・ p.559

9 つの段階と、この資料で学んだページ（2/2）

段階	学ぶこと	この資料のページ
5 Web ・ DB	要求を受けて応答を返す流れ・SQL	p.326 ・ p.160 ・ p.163
6 仮想化・クラウド	物理と仮想の資源の関係	p.413 ・ p.416 ・ p.443
7 監視・バックアップ・冗長化	故障を見つけ、復旧する	p.559 ・ p.379 ・ p.552
8 スクリプト・Git・自動化	変更を記録し、再現できる形に	p.664 ・ p.670
9 コンテナ・AI 基盤	多数のアプリ・GPU を効率よく管理	p.425 ・ p.433 ・ p.512

1 つずつ完璧にせず、行き来しながら理解する

1 つの段階を完璧にしてから次へ、ではなく、実習で行き来しながら理解を深めます。後の段階で分からないことが出たら、前の段階のページに戻ります。

📖 行き来する例

DNS（段階 3）の問い合わせを観察して「IP アドレスが返ってくる」の意味が分からなければ、IP アドレス（段階 2）に戻ります。

Web サーバー（段階 5）が応答しないときは、サービス（段階 4）が動いているか、ポート番号（段階 3）が合っているかを順に確かめます。

→ 次のつながり

次のページから、自分の PC で「見るだけ」の観察を始めます。そのために、まずコマンドとシェルを説明します。

シェルとは：文字の命令で OS を操作する窓口

シェル (しえる)

分類：ソフトウェア

何か	人が文字で打ち込んだ命令（コマンド）を受け取り、OS に実行させ、結果を文字で表示するソフトウェア。
なぜ必要か	画面のボタンを探さなくても、決まった操作を正確に、何度でも同じように行える。操作を記録して再現しやすい。
どこにあるか	OS の上で動くアプリの 1 つ。Windows のコマンドプロンプト・PowerShell、Linux の Bash（バッシュ）など。
受け取る→する→返す	「ipconfig /all」という文字を受け取り → OS に通信の設定を問い合わせ → IP アドレスなどの一覧を文字で表示する。
似た言葉との違い	OS の中心部分（カーネル）とは別。シェルは、人と OS の間の「窓口」にあたる（殻 = shell という名前の由来）。

通信の用語を、コマンドで「見るだけ」観察する

コマンド	見るもの	学んだページ
ipconfig /all	自分の IP ・ DNS ・ DHCP ・ ゲートウェイ	p.225 ・ p.244
route print	OS が使う経路の一覧	p.256
arp -a	IPv4 の、近くの機器の IP と MAC の対応	p.227
nslookup example.com	DNS で調べた IP アドレス	p.247
ping example.com	相手に届くか・往復時間（RTT）	p.295
tracert -d example.com	経路上の中継点から得られる応答	p.257
Test-NetConnection example.com -Port 443	TCP 443 番への接続（PowerShell）	p.258
netsh wlan show interfaces	Wi-Fi 接続の状態・リンク速度	p.218

ipconfig /all の表示を読む（説明用の例）

佐藤さんの家の PC（192.168.1.20）で実行した場合の表示の一部を、簡略化した説明用の例です。

```
C:\> ipconfig /all
DHCP 有効 ..... : はい
IPv4 アドレス .... : 192.168.1.20
サブネット マスク  : 255.255.255.0
デフォルト ゲートウェイ : 192.168.1.1
DNS サーバー ..... : 192.168.1.1
```

通信の設定をすべて表示して
DHCP で設定を受け取っている
この PC の IP アドレス
同じ LAN の範囲（/24）
LAN の外へ出るときの渡し先（ルーター）
名前を問い合わせる先（ここではルーター）

⚠ 注意

前のページの表について：443 番に接続できても、サービスの全機能が正常とは限りません。
Wi-Fi のリンク速度は、インターネットまでの速度とは別です。

4つの道具は、操作する対象が違う

最初は Linux のシェルと Python から始め、PowerShell や SQL は担当に合わせて加えます。

>_ Bash (シェル)

Linux を操作する。
決まった作業を**スクリプト**（命令を並べたファイル）にして自動実行。

>_ PowerShell

Windows ・
Microsoft 系の管理に使うシェル兼スクリプト言語。

⚙️ Python

汎用のプログラミング言語。API ・ログ解析 ・構成情報の取得に使う。

🗄️ SQL

データベースへの問い合わせ言語。DB の確認 ・集計に使う（ p.163 ）。

4 つの道具の分類・使い道・最初の目標

名前	分類	主な使い道	最初の目標
Bash ／シェル	コマンド処理・スクリプト	Linux の操作・定期的な自動実行	変数・条件分岐・繰り返し・終了コード
PowerShell	シェル・スクリプト言語	Windows ・ Microsoft 系の管理	オブジェクト・パイプライン・一覧出力
Python	汎用のプログラミング言語	API ・ ログ解析・構成情報の取得	関数・ファイル・ CSV ・ JSON ・ HTTP
SQL	データの問い合わせ言語	DB の確認・集計	SELECT ・ WHERE ・ JOIN ・ 集計

「最初の目標」に出てくる言葉は、この後のページで説明します。

HTTP ・ REST ・ JSON は、プログラミング言語ではない

⇄ HTTP : 通信の約束

要求と応答をやり取りする手順の決まり（p.321）。

REST : 設計の考え方

API の作り方の考え方。データの場所を URL で、操作を HTTP のメソッド（GET = 取得など）で表す。

📄 JSON : データの形式

「項目名：値」の組で書く文字の形式。人もプログラムも読みやすい。

🔑 ポイント

Python は「言語」で、HTTP ・ REST ・ JSON はその言語のプログラムが使う「約束・考え方・形式」です。種類が違うので、同じ表で比べません。

HPE の例：Python から iLO の状態を取得する

Python のプログラムが iLO の Redfish API （ p.604 ） に要求を送り、JSON で答えを受け取ります。



③ で返る JSON （一部・説明用）

```
{
  "PowerState": "On",
  "Status": {
    "Health": "OK"
  }
}
```

電源：入っている
状態：
健全（正常）

URL の例： /redfish/v1/Systems/1 （「1 台目のシステム」の場所）。実際の応答はもっと多くの項目を含みます。

3 つのコマンド例を読む

```
# Bash：エラー行を数える
grep -c ERROR app.log
# PowerShell：動作中のサービス
Get-Service |
    Where-Object Status -eq Running
# Python：JSON を読む
import json
d = json.load(open('s.json'))
```

（# から後は説明のメモ）

app.log の中で ERROR を含む行の数を表示

サービスの一覧を取り出し、次へ渡す
状態が「実行中」のものだけ残す

JSON を扱う部品を読み込む
s.json を開いて、中身を d に入れる

「|」（パイプ）は、前のコマンドの結果を次のコマンドに渡す記号です。

「最初の目標」に出てくる言葉

言葉	やさしい意味
変数	値に名前を付けて入れておく入れ物（例：count に 3 を入れる）
条件分岐・繰り返し	条件によって処理を変える／同じ処理を何度も行う
終了コード	コマンドが成功したか失敗したかを表す数（0 が成功のことが多い）
オブジェクト・パイプライン	項目をまとめたデータ／前の結果を次の命令へ渡すつなぎ方
関数	決まった処理に名前を付け、何度も呼び出せるようにしたもの
CSV	値をカンマで区切って並べた、表の形のデータ
集計	件数を数える・合計や平均を出すこと

IaC とは：インフラの構成をコードで書く

IaC（あいえーしー）

分類：考え方

略語：IaC = Infrastructure as Code

何か	サーバー・ネットワーク・クラウドの資源の構成を、画面で 1 つずつ操作する代わりに、設定ファイル（コード）に書き、ツールで作ったり変えたりする方法。
なぜ必要か	手作業だと手順の抜けや人による違いが出る。ファイルにすれば、同じ環境を何度でも再現でき、変更の履歴とレビューが残る。
どこにあるか	クラウドの資源の作成（Terraform など）、OS やソフトの設定（Ansible など）。
受け取る→する→返す	「Web サーバー用の VM を 2 台」と書いた設定を受け取り → ツールが今の状態との差を計算して作る → 結果の資源が残る（説明用の例）。
似た言葉との違い	手順を並べるスクリプトに対し、Terraform のように「あるべき状態」を書き、差分をツールが反映する方式が代表的。

設定の形式・履歴管理・自動化ツールを区別する

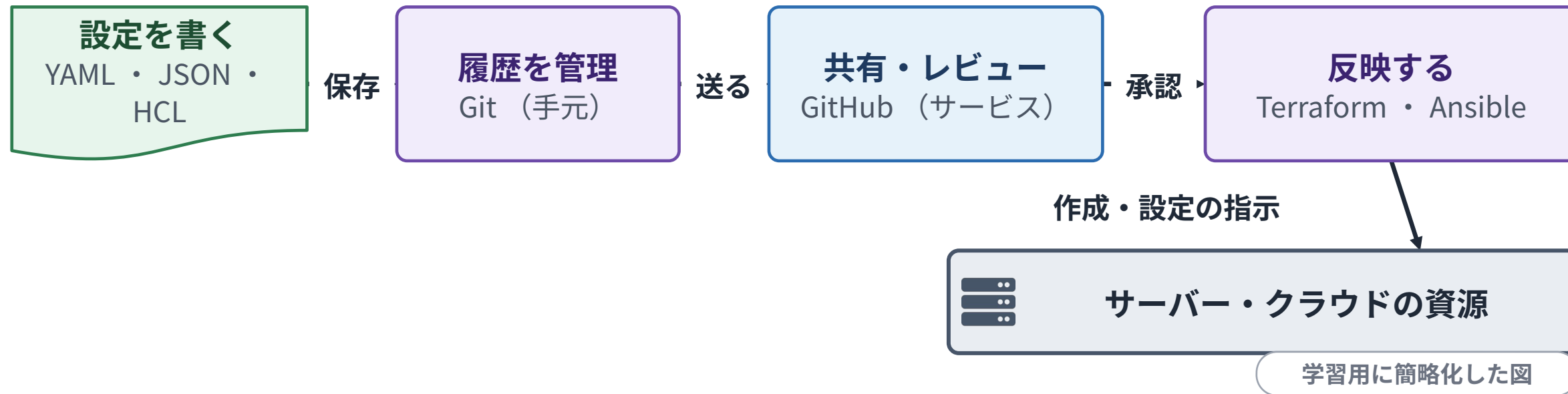
YAML・JSON は形式、Terraform・Ansible はツール、Git は履歴の管理、GitHub は共有のサービスです。

名前	分類	使い道
YAML / JSON	記述の形式	API のデータ・構成の設定
HCL / Terraform	記述言語とツール	クラウド等の資源をコードで定義・作成
Ansible	自動化ツール	OS やソフトの設定をそろえる
Git / GitHub	仕組み／サービス	変更履歴の管理／共有・共同作業

HCL は Terraform 用の記述言語です。Git は手元の PC で動く仕組みで、GitHub はその履歴をインターネット上で共有し、レビューするためのサービスです。

書く→記録→共有・レビュー→反映の流れ

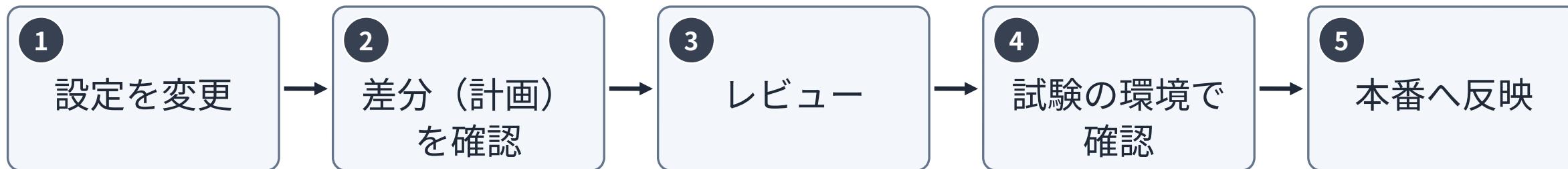
関係図。黒い矢印は、設定ファイルや指示がどこへ渡るかを表します。まとめて **laC** を支えます。



📖 具体例

サンプル商店の検証用の環境を、本番と同じ構成で、設定ファイルから作り直せるようにする（説明用の例）。手作業の「やったつもり」の抜けを防げます。

自動化の注意：誤りも一度に広く反映される



⚠ 注意

自動化は便利な反面、誤りも一度に広く反映されます。計画（差分）の確認・試験・レビューを組み込みます。

Git や GitHub は、すべてのデータを自動でバックアップするものではありません。

i 補足

パスワードなどの秘密の情報を設定ファイルに直接書いて共有しないことも大切です（再構築時に追加した一般的な注意）。

自分で管理する環境で行う 7 つの実習

観察→動かす→わざと止めて比べる→記録して再現する、の順に進めます。

実習	すること	関係するページ
1 自分の環境を記録	ipconfig ・ route ・ nslookup	p.662
2 Linux を観察	ファイル・権限・プロセス・サービス・ログ	p.110 ・ p.112
3 Web サーバーを起動	自分のブラウザで開く	p.321
4 止めて比べる	サービス停止・名前解決の不良と正常時の差	p.247 ・ p.562
5 ログを集計	スクリプトで CSV にする	p.559 ・ p.668
6 バックアップと復元	別の場所へ戻せるか試す	p.379
7 構成図と手順書	同じ環境を再現する	p.195 ・ p.670

実習の準備と、守ること

準備（先に）

更新（OS を最新に）・[多要素認証](#)・暗号化（BitLocker 回復キーの保管）・[バックアップ](#)・WSL または VM の用意。BitLocker は Windows のディスク暗号化機能で、回復キーをなくすとデータを読めなくなります。

注意

本番環境や他人の環境は変更しません。会社支給の PC は会社の指示に従い、機密情報を外部の AI へ入力しません。

補足

WSL（Windows Subsystem for Linux）は、Windows の中で Linux を動かす仕組みです。WSL と VM はネットワークの仕組みが違うので、別の端末から届くかどうかを同じと考えないようにします。

資格は知識の整理に使う

「取得済み」「学習中」「実務でできる」を分けて書きます。

項目	内容	仕事で役立つ場面
IT パスポート試験	IT と経営の基礎（IPA の国家試験）	用語の全体像をつかむ
基本情報技術者試験	技術者の基礎（アルゴリズム・ネットワーク等）	設計書・手順書の読み書きの土台
応用情報技術者試験	より広く深い技術と管理	設計・PM の判断の根拠
MOS	Word ・ Excel 等の操作の資格	資料作成・集計の効率（IT 国家試験とは目的が違う）
HPE 関連の認定	HPE の技術トレーニング・認定（S59）	担当製品の知識を体系的に

IPA は情報処理推進機構（国の機関）です。

試験の制度は変わる：今回確認できていること・いないこと

⚠ 制度は受験時に公式情報で確認

試験の名称・範囲・制度は変わることがあります。受験時に、IPA や各社の公式情報で確認します（元資料 p.119）。この資料は IPA の制度を確認していないため、ここに書く内容を「確認済み」とはしません。

i 再構築時に検索結果で見た情報（未確認）

- ・ 基本情報技術者試験のシラバスは Ver.9.2（2026 年 1 月公開とされる）。
- ・ 2027 年度からの新制度の案が公表されている。

どちらも、検索結果で版名・公開時期を確認したもので、IPA のページ本文はこの作業環境から閲覧できず未確認です。

この資料の青字の用語は、上のシラバスの範囲を目安に選びましたが、範囲を照合して確かめたものではありません。

英語・発表・業務ツールも、使う場面と結び付ける

資格は知識の整理に、発表は説明する力に、Excel や Git は日々の作業に役立ちます。

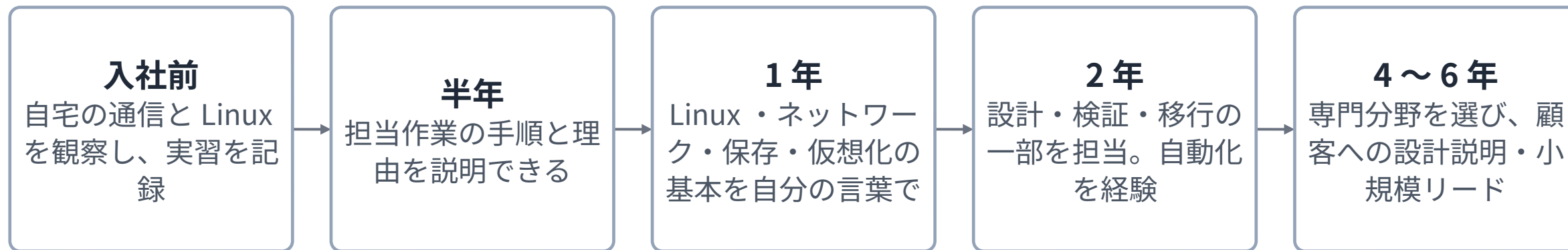
項目	内容	仕事で役立つ場面
英語	マニュアル・エラー・チケットを読む	海外の技術情報・グローバルの窓口
口頭／ポスター発表	時間を区切った全体への説明／掲示物での個別の対話	顧客への説明・設計レビュー
Excel ・データ分析	表計算・集計・グラフ	ログや性能値の整理・報告

「チケット」は、問い合わせや作業依頼を 1 件ずつ記録する票です。国内／海外と、口頭／ポスターは別の分類で、国際会議が日本で開かれることもあります。

入社前から 4 ～ 6 年目まで、説明できる範囲を広げる

時期ごとの目安です。会社の公式な評価基準ではありません。

灰色の矢印は時間の流れです。



🔑 ポイント

各段階で「何を説明・設計・検証できるか」と「何を記録したか」を残すと、成長を確かめやすくなります。

2 年目に強くしたい 5 つの力

- ① 構成図を描く：機器・線・矢印の意味をそろえて描く（p.195）。
- ② ログと事実から原因を絞る：記録で切り分ける（p.562）。
- ③ 変更の影響と戻し方を説明する：何が止まり、どう戻すか（p.565）。
- ④ 作業を再現できる形にする：手順書・スクリプト・IaC（p.670）。
- ⑤ 必要な情報を短く報告する：事実・影響・次の行動を先に（p.568）。

キャリアの方向：必要な経験と仕事の中身の違い（1/2）

ケース例（学部卒・2027 年入社・インフラ／AI 基盤志望）で、HPE での経験から広がる方向を整理した、原資料作成者の想定です。

方向	例	必要になりやすい経験	仕事の違い
クラウド技術職	AWS ・ Microsoft	クラウド設計、顧客課題の整理、英語	技術提案・設計支援が中心
ネットワーク・仮想化	Cisco ・ Nutanix	DC ネットワーク・HCI の深さ、検証	製品の技術提案・検証
GPU ・ データ基盤	NVIDIA ・ Databricks	GPU クラスタ・HPC ／ データ基盤・SQL/Python	Databricks はデータ基盤中心

キャリアの方向：必要な経験と仕事の中身の違い（2/2）

ケース例（学部卒・2027年入社・インフラ／AI基盤志望）で、HPEでの経験から広がる方向を整理した、原資料作成者の想定です。

方向	例	必要になりやすい経験	仕事の違い
DC・設備	Equinix	DC・ネットワーク・設備	GPUサーバー設計とは別の領域
日系の基盤職	NRI・CTC・NTT データ等	大規模設計・上流・リード	顧客案件の推進・調整
コンサル	Big4・アクセンチュア	技術を変革計画に結び付ける説明力	資料作成・顧客調整の比重が大きい

HCI＝サーバー・保存・仮想化を一体にした基盤。Big4＝大手4会計事務所系のコンサル。詳しくは付録（p.737）。

個人の転職例と、転職者の集計は別物

！ 公開された個別例（個別の体験談）

日本 HP に新卒でテクニカルスペシャリストとして入社し、構築・コンサル・グローバル組織を経て AWS のシニア SA（ソリューションアーキテクト）になった例（S73）。HP/HPE 分社をまたぐ経歴で、現在と同じ条件ではありません。

🧪 転職者の集計（公開投稿の集計）

OpenMoney の Microsoft への転職者 23 人：転職前平均 797 万円→転職後 1,144 万円（差 347 万円）（S72）。HPE 出身者・SE に限った集計ではなく、内定確率でもありません。

🔑 ポイント

個人の転職例があることと、転職先の分布は別物です。個別例・集計・推定を区別して読みます。

原資料のケース例：6 年計画（原資料の提案）

元資料 p.145 のノートにある、原資料 A の 6 年計画です。統計や会社の制度ではありません。

時期	内容
1 ～ 2 年目	Linux ・ ネットワーク ・ ストレージ ・ 仮想化を理解し、作業の理由を説明
2 ～ 4 年目	設計 ・ 検証 ・ 移行。 Python ・ IaC ・ Git で自動化
3 ～ 5 年目	GPU/HPC ・ クラウド ・ DC ネットワークなど専門分野を選ぶ
4 ～ 6 年目	顧客への設計説明 ・ 性能と障害の改善 ・ 小規模リード ・ 英語
5 ～ 7 年目	社内昇格と外部オファーを比較

転職時期の目安（原資料の提案）：クラウド領域は 3 ～ 5 年目から市場を確認、GPU/HPC は 4 ～ 7 年目。時期は重なりながら進みます。

第 16 章のまとめ

✓ ここまでで説明できること

- ・ 学ぶ順番と、行き来しながら実習で確かめる進め方。
- ・ コマンドで通信を観察する方法と、表示の読み方。
- ・ シェル・ PowerShell・ Python・ SQL（道具）と、HTTP・ REST・ JSON（約束・考え方・形式）の区別。
- ・ IaC と Git で変更を記録・再現すること、自動化の注意。
- ・ 時期ごとに説明できるようにすることと、個別例・集計・推定の区別。

この資料全体の振り返り

まとめり	学んだこと	代表のページ
基礎（第 1 ～ 4 章）	データ・コンピューターの中・OS・サーバーと全体像	p.16 ・ p.50 ・ p.106 ・ p.129
データと通信（第 5 ～ 7 章）	DB・ネットワーク・Web で注文履歴が表示されるまで	p.151 ・ p.269 ・ p.326
土台（第 8 ～ 11 章）	保存方式・設備・仮想化とクラウド・セキュリティ	p.350 ・ p.390 ・ p.413 ・ p.462
AI と仕事（第 12 ～ 13 章）	AI 基盤と、SE の設計・構築・運用	p.512 ・ p.540
会社と学び（第 14 ～ 16 章）	会社・製品・給与の読み方と、学び方	p.582 ・ p.632 ・ p.658

「注文履歴を見る」から一覧が出るまでを、自分の言葉で説明してみましょう。

確認の問い（１）

自分の言葉で答えてから、次のページの解説を読みましょう。

- ❶ `ipconfig /all` ・ `nslookup` ・ `ping` は、それぞれ何を確かめますか。
- ❷ Python ・ HTTP ・ REST ・ JSON の違いを説明してください。

確認の問い（１）：解説

前のページの問いへの答えと、考え方です。

✓ 問 1 `ipconfig /all` ・ `nslookup` ・ `ping` は、それぞれ何を確認めますか。

`ipconfig /all` は自分の IP アドレス ・ DNS ・ DHCP ・ ゲートウェイの設定、`nslookup` は DNS で名前から調べた IP アドレス、`ping` は相手に届くかと往復時間（RTT）を確認めます。どれも設定を変えない「見るだけ」のコマンドです。 p.662

✓ 問 2 Python ・ HTTP ・ REST ・ JSON の違いを説明してください。

Python はプログラミング言語、HTTP は通信の約束、REST は API の設計の考え方、JSON はデータの形式です。例えば Python のプログラムが、HTTP で REST 型の API に要求し、JSON で答えを受け取ります。 p.666

確認の問い（2）

自分の言葉で答えてから、次のページの解説を読みましょう。

- 1 IaC で自動化する利点と危険、その防ぎ方は？
- 2 「転職者の平均が上がっている＝自分も上がる」と言えない理由は？

確認の問い（２）：解説

前のページの問いへの答えと、考え方です。

✓ 問 1 IaC で自動化する利点と危険、その防ぎ方は？

同じ環境を再現でき、変更の履歴とレビューが残るのが利点です。一方、誤りも一度に広く反映されます。差分（計画）の確認・試験・レビューを組み込んで防ぎます。 p.673

✓ 問 2 「転職者の平均が上がっている＝自分も上がる」と言えない理由は？

OpenMoney の集計は Microsoft への転職者 23 人の投稿で、HPE 出身者や SE に限りません。内定確率でもなく、個人の結果を予測する値ではないからです。 p.683

付録

資料・出典・索引

付録に載っているもの

付録 B 企業・キャリア：会社の公表値・口コミサイトの投稿の集計・原資料作成者の想定（推定）の表を、条件と一緒に載せています。

付録 C AI サービス：会社・サービス・モデル・ツール・実行環境の違い、モデルの選び方、料金の数え方を整理しています。

続いて、出典一覧・確認できていないこと・更新の方法・用語索引があります。

付録の使い方

付録は本文を読んだあとで調べ直すための補助で、本文の説明の代わりではありません。
第 15 章（給与の読み方）・第 12 章（生成 AI ・ [トークン](#)）の後に使います。

付録 B の数字は「誰が・どう出した値か」で読む

付録 B には出所の違う数字が並びます。各ページ右上の区分ラベルで、まず出所を確かめます。金額は税引前（額面）で、表の単位は多くが万円です。

区分ラベル	何の数字か・読むときの注意
企業公表	会社が公表した値（初任給・年収モデルなど）。年度・学歴などの条件付きで、保証額ではない
原資料の提示条件	原資料の作成者に示された条件。公開ページで確かめた値ではない
公開投稿の集計	口コミサイトへの投稿をサイトが集計した値。投稿者の年齢・職種がそろっていない
個別の体験談	1 人の経歴の例。ほかの人に一般化できない
原資料の想定	原資料の作成者が条件を置いて見積もった値。実績でも統計でもない
今回未確認	原資料にあるが、今回確かめ直していない内容。使う前に公式情報で確認

投稿の集計を読む用語：平均・中央値・人数

公開投稿の集計とは、OpenMoneyのような口コミサイトに、社員や元社員が自分の年収などを書き込み、それをサイトが会社ごとにまとめた値です。会社の公式な値ではなく、書き込んだ人だけの集計です。

- **平均**：全員の値を足して人数で割った値。少数のとても高い値に引っ張られやすい。
- **中央値**：小さい順に並べたときの、ちょうど真ん中の値。極端な値の影響を受けにくい。
- **平均年齢・人数**：投稿した人の平均の年齢と、投稿の数。人数が少ないと、数人の投稿で平均が大きく動く。

平均と中央値は第 15 章（p.637）でも説明しています。

説明用の例

5 人の年収が 500 ・ 600 ・ 700 ・
800 ・ 2,000 万円なら、

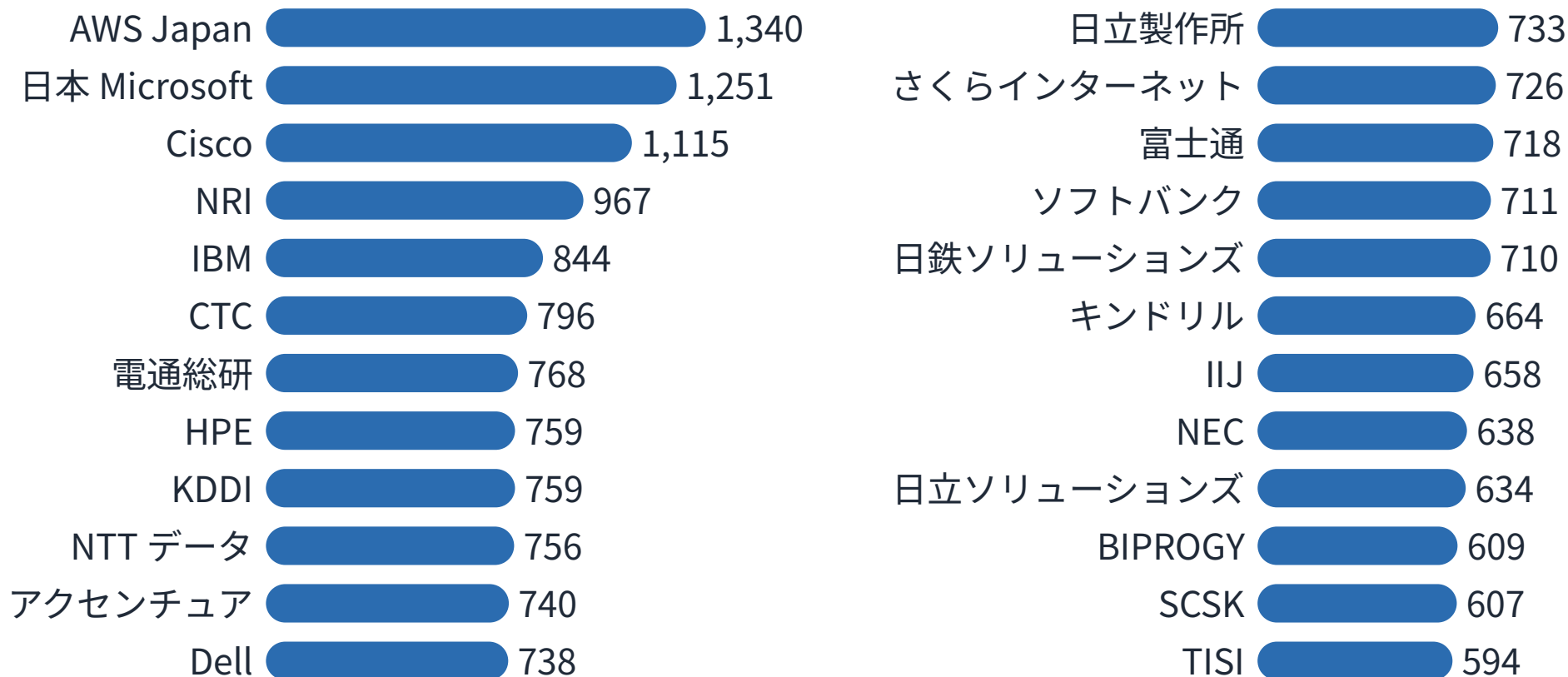
平均 = $4,600 \div 5 =$ **920 万円**

中央値 = 真ん中の 3 番目 = **700 万円**

2,000 万円の 1 人が、平均だけを大きく
押し上げます。

IT エンジニア投稿の平均年収（24 社・棒グラフ）

横棒の長さ＝OpenMoney「IT エンジニア」投稿の平均（万円、原資料 A が 2026 年 10 月 3 日に確認）。左右は同じ目盛りです。年齢・役職・職種がそろっていないので、待遇の順位表ではありません。



IT エンジニア投稿の年収（全 24 社の表）（ 1/3 ）

OpenMoney 「IT エンジニア」投稿の集計（原資料 A が 2026 年 10 月 3 日に確認）。平均・中央値の単位は万円。「—」は値の記載なし。

企業	平均	中央値	平均年齢	人数
AWS Japan	1,340	1,270	32.8 歳	234 人
日本 Microsoft	1,251	1,100	34.0 歳	159 人
Cisco	1,115	1,027	34.1 歳	84 人
NRI	967	940	30.1 歳	619 人
IBM	844	750	31.1 歳	270 人
CTC	796	780	30.6 歳	219 人
電通総研	768	—	28.3 歳	82 人
HPE	759	717	31.1 歳	78 人

IT エンジニア投稿の年収（全 24 社の表）（2/3）

OpenMoney 「IT エンジニア」投稿の集計（原資料 A が 2026 年 10 月 3 日に確認）。平均・中央値の単位は万円。「―」は値の記載なし。

企業	平均	中央値	平均年齢	人数
KDDI	759	776	30.2 歳	139 人
NTT データ	756	750	29.5 歳	684 人
アクセンチュア	740	670	30.2 歳	713 人
Dell	738	640	33.0 歳	34 人
日立製作所	733	713	30.7 歳	459 人
さくらインターネット	726	650	35.0 歳	17 人
富士通	718	680	29.8 歳	669 人
ソフトバンク	711	700	30.6 歳	378 人

IT エンジニア投稿の年収（全 24 社の表）（ 3/3 ）

OpenMoney 「IT エンジニア」投稿の集計（原資料 A が 2026 年 10 月 3 日に確認）。平均・中央値の単位は万円。「―」は値の記載なし。

企業	平均	中央値	平均年齢	人数
日鉄ソリューションズ	710	706	30.0 歳	145 人
キンドリル	664	592	27.7 歳	88 人
IIJ	658	605	31.7 歳	54 人
NEC	638	608	29.6 歳	407 人
日立ソリューションズ	634	600	29.7 歳	103 人
BIPROGY	609	600	28.0 歳	69 人
SCSK	607	570	28.9 歳	204 人
TISI	594	580	29.7 歳	239 人

投稿の年収を比べる時の注意

前の表の数字は、投稿した人だけの集計です。投稿者には新卒・中途、設計・運用・サポートが混在し、年齢・役職もそろっていません。さくらインターネット（17人）やDell（34人）のように人数が少ない会社は、数人の投稿で平均が大きく動きます。

⚠ 職種を分ける

アクセンチュアは、全職種の投稿平均 932 万円に対し、IT エンジニアは 740 万円です。同じ会社でも職種で大きく違うので、職種を分けて比べます。

ℹ 別の出所の数字は混ぜない

クラスメソッドが公表した 30 代エンジニアの平均 924 万円・中央値 915 万円は企業公表の値です。投稿集計とは出所も条件も違うので、表に混ぜずに別に読みます。

初任給の表を読む用語：手当・固定残業代・賞与

初任給は入社1年目の毎月の給与です。会社ごとに月額に含めるものが違うので、中身を分けて読みます。

言葉	意味	比べるときの注意
学部卒・27卒	大学の学部を卒業して入社する場合。27卒＝2027年3月卒業で入社	大学院卒・別の年度の値と混ぜない
住宅手当	住まいの費用の補助として払われる額	全員に一律か、一人暮らしなど条件付きか
固定残業代	一定時間分の残業代を、あらかじめ月給に含めた額	何時間分か。除くと基本の額は低くなる
賞与	月給とは別に年に数回払う額。「基本月額○か月分」と書く	「賞与別」は月額に含まない意味。業績で変わる分は保証ではない

「×18 か月」「満額換算」は年額に直す計算

次の表には、月額に「何か月分」を掛けて1年分に直した値が出てきます。賞与を毎月の給与の何か月分と決めている会社では、「12 か月分の月給＋賞与の月数」を掛けると年額の目安になります。

計算の例

HPE（原資料の提示条件）：月 26.272 万円 × （12 か月＋固定賞与 6 か月）
＝ 26.272 × 18 ＝ 約 472.9 万円 → 約 473 万円

ソフトバンク：月 30.6 万円 × （12 か月＋標準賞与 4.5 か月）
＝ 30.6 × 16.5 ＝ 約 504.9 万円

注意

どちらも賞与を満額もらえると仮定した年換算です。初年度の実際の支給額とは限らず、残業代・変動する報酬を含む総年収でもなく、保証額でもありません。

初任給の内訳：住宅手当・固定残業代・賞与を分ける（1/3）

学部卒の公開初任給など（原資料 A の確認時点）。月額のみだけで比べると、手当や固定残業代の違いを見落とします。

企業	学部卒の初任給など	比較の注意
HPE（公開実績）	月 25.63 万円（2025 年 4 月実績）	固定賞与：基本月額 6 か月。初年度夏季は在籍期間で調整
HPE（原資料の提示条件）	月 26.272 万円（27 卒）	×18 か月で約 473 万円（満額賞与の仮定）。残業・変動報酬は別
NRI・総合職	月 33.65 万円（27 卒予定）	一律住宅手当 6 万円を含む（除くと 27.65 万円）。賞与別
CTC	月 30.15 万円	Grow + 手当 6,000 円を含む。賞与別

初任給の内訳：住宅手当・固定残業代・賞与を分ける（2/3）

学部卒の公開初任給など（原資料 A の確認時点）。月額のみで比べると、手当や固定残業代の違いを見落とします。

企業	学部卒の初任給など	比較の注意
クラスメソッド・専門エンジニア	想定年収 500 ～ 550 万円	一人暮らしは住宅補助月 4.5 ～ 5 万円。 実務・プログラミング経験等を求める
KDDI	月 31.3 万円～（26 年入社 の公表）	27 卒の最終条件は要確認
ソフトバンク	月 30.6 万円～（2026 年度 見込み）	標準賞与 4.5 か月＋業績加算。満額換算 約 504.9 万円は保証ではない

初任給の内訳：住宅手当・固定残業代・賞与を分ける（3/3）

学部卒の公開初任給など（原資料 A の確認時点）。月額のみで比べると、手当や固定残業代の違いを見落とします。

企業	学部卒の初任給など	比較の注意
TISI	月 29.46 万円（合併後予定）	20 時間の固定残業制度あり。手当との関係は内訳確認が必要
BIPROGY / 電通総研	月 28.4 万円 / 月 28 万円（25 年度実績）	いずれも賞与別
IIJ・総合職	月 28 万円	20 時間分の固定残業代を含む
さくら・エンジニア	月 29 万 2,557 円～	固定残業 20 時間分（3 万 6,157 円～）と住宅・通信等の手当を含む

会社公表の年収モデル（CTC）

CTC は、学部卒・標準評価の人の年次別の**年収モデル**を公表しています。年収モデルは「この条件ならこのくらい」という会社の例示で、全社員の平均でも保証額でもありません。ただし条件がはっきりしているので、口コミの投稿より比べやすい情報です。

¥ 683 万円

CTC 学部卒・標準評価 2 年
目

¥ 775 万円

CTC 学部卒・標準評価 5 年
目

¥ 906 万円

CTC 学部卒・標準評価 7 年
目

i 補足

モデルの条件：2024 年度業績に基づく賞与、月 12 時間の残業、各種手当・条件付き家賃補助などを含みます。

日系企業の賃上げと、その読み方

賃上げは、会社が社員の給与の水準を引き上げることです。賃上げ率は、会社全体で平均して何%上げたかを表します。

目 日系企業の賃上げ（企業公表）

NEC は 2026 年度も約 7 % の賃上げ、KDDI は平均 5.1 % の賃上げと初任給の引き上げを公表しています。「昔の日系企業の給与感」だけで判断しないようにします。

⚠ 読み方の注意

会社平均の賃上げ率が、本人に毎年そのまま当てはまるわけではありません。年次・評価・等級（社内の職位の段階）で変わります。

年次の投稿で分かること・分からないこと

「2年目の人の投稿がある」とことと「2年目の給与額が公開されている」ことは別です。公開範囲で見えるのは、投稿者の年齢・年次・等級（Entry・L2・Band6は各社の職位の段階の名前）と、その年齢帯のITエンジニア全体の平均です。

確認できた投稿	公開範囲の数値（年齢帯のITエンジニア平均）	読み方
HPE・23歳・2年目・Entry	20代前半 517万円	本人の年収ではない
HPE・28歳・5年目・L2	20代後半 654万円	5年目の平均ではない
HPE・30歳・6年目・Entry	30代前半 762万円	6年目でもEntryの投稿がある
IBM・26歳・2年目・Band6	20代後半 687万円	修士卒等も含む年齢帯
日立・24歳／26歳・2年目	20代前半 467万円／20代後半 618万円	過去給与を含む年齢帯集計

HPE の勤続年数：公開情報で分かる範囲

平均勤続年数は、いま在籍している社員が、入社してから平均で何年働いているかです。すでに辞めた人は数に入らず、中途入社の人でも長く勤めている人も混ざります。そのため、在籍者の平均勤続年数から「新卒が何年で辞めるか」は推定できません。

HPE の勤続年数	確認結果
新卒 SE だけの平均勤続年数	公開情報では確認できない
技術職全体の平均勤続年数	女性 11.92 年、男性 16.35 年（中途入社者・長期勤続者を含む）
新卒入社者の離職率	採用人数の掲載はあるが、離職人数・定着率は未掲載

⚠ 注意

新卒の平均勤続年数を「16 年」とも「平均 6 年で辞める」とも説明できません。

原資料の想定（推定）の読み方

想定の種類	何を見積もったか	置いた条件の例	掲載
比較対象 23 社の一覧	各社の仕事の傾向と「AI 基盤との相性」	会社全体ではなく対象職種で比べる	p.710
2 ～ 20 年目の総年収レンジ	年次ごとの年収の幅	学部卒・同じ会社に在籍・通常の昇格	p.719
就職難易度の位置	選考の厳しさと並べたときの位置	新卒 IT エンジニアを採用する国内企業の中で	p.726
6 年後の転職先と内定確率	転職先の候補と内定の見込み	HPE で 6 年経験・求人 1 件への応募	p.731

ここから先の数値を読むときの 3 つの約束

- ① 原資料の作成者が条件を置いて見積もった「想定」で、実測でも統計でもない ② 「500 ～ 580」のような幅（レンジ）で示している ③ 条件が違えば当てはまらない

23 社の一覧を読む用語：立場・傾向・相性

原資料は 23 社を、「HPE・システムエンジニア」のように職種を決めて比べています。

列	意味
立場	会社の主な役割。メーカー＝自社の機器やソフトを作って売る。Sler＝顧客のシステムを組み立てて納める。通信キャリア＝通信網を持つ事業者。ほかにクラウド専門・コンサルなど
アプリ／インフラの傾向	対象職種の仕事が、業務の機能を作るアプリ寄りか、サーバー・ネットワークなどの基盤（インフラ）寄りか
AI 基盤との相性	AI 基盤（GPU サーバー・ネットワーク・保存装置など）の仕事にどれだけつながるか。原資料作成者の判断
経験を積んだ後の強み	その職種で経験しやすい技術・仕事

表中の略語：DC＝データセンター、プリセールス＝顧客が買う前に技術の説明や構成の提案をする職種、laC＝設定をコードで書いて再現する方法、PM＝プロジェクトの管理。

原資料の比較対象 23 社：立場・傾向・相性（1/4）

「AI 基盤との相性」は原資料作成者の判断。会社全体でなく対象職種で比較。

企業・対象職種	立場	アプリ／インフラの傾向	AI 基盤との相性
HPE ・ システムエンジニア	メーカー	強くインフラ寄り	非常に高い
Cisco ・ 技術職	メーカー	強くインフラ寄り	非常に高い
CTC ・ エンジニア	Sler	インフラに強い。アプリもある	非常に高い
クラスメソッド・専門エンジニア	クラウド専門	クラウド基盤＋アプリ	非常に高い
さくらインターネット・エンジニア	通信・ DC ・ 自社サービス	インフラ／自社サービス	非常に高い

原資料の比較対象 23 社：立場・傾向・相性（2/4）

「AI 基盤との相性」は原資料作成者の判断。会社全体でなく対象職種で比較。

企業・対象職種	立場	アプリ／インフラの傾向	AI 基盤との相性
IIJ ・ 技術職	通信・自社サービス	強くインフラ寄り	高い
キンドリル・技術職	基盤サービス	強くインフラ寄り	高い
Dell ・ 設計／プリセールス（テクニカルサポート職とは別に比較）	メーカー	強くインフラ寄り	高い
IBM ・ IT スペシャリスト	メーカー兼 SI	両方。配属による	高い
日鉄ソリューションズ	Sler	両方。基盤部門は強い	高い
NRI ・ テクニカルエンジニア	Sler ・ コンサル	対象職種はインフラ寄り	高い

原資料の比較対象 23 社：立場・傾向・相性（3/4）

「AI 基盤との相性」は原資料作成者の判断。会社全体でなく対象職種で比較。

企業・対象職種	立場	アプリ／インフラの傾向	AI 基盤との相性
NTT データ・基盤系 SE	Sler	全社では両方	高い
日立製作所・IT 基盤系 SE	メーカー兼 SI	全社では両方	高い
富士通・基盤系 SE	メーカー兼 SI	全社では両方	高い
NEC・基盤系 SE	メーカー兼 SI	全社では両方	高い
KDDI・ネットワーク／基盤系	通信キャリア	対象職種はインフラ寄り	高い
ソフトバンク・基盤系	通信キャリア	対象職種はインフラ寄り	高い
アクセントチュア・SE	コンサル・SI	アプリ／業務寄り。クラウド部門も	部門次第

原資料の比較対象 23 社：立場・傾向・相性（4/4）

「AI 基盤との相性」は原資料作成者の判断。会社全体でなく対象職種で比較。

企業・対象職種	立場	アプリ／インフラの傾向	AI 基盤との相性
SCSK ・ SE	Sler	両方	部門次第
日立ソリューションズ ・ SE	Sler	アプリ／ソリューション寄り。基盤も	部門次第
TISI（旧 TIS ・ インテック。2026 年 7 月 1 日に合併）	Sler	アプリ／業務寄り。基盤も	部門次第
BIPROGY ・ エンジニア	Sler	アプリ／業務寄り。基盤も	部門次第
電通総研 ・ 技術職	Sler ・ ソリューション	アプリ／業務ソリューション寄り	部門次第

原資料の比較対象 23 社：経験を積んだ後の強み（1/4）

その職種で経験しやすい技術・仕事（原資料作成者の整理）。立場・相性は p.710 から。

企業・対象職種	経験を積んだ後の強み
HPE ・ システムエンジニア	サーバー、ストレージ、ネットワーク、HPC、顧客への基盤設計
Cisco ・ 技術職	ネットワーク、セキュリティ、技術提案。DC ネットワーク
CTC ・ エンジニア	複数ベンダー、クラウド、基盤全体の設計・統合
クラスメソッド・専門エンジニア	AWS、IaC、自動化、クラウド設計。コードも重要
さくらインターネット・エンジニア	Linux、ネットワーク、クラウド、GPU 基盤。配属差が大きい

原資料の比較対象 23 社：経験を積んだ後の強み（2/4）

その職種で経験しやすい技術・仕事（原資料作成者の整理）。立場・相性は p.710 から。

企業・対象職種	経験を積んだ後の強み
IIJ・技術職	ネットワーク、クラウド、セキュリティ、サービス運営
キンドリル・技術職	大規模基盤、クラウド移行、運用設計
Dell・設計／プリセールス（テクニカルサポート職とは別に比較）	サーバー、ストレージ、仮想化、技術提案
IBM・IT スペシャリスト	ハイブリッドクラウド、アーキテクチャ、大規模案件
日鉄ソリューションズ	大規模基盤、クラウド、金融・産業、設計／PM

原資料の比較対象 23 社：経験を積んだ後の強み（3/4）

その職種で経験しやすい技術・仕事（原資料作成者の整理）。立場・相性は p.710 から。

企業・対象職種	経験を積んだ後の強み
NRI ・テクニカルエンジニア	金融等の大規模基盤、クラウド、セキュリティ、上流設計
NTT データ・基盤系 SE	大規模クラウド、金融・公共基盤、設計／PM
日立製作所・IT 基盤系 SE	大規模基盤、クラウド、社会システム、設計／PM
富士通・基盤系 SE	クラウド、HPC、基盤アーキテクチャ
NEC ・基盤系 SE	クラウド、ネットワーク、セキュリティ、公共・企業基盤
KDDI ・ネットワーク／基盤系	通信網、クラウド、データセンター、大規模サービス
ソフトバンク・基盤系	通信、クラウド、AI 関連基盤。担当業務の確認が重要

原資料の比較対象 23 社：経験を積んだ後の強み（4/4）

その職種で経験しやすい技術・仕事（原資料作成者の整理）。立場・相性は p.710 から。

企業・対象職種	経験を積んだ後の強み
アクセンチュア・SE	クラウド移行、設計、プロジェクト推進
SCSK・SE	基盤設計、クラウド、セキュリティ、顧客業務理解
日立ソリューションズ・SE	クラウド、セキュリティ、業務システム、設計／PM
TISI（旧 TIS・インテック。2026 年 7 月 1 日に合併）	クラウド、基盤設計、金融・決済システム
BIPROGY・エンジニア	大規模システム、クラウド、設計／PM
電通総研・技術職	製造・金融の専門性、クラウド、上流設計

年収レンジの表を読む前に：置いた条件

原資料作成者が想定した**総年収**（賞与・残業代・手当を含む年間の給与）の幅。

置いた条件	意味
学部卒・インフラ系 SE	大学の学部卒で、サーバーやネットワークなどの基盤を担当する SE
同じ企業に在籍	転職しない場合
通常の昇格	標準的な速さで等級（社内の職位の段階）が上がる
現在の給与水準	将来の物価上昇やベア（会社全体で基本給の水準を上げること）は加えていない

⚠ 注意

実際の給与は滑らかには増えません。昇格時に増え、同じ等級に長く留まると伸びが鈍ります。10年目以降は年次より役職の差が大きくなります。Dell・AWS・Microsoftは職種・等級の差が大きく、原資料でも数値なし。

原資料の想定：年収レンジ（２・３・６年目）（１/３）

【原資料の想定・実績ではない】単位：万円／年。置いた条件は p.718。10 年目以降は p.722 から。

企業	２年目	３年目	６年目
HPE	500 ～ 580	530 ～ 620	600 ～ 780
IBM	530 ～ 650	570 ～ 720	700 ～ 950
キンドリル	500 ～ 600	530 ～ 650	650 ～ 850
アクセンチュア・ SE	520 ～ 620	560 ～ 680	700 ～ 950
日立製作所	530 ～ 640	560 ～ 700	680 ～ 900
日立ソリューションズ	500 ～ 580	530 ～ 640	650 ～ 820
NEC	520 ～ 630	550 ～ 690	650 ～ 850
富士通	530 ～ 650	560 ～ 720	700 ～ 950

原資料の想定：年収レンジ（２・３・６年目）（２/３）

【原資料の想定・実績ではない】単位：万円／年。置いた条件は p.718。10 年目以降は p.722 から。

企業	２年目	３年目	６年目
Cisco	650 ～ 800	700 ～ 900	900 ～ 1,250
日鉄ソリューションズ	550 ～ 680	580 ～ 740	750 ～ 950
SCSK	490 ～ 590	520 ～ 650	630 ～ 800
CTC	600 ～ 700	650 ～ 750	780 ～ 900
NTT データ	550 ～ 680	580 ～ 740	750 ～ 950
NRI	550 ～ 700	600 ～ 800	850 ～ 1,100
TISI	480 ～ 580	520 ～ 650	650 ～ 830
BIPROGY	500 ～ 600	530 ～ 670	650 ～ 850

原資料の想定：年収レンジ（２・３・６年目）（３/３）

【原資料の想定・実績ではない】単位：万円／年。置いた条件は p.718。10 年目以降は p.722 から。

企業	２年目	３年目	６年目
IIJ	440 ～ 540	470 ～ 600	580 ～ 780
電通総研	550 ～ 700	620 ～ 800	800 ～ 1,050
KDDI	530 ～ 650	560 ～ 720	700 ～ 900
ソフトバンク	530 ～ 650	560 ～ 700	680 ～ 900
さくらインターネット	430 ～ 540	480 ～ 610	600 ～ 800
クラスメソッド	560 ～ 670	600 ～ 730	750 ～ 1,000

原資料の想定：年収レンジ（10・15・20年目）（1/3）

【原資料の想定・実績ではない】単位：万円／年。置いた条件は p.718。10年目以降は年次より役職の差が大きくなります。

企業	10年目	15年目	20年目
HPE	750 ～ 1,000	850 ～ 1,200	900 ～ 1,350
IBM	900 ～ 1,250	1,050 ～ 1,500	1,100 ～ 1,650
キンドリル	800 ～ 1,100	900 ～ 1,250	950 ～ 1,400
アクセンチュア・SE	900 ～ 1,300	1,050 ～ 1,600	1,100 ～ 1,800
日立製作所	850 ～ 1,150	1,000 ～ 1,450	1,050 ～ 1,550
日立ソリューションズ	800 ～ 1,050	900 ～ 1,200	950 ～ 1,350
NEC	800 ～ 1,100	900 ～ 1,300	950 ～ 1,450
富士通	900 ～ 1,200	1,000 ～ 1,450	1,100 ～ 1,600

原資料の想定：年収レンジ（10・15・20年目）（2/3）

【原資料の想定・実績ではない】単位：万円／年。置いた条件は p.718。10年目以降は年次より役職の差が大きくなります。

企業	10年目	15年目	20年目
Cisco	1,100 ～ 1,500	1,300 ～ 1,800	1,400 ～ 2,000
日鉄ソリューションズ	950 ～ 1,250	1,100 ～ 1,450	1,150 ～ 1,600
SCSK	780 ～ 1,000	850 ～ 1,150	900 ～ 1,250
CTC	950 ～ 1,250	1,050 ～ 1,450	1,100 ～ 1,550
NTT データ	950 ～ 1,250	1,050 ～ 1,450	1,100 ～ 1,550
NRI	1,100 ～ 1,450	1,200 ～ 1,650	1,300 ～ 1,800
TISI	800 ～ 1,050	900 ～ 1,200	950 ～ 1,300
BIPROGY	800 ～ 1,100	900 ～ 1,250	950 ～ 1,350

原資料の想定：年収レンジ（10・15・20年目）（3/3）

【原資料の想定・実績ではない】単位：万円／年。置いた条件は p.718。10年目以降は年次より役職の差が大きくなります。

企業	10年目	15年目	20年目
IIJ	700 ～ 950	800 ～ 1,100	850 ～ 1,200
電通総研	1,000 ～ 1,350	1,200 ～ 1,650	1,250 ～ 1,800
KDDI	850 ～ 1,150	1,000 ～ 1,350	1,050 ～ 1,450
ソフトバンク	800 ～ 1,100	900 ～ 1,300	950 ～ 1,450
さくらインターネット	750 ～ 1,000	850 ～ 1,200	900 ～ 1,350
クラスメソッド	900 ～ 1,250	1,000 ～ 1,400	1,100 ～ 1,600

就職難易度の「上位 0.5 割」とは

原資料は、新卒 IT エンジニアを採用する国内企業を選考の厳しさに並べたとき、各社が難しい側からどのあたりかを推定しています。実測の順位ではありません。

「割」は 10 分の 1。上位 1 割＝上位 10 %、上位 0.5 割＝上位 5 %です。

説明用の例

採用する企業が仮に 1,000 社なら、「上位 0.5 ～ 1 割」は難しい側から 50 ～ 100 番目あたりの位置です。

注意

学生の能力の順位でも、内定率（応募した人のうち内定した人の割合）でもありません。企業別の応募者数・内定者数はそろっておらず、学部・職種・採用コースでも変わります。

明治大学は HPE の採用実績に掲載されています。学歴だけで判断せず、SPI（多くの会社を使う適性検査）・面接・インフラ志望の説明を準備する対象です（原資料の記載）。

原資料作成者の想定：就職難易度の位置（23 社）（1/3）

【実測順位ではない】選考の厳しさで並べたときの、難しい側からの位置の推定。上位 0.5 割＝上位 5 %。

企業・対象職種	位置（推定）
HPE ・ システムエンジニア	上位 0.5 ～ 1 割
Cisco ・ 技術職	上位 0.1 ～ 0.3 割
CTC ・ エンジニア	上位 0.5 ～ 1 割
クラスメソッド・専門エンジニア	上位 0.5 ～ 1.5 割
さくらインターネット・エンジニア	上位 1 ～ 2 割
IJJ ・ 技術職	上位 1 ～ 2 割
キンドリル・技術職	上位 1 ～ 2 割
Dell ・ 設計／プリセールス	上位 0.5 ～ 1.5 割

原資料作成者の想定：就職難易度の位置（23 社）（2/3）

【実測順位ではない】選考の厳しさで並べたときの、難しい側からの位置の推定。上位 0.5 割＝上位 5 %。

企業・対象職種	位置（推定）
IBM ・ IT スペシャリスト	上位 0.5 ～ 1 割
日鉄ソリューションズ	上位 0.5 ～ 1 割
NRI ・ テクニカルエンジニア	上位 0.3 ～ 0.7 割
NTT データ・基盤系 SE	上位 0.5 ～ 1 割
日立製作所・ IT 基盤系 SE	上位 0.5 ～ 1 割
富士通・基盤系 SE	上位 0.7 ～ 1.5 割
NEC ・ 基盤系 SE	上位 0.7 ～ 1.5 割
KDDI ・ ネットワーク／基盤系	上位 0.5 ～ 1 割

原資料作成者の想定：就職難易度の位置（23 社）（3/3）

【実測順位ではない】選考の厳しさで並べたときの、難しい側からの位置の推定。上位 0.5 割＝上位 5 %。

企業・対象職種	位置（推定）
ソフトバンク・基盤系	上位 0.7 ～ 1.5 割
アクセンチュア・SE	上位 1 ～ 2 割
SCSK ・ SE	上位 1 ～ 2 割
日立ソリューションズ・SE	上位 1 ～ 2 割
TISI（旧 TIS ・インテック）	上位 1 ～ 2 割
BIPROGY ・エンジニア	上位 1 ～ 2 割
電通総研・技術職	上位 0.5 ～ 1 割

内定確率の表を読む前に：A・Bの条件

HPE で 6 年働いた人の転職候補と内定の見込みを、原資料の作成者が想定した表です。

言葉	意味
確率 A	標準的な基盤の経験をした場合
確率 B	設計～構築、クラウド／GPU、自動化、小さなチームのリード、英語の経験がある場合
確率の範囲	必須条件を満たす求人 1 件に応募してから、内定までの見込み
狙う年収	転職前の年収を 700 万円とした場合の、原資料の想定（万円）

⚠ 注意

統計で確かめた確率ではありません。原資料自身が「実際には 10 ～ 20 ポイント以上ずれる可能性」と書いています（15 %とあっても 5 %や 30 %でもおかしくない）。

複数の応募は同じ経歴・面接力の影響を受けるので、独立したくじとして合算できません。

転職先の表に出てくる職種名・略語

略語・言葉	意味
SA（ソリューションアーキテクト）	顧客の課題に合わせて、どの技術をどう組み合わせるか（構成）を設計・提案する技術職
プリセールス	顧客が買う前に、技術の説明・構成の提案・検証をする職種
HCI	サーバーの計算と保存（ストレージ）を、同じ種類の機器の集まりとしてソフトウェアで一体に扱う構成
Big4	PwC・デロイト・EY・KPMGの4つの大きな会計・コンサルティングのグループ
上流工程	要件定義や設計など、システムづくりの前半の工程

※NVIDIA・基盤系 SA の確率 B 「5～15 %※」：GPU クラスタを本番で設計・構築・性能改善し、英語で説明できる場合、原資料は 15～30 %程度に引き上げています。

原資料作成者の想定：6年後の転職先と内定確率（1/6）

【統計的に検証された確率ではない】年収は転職前 700 万円を基準にした想定（万円）で、2032 ～ 33 年の市場の予言ではありません。

転職候補・職種	狙う年収	確率 A	確率 B	主な条件
AWS ・ SA ／技術職	1,000 ～ 1,500	5 ～ 15 %	15 ～ 30 %	クラウド設計、顧客課題の整理、技術・行動面接
Microsoft ・ クラウド技術職	1,000 ～ 1,500	5 ～ 15 %	15 ～ 30 %	Azure ／基盤設計、顧客対応、英語
Cisco ・ SE ／技術職	1,000 ～ 1,400	10 ～ 20 %	20 ～ 40 %	ネットワークの深さ、技術説明、顧客対応
Nutanix ・ Systems Engineer	1,000 ～ 1,500	10 ～ 20 %	20 ～ 40 %	仮想化、HCI、クラウド、提案・検証

原資料作成者の想定： 6 年後の転職先と内定確率（ 2/6 ）

【統計的に検証された確率ではない】 年収は転職前 700 万円を基準にした想定（万円）で、2032 ～ 33 年の市場の予言ではありません。

転職候補・職種	狙う年収	確率 A	確率 B	主な条件
NVIDIA ・ 基盤系 SA	1,200 ～ 1,800	1 ～ 5 %	5 ～ 15 %※	GPU 、 Linux 、 HPC 、 ネットワーク、英語等の深い専門性（※は p.730 ）
Databricks ・ SA ／ 技術職	1,100 ～ 1,700	3 ～ 10 %	10 ～ 25 %	データ基盤、 SQL ／ Python 、 分散処理、クラウド
Dell ・ プリセールス	900 ～ 1,300	15 ～ 30 %	30 ～ 50 %	製品領域の知識と技術提案

原資料作成者の想定：6年後の転職先と内定確率（3/6）

【統計的に検証された確率ではない】年収は転職前 700 万円を基準にした想定（万円）で、2032 ～ 33 年の市場の予言ではありません。

転職候補・職種	狙う年収	確率 A	確率 B	主な条件
Equinix ・基盤／DC 技術職	800 ～ 1,100	15 ～ 30 %	25 ～ 45 %	DC ・ネットワーク・設備 等と求人の一致
NRI ・基盤系	900 ～ 1,200	15 ～ 30 %	25 ～ 45 %	大規模設計、上流工程、説 明・調整能力
CTC ・基盤設計／ リード	800 ～ 1,050	25 ～ 40 %	35 ～ 55 %	複数技術を組み合わせる設 計・顧客対応
日鉄ソリューションズ	800 ～ 1,050	20 ～ 35 %	30 ～ 50 %	基盤設計、上流、リード経 験

原資料作成者の想定：6年後の転職先と内定確率（4/6）

【統計的に検証された確率ではない】年収は転職前 700 万円を基準にした想定（万円）で、2032 ～ 33 年の市場の予言ではありません。

転職候補・職種	狙う年収	確率 A	確率 B	主な条件
NTT データ・基盤系	850 ～ 1,100	25 ～ 40 %	35 ～ 55 %	大規模案件、設計、推進・調整
日立・NEC・富士通の基盤系	800 ～ 1,100	25 ～ 40 %	35 ～ 55 %	担当ポジションと等級が経験に合うこと
KDDI・ソフトバンクの基盤系	800 ～ 1,100	20 ～ 35 %	30 ～ 50 %	通信／クラウド／AI 基盤と経験が一致
クラスメソッド・クラウド	800 ～ 1,100	10 ～ 25 %	30 ～ 50 %	AWS、IaC、コード、設計の実力

原資料作成者の想定：6年後の転職先と内定確率（5/6）

【統計的に検証された確率ではない】年収は転職前 700 万円を基準にした想定（万円）で、2032 ～ 33 年の市場の予言ではありません。

転職候補・職種	狙う年収	確率 A	確率 B	主な条件
さくら・GPU／クラウド基盤	800 ～ 1,100	10 ～ 25 %	25 ～ 45 %	Linux、ネットワーク、クラスタ、コード等
IIJ・基盤技術職	700 ～ 950	20 ～ 35 %	35 ～ 55 %	ネットワーク／クラウド／サービス設計
PwC・クラウド／テクノロジー	800 ～ 1,200	15 ～ 30 %	30 ～ 50 %	技術経験を顧客課題・変革計画に結び付ける
デロイト・クラウド／テクノロジー	800 ～ 1,200	15 ～ 30 %	30 ～ 50 %	所属法人・職種の確認が必要

原資料作成者の想定： 6 年後の転職先と内定確率（ 6/6 ）

【統計的に検証された確率ではない】 年収は転職前 700 万円を基準にした想定（万円）で、2032 ～ 33 年の市場の予言ではありません。

転職候補・職種	狙う年収	確率 A	確率 B	主な条件
EY ・テクノロジー系	800 ～ 1,200	15 ～ 30 %	30 ～ 50 %	設計経験、説明力、プロジェクト推進
KPMG ・テクノロジー系	800 ～ 1,200	15 ～ 30 %	30 ～ 50 %	コンサル職への適応が必要
アクセンチュア・クラウド系	800 ～ 1,200	20 ～ 35 %	30 ～ 50 %	設計・推進経験。採用職種・レベルで給与差

キャリアの方向：必要な経験と仕事の中身の違い（1/2）

ケース例（学部卒・2027年入社・インフラ／AI基盤志望）でHPEの経験から広がる方向。

方向	例	必要になりやすい経験	仕事の違い
クラウド技術職	AWS ・ Microsoft	クラウド設計、顧客課題の整理、英語	技術提案・設計支援が中心
ネットワーク・仮想化	Cisco ・ Nutanix	DC ネットワーク・HCI の深さ、検証	製品の技術提案・検証
GPU ・ データ基盤	NVIDIA ・ Databricks	GPU クラスタ・HPC ／ データ基盤・SQL/Python	Databricks はデータ基盤中心

キャリアの方向：必要な経験と仕事の中身の違い（2/2）

ケース例（学部卒・2027年入社・インフラ／AI基盤志望）でHPEの経験から広がる方向。

方向	例	必要になりやすい経験	仕事の違い
DC・設備	Equinix	DC・ネットワーク・設備	GPUサーバー設計とは別の領域
日系の基盤職	NRI・CTC・NTTデータ等	大規模設計・上流・リード	顧客案件の推進・調整
コンサル	Big4・アクセンチュア	技術を変革計画に結び付ける説明力	資料作成・顧客調整の比重が大きい

個人の転職例と、転職者の集計は別物

「この会社からあの会社へ移った人がある」という 1 人の例があっても、多くの人と同じ道に進めるとは言えません。個人の転職例があることと、転職先の分布（どこへ何人移ったか）は別物です。

⚠ 公開された個別例（個別の体験談）

日本 HP に新卒でテクニカルスペシャリストとして入社し、構築・コンサル・グローバル組織を経て AWS のシニア SA になった例。HP/HPE 分社をまたぐ経歴で、現在と同一条件ではありません。

ℹ 転職者の集計（公開投稿の集計）

OpenMoney の Microsoft への転職者 23 人：転職前平均 797 万円→転職後 1,144 万円（差 347 万円）。HPE 出身者・SE に限らず、内定確率でもありません。

「ChatGPT」と「GPT」の違い：5つの層で整理

学習用の層の図。上ほど「提供する側」、下ほど「実際に使う場所」に近い層です。

提供企業

サービスやモデルを作って提供する会社

サービス

利用者が契約して使う製品（ChatGPT、Claude など）

モデル

文章を読んで書く LLM などのモデル本体

ツール

モデルが呼び出す機能（Web 検索・ファイル作成など）

実行環境

実際に使う場所（チャット画面・開発環境など）

i 補足

画像生成は、文章のモデルが画像生成の機能（ツール）を呼び出す構成を含みます。スライドの作成は、モデルの構成力とファイル作成の機能の組み合わせです。

層ごとの例：OpenAI ・ Anthropic ・ Google

サービス名・モデル名は原資料（2026年9月28日～30日時点）の記載で今回未確認。
頻繁に変わるので、使う時点で公式ページを確認します。

層	OpenAI の例	Anthropic の例	Google の例
提供企業	OpenAI	Anthropic	Google
サービス	ChatGPT ・ Codex	Claude ・ Claude Code	Gemini
モデル	GPT 系の各モデル	Claude Opus ・ Sonnet ・ Haiku 等	Gemini 系の各モデル
ツール	Web 検索・画像生成・ファイル作成・コード実行	Web 検索・ファイル作成・コード実行	Workspace 連携など
実行環境	チャット・開発環境・ブラウザー	チャット・ Claude Code ・ ファイル	Gmail ・ ドキュメント等の中

主な AI サービスの公開機能（順位ではない）（1/2）

機能は原資料（2026年9月28日～30日時点）の記載で今回未確認。公式説明をもとにした用途の例で、実験で決めた優劣ではありません。

サービス	公開されている主な機能・用途の例	利用条件の注意
ChatGPT / Work	調査・分析・コード実行・ファイル作成・画像生成を一連の作業で	Work は使用量・クレジット・利用枠を共有
Claude / Claude Code	長い資料の整理・文章・コードの編集と実行・文書やスライド作成	機能はプランで異なる
Gemini	Google Workspace（Google のメール・文書などの仕事用サービス）連携、画像や音声などの機能	提供地域・プランで異なる

プラン・契約・権限で使える機能も変わります。どれも万能ではありません。

主な AI サービスの公開機能（順位ではない）（2/2）

機能は原資料（2026年9月28日～30日時点）の記載で今回未確認。公式説明をもとにした用途の例で、実験で決めた優劣ではありません。

サービス	公開されている主な機能・用途の例	利用条件の注意
Microsoft 365 Copilot	Word・Excel・PowerPointの中で作成・編集	組織の契約・権限が必要
Perplexity	出典をたどりながら検索・調査	検索結果の質は出典次第
Grok	X（旧 Twitter）の投稿・話題を含む調査	投稿の人気は正しさの根拠ではない

プラン・契約・権限で使える機能も変わります。どれも万能ではありません。

モデルは、仕事の難しさ・速さ・費用・条件で選ぶ

同じ会社にも、難しい仕事向け・普段使い向け・軽い作業向けのモデルがあります。

品質

曖昧・複雑な課題をどこまで扱えるか

速さ

TTFT（最初の出力が出るまでの時間）と生成速度

費用

トークンあたりの単価と、契約の利用枠

入出力

画像・長い文書・ファイルを扱えるか

ツール

検索・コード実行・ファイル作成を使えるか

契約条件

データの扱い・提供地域・権限

→ 提案（学習の提案）

普段は中位のモデル、行き詰まる難所だけ上位、単純作業は軽量のモデルを試し、同じ入力・同じ評価基準で修正の回数まで比べます。強いモデルほど必ず正解になるわけではありません。

原資料が挙げたモデルの例と、試し方の提案

モデル名と用途は原資料（2026 年 9 月 28 日～30 日時点。この表は 9 月 28 日時点）の記載で、今回未確認です。

原資料の例（今回未確認）	原資料が挙げた用途の例
GPT-6.1 Sol （ OpenAI ）	複雑なコード・実務を費用も考えて進める
GPT-6 Astra （ OpenAI ）	曖昧な課題・深い分析が要る難しい仕事
GPT-6 Luna （ OpenAI ）	範囲の明確な作業・抽出・定型処理
Claude Opus 5.5 （ Anthropic ）	長時間の複雑な開発・知識作業・慎重な判断
Claude Sonnet 5.5 （ Anthropic ）	日常のコード・文書・スライド作成の速度と品質の両立
Claude Haiku 4.5 （ Anthropic ）	軽量・高速・費用を抑えた多量の処理

トークン・利用枠・クレジット・API 料金は別の指標

AI サービスの「使った量」は、別々の物差しで数えます。量の単位・割合・お金に近い単位を混ぜないことが大切です。

指標	意味	単位の例
トークン	入出力を数える単位。文字数とは一致しない	トークン
コンテキスト上限	1 回に扱える入力・会話の情報量の上限	トークン
利用枠の使用率	契約・期間ごとに決まった枠のうち、使った割合	%
クレジット	使った量に応じて減っていく、サービスの中だけの単位	credits
API 料金	開発者がプログラムから API 経由でモデルを使うときの料金	米ドル／ 100 万トークン

画面の「4 %」は何の割合か：確認すべきこと

原資料に、画面に「4 %」と出た例があります。「利用枠の4 %を使った」と考えられますが、契約・期間・内訳が不明なので断定しません。

⚠ 「4 %」で確認すべきこと

- ・ どの画面・どの契約プランの表示か
- ・ 集計期間（日・週・月）
- ・ 分母（何に対する4 %か）と内訳

⚠ 言えないこと

- ・ AI の賢さや記憶容量の4 %ではない
- ・ 「あと24回できる」とは言えない（質問ごとの使用量は一定でない）

i 補足

使う量は、モデル・入出力の量・添付ファイル・ツールの結果・推論の強さ（答える前にどれだけ考えさせるかの設定）・反復回数・キャッシュの再利用で変わり、質問1回の固定料金はありません（原資料）。OpenAI の公式説明では、ChatGPT Work と Codex は使用量・料金・クレジット・利用枠を共有（S23）。

API 料金の計算のしかた：入力と出力を分ける

API 料金は、入力（送った文章）と出力（返ってきた文章）で単価が違い、「100 万トークンあたり何ドル」で示されます。

🔑 計算式

入力トークン数 ÷ 100 万 × 入力単価 + 出力トークン数 ÷ 100 万 × 出力単価

📖 計算例（算数の例）

入力 2 万・出力 1 万トークン、単価が入力 \$2・出力 \$10 のモデルの場合

$20,000 \div 1,000,000 \times \$2 = \$0.04$ / $10,000 \div 1,000,000 \times \$10 = \$0.10$

合計：\$0.04 + \$0.10 = **\$0.14**

⚠️ 注意

実際には、キャッシュ（同じ入力の再利用で単価が変わる）・推論トークン（答える前に考える過程の分）・ツール料金で条件が変わります。

原資料の単価で計算した例（算数の例）

モデル名・単価は原資料（2026年9月28日～30日時点）の記載で今回未確認。

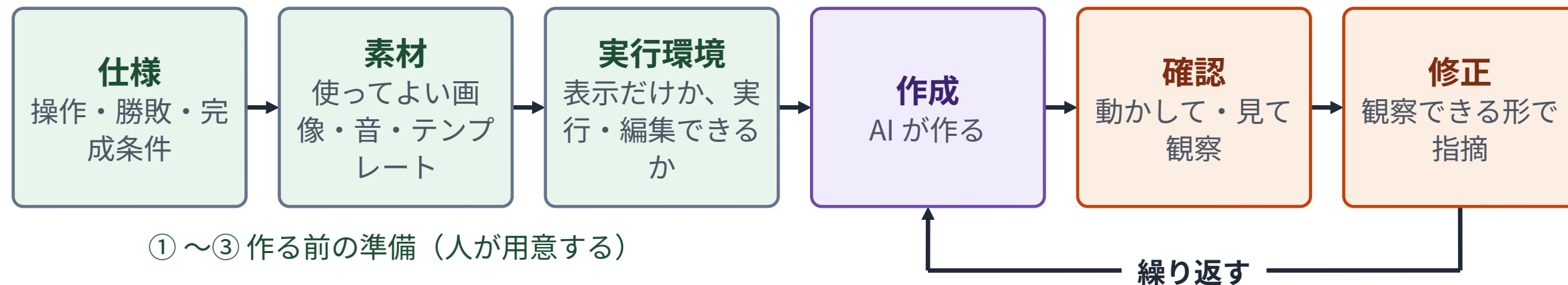
モデル（原資料の単価）	入力（\$/100万）	出力（\$/100万）	計算例
GPT-6.1 Sol	\$2	\$10	\$0.14
GPT-6 Astra	\$10	\$50	\$0.70
GPT-6 Luna	\$0.10	\$0.50	\$0.007
Claude Opus 5.5	\$4	\$20	\$0.28
Claude Sonnet 5.5	\$2	\$10	\$0.14
Claude Haiku 4.5	\$1	\$5	\$0.07

計算例は入力2万＋出力1万トークンの場合で、実費ではなく算数の例です。モデルごとに使うトークン数は違い得ます。ChatGPT Workのクレジットは、APIの米ドル価格と別の表で計算します。

AIで成果物を作る工程：仕様から確認・修正へ

上手な例との差は、プロンプト（AIへの指示文）だけでなく準備と反復で生まれます。

緑＝人が用意する準備、紫＝AIの作業、橙＝人の確認と指摘。黒い矢印は作業の順番で、作成→確認→修正を繰り返します。



i 差が生まれる要因

モデルと設定／実行環境／仕様の具体性／素材／反復の回数／人の知識／公開された範囲（成功場面だけの動画か）

修正の指摘は「観察できる形」で書く

「確認→修正」を回すとき、AI への指摘が具体的でないと、何を直せばよいかが伝わりません。画面を見たり動かしたりして、誰でも同じように確かめられる形で書きます。

指摘の例	判定
「もっとよく」	観察できない
「文字が切れる」	観察できる
「当たり判定が見た目より大きい」	観察できる
「低 FPS でも移動速度が変わらないように」	観察できる

補足

最初の題材は、一人用の小さな 2D ゲームがよい。オンライン対戦は同期・認証・攻撃対策が加わります。FPS = 1 秒あたりに画面を描き直す回数。

出典一覧の読み方

各ページ下の「出典：S〇〇」は、この一覧の ID です。元資料と同じ番号を使い、再構築で足した説明は S79 ・ S80 です。

- **依頼者の追加調査で確認（2026/10/3）**：元資料の作成時に、依頼者が公式ページを開いて確認した資料です。
- **原資料に記載。今回は未再確認**：元資料に載っていたが、元資料の作成時にも、この再構築でも開き直していない資料です。
- **S78**：元資料の編集者による一般的な技術説明（図・たとえ）。公式の定義とは言い回しが違うことがあります。
- **S79**：この再構築で追加した一般的な基礎説明（図・たとえ・説明用の例）。外部資料で1つずつ確認したものではありません。
- **S80**：IPAの試験シラバス。版名と公開時期は検索結果で確認しましたが、IPAのページ本文はこの作業環境から閲覧できず、未確認です。

出典を使うときの注意

⚠ 注意

会社・製品・給与・AI サービス・料金などは変わりやすい情報です。使う時点で、各社の公式ページで確認してください。

一覧の URL は、https:// を省いた形で示しています。元資料の確認基準日は 2026 年 10 月 3 日です。

i 補足

元資料では、完全な URL を各スライドのノートに載せていました。この再構築版では、出典一覧に URL をまとめ、各ページの下には出典 ID だけを示しています。

出典：技術説明と製品（1/7）

ID	資料名	掲載先（URL）	確認の状態
S06	HPE : Integrated Lights-Out（iLO）	www.hpe.com/us/en/products/compute/integrated-lights-out-ilo.html	依頼者の追加調査で確認（2026/10/3）
S07	HPE : Private Cloud AI	www.hpe.com/us/en/products/private-cloud/pcai.html	依頼者の追加調査で確認（2026/10/3）
S08	Intel : What is a GPU?	www.intel.com/content/www/us/en/products/docs/processors/what-is-a-gpu.html	依頼者の追加調査で確認（2026/10/3）
S09	NVM Express : Specifications	nvmexpress.org/specifications/	依頼者の追加調査で確認（2026/10/3）
S10	NIST SP 800-145 : The NIST Definition of Cloud Computing	csrc.nist.gov/pubs/sp/800/145/final	依頼者の追加調査で確認（2026/10/3）

出典：技術説明と製品（2/7）

ID	資料名	掲載先（URL）	確認の状態
S11	IBM：Object vs. file vs. block storage	www.ibm.com/think/topics/object-vs-file-vs-block-storage	依頼者の追加調査で確認（2026/10/3）
S12	Docker：Docker overview	docs.docker.com/get-started/docker-overview/	依頼者の追加調査で確認（2026/10/3）
S13	Kubernetes：Overview	kubernetes.io/docs/concepts/overview/	依頼者の追加調査で確認（2026/10/3）
S14	PostgreSQL：Tutorial	www.postgresql.org/docs/current/tutorial.html	依頼者の追加調査で確認（2026/10/3）
S15	AWS：What is Amazon S3?	docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html	依頼者の追加調査で確認（2026/10/3）

出典：技術説明と製品（3/7）

ID	資料名	掲載先（URL）	確認の状態
S16	Cloudflare：What is DNS?	www.cloudflare.com/learning/dns/what-is-dns/	依頼者の追加調査で確認（2026/10/3）
S17	RFC 3022：Traditional NAT / NAT	www.rfc-editor.org/rfc/rfc3022.html	依頼者の追加調査で確認（2026/10/3）
S18	RFC 9000：QUIC	www.rfc-editor.org/rfc/rfc9000.html	依頼者の追加調査で確認（2026/10/3）
S19	RFC 8446：TLS 1.3	www.rfc-editor.org/rfc/rfc8446.html	依頼者の追加調査で確認（2026/10/3）
S20	AWS：RAG と ファインチューニングの比較	docs.aws.amazon.com/prescriptive-guidance/latest/retrieval-augmented-generation-options/rag-vs-fine-tuning.html	依頼者の追加調査で確認（2026/10/3）

出典：技術説明と製品（4/7）

ID	資料名	掲載先（URL）	確認の状態
S24	RFC 5905 : NTP version 4	www.rfc-editor.org/info/rfc5905/	原資料に記載。今回は未再確認
S25	Cloudflare : What is HTTP/3?	www.cloudflare.com/en-ca/learning/performance/what-is-http3/	原資料に記載。今回は未再確認
S27	NTT ドコモ：電波に影響する環境	www.docomo.ne.jp/area/connect/environs/	原資料に記載。今回は未再確認
S28	Supercell : Brawl Stars 対戦地域・マッチング	support.supercell.com/brawl-stars/en/articles/battle-sites.html	原資料に記載。今回は未再確認
S29	Unity : ゲームネットワークと遅延への対処	docs-multiplayer.unity3d.com/netcode/1.11.0/learn/dealing-with-latency/	原資料に記載。今回は未再確認

出典：技術説明と製品（5/7）

ID	資料名	掲載先（URL）	確認の状態
S31	NVIDIA：SK Group との AI 基盤・メモリの協業	nvidianews.nvidia.com/news/sk-group-and-nvidia-expand-strategic-partnership-across-ai-factories-and-next-generation-memory	原資料に記載。今回は未再確認
S34	Microsoft：データセンターの水利用	local.microsoft.com/blog/understanding-water-use-at-microsoft-datacenters/	原資料に記載。今回は未再確認
S40	Microsoft：Test-NetConnection / DNS クライアントのトラブルシューティング	learn.microsoft.com/en-us/powershell/module/nettcpip/test-netconnection	原資料に記載。今回は未再確認
S55	HPE：iLO RESTful API（Redfish）と Python	developer.hpe.com/platform/ilo-restful-api/home/	原資料に記載。今回は未再確認

出典：技術説明と製品（6/7）

ID	資料名	掲載先（URL）	確認の状態
S57	Microsoft：BitLocker 回復キーのバックアップ	support.microsoft.com/en-us/windows/security/encryption/backup-your-bitlocker-recovery-key	原資料に記載。今回は未再確認
S58	Microsoft：WSL 開発環境の設定	learn.microsoft.com/en-us/windows/wsl/setup/environment	原資料に記載。今回は未再確認
S74	IPA 試験レベル区分の解説（二次資料）	www.cyber-u.ac.jp/ikou/article/006.html	原資料に記載。今回は未再確認
S77	Cloudflare：DNS over TLS	www.cloudflare.com/en-ca/learning/dns/dns-over-tls/	原資料に記載。今回は未再確認
S78	原資料 B・C の編集者による一般的な技術説明（図・たとえ）	—	原資料に記載。今回は未再確認

出典：技術説明と製品（7/7）

ID	資料名	掲載先（URL）	確認の状態
S79	再構築時に追加した一般的な基礎説明（図・たとえ・説明用の例）	—	再構築時に追加。外部資料での個別確認はしていない
S80	IPA：基本情報技術者試験（レベル2）シラバス Ver.9.2（2026年1月公開とされる）／2027年度からの新試験制度のシラバス案	www.ipa.go.jp/shiken/syllabus/gaiyou.html	2026/10/4 検索結果で版名・公開時期を確認。IPAのページ本文はこの作業環境から閲覧できず未確認

出典：企業・採用・給与・投稿集計（1/6）

ID	資料名	掲載先（URL）	確認の状態
S01	HPE：企業向け技術と HP Inc. との分社	www.hpe.com/us/en/about/enterprise-technology-us.html	依頼者の追加調査で確認（2026/10/3）
S02	HPE：日本の会社情報（日本ヒューレット・パカード合同会社）	www.hpe.com/jp/ja/collaterals/collateral.a50011874jpn.html	依頼者の追加調査で確認（2026/10/3）
S03	マイナビ 2027：日本 HPE SE 募集・待遇（給与は 2025 年 4 月実績、最終更新 2026/8/6）	job.mynavi.jp/27/pc/search/corp515/recruiting_course27058348/recruiting_course.html	依頼者の追加調査で確認（2026/10/3）
S04	CTC：新卒の初任給と年収モデル	www.ctc-g.co.jp/recruit/newgrads/salary-model/	依頼者の追加調査で確認（2026/10/3）
S05	TISI：2026 年 7 月 1 日の合併完了・商号変更	www.tisi.jp/news/2026/tisi_info/20260701_1.html	依頼者の追加調査で確認（2026/10/3）

出典：企業・採用・給与・投稿集計（2/6）

ID	資料名	掲載先（URL）	確認の状態
S32	キオクシア：2026年7月30日製品発表	www.kioxia.com/ja-jp/business/news/2026/20260730-1.html	原資料に記載。今回は未再確認
S33	東京エレクトロン：2026年6月26日発表	www.tel.co.jp/news/topics/2026/20260626_001.html	原資料に記載。今回は未再確認
S39	ASML：EUV lithography systems	www.asml.com/en/products/euv-lithography-systems	原資料に記載。今回は未再確認
S52	LabBase：HPE 2027年新卒SE採用情報	compass.labbase.jp/company/1916/recruit/41029	原資料に記載。今回は未再確認
S53	OpenWork：日本ヒューレット・パッカー（公開集計）	www.openwork.jp/company.php?m_id=a0910000000G22E	原資料に記載。今回は未再確認

出典：企業・採用・給与・投稿集計（3/6）

ID	資料名	掲載先（URL）	確認の状態
S54	HPE：Juniper Networks 買収完了（2025年7月2日）	www.hpe.com/us/en/newsroom/press-release/2025/07/hewlett-packard-enterprise-closes-acquisition-of-juniper-networks-to-offer-industry-leading-comprehensive-cloud-native-ai-driven-portfolio.html	原資料に記載。今回は未再確認
S56	HPE：NVIDIA との AI 基盤の発表（2026年3月）	www.hpe.com/us/en/newsroom/press-release/2026/03/hpe-accelerates-secure-scalable-production-ready-ai-through-new-innovations-with-nvidia.html	原資料に記載。今回は未再確認

出典：企業・採用・給与・投稿集計（4/6）

ID	資料名	掲載先（URL）	確認の状態
S59	HPE：技術トレーニングと認定	education.hpe.com/ww/en/training/certification.html	原資料に記載。今回は未再確認
S63	HPE：日本向け学生・新卒採用	careers.hpe.com/us/en/students-graduates-japan	原資料に記載。今回は未再確認
S64	HPE：Morpheus ソフトウェア	www.hpe.com/jp/ja/products/software/morpheus-software.html	原資料に記載。今回は未再確認
S65	HPE：NonStop 製品資料	www.hpe.com/jp/ja/collaterals/collateral.a50013513jpn.html	原資料に記載。今回は未再確認
S66	OpenMoney：各社「IT エンジニア」年収ページ（原文章 A の集計、2026/10/3 時点）	openmoney.jp/	原資料に記載。今回は未再確認

出典：企業・採用・給与・投稿集計（5/6）

ID	資料名	掲載先（URL）	確認の状態
S67	クラスメソッド：給与（会社公表の30代平均・等級別平均）	careers.classmethod.jp/feature/salary/	原資料に記載。今回は未再確認
S68	NRI：2027年度新卒募集要項	working.nri.co.jp/recruit/2027/contents/recruiting/application.html	原資料に記載。今回は未再確認
S69	KDDI：賃上げ・初任給引き上げの発表	newsroom.kddi.com/news/detail/kddi_nr-965_4372.html	原資料に記載。今回は未再確認
S70	NEC：労働・賃金に関する開示	group.nec/global/ja/about/sustainability/social/labour	原資料に記載。今回は未再確認
S71	マイナビ 2027：日本 HPE 会社概要（技術職の平均勤続年数）	job.mynavi.jp/27/pc/search/corp515/outline.html	原資料に記載。今回は未再確認

出典：企業・採用・給与・投稿集計（6/6）

ID	資料名	掲載先（URL）	確認の状態
S72	OpenMoney：日本マイクロソフトへの転職者集計	openmoney.jp/corporations/230/career_path_reports/to_corporation_list	原資料に記載。今回は未再確認
S73	早稲田大学ビジネススクール関連記事：日本 HP→AWS の経歴例	www.wbs-entre.com/	原資料に記載。今回は未再確認
S75	NVIDIA：日本向け基盤系ソリューションアーキテクト募集	jobs.nvidia.com/careers/job/893382379753	原資料に記載。今回は未再確認
S76	さくらインターネット：新卒採用	www.sakura.ad.jp/recruit/graduates/	原資料に記載。今回は未再確認

出典：統計・AI サービス（1/5）

ID	資料名	掲載先（URL）	確認の状態
S21	IEA：Energy and AI（2025 年報告）	www.iea.org/reports/energy-and-ai/executive-summary	依頼者の追加調査で確認（2026/10/3）
S22	Stanford HAI：AI Index 2026 研究開発	hai.stanford.edu/ai-index/2026-ai-index-report/research-and-development	依頼者の追加調査で確認（2026/10/3）
S23	OpenAI：ChatGPT Work / Codex 料金・使用量	learn.chatgpt.com/docs/pricing / https://learn.chatgpt.com/docs/enterprise/chatgpt-work-usage-and-cost	依頼者の追加調査で確認（2026/10/3）
S26	Microsoft：Microsoft と OpenAI の協業（2026 年 4 月）	blogs.microsoft.com/blog/2026/04/27/the-next-phase-of-the-microsoft-openai-partnership/	原資料に記載。今回は未再確認

出典：統計・AI サービス（2/5）

ID	資料名	掲載先（URL）	確認の状態
S30	OpenAI：応答のストリーミング	developers.openai.com/api/docs/guides/streaming-responses	原資料に記載。今回は未再確認
S35	Stanford HAI：AI Index 2026 経済	hai.stanford.edu/ai-index/2026-ai-index-report/economy	原資料に記載。今回は未再確認
S36	米商務省 BIS：半導体の輸出管理	www.bis.gov/press-release/commerce-strengthens-export-controls-restrict-chinas-capability-produce-advanced-semiconductors-military	原資料に記載。今回は未再確認
S37	欧州投資銀行：The scale-up gap	www.eib.org/en/publications/online/all/the-scale-up-gap	原資料に記載。今回は未再確認

出典：統計・AI サービス（3/5）

ID	資料名	掲載先（URL）	確認の状態
S38	欧州委員会：AI Gigafactories 公募	digital-strategy.ec.europa.eu/en/news/eu-launches-ai-gigafactories-call-boost-europes-computing-capacity-and-unlock-more-eu30-billion	原資料に記載。今回は未再確認
S41	OpenAI：モデルの選び方	learn.chatgpt.com/docs/model-selection	原資料に記載。今回は未再確認
S42	OpenAI：画像生成機能	learn.chatgpt.com/docs/image-generation	原資料に記載。今回は未再確認
S43	Anthropic：Claude でファイルを作成・編集	support.claude.com/en/articles/12111783-create-and-edit-files-with-claude	原資料に記載。今回は未再確認
S44	Anthropic：Claude Sonnet 5.5（2026年9月28日発表）	www.anthropic.com/claude-sonnet-5-5	原資料に記載。今回は未再確認

出典：統計・AI サービス（4/5）

ID	資料名	掲載先（URL）	確認の状態
S45	Anthropic：Claude の Web 検索	support.claude.com/en/articles/10684626-enable-and-use-web-search	原資料に記載。今回は未再確認
S46	Anthropic：Claude Opus 5.5（2026年9月22日発表）	www.anthropic.com/claude-opus-5-5	原資料に記載。今回は未再確認
S47	Microsoft：Copilot in PowerPoint FAQ	support.microsoft.com/en-us/powerpoint/frequently-asked-questions-about-copilot-in-powerpoint	原資料に記載。今回は未再確認
S48	Anthropic：Claude Haiku	www.anthropic.com/claude/haiku	原資料に記載。今回は未再確認
S49	OpenAI：API 料金	developers.openai.com/api/docs/pricing	原資料に記載。今回は未再確認

出典：統計・AI サービス（5/5）

ID	資料名	掲載先（URL）	確認の状態
S50	Anthropic：API 料金	platform.claude.com/docs/en/about-claude/pricing	原資料に記載。今回は未再確認
S51	Anthropic：Claude Code（Team／Enterprise）	www.anthropic.com/news/claude-code-on-team-and-enterprise	原資料に記載。今回は未再確認
S60	Google：Gemini・WorkspaceのAI機能	blog.google/products-and-platforms/products/google-one/fall-2026-ai-plan-updates/	原資料に記載。今回は未再確認
S61	Perplexity：Research モードと提供プラン	www.perplexity.ai/help-center/en/articles/10738684-what-is-research-mode	原資料に記載。今回は未再確認
S62	xAI：X Search	docs.x.ai/developers/tools/x-search	原資料に記載。今回は未再確認

確認できていないこと・推定の範囲

この資料で確認できていない事項を先に示します。条件の違う数字を平均して1つにすることはしていません。

⚠️ 今回確認できていないこと

- ・ S24 以降の出典（原資料の記載のまま）
- ・ 27 卒の提示額 262,720 円の正式な通知
- ・ 画面の「4 %」の契約・期間・内訳
- ・ モデル名・API 単価・機能の現在の値
- ・ 本人の配属・チーム別の制度
- ・ IPA の試験シラバスの本文（版名・時期のみ検索で確認）

🧪 推定の範囲（原資料作成者の想定）

- ・ 就職難易度の位置（上位〇割）
 - ・ 2 ～ 20 年目の年収レンジ
 - ・ 6 年後の転職年収と内定確率
- いずれも原資料作成者の想定で、統計的に検証された値ではありません。

数値を比べる時の制約

- 投稿平均（OpenMoney・OpenWork）は、年齢・職種・入社の経路（新卒・中途）が混ざっています。
- OpenWork の会社全体の平均（801 万円／820 万円）は、取得した時点が違います。
- 年齢帯の平均は、勤続年数ごとの平均ではありません。
- 在籍者の平均勤続年数は、新卒が辞めるまでの期間を表しません。

説明用の例

説明用の例（IP アドレス、顧客表と注文表、RTT 200ms／30ms、RPO・RTO の時刻、サンプル商店、佐藤さん）は、各ページで「説明用の例」と表示しています。実在の会社・人・測定値ではありません。

今後の追記・更新の方法

新しい質問や情報が出たら、回答と根拠を記録し、影響するページを更新します。将来の質問と回答を想像で埋めることはしません。



更新の記録に書く項目

記録する項目	書く内容	記入例（形式の例）
質問	何を知りたいか	「HPE の配属面談はいつか」
回答	確認できた内容	（確認後に記入）
根拠	出典 ID ・ 資料名 ・ URL ・ 確認日	S03 （ 2026/○/○ 確認）
情報の区分	企業公表・公的統計・投稿集計・体験談・推定など	企業公表
影響するページ	更新が必要なページ番号	給与の章、付録の初任給の表
更新日・更新者	いつ誰が直したか	2026/○/○

用語索引の使い方

本文で説明した用語を、読みの 50 音順に並べています。「説明ページ」が、その用語を初めて詳しく説明したページです。

- 濃い青（青字）で示した用語は、重点的に覚えたい用語（基本情報技術者試験の学習にも役立つ用語）です。黒字の用語も、資料の理解に必要な用語です。
- 意味は短くまとめています。仕組みや具体例は、説明ページに戻って確認してください。
- 元資料の付録 A（用語の一覧、元資料 p.121 ～ 131）の用語は、本文の各章で説明し、この索引にまとめました。

用語索引（1/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
ARP	あーぷ Address Resolution Protocol	IPv4 で、IP アドレスからその機器の MAC アドレスを調べる約束	p.237
RTO	あーる・ていー・おー Recovery Time Objective	目標復旧時間。障害からどれだけの時間で使えるようにするかの目標	p.556
RTT	あーるていーていー Round Trip Time（往復時間）	データを送ってから、相手の返事が戻ってくるまでの往復の時間。単位は ms	p.297
RPO	あーる・ぴー・おー Recovery Point Objective	目標復旧時点。障害のとき、どの時点までのデータを戻せればよいかの目標	p.556

用語索引（2/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
IaC	あいえーしー Infrastructure as Code	インフラの構成を設定ファイル（コード）に書き、ツールで作成・変更する方法	p.670
ISP	あいえすぴー Internet Service Provider	インターネット接続事業者（プロバイダー）。家庭や会社の回線をインターネットへつなぐサービスを提供する会社	p.214
IOPS	あいおぷす Input/Output Operations Per Second	保存装置が 1 秒あたりに行える読み書きの回数	p.96
iSCSI	あいすかじー	保存装置への読み書きの命令（SCSI）を、IP ネットワークで運ぶ約束。通常の LAN の機器で使える	p.363

用語索引（3/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
IT インフラ	あいていーいんふら	アプリやサービスを動かすための、計算・保存・通信・電力と冷却・運用の土台	p.128
IT パスポート試験	あいていーぱすぽーとしけん	IPA（情報処理推進機構）が実施する国家試験の1つ。ITと経営の基礎を問う	p.676
IP アドレス	あいぴーあどれす IP = Internet Protocol address	ネットワーク上の機器（の接続口）を区別する番号。データの最終的な宛先と送り元に使う	p.225
IPv6	あいぴーぶいしっくす	IP アドレスを 128 ビットで表す IP の版。番号の数が非常に多い。例 2001:db8::1	p.236
IPv4	あいぴーぶいふおー	現在広く使われる IP の版。IP アドレスを 32 ビットで表し、0～255 の 4 つの数をピリオドで区切って書く	p.226

用語索引（4/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
アクセスポイント	あくせすぽいんと AP = Access Point	無線で通信する端末を電波で受け止め、ネットワークにつなぐ機器	p.210
ACID 特性	あしっどとくせい	トランザクションが満たすべき 4 つの性質：原子性・一貫性・独立性・耐久性	p.175
UPDATE	アップデート	SQL で、表の行の値を変更する命令	p.164
アドレス	あどれす	メモリの中の置き場 1 つ 1 つに付いた通し番号（番地）。CPU はこの番号で読み書きする場所を指定する	p.53
アプリケーション	あぷりけーしょん	利用者の目的に合わせた機能を提供するソフトウェア。例：買い物アプリ、表計算、地図	p.116

用語索引（5/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
アプリケーション サーバー	あぷりけーしょんさー ばー	業務の処理をするプログラムを動かし、DB に 問い合わせで動的なページを作る役割（アプリ サーバー）	p.325
暗号化	あんごうか	決まった手順と鍵で、データを第三者に読めな い形に変えること。鍵を持つ相手だけが元に戻 せる	p.261
IaaS	いあーす／あいあーず Infrastructure as a Service	計算・保存・通信の基盤（仮想サーバーなど） をサービスとして借りる形。OS から上は利用 者が管理する	p.449
移行	いこう	古いシステム（環境）から新しいシステムへ、 データや利用を切り替えること	p.567

用語索引（6/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
イメージ	いめーじ	アプリと必要な部品・設定を固めた、変更されないコンテナの雛形	p.427
INSERT	インサート	SQL で、表に新しい行を追加する命令	p.164
インシデント	いんしでんと	サービスの中断や品質の低下という出来事。まず影響を抑え、復旧させる対象	p.564
インターネット	いんたーねっと	世界中の多数のネットワーク（通信会社・企業・大学など）を相互につないだ仕組み。特定の1社の網ではない	p.206
インターフェース	いんたーふえーす	2つのものが接する部分と、そこでのやり取りの決まり（差し込み口の形・信号・受け渡し方など）	p.84

用語索引（7/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
WHERE	ウェア	SQL で「どの行を対象にするか」の条件を書く部分	p.165
Web	うぇぶ WWW = World Wide Web	ブラウザで文書（Web ページ）を見たり、リンクで別のページへ移ったりできる、インターネット上のサービス	p.318
Web サーバー	うぇぶさーばー	ブラウザからの HTTP の要求を受け付け、ページや画像を返す（または担当へ転送する）役割	p.323
運用	うんよう	システムを止めずに動かし続けるための活動。監視・保守・バックアップ・変更管理など	p.132

用語索引（8/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
HTML	えいちていーえむえる HyperText Markup Language	Web ページの文章・見出し・箇条書き・画像 などの構造を書くための言語	p.319
HDD	えいちでいーでいー Hard Disk Drive	回転する円盤に磁気でデータを記録する保存装 置	p.45
HTTP	えいちていーていー ぴー HyperText Transfer Protocol	ブラウザと Web サーバーが要求と応答をや り取りするときの約束（プロトコル）	p.321
HTTPS	えいちていーていー ぴーえす	Web のページなどをやり取りする約束 （HTTP）を、TLS で保護した通信	p.263

用語索引（9/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
HBA	えいちびーえー Host Bus Adapter（ホストバスアダプター）	サーバーを、SAN などを通じて保存装置へつなぐための部品（アダプター）	p.394
HBM	えいちびーえむ High Bandwidth Memory	演算チップのすぐ近くに積み重ねて置き、GPU などへ大量のデータを高速に供給するメモリ。長期保存用ではない	p.93
HPC	えいちぴーしー High Performance Computing	多数の計算機や GPU を高速な網でつなぎ、大規模な計算を短時間で行うこと。スーパーコンピューターが代表例	p.589
AI	えーあい Artificial Intelligence	認識・予測・計画・生成などを機械で行う技術の広い分野	p.486

用語索引（10/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
ALU	えーえるゆー Arithmetic Logic Unit	算術論理演算装置。足し算などの算術演算と、比較などの論理演算を行う、演算装置の中心部分	p.67
API	えーぴーあい Application Programming Interface	プログラムが別のソフトやサービスの機能呼び出すための、決まった窓口と呼び出し方	p.605
Sler	えすあいやー System Integrator	顧客の目的に合わせて、複数の会社の製品やサービスを組み合わせ、システムを設計・構築（必要なら運用も）する会社	p.582
SSD	えすえすでいー Solid State Drive	NAND フラッシュというチップに電氣的にデータを記録する保存装置。動く部品がない	p.45

用語索引（11/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
SLA	えす・える・えー Service Level Agreement	サービスの水準（稼働率・応答時間など）について、提供者と利用者が契約で合意したもの	p.571
SLO	えす・える・おー Service Level Objective	サービスの水準について、運用する側が目指す目標	p.571
エスカレーション	えすかれーしょん	自分で解決・判断できないとき、上位の担当者や専門の担当に支援や判断を求めること	p.571
SQL	えすきゅーえる	DBMS にデータの検索・追加・変更・削除を指示するための言語。製品名ではない	p.163
エッジコンピューティング	えっじこんぴゅーていんぐ	データが発生する場所（工場・店舗・基地局など）の近くで処理する考え方。略してエッジ	p.447

用語索引（12/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
NTP	えぬていーピー Network Time Protocol	ネットワークを通じて、時刻を配る機器（時刻サーバー）から正しい時刻を受け取り、機器の時計を合わせる約束	p.265
NVMe	えぬぶいえむいー NVM Express（Non-Volatile Memory Express）	SSD などの保存装置へ命令を出す手順を決めた通信規約の体系。多くは PCIe 上で使う	p.87
M.2	えむどっとつー	板状の小さな部品（SSD など）の、基板の大きさと差し込み口の形を決めた規格	p.86
LLM	える・える・えむ Large Language Model	大量の文章で学習した、非常に多くのパラメータを持つ言語のモデル	p.500

用語索引（13/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
演算装置	えんざんそうち	足し算や比較などの計算をする装置。CPU の中にある	p.18
応答	おうとう	提供する側が、要求を受けて返す結果のデータ	p.123
応用情報技術者試験	おうようじょうほうぎ じゅつしゃしけん	IPA が実施する国家試験の 1 つ。より広く深い技術と管理の知識を問う	p.676
OS	オーエス Operating System（オペレーティングシステム）	機械の資源（CPU・メモリ・ストレージなど）を管理し、アプリを動かす土台になる基本ソフト	p.106
ONU	おーえぬゆー Optical Network Unit	光回線終端装置。光ファイバーの光の信号と、家の機器が扱う電気の信号を相互に変換する機器	p.213

用語索引（14/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
OT	おーていー Operational Technology	工場の設備や制御装置を動かし、監視するための技術。オフィスの IT 機器とは更新の頻度や停止の許容が異なる	p.597
オープンソースソフトウェア	おーぷんそーすそふと うえあ OSS = Open Source Software	プログラムの元の文章（ソースコード）が公開され、決められた条件の下で誰でも使える・直せるソフトウェア	p.180
オブジェクトストレージ	おぶじえくとすとれー じ	データ本体とメタデータを 1 つのまとまり（オブジェクト）にし、キーで保存・取得させる保存方式。大量の画像やバックアップ向き	p.350
オンプレミス	おんぷれみす	組織が自分で管理する環境に設備を置いて運用する考え方。借りたデータセンターで自社管理する形も含む	p.441

用語索引（15/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
カーネル	かーねる	OS の中心部分。CPU ・メモリ ・入出力を直接管理する	p.109
外部キー	がいぶきー	別の表の主キーを指す列。存在しない相手を指す行を防げる	p.172
可観測性	かかんそくせい	外から得られる記録や測定値から、システムの内部の状態を調べられる性質（オブザーバビリティ）	p.558
学習	がくしゅう	例題を使って、答えが正解に近づくようにモデルのパラメータを調整する工程	p.492
仮想化	かそうか	ソフトの力で 1 台の物理的な機械を、複数の独立した論理的な環境に分けて使う技術	p.413

用語索引（16/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
仮想記憶	かそうきおく	保存装置の一部をメモリの延長として使い、実際より大きな置き場があるように見せる仕組み。内容の出し入れをスワップという	p.81
仮想マシン	かそうましん VM = Virtual Machine	ソフトで作った独立したコンピューター環境。仮想的な CPU ・メモリ ・ディスクを持ち、中で OS とアプリが動く	p.416
稼働率	かどうりつ	使えるはずの時間のうち、実際に使えた時間の割合	p.550
可用性	かようせい	使いたいときに使える状態を保つ性質	p.550
監視	かんし	機械やソフトの状態を見続け、異常を早く見つけること	p.132

用語索引（17/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
記憶階層	きおくかいそう	レジスタ→キャッシュ→メモリ→保存装置と、速く小さい記憶から遅く大きい記憶へ段々に組み合わせた構造	p.71
記憶装置	きおくそうち	プログラムやデータを覚えておく装置	p.18
機械学習	きかいがくしゅう	データからパターン（規則性）をコンピューターに見つけさせる方法。ルールを人が全部書かない	p.487
規格	きかく	多くの会社や団体が合わせる、形・大きさ・信号・手順などの決まり。同じ規格に合わせれば別の会社の製品どうしてもつなげる	p.85

用語索引 (18/64)

用語	読み／略語＝正式名称	やさしい意味	説明ページ
基地局	きちきよく	携帯電話の電波を送受信するアンテナと、その電波を有線の回線へつなぐ装置の組。ビルの屋上や鉄塔などに置かれ、周りの一定の範囲を受け持つ	p.282
Git	ぎっと	ファイルの変更履歴を記録し、いつ誰が何を変えたかをたどったり、前の状態に戻したりできる仕組み	p.671
機能要件	きのうようけん	システムが「何をするか」についての要件	p.541
基本情報技術者試験	きほんじょうほうぎ じゅつしゃしけん	IPA が実施する国家試験の 1 つ。アルゴリズム・ネットワークなど技術者の基礎を問う	p.676
基本設計	きほんせっけい	システム全体の方式・構成と、それを選んだ理由を決める設計	p.545

用語索引（19/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
キャッシュメモリ	きゃっしゅめもり	CPU の中の、メモリより小さく速い記憶。最近使った命令やデータの写しを置き、メモリを待つ時間を減らす	p.70
行	ぎょう	表の横の 1 並び。1 件の記録（例：注文 1 件）にあたる。レコードとも呼ぶ	p.140
共通鍵暗号方式	きょうつうかぎあんごうほうしき	暗号化と復号に同じ鍵を使う方式。処理は速いが、鍵を相手に安全に渡す方法が課題になる	p.471
切戻し	きりもどし	変更失敗したとき、変更前の状態へ戻すこと	p.565
QUIC	くいっく	UDP の上で、送り直し・暗号化・複数の流れの管理を行う届け方の約束。HTTP/3 が使う	p.258
Kubernetes	くーばねていす	コンテナにしたアプリの配置・数の増減・故障時の再起動を自動で行う基盤	p.433

用語索引 (20/64)

用語	読み／略語＝正式名称	やさしい意味	説明ページ
クライアント	くらいあんと	サービスを利用する側。要求を送って応答を受け取る役割の名前	p.124
クライアントサーバシステム	くらいあんとさーばしすてむ	要求を送るクライアントと、応答を返すサーバーに役割を分けて作る仕組み	p.124
クラウドコンピューティング	—	計算・保存などの資源を、ネット経由で必要なときに必要な分だけ申し込んで使い、解約できる提供方式	p.442
クラスター	くらすたー	複数の機器や環境をまとめて、1つのもののよう に扱う単位	p.435
クラスタリング	くらすたりんぐ	複数の機器をまとめて1つのシステムのように 動かし、性能や止まりにくさを高める構成に すること	p.552

用語索引（21/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
グローバル IP アドレス	ぐろーばるあいぴーあどれす	インターネット上で通じる IP アドレス。世界で重ならないように割り当てられる。公開 IP とも	p.233
クロック	くろっく	CPU の各部分の動きをそろえる、一定間隔の電気の合図。1 秒間に刻む回数をクロック周波数（GHz など）という	p.73
KV キャッシュ	けー・ぶい・きゃっしゅ	LLM が文章を生成する途中の計算結果を保持し、再計算を減らすための GPU メモリ上のデータ	p.510
ゲスト OS	げすとおーえす	VM の中で動く OS。VM を「お客（ゲスト）」に見立てた呼び方	p.419
結合	けつごう	2 つの表を、共通の列の値が同じ行どうしてつながい合わせて取り出すこと（SQL では JOIN）	p.177

用語索引（22/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
権限	けんげん	どの利用者やプログラムが、どのファイルや機能に対して、読む・書く・実行するのどれをしてよいかの決まり	p.114
検索	けんさく	たくさんの行の中から、条件に合う行を探して取り出すこと	p.156
コア	こあ	命令の取り出し・解読・実行を行う一式。多くの CPU は 1 つのチップに複数のコアを持つ（マルチコア）	p.64
コア網	こあもう	携帯電話会社の中心の設備の網。契約の確認・移動の管理・料金の計算などをし、インターネットへの出口になる	p.283

用語索引（23/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
公開鍵暗号方式	こうかいかぎあんごうほうしき	公開してよい鍵と、本人だけが持つ秘密の鍵の組を使う方式。公開鍵で暗号化したものは秘密鍵でしか復号できない	p.471
五大装置	ごだいそうち	コンピューターの働きを、入力・出力・記憶・演算・制御の 5 つに分けた基本の整理	p.18
コンテキスト	こんてきすと	LLM が 1 回の処理で参照する入力の範囲（会話・添付資料など）。上限があり、永続的な記憶ではない	p.510
コンテナ	こんてな	アプリと必要な部品をまとめ、ホスト OS のカーネルを共有しながら、ほかと分けて動かす実行の環境	p.425

用語索引（24/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
コンピューター	こんぴゅーたー	数や文字などを受け取り、決められた手順で処理して、結果を出す機械。電卓・スマホ・PC・サーバーもその仲間	p.14
SaaS	さーす Software as a Service	完成したソフトを、ネット経由ですぐ使えるサービスとして利用する形	p.449
サーバー	さーばー	サービスを提供する側。要求を受けて応答を返す役割の名前で、機械にもソフトにも使う	p.124
サービス	さーびす	（OSの用語）画面に出ず、裏で動き続けて機能を提供するプログラム。Linuxなどではデーモンと呼ぶ。利用者に提供される「ネットのサービス」とは別の意味	p.111

用語索引（25/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
サービスマネジメント	さーびすまねじめんと	IT サービスを、決めた品質で安定して提供し続けるための管理の仕組み全体	p.564
最小権限	さいしょうけんげん	人やプログラムに、仕事に必要な範囲だけの権限を与え、それ以上は与えない考え方	p.468
サイジング	さいじんぐ	必要な容量・処理能力・台数を、根拠と前提を付けて見積もること	p.546
SATA	さた Serial ATA	SSD や HDD などの保存装置と PC をつなぐ接続の規格	p.86
サブネット	さぶねっと	同じ LAN とみなす IP アドレスの範囲。範囲はサブネットマスク（例 255.255.255.0 = /24 ）で表す	p.234

用語索引（26/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
差分バックアップ	さぶんばっくあっぷ	最後のフルバックアップから変わった部分を、すべてコピーするバックアップ	p.382
SAN	さん Storage Area Network（保存装置のための網）	サーバーと保存装置の間をつなぐ、保存のための専用のネットワーク。サーバーからはディスクに見える	p.357
3層クライアントサーバシステム	さんそうくらいあんと さーばしすてむ	表示（プレゼンテーション層）・業務の処理（ファンクション層）・データ管理（データ層）の3つに役割を分けたシステムの形	p.326
CSS	しーえすえす Cascading Style Sheets	Web ページの色・大きさ・配置などの見た目を指定するための言語	p.319

用語索引（27/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
CPI	シーピーあい Cycles Per Instruction	1 つの命令の実行に平均何回のクロック（リズム）が必要かを表す値	p.75
CPU	シーピーゆー Central Processing Unit	プログラムの命令を順に読み取り、計算・比較・データの移動を行う汎用の部品（中央処理装置）	p.46
GPU	ジーピーゆー Graphics Processing Unit	同じ種類の計算を多数の計算ユニットで一斉に進めるのが得意な部品。画像処理や AI の計算に使う	p.89
JSON	じえいそん JavaScript Object Notation	データを「項目名：値」の組で書く、文字だけの形式。API のデータの受け渡しに広く使われる	p.666

用語索引（28/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
シェル	しえる	人が文字で打ち込んだコマンドを受け取り、OS に実行させて結果を表示するソフトウェア	p.661
資源	しげん	プログラムが動くために使う、限りのある機械の能力や場所。CPU を使う時間、メモリの場所、ストレージの領域など	p.107
ジッター	じったー	遅延のばらつき。届くまでの時間が毎回大きく変わると、ゲームの動きがカクつく・音声が乱れるなどの症状が出る	p.298
SIM	しむ Subscriber Identity Module（加入者を識別する部品）	携帯電話の契約者（加入者）を識別する情報を持つ部品。スマホに差す小さなカード型のほか、本体に組み込まれた型（eSIM）もある。	p.285

用語索引（29/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
Java	じゃば	Web のアプリなどを作るのによく使われるプログラミング言語。JVM の上で動く	p.339
周波数	しゅうはすう	電波などの波が 1 秒間に振動する回数。単位は Hz（ヘルツ）。携帯電話は複数の周波数の電波（周波数帯）を使い分ける	p.292
主キー	しゅきー	表の中で 1 行を特定するための値（列）。重複せず、空にもできない	p.171
主記憶	しゅきおく	CPU が実行中のプログラムとデータを置き、直接読み書きする記憶。実物はメモリ。電源を切ると内容が消える	p.43
出力装置	しゅつりよくそうち	処理の結果を外へ出す装置。例：画面、スピーカー、プリンター	p.18

用語索引（30/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
障害	しょうがい	システムや機器が本来の働きをできなくなる こと、またはその状態	p.549
詳細設計	しょうさいせつけい	基本設計にもとづき、具体的な設定値や手順を 決める設計	p.545
冗長化	じょうちょうか	予備の部品・機器・経路を用意し、一部が壊れ ても全体が止まらない・データを失わないよう にすること	p.367
情報	じょうほう	データを目的に合わせて集めたり計算したりし て、人が判断に使える意味を持たせたもの	p.17
情報セキュリティの 3 要素	じょうほうせきゅり ていのさんようそ	機密性（見てよい人だけが見られる）・完全性 （内容が正しく保たれる）・可用性（必要なと きに使える）の 3 つ	p.460

用語索引（31/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
証明書	しょうめいしょ	サーバーなどが本物であることを、信頼できる発行元が電子的な署名で保証するデータ	p.261
使用率	しょうりつ	CPU や GPU などの部品が働いている時間の割合（%）	p.96
処理	しより	受け取った数や文字を、決まった手順で計算したり、比べたり、探したり、形を変えたりすること	p.15
深層学習	しんそうがくしゅう	層を深く重ねたニューラルネットワークを使う機械学習	p.498
スイッチ	すいっち	LAN 端子を複数持ち、同じ LAN の中の機器どうしの通信を中継する機器	p.211

用語索引（32/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
推論	すいろん	学習済みのモデルに新しい入力を渡し、計算して出力を得る工程	p.493
スクリプト	すくりぷと	実行したいコマンドや処理を順に書いてファイルに保存し、まとめて自動で実行できるようにした短いプログラム	p.664
スケールアウト	すけーるあうと	同じ役割の機械（または VM ・コンテナ）の台数を増やして、仕事を分け合うこと	p.445
スケールアップ	すけーるあっぷ	1 台の機械（または VM ）の CPU やメモリを増やして、性能を上げること	p.445
スループット	すするーぷっと	一定時間に処理・転送できる量。1 秒に運べるデータ量（GB/s など）や 1 秒に処理できる件数	p.96

用語索引（33/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
ストライピング	すらいぴんぐ	データを小さく分け、複数のドライブに順番に並べて書くこと。同時に読み書きできるので速くなる	p.368
ストレージ	すとれーじ	電源を切っても消えないように、プログラムやデータを長く保存する装置（補助記憶）	p.45
スナップショット	すなっぷしょっと	ある時点のデータの状態を記録しておき、その時点に戻せるようにする仕組み	p.378
制御装置	せいぎょそうち	ほかの装置に指示を出し、仕事の順番をとりしきる装置。CPU の中にある	p.18
脆弱性	ぜいじゃくせい	ソフトや設定にある、攻撃に悪用できる弱点（欠陥）	p.476

用語索引（34/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
生成 AI	せいせいえーあい	文章・画像・音声などを新しく作り出す AI。 用途による分類	p.499
静的コンテンツ	せいてきこんてんつ	誰が見ても同じ中身で、前もって作って置いてあるファイル（画像・CSS など）	p.324
責任共有モデル	せきにんきょうゆうも でる	クラウドで、どこまでを事業者が守り、どこからを利用者が守るかの分担を示した考え方	p.451
SELECT	セレクト	SQL で、表から条件に合うデータを取り出す命令	p.164
総年収	そうねんしゅう	1 年間に受け取る給与の合計（基本給・手当・賞与・時間外手当などを含む、税引前の額）	p.630
増分バックアップ	ぞうぶんばっくあっぷ	前回のバックアップ（種類を問わない）から変わった部分だけをコピーするバックアップ	p.382

用語索引（35/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
ソフトウェア	そふとうえあ	ハードウェアに何をさせるかを定めるプログラムと設定。例：アプリ、OS	p.30
帯域	たいいき	1 秒あたりに運べるデータの量（通信の容量）。単位は bps（Mbps・Gbps など）	p.295
DAS	だす Direct Attached Storage（直接つないだ保存装置）	保存装置を、ネットワークを通さずにサーバーや PC へケーブルで直接つなぐ方式	p.357
WBS	だぶりゅー・びー・えす Work Breakdown Structure	プロジェクトの作業を、木の枝のように細かく分解して整理したもの	p.569

用語索引（36/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
多要素認証	たようそにんしょう MFA = Multi-Factor Authentication	知っているもの・持っているもの・本人の特徴のうち、種類の違う要素を 2 つ以上組み合わせて本人を確かめる方法	p.466
遅延	ちえん	データを送ってから相手に届くまでにかかる時間。単位は ms（ミリ秒＝1000 分の 1 秒）	p.295
中央値	ちゅうおうち	値を小さい順に並べたときの真ん中の値。少数の極端な値の影響を受けにくい	p.637
通信	つうしん	機器どうしが、線や電波を通じてデータを送り合うこと	p.123
DR	でいー・あーる Disaster Recovery	災害復旧。建物ごと使えなくなる場合に備え、離れた場所でシステムを復旧させる計画と仕組み	p.557

用語索引（37/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
DHCP	でいーえいちしーピー Dynamic Host Configuration Protocol	IP アドレス・出口・DNS サーバーなどの設定を、つながった機器に自動で配る仕組み	p.244
DNS	でいーえぬえす Domain Name System	ドメイン名から対応する IP アドレスなどを調べて答える仕組み（名前解決）	p.247
TLS	ていーえるえす Transport Layer Security	相手を証明書で確かめ、通信の中身を暗号化し、改ざんに気づけるようにする約束	p.262
TCP	ていーしーピー Transmission Control Protocol	順番の管理と送り直しで、データを確実に届ける届け方の約束	p.258

用語索引（38/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
TCP/IP	ていーしーぴーあい ぴー	IP ・ TCP ・ UDP など、インターネットで使う 通信の約束の一式の呼び名	p.258
TTFT	ていー・ていー・え ふ・ていー Time to First Token	質問を送ってから、最初の出力（トークン）が 現れるまでの時間	p.506
DBMS	でいーびーえむえす Database Management System	データベースの検索・追加・変更・削除、同時 利用の調整、権限の確認、記録の保護をまとめ て行うソフトウェア	p.145
DB サーバー	でいーびーさーばー	データベースについての依頼（探す・追加す る・書き換える・消す）を受け付け、DBMS を 使って実行し、結果を返す役割	p.148

用語索引（39/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
ディストリビューション	でいすとりびゅーしょん	Linux のカーネルに周辺のソフトを組み合わせ、すぐ使える形にまとめて配布したもの。例：Ubuntu、Red Hat Enterprise Linux	p.109
データ	でーた	コンピューターで扱うために記録された、数や文字などの値	p.16
データセンター	でーたせんたー	多数のサーバー・保存装置・通信機器を動かすための、電源・冷却・通信・入退室管理を備えた施設	p.133
データベース	でーたべーす DB = Database	目的に合わせて整理し、まとめて蓄えたデータの集まり（例：顧客表と注文表）	p.142

用語索引（40/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
デコード	でこーど	命令の解読。取り出した命令が何を求めているかを調べ、どの部分をどう動かすかを決めること	p.55
テスト	てすと	設計どおりに動くか、障害時にどうなるかを確認かめること	p.546
デフォルトゲートウェイ	でふおるとげーとうえい	LAN の外へデータを送るときに渡す出口の機器（家庭ではルーター）の IP アドレス	p.235
デプロイ	でぷろい	作ったアプリを実行環境に配置し、設定・起動して、利用者が使える状態にする工程	p.341
DELETE	デリート	SQL で、表から行を削除する命令	p.164
動的コンテンツ	どうてきこんてんつ	要求のたびにプログラムが中身を作るページ。見る人や時刻で中身が変わる	p.324

用語索引（41/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
トークン	とーくん	LLM が文章を扱う区切りの単位。料金や上限の 数え方にも使われる	p.503
Docker	どっかー	コンテナのイメージを作る・配る・コンテナを 実行するための代表的な道具（ソフトウェア）	p.427
ドメイン名	どめいんめい	shop.example.com のように、インターネッ ト上の機器やサービスを人が覚えやすい文字で 表した名前	p.246
トランザクション	とらんざくしょん	複数の操作をひとまとまりとして扱い、すべて 成功したら確定し、途中で失敗したら全体を取り 消して整合性を守る仕組み。DBMS の機能	p.174
トレース	とれーす	1 つの要求が、複数の処理をどの順に通る、ど こで時間がかかったかの記録	p.559

用語索引（42/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
NAS	なす Network Attached Storage（ネットワークにつないだ保存装置）	LAN などのネットワーク越しに、ファイルを提供する保存装置・システム	p.357
NAT	なっと Network Address Translation	アドレス変換。ルーターなどがパケットの IP アドレスを書き換え、対応を覚えておく仕組み	p.252
NAPT	なぶと Network Address Port Translation	IP アドレスに加えてポート番号も書き換え、複数の端末が 1 つの公開 IP を共有できるようにする NAT の一種	p.253
2 進数	にしんすう	0 と 1 の 2 つの数字だけで表す数の書き方。1 桁上がるごとに重みが 2 倍になる	p.24

用語索引（43/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
NIC	ニック Network Interface Card	機器をネットワークにつなぐ通信用の部品。0と1と信号を相互に変換して送受信する	p.194
ニューラルネットワーク	にゅーらるねっとわーく	入力に重みを掛けて足し、次へ渡す小さな計算を、層状に多数つないだモデルの形	p.497
入力装置	にゅうりよくそうち	外から数や文字、操作を受け取る装置。例：タッチパネル、キーボード、マイク、カメラ	p.18
認可	にんか	認証で確かめた相手に、どの操作・情報へのアクセスを許すかを判断すること	p.462
認証	にんしょう	要求や利用者が「誰か」を確かめること（ログインなど）	p.335

用語索引（44/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
ネットワーク	ねっとわーく	複数の機器をケーブルや電波でつなぎ、その間でデータを運べるようにした仕組み全体	p.200
NoSQL	のーえすきゅーえる	RDB（表と表の結び付き）以外の方式のデータベースをまとめた呼び名。文書型やキーと値の型など	p.180
PaaS	ぱーす Platform as a Service	アプリを動かすための、管理された実行の基盤を借りる形。利用者はアプリとデータを用意する	p.449
ハードウェア	はーどうえあ	手で触れる部品や機器。例：PC、サーバー、CPU、メモリ、SSD	p.30

用語索引（45/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
Python	ぱいそん	読みやすい文法の汎用プログラミング言語。ログの解析、API を使った情報の取得、AI などに広く使われる	p.664
バイト	ばいと B	8 ビットをまとめた単位。データの大きさを表すのに使う	p.24
ハイパーバイザー	はいぱーばいざー	物理的な機械の CPU ・メモリ・ディスクなどを区切り、複数の VM に割り当てるソフト	p.417
ハイブリッドクラウド	はいぶりっどくらうど	オンプレミス・プライベートクラウド・パブリッククラウドなど、複数の環境を組み合わせで使う形	p.447
パケット	ぱけっと	ネットワークで運ぶデータの小さなまとまり。宛先・送り元などの情報（付け札）が付く	p.203

用語索引（46/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
パケットロス	ぱけっとろす	送ったパケットの一部が途中で失われ、相手に届かないこと。割合（％）で表す	p.298
バックアップ	ばっくあっぷ	データの複製を別の場所に残し、失っても元に戻せるようにすること	p.132
バックホール	ばっくほーる	基地局と携帯電話会社の中心の設備（コア網）を結ぶ、基地局の先の回線	p.283
パブリッククラウド	ぱぶりっくくらうど	事業者の資源を、広く多数の顧客が分け合って使うクラウド	p.446
パラメータ	ぱらめーた	モデルの計算に使う数値（重み）。学習で調整される	p.490
パリティ	ぱりてい	複数のデータから計算して作る、失ったデータを計算で戻すための情報	p.370

用語索引（47/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
ハルシネーション	はるしねーしょん	AI が、もっともらしいが事実と違う内容を作ってしまうこと	p.507
PowerShell	パワーしえる	Microsoft のシェル兼スクリプト言語。Windows やマイクロソフト系サービスの管理に使い、結果を項目のまとまり（オブジェクト）として受け渡す	p.664
半導体	はんどうたい	電気を通す性質を制御できる材料と、それで作ったチップ（CPU ・ GPU ・ メモリなど）の総称	p.517
BMC	びーえむしー Baseboard Management Controller（基板の管理用コントローラー）	マザーボードに載った、業務用の OS から独立して動く管理用の小さなコンピューター。専用の口から管理 LAN につながる。	p.397

用語索引（48/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
PoC	ピー・オー・シー Proof of Concept	その方式で本当にできそうかを、小さく試して確かめること（実現性の検証）	p.546
PCIe	ピーシーあいえくすぷ れす PCI Express	GPU ・ ネットワーク部品 ・ NVMe SSD などが使う、部品どうしの高速な接続の規格	p.87
PDU	ピーでいーゆー Power Distribution Unit （電源分配装置）	ラックの中の機器へ電気を配る、コンセントがたくさん付いた装置	p.399
非機能要件	ひきのうようけん	性能・止まりにくさ・安全・運用など「どの条件で動くか」についての要件	p.541
ビット	びっと bit	0 か 1 かを表す、データの最小の単位（1 桁）	p.24

用語索引（49/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
表	ひょう	同じ項目をもつ記録を、行と列に並べて整理したもの。データベースではテーブルとも呼ぶ	p.140
ping	ピン	相手に小さな確認のデータを送り、返事が戻るまでの往復時間（RTT）と、届かなかった割合を調べるコマンド	p.312
ファームウェア	ふあーむうえあ	機器に組み込まれ、その機器の基本動作を制御するソフトウェア。OS・アプリとは別に更新が必要なことがある	p.32
ファイアウォール	ふあいあうおーる	決めた条件（送り元・あて先・ポート番号など）に合う通信だけを通し、それ以外を止める仕組み	p.335

用語索引 (50/64)

用語	読み／略語＝正式名称	やさしい意味	説明ページ
ファイバチャネル	ふあいばちゃねる	保存装置へのアクセスのために設計された、高速な専用の網と約束。 SAN で使われる	p.363
ファイル	ふあいる	名前を付けてひとまとまりにしたデータ。例：「注文一覧.xlsx」「写真 001.jpg」	p.112
ファイルシステム	ふあいるしすてむ	ストレージの中をファイルとフォルダーに整理し、名前・場所・権限・空き領域を管理する仕組み。例：NTFS、ext4、APFS	p.112
ファイルストレージ	ふあいるすとれーじ	保存したデータを、フォルダーとファイル名で扱えるように見せる保存方式。共有フォルダーなど	p.350
ファインチューニング	ふあいんちゅーにんぐ	学習済みモデルに追加のデータで学習させ、パラメータを調整すること	p.524

用語索引（51/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
ファウンドリ	ふあうんどりー	他社が設計した半導体の製造を請け負う会社	p.519
VLAN	ぶいらん Virtual LAN	1つの物理的なネットワーク（スイッチやケーブル）を、設定によって論理的に複数の LAN に分ける仕組み	p.609
フェールオーバー	ふえーるおーばー	障害が起きたとき、処理を予備の機器へ自動で切り替えること	p.552
フェッチ	ふえっち	命令の取り出し。CPU が、次に実行する命令をメモリ（実際はキャッシュ）から読み込むこと	p.54
負荷分散	ふかぶんさん	届いた要求を、同じ仕事をする複数の機械に振り分けること。振り分ける装置やソフトをロードバランサーと呼ぶ	p.334

用語索引 (52/64)

用語	読み／略語＝正式名称	やさしい意味	説明ページ
不正アクセス	ふせいあくせす	使う権限のない人が、他人の ID を使ったり弱点を突いたりして、システムやデータに入り込むこと	p.461
物理	ぶつり	IT では、実際に存在して手で触れられる機械・部品・ケーブルなどの実体を指す言葉	p.411
プライベート IP アドレス	ぷらいべーとあいぴーあどれす	家や会社の LAN の中だけで使う IP アドレス。192.168. ～などの範囲が決められ、別の LAN でも同じ番号を使える	p.233
プライベートクラウド	ぷらいべーとくらうど	1 つの組織だけが使うクラウド。自社のデータセンターでも、外部の施設や委託管理でも成り立つ	p.446
フルバックアップ	ふるばっくあっぷ	対象のデータをすべてコピーするバックアップ	p.382

用語索引（53/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
ブレードサーバ	ぶれーどさーば	電源・通信・管理の部品を持つ共用の箱（筐体）に、薄い板状の計算機（ブレード）を何枚も差し込んで使うサーバー	p.387
プログラム	ぷろぐらむ	コンピューターにさせる仕事の手順を、命令の並びとして書いたもの	p.20
プログラムカウンタ	ぷろぐらむかうんた	次に取り出す命令のアドレスを覚えているレジスタ	p.69
プロジェクトマネジメント	PM = Project Management	期限のある仕事のまとまりを、範囲・期限・品質・費用・リスク・関係者を管理して完了させること	p.568
プロセス	ぷろせす	実行中のプログラムと、その状態・資源のまとまり。保存されているだけのファイルとは違う	p.110

用語索引（54/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
ブロックストレージ	ぶろっくすとれーじ	保存領域を同じ大きさの区画（ブロック）に分け、番号で読み書きさせる保存方式。使う側からはディスクのように見える	p.350
プロトコル	ぷろところ	機器どうしが通信するための約束ごと（信号・形式・宛先・順番・失敗時の扱いなど）	p.201
平均	へいきん	全員の値を足して人数で割った値	p.637
変更管理	へんこうかんり	設定や構成の変更を、計画・確認・記録して安全に行うこと	p.132
ポート番号	ぽーとばんごう	1 台の機器の中で、どのプログラムの通信の受け口かを示す番号（0 ～ 65535）。例：443	p.229
保守	ほしゅ	部品の交換やソフトの更新をして、正常に動く状態を保つこと	p.132

用語索引（55/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
補助記憶	ほじょきおく	電源を切っても内容が残るように、プログラムやデータを長く保存する記憶。実物は SSD や HDD	p.43
ホスト OS	ほすとおえす	物理的な機械の上で直接動く OS。その上で仮想化ソフトやコンテナが動くときの土台になる	p.420
ボトルネック	ぼとるねっく	処理の流れの中で最も遅く、全体の速さを決めてしまう箇所（瓶の首のように細い所）	p.98
マザーボード	まざーぼーど	CPU ・メモリ ・保存装置などを差し込み、配線で互いにつなぐ中心の基板	p.39
MAC アドレス	まっくあどれす MAC = Media Access Control address	NIC ごとの番号。同じ区間の中で、次にデータを手渡す相手を指定するのに使う	p.227

用語索引（56/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
マネージドサービス	まねーじどさーびす	機械の用意やソフトウェアの更新などの運用の一部を事業者が担い、利用者は通信回線越しに使うサービス	p.180
ミドルウェア	みどるうえあ	OS とアプリの中間で、多くのアプリが共通に使う機能や実行環境を提供するソフトウェア。 例：データを管理するソフト、Web ページを返すソフト	p.115
ミラーリング	みらーりんぐ	同じデータを 2 台以上のドライブに同時に書き、鏡のように同じ内容を持たせること	p.369
無線 LAN	むせんらん	電波でつなぐ LAN 。 Wi-Fi はその代表的な規格の名前	p.210

用語索引（57/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
命令	めいれい	CPU が実行できる小さな指示。読み込む・足す・比べる・書き込むなど	p.21
命令実行サイクル	めいれいじっこうさいくる	CPU が命令の取り出し→解読→実行（→書き戻し）を 1 命令ずつ繰り返すこと	p.61
メトリクス	めとりくす	定期的に測った数値（CPU 使用率・応答時間・エラー率など）。グラフで傾向を見る	p.559
メモリ	めもり RAM ・ DRAM = Random Access Memory ・ Dynamic RAM	実行中のプログラムと作業中のデータを一時的に置く部品（主記憶）。一般的な DRAM は電源を切ると内容が消える	p.44
モデム	もでむ	テレビ用のケーブルや電話線などの回線の信号と、家の機器が扱う信号を相互に変換する機器	p.213

用語索引（58/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
モデル	もでる	入力を受け取り、決まった計算をして出力を返す仕組み。計算の構造と数値の組でできている	p.488
問題管理	もんだいかんり	インシデントの根本原因を特定し、再発防止策を実施する活動	p.564
YAML	やむる	設定やデータを、字下げ（行頭の空白）と「項目名：値」で書く、人が読みやすい文字の形式	p.671
URL	ゆーあーるえる Uniform Resource Locator	Web 上のページなどの場所を「通信の約束・サーバーの名前・サーバーの中の場所」で示す書き方	p.320
UDP	ゆーでいーピー User Datagram Protocol	順番の管理や送り直しをせず、小さなまとまりを軽く送る届け方の約束	p.258

用語索引（59/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
UPS	ゆーぴーえす Uninterruptible Power Supply（無停電電源装置）	停電したとき、電池にためた電気で一定時間、給電を続ける装置	p.399
要求	ようきゅう	使う側が提供する側へ送る「これをして」という依頼のデータ	p.123
要件定義	ようけんていぎ	システムに何を、どの条件で実現させるかを顧客と合意して決める工程	p.540
RAG	らぐ Retrieval-Augmented Generation	質問に関係する文書を検索し、モデルへの入力に添えて回答させる方法	p.523

用語索引（60/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
ラック	らっく	サーバーやスイッチなどの機器を、上下に積み重ねて載せる金属の棚。高さは U（1U = 44.45mm）で数える	p.387
LAN	らん Local Area Network	家や 1 つの建物の中など、近い範囲の機器をつなぐネットワーク。有線でも Wi-Fi でもよい	p.205
リレーショナルデータベース	りれーしょなるでーたべーす RDB = Relational Database	データを行と列の表で持ち、表どうしを共通の値で結び付けて使うデータベース	p.162
ルーター	るーたー	ネットワークとネットワークの境目で、宛先を見て次の渡し先を決めて中継する機器	p.212

用語索引（61/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
ルーティング	るーていんぐ	経路制御。宛先に応じて、パケットの次の渡し先を決めること。ルーターどうしが経路の情報を交換する	p.256
冷却	れいきゃく	機器が出す熱を、空気や液体で運び、建物の外へ捨てて、部品を適切な温度に保つこと	p.402
RACI	れいしー Responsible ・ Accountable ・ Consulted ・ Informed	作業ごとに、実行・最終責任・相談・共有の担当を整理した表	p.570
レイテンシー	れいてんしー	1 回の処理や読み書きを頼んでから結果が返るまでの待ち時間（応答時間）。ms ・ μ s で表す	p.96

用語索引 (62/64)

用語	読み／略語＝正式名称	やさしい意味	説明ページ
RAID	れいど Redundant Arrays of Independent Disks (独立したディスクを 冗長に並べたもの)	複数のドライブにデータを分けたり複製したり して、速さや故障への強さを得る技術。	p.366
RAID 6	れいどしっくす	2 種類のパリティを持つ RAID 。通常 2 台の同 時故障まで耐え、使える容量は $(N-2) \times D$	p.373
RAID 0	れいどぜろ	ストライピングだけを行う RAID 。速いが、1 台でも壊れるとデータを失う	p.368
RAID 10	れいどてん	ミラーリングした組を複数作り、その組にスト ライピングで分けて書く RAID 。使える容量は $N \times D \div 2$	p.374

用語索引（63/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
RAID 5	れいどふあいぶ	データとパリティを 3 台以上のドライブに分散して置く RAID。通常 1 台の故障まで耐え、使える容量は $(N-1) \times D$	p.372
RAID 1	れいどわん	ミラーリングを行う RAID。2 台構成なら 1 台が壊れてもデータが残る。使える容量は 1 台分	p.369
レジスタ	れじすた	CPU（コア）の中のごく小さく非常に速い記憶。計算に使う値や途中の結果を置く	p.68
REST	れすと	Web の仕組み（URL と HTTP）に合わせて API を設計する考え方。データの場所を URL で、操作を HTTP のメソッド（GET＝取得など）で表す	p.666
列	れつ	表の縦の 1 並び。1 つの項目（例：数量）にあたる。フィールドとも呼ぶ	p.140

用語索引（64/64）

用語	読み／略語＝正式名称	やさしい意味	説明ページ
レプリケーション	れぷりけーしょん	データを別の装置や別の場所へ写し続け、同じ内容を持たせる仕組み。複製	p.378
ログ	ろぐ	システムで起きた出来事（いつ・誰が・何をしたか・結果）を順に書き残した記録	p.480
論理	ろんり	IT では、ソフトや設定で作った区切り・見え方を指す言葉（例：1 台の SSD を分けた C ドライブと D ドライブ）	p.411
WAF	わふ Web Application Firewall	Web アプリへの不正な要求（攻撃の書き方を含む要求など）を見分けて止める仕組み	p.335
WAN	わん Wide Area Network	離れた拠点どうしをつなぐ広い範囲のネットワーク	p.205